AIM Foundations

Curvature-Driven Modelling, Classification, Runtime Evaluation, Realization, and Optimization of Structured Railway Alignments

Uwe Falz

29. Mai 2026

Inhaltsverzeichnis

Preface

This manuscript is intended as a long-term scientific foundation for alignment-based information modelling.

It is not merely software documentation, but a structured knowledge base covering:

- intrinsic railway geometry,
- curvature-driven reconstruction,
- runtime evaluation,
- realization processes,
- vehicle-space interpretation,
- optimization,
- and Earth-attached railway geometry.

The AIM framework treats railway alignments not merely as static curves, but as coupled operator-based state systems connecting geometry, construction, runtime interpretation, and operational behaviour.

Executive Summary

Alignment-Based Information Modelling (AIM)

Railway alignments are traditionally described as geometric curves in space. This perspective is insufficient for modern engineering tasks, which require the integration of design, measurement, construction, and optimization.

Alignment is not a curve. It is a curvature-driven system whose geometry emerges from a structured process.

This work introduces *Alignment-Based Information Modelling (AIM)* as a unified framework for this perspective.

—

Core Idea

AIM represents alignments intrinsically by curvature:

$$\kappa(s).$$

This formulation:

- unifies geometry and dynamics,
- reduces model complexity,
- enables efficient computation and optimization.

—

From Representation to Process

AIM replaces static modelling by an operator pipeline:

$$\mathcal{P} = \mathcal{W}^{-1} \circ \mathcal{R} \circ \mathcal{M} \circ \mathcal{W} \circ \mathcal{C}.$$

This pipeline connects:

- coordinate systems,
- world geometry,

- intrinsic track coordinates,
- modelling,
- realization,
- and optimization.

Alignment modelling becomes a process.

—

Hybrid Modelling

AIM combines structure and flexibility:

$$\kappa(s) = \kappa_{\text{param}}(s; \mathbf{T}) + \delta\kappa(s).$$

This hybrid representation:

- preserves engineering intent,
- integrates measurement data,
- avoids overfitting.

—

Realization as a First-Class Concept

The designed alignment is not identical to the constructed track.

A realization operator

$$\mathcal{R}$$

maps the theoretical model to physical geometry, introducing smoothing and constraints.

Design must therefore be performed in realized space.

—

Optimization

Alignment design is formulated as a structured nonlinear least-squares problem.

Due to:

- locality in arc-length,
- sparse Jacobians,
- intrinsic block structure,

the problem is efficiently solvable using SQP methods.

—

World-to-Track Mapping

A central component is the transformation:

$$x \leftrightarrow (s, d).$$

This mapping:

- links measurement data to the model,
- introduces nonlinear complexity,
- motivates hybrid and LOD-based strategies.

—

Key Principles

- curvature-centric modelling,
- separation of model and realization,
- hybrid parametric–discrete representation,
- operator-based architecture,
- optimization in realized space,
- exploitation of sparsity and structure.

—

Outcome

AIM provides a coherent framework for:

- reconstruction from imperfect data,
- physically meaningful optimization,
- integration of design, measurement, and construction,
- scalable computational implementation.

—

Conclusion

Alignment modelling is not a purely geometric task.

It is the integration of:

- geometry,
- physics,
- data,
- and computation.

AIM formalizes this integration as a structured operator framework, establishing a new foundation for railway alignment engineering.

Core Thesis

Alignment is not a curve.
It is a curvature-driven, realization-aware generative system.

Remark 0.1. Classical railway geometry represents track layouts as curves in Euclidean space. While sufficient for visualization, this perspective is inadequate for design, construction, and dynamic analysis.

Remark 0.2. Within the AIM framework, the fundamental object is not the curve $\gamma(s)$, but the curvature law $\kappa(s)$, complemented by the cant function $\phi(s)$. All geometric quantities are derived by integration.

Remark 0.3. This shifts the modelling paradigm from descriptive geometry to generative structure: alignments are defined by the laws that produce curves, not by the curves themselves.

—

Key Principles

Curvature as Primary Variable The alignment is defined by curvature $\kappa(s)$. Position $\gamma(s)$ is a derived quantity. This inversion simplifies modelling and exposes the intrinsic structure of railway geometry.

Realization as Operator The physically constructed track differs from the theoretical alignment. This transformation is captured by a realization operator $\mathcal{R}$, which includes discretization, interpolation, and mechanical response. Design must therefore be formulated in realized space.

Hybrid Representation Real-world data and engineering design require both structure and flexibility. A hybrid model combines parametric representations with local corrections, enabling robust reconstruction from imperfect data.

Analytical Structure All relevant quantities arise from arc-length based relations. As a consequence, derivatives with respect to parameters are available in closed form. This eliminates the need for numerical differentiation.

Efficient Optimization Alignment optimization reduces to a structured nonlinear least-squares problem with sparse and banded Jacobians. This enables efficient solution via Gauss–Newton or SQP methods.

—

Implication

Remark 0.4. The AIM framework unifies geometry, dynamics, realization, and data processing within a single coherent model.

It replaces curve-based thinking by a system-based perspective in which alignment, realization, and optimization are intrinsically linked.

Remark 0.5. This provides a foundation for alignment-based information modelling, capable of bridging theoretical design and practical railway engineering.

Abstract

Railway alignments are traditionally represented as geometric curves in space. This perspective is insufficient for modern engineering tasks, which require the integration of design, measurement, construction, and optimization.

This work introduces *Alignment-Based Information Modelling (AIM)*, a unified framework that reformulates alignment modelling as a curvature-driven and realization-aware system. Instead of treating geometry as a primary object, AIM represents alignments intrinsically through curvature and cant, from which all geometric and dynamic quantities are derived.

A central contribution is the introduction of a realization operator, which models the transformation from theoretical alignment to physically constructed track. This leads to the formulation of alignment design in realized space rather than in purely geometric terms.

The framework combines parametric structure with discrete corrections in a hybrid representation, enabling robust reconstruction from imperfect data. Alignment optimization is formulated as a structured nonlinear least-squares problem with sparse analytical Jacobians, allowing efficient solution via SQP methods.

AIM provides a coherent foundation that unifies geometry, dynamics, data, and computation, establishing a new perspective on railway alignment engineering.

Introduction

1. A Shift in Perspective

Remark 0.1. Railway alignment is commonly described as a geometric curve $\gamma(s)$ embedded in space.

Remark 0.2. This view is sufficient for visualization, but it fails to capture the quantities that actually govern railway design: curvature, cant, dynamics, and realization.

Remark 0.3. In practice, railway tracks are not designed, constructed, or maintained as curves, but through operations acting on curvature and discrete control points.

Summary 0.4. Alignment is not a curve. It is a curvature-driven system whose geometry emerges from its generative structure.

—

2. Limitations of Current Practice

Remark 0.5. Despite a long tradition of railway engineering, alignment modelling remains fragmented across several domains:

- geometric construction of curves,
- rule-based engineering design,
- data exchange formats such as LandXML and IFC,
- numerical evaluation and simulation.

Remark 0.6. These aspects are typically handled in separate tools and workflows, leading to discontinuities between modelling, analysis, and design.

Remark 0.7. In particular:

- geometry is treated independently of dynamics,
- transition curves are selected from predefined catalogues,
- data formats focus on representation rather than computation,
- optimization is not part of the core modelling paradigm.

Remark 0.8. As a consequence, alignment design lacks a unified internal model that connects data, geometry, dynamics, and realization.

—

3. Problem Statement

Remark 0.9. The absence of a unified modelling framework leads to fundamental limitations:

- inconsistent interpretation of alignment data,
- separation of geometry and dynamics,
- limited support for optimization-based design,
- high effort for integrating heterogeneous data sources.

Remark 0.10. These issues become increasingly critical as infrastructure systems grow in complexity and demand higher levels of automation, precision, and traceability.

—

4. AIM: A Unified Framework

Remark 0.11. This work introduces **Alignment-Based Information Modelling (AIM)** as a framework addressing these limitations.

Remark 0.12. The central idea is to represent alignment as a structured, state-based object defined by curvature and cant:

$$(\kappa(s), \phi(s)),$$

from which all geometric and dynamic quantities are derived.

Remark 0.13. Within this framework:

- curvature is the primary modelling variable,
- geometry is obtained by integration,
- transition curves are interpreted as control functions,
- alignment becomes a generative system rather than a static object.

—

5. Core Claim

Proposition 0.14. *Railway alignment modelling can be reformulated as a curvature-driven, realization-aware, and optimization-ready system.*

Remark 0.15. This reformulation enables a unified treatment of:

- geometry,
- dynamics,

- construction and realization,
- data integration,
- and optimization.

—

6. Consequences

Remark 0.16. Adopting this perspective leads to several key consequences:

- alignment design becomes an optimization problem,
- realization is treated as an operator acting on the model,
- data reconstruction is formulated as an inverse problem,
- hybrid models bridge parametric structure and measured data,
- analytical derivatives arise naturally from the formulation.

—

7. Scope of the Work

Remark 0.17. The present manuscript develops the AIM framework across the following dimensions:

- mathematical foundations of curvature-based modelling,
- structured representation of alignment elements,
- classification and interpretation of transition curves,
- modelling of realization and construction processes,
- formulation of optimization problems and solution methods,
- integration of real-world data and coordinate systems,
- application to railway engineering use cases.

—

8. Contribution

Remark 0.18. The main contributions of this work are:

- a unified state-based alignment model,
- the reinterpretation of transition curves as curvature control functions,
- the introduction of a realization-aware modelling perspective,
- the formulation of alignment design as a structured optimization problem,
- the development of a pipeline connecting data, model, and computation.

Remark 0.19. Together, these contributions establish a coherent framework linking theoretical modelling and practical engineering.

—

9. Structure of the Manuscript

Remark 0.20. The manuscript is organized into several parts, progressing from conceptual foundations to practical applications:

- motivation and conceptual shift,
- mathematical and state-based foundations,
- modelling and representation,
- realization and system perspective,
- optimization methods,
- engineering constraints and standards,
- applications and outlook.

—

10. Outlook

Remark 0.21. The AIM framework is intended as a foundation for future development.

Remark 0.22. It opens the path toward:

- optimization-driven design systems,
- deeper integration with BIM,

- network-level alignment modelling,
- interactive and real-time engineering tools.

Remark 0.23. In this sense, the work contributes to a broader transition from static geometry toward computational infrastructure modelling.

Glossary

Core Concepts

Alignment A structured representation of railway track geometry defined primarily through curvature $\kappa(s)$, together with associated state variables such as position, orientation, profile, and cant. An alignment is not merely a curve, but a system linking geometry, dynamics, realization, and operational behaviour.

Arc-Length Domain The intrinsic one-dimensional domain s shared by all synchronized longitudinal alignment bands. Within AIM, arc-length acts as the primary reference structure for geometry, cant, profile, dynamics, and optimization.

Curvature $\kappa(s)$ The fundamental geometric quantity describing the rate of change of direction along the track. Within AIM, curvature is the primary design variable.

In planar formulations this coincides with the standard curvature. On curved surfaces, the physically relevant quantity becomes geodesic curvature.

Cant The rail height difference between both rails, represented as a longitudinal band along the alignment.

The corresponding roll angle

$$\phi(s) = \arctan\left(\frac{u(s)}{g}\right)$$

is derived using the applicable gauge g.

Pose2 A minimal planar alignment state consisting of:

$$\gamma(s), \qquad \theta(s).$$

Pose2 forms the reconstruction kernel of planar alignment geometry.

Pose3 A runtime-derived spatial railway state combining:

- planar geometry,
- profile evaluation,
- cant evaluation,
- and spatial frame construction.

State A geometric or physical description of the railway system at a given arc-length position s.

Sparse Alignment Structure

cGeom The central curvature-driven geometric band of the sparse alignment model. It contains:

- fixed elements,
- transition elements,
- pose2-based reconstruction anchors.

Cant Band A longitudinal band storing cant information correlated with fixed alignment elements.

The cant band:

- has no independent stationing system,
- stores rail height differences,
- follows the same normalized curvature family as the associated cGeom element,
- may contain local station offsets.

Profile Band A longitudinal band representing vertical geometry independently from planar curvature geometry.

Fixed Element An alignment element with constant curvature.

Typical special cases are:

- straight segments,
- circular arcs.

Transition Element An alignment element with non-constant curvature evolution.

Transition elements describe controlled changes between curvature states.

Sparse Representation A compact, reconstruction-oriented representation of an alignment using semantically meaningful elements rather than dense sampled geometry.

The sparse representation is reconstruction-oriented rather than sample-oriented.

Runtime Pose3 A pose3 state evaluated dynamically from cGeom, cant, profile, and runtime operators rather than stored explicitly as persistent truth.

Modeling and Representation

Parametric Model A low-dimensional representation of curvature or cant defined by a small set of parameters $\mathbf{T}$, such as transition types or segment lengths.

Sample-Based Model A discrete representation of curvature or geometry defined at a set of points $\{s_i\}$, typically used for measurement data or numerical optimization.

Hybrid Model A combination of parametric structure and discrete corrections:

$$\kappa(s) = \kappa_{\text{param}}(s; \mathbf{T}) + \delta\kappa(s).$$

It allows structured modelling with local flexibility.

Normalized Curvature Law A curvature law defined on the normalized interval

$$u \in [0, 1],$$

separating shape from physical scale.

Operators and Runtime Evaluation

Sampling Operator $\mathcal{S}$ Maps a continuous function to discrete values:

$$\mathcal{S}(\kappa) = \{\kappa(s_i)\}.$$

Interpolation Operator $\mathcal{I}$ Reconstructs a continuous curve or function from discrete data.

Mechanical Operator $\mathcal{M}$ Represents the physical response of the track structure, including smoothing and deformation effects.

Realization Operator $\mathcal{R}$ Maps a theoretical alignment to its physically realized form:

$$\mathcal{R} = \mathcal{M} \circ \mathcal{I} \circ \mathcal{S}.$$

Adjustment Operator $\mathcal{T}$ A local operator representing one construction or maintenance step, for example tamping:

$$\gamma_{k+1} = \mathcal{T}(\gamma_k).$$

Runtime Evaluation Dynamic computation of geometric or physical states from sparse bands and mathematical operators rather than from explicitly stored geometry.

Spaces and Transformations

Track Space A coordinate system aligned with the track, typically parameterized by arc-length s, where geometry and dynamics are defined.

World Space A global coordinate system, often CRS-based, used for georeferencing and data integration.

World-to-Track Transformation The mapping from world coordinates to intrinsic alignment coordinates, typically requiring projection onto the alignment and evaluation of s, d, κ, and cant.

Kilometre Line A longitudinal reference structure used for stationing, independent of the physical geometric alignment.

Optimization

Residual A component of the objective function representing deviation from a desired property, such as smoothness, dynamic equilibrium, or data fit.

Jacobian The matrix of derivatives of residuals with respect to parameters.

Within AIM, Jacobians are preferably derived analytically from arc-length-based relations.

Gauss–Newton Method An optimization method using the approximation

$$H \approx J^T J$$

for solving nonlinear least-squares problems.

Sequential Quadratic Programming (SQP) An iterative optimization framework solving a sequence of quadratic subproblems based on linearized constraints.

Realization and Construction

Realized Alignment The physically constructed track geometry, which differs from the theoretical alignment due to discretization, construction, and mechanical effects.

Control Points Discrete positions $\{s_i\}$ at which geometry is specified or adjusted during construction or maintenance.

Realization Error The deviation between target and realized curvature or geometry.

Inverse Realization Problem The problem of finding a target alignment such that

$$\mathcal{R}(\kappa_{\text{target}}) \approx \kappa_{\text{desired}}.$$

Advanced Geometry

Geodesic Curvature κ_g The intrinsic curvature of a curve on a surface, measuring the change of direction within the surface.

For railway alignment on the Earth, this is the physically relevant curvature.

Normal Curvature κ_n The component of curvature induced by the curvature of the underlying surface, for example the Earth ellipsoid.

Ellipsoidal Alignment An alignment defined directly on a reference ellipsoid rather than in a projected coordinate system.

Surface Frame A local coordinate frame on the ellipsoid consisting of tangent, lateral, and surface-normal directions.

Cant Rotation Rotation of the track cross-section around the tangent direction, defining superelevation.

Effective Lateral Acceleration The physically relevant lateral acceleration acting on a vehicle:

$$a_{\text{eff}} = v^2\kappa_g - \mathbf{g} \cdot \mathbf{b}_\phi.$$

Pipeline

Pipeline The sequence of transformations from raw data to optimized alignment:

$$\text{data} \to \text{model} \to \text{state} \to \text{realization} \to \text{optimization}.$$

Failure Modes Typical breakdown points within the pipeline, such as inconsistent data, invalid coordinate systems, unstable projection, or failed optimization.

Design Principles Guiding principles ensuring robustness, interpretability, and consistency across the pipeline.

Teil I.

Why Alignments Matter

Remark 0.24. This part introduces the conceptual motivation of the AIM framework. It explains why railway alignments cannot be treated as mere geometric curves, but must be understood as structured engineering objects.

Remark 0.25. The central claim is that alignment modelling must integrate geometry, dynamics, realization, and data interpretation within a single coherent framework.

Remark 0.26. The following chapters therefore develop the problem setting, the need for a new perspective, and the guiding vision behind alignment-based information modelling.

1. Motivation

1.1. The Hidden Complexity of Railway Alignments

Remark 1.1. At first glance, a railway alignment appears to be a simple geometric object: a curve connecting two points in space.

Remark 1.2. However, this view is misleading. An alignment is not merely a curve, but the result of a complex interaction between geometry, physics, engineering constraints, and operational requirements.

Remark 1.3. Even small changes in curvature or cant can have significant effects on:

- passenger comfort,
- vehicle stability,
- track wear,
- and operational safety.

1.2. Fragmentation of Current Practice

Remark 1.4. In current engineering practice, alignment design is typically divided into separate domains:

- geometric design (plan and profile),
- cant design,
- dynamic evaluation,
- and regulatory verification.

Remark 1.5. These aspects are often treated sequentially rather than jointly. As a consequence:

- design iterations are required,
- optimal solutions are rarely achieved,
- and knowledge remains fragmented across tools and disciplines.

1.3. Limitations of Existing Models

Remark 1.6. Traditional alignment models are primarily geometric. They describe the track as a sequence of elements such as:

- straight lines,
- circular arcs,
- and transition curves.

Remark 1.7. While this representation is sufficient for construction, it is insufficient for:

- capturing dynamic behaviour,
- integrating measurement data,
- or supporting automated optimization.

Remark 1.8. In particular, curvature is often treated as a derived quantity, although it is the fundamental driver of railway dynamics.

1.4. Need for a Unified Representation

Remark 1.9. A modern alignment model must integrate:

- geometry,
- curvature evolution,
- cant,
- and dynamic effects.

Remark 1.10. Such a model should not treat these aspects independently, but as components of a single coherent system.

Remark 1.11. This requires a shift from element-based descriptions to function-based representations.

1.5. Alignment as a State Trajectory

Remark 1.12. Instead of viewing an alignment as a static geometric object, it can be interpreted as a trajectory of a system evolving along arc length.

Remark 1.13. In this interpretation:

- curvature acts as a control variable,
- cant represents an additional state component,

- and the resulting motion determines physical behaviour.

Remark 1.14. This perspective naturally connects alignment design with modern concepts from:

- differential geometry,
- numerical analysis,
- and optimal control theory.

1.6. Towards Alignment-Based Information Modelling

Remark 1.15. The observations above motivate the development of a new modelling paradigm: *alignment-based information modelling* (AIM).

Remark 1.16. The central idea of AIM is to treat alignment not as a collection of geometric primitives, but as a structured, functional object defined by curvature and its derived quantities.

Remark 1.17. Within this framework:

- transitions are primary modelling elements,
- dynamics are embedded in the representation,
- and optimization becomes a natural design tool.

1.7. Connection to Practical Engineering

Remark 1.18. Despite its mathematical formulation, the AIM approach is not purely theoretical.

Remark 1.19. It directly addresses practical challenges such as:

- handling imperfect measurement data,
- integrating multiple coordinate systems,
- modelling complex track configurations,
- and supporting modern BIM workflows.

1.8. Objective of this Work

Remark 1.20. The objective of this manuscript is to provide a coherent foundation for alignment-based information modelling.

Remark 1.21. This includes:

- a mathematical description of alignment structures,
- a unified state-based representation,
- methods for numerical reconstruction,
- integration of engineering constraints,
- and approaches to optimization.

Summary 1.22. Railway alignments are not simple geometric curves, but complex systems governed by curvature, cant, and dynamic effects.

A unified, curvature-driven modelling approach is required to capture this complexity and to enable modern, optimization-based design methods.

This work develops the foundations of such an approach.

2. Problem Statement

2.1. From Motivation to Formalization

Remark 2.1. The previous chapter has argued that railway alignment design is intrinsically a coupled problem involving geometry, dynamics, and engineering constraints.

Remark 2.2. We now formulate this insight as a precise problem statement.

2.2. Core Deficiency of Existing Approaches

Remark 2.3. Current alignment models are predominantly:

- element-based,
- geometry-centered,
- and weakly connected to dynamic behaviour.

Remark 2.4. As a consequence, they exhibit the following structural deficiencies:

- curvature is not treated as a primary design variable,
- cant is handled in a separate design process,
- dynamic quantities are evaluated only after geometric design,
- optimization is indirect and iterative rather than intrinsic.

2.3. Formal Gap

Remark 2.5. Let

$$\gamma : [0, L] \to \mathbb{R}^2$$

denote a planar alignment.

Remark 2.6. In classical formulations, the curve γ is the primary object, while curvature is derived as

$$\kappa(s) = \theta'(s).$$

Remark 2.7. However, from a dynamic perspective, the situation is reversed:

- curvature determines acceleration,

- curvature variation determines jerk,
- and therefore curvature governs physical behaviour.

Remark 2.8. This reveals a fundamental mismatch between:

- geometric modelling (curve-first),
- and physical interpretation (curvature-first).

2.4. Missing Coupling of State Variables

Remark 2.9. Let

$$\kappa(s), \quad \phi(s)$$

denote curvature and cant.

Remark 2.10. In practice, these variables are strongly coupled through

$$a_{\mathrm{eff}}(s) = v^2\kappa(s) - g\sin\phi(s).$$

Remark 2.11. Despite this coupling:

- κ is typically designed geometrically,
- ϕ is designed in a separate process,
- and their interaction is only checked afterwards.

Remark 2.12. This separation prevents a unified optimization of the system.

2.5. Data and Reality Gap

Remark 2.13. Modern railway projects rely heavily on real-world data, including:

- measurement polylines,
- legacy alignment descriptions,
- and heterogeneous coordinate systems.

Remark 2.14. Existing models struggle to:

- integrate noisy and incomplete data,
- reconcile different coordinate reference systems,
- and maintain a consistent representation across scales.

Remark 2.15. This leads to a disconnect between:

- theoretical alignment models,
- and real-world engineering data.

2.6. Lack of a Unified Optimization Framework

Remark 2.16. Alignment design problems naturally take the form:

$$\min J(\kappa, \phi)$$

subject to geometric and engineering constraints.

Remark 2.17. However, in current practice:

- the objective function is implicit or fragmented,
- constraints are applied in separate stages,
- and optimization is not formulated as a single problem.

Remark 2.18. This prevents the use of systematic numerical optimization methods.

2.7. Target Formulation

Remark 2.19. We aim to formulate alignment design as a unified problem:

$$\text{find } (\kappa, \phi)$$

such that:

- geometric consistency is satisfied,
- engineering constraints are fulfilled,
- and a global objective functional is minimized.

Remark 2.20. This requires:

- a curvature-based representation,
- a coupled state model,
- and a consistent treatment of data and constraints.

2.8. Scope of the Problem

Remark 2.21. The problem addressed in this work includes:

- modelling of alignments via curvature functions,
- integration of cant as a state variable,
- reconstruction of geometry from curvature,

- incorporation of engineering rules as constraints,
- and formulation of optimization problems.

Remark 2.22. The problem explicitly excludes:

- detailed multibody vehicle dynamics,
- full infrastructure modelling,
- and construction-specific considerations.

2.9. Research Questions

Remark 2.23. The following central questions arise:

- How can alignment be represented in a curvature-driven manner?
- How can curvature and cant be modelled as a coupled state?
- How can real-world data be integrated into such a model?
- How can engineering rules be expressed as mathematical constraints?
- How can alignment design be formulated as an optimization problem?

2.10. Working Hypothesis

Remark 2.24. We hypothesize that:

- a curvature-based state representation,
- combined with a unified optimization framework,

provides a more consistent and powerful foundation for railway alignment design.

Remark 2.25. In particular, we assume that:

- transition curves emerge naturally from optimization,
- geometry and dynamics can be unified,
- and real-world data can be systematically integrated.

2.11. Alignment

2.11.1. Motivation

Remark 2.26. In classical geometry, a railway track is often represented as a curve $\gamma(s)$ in space.

Remark 2.27. This representation is insufficient for engineering purposes, as it does not capture the intrinsic quantities governing construction, dynamics, and realization.

Remark 2.28. In particular, curvature and cant are not derived properties, but primary design variables.

—

2.11.2. Definition

Definition 2.29 (Alignment)**.** An **alignment** is a structured object defined over arc-length $s \in [0, L]$, consisting of the following components:

- a curvature function
$$\kappa : [0, L] \to \mathbb{R},$$
- a cant function
$$\phi : [0, L] \to \mathbb{R},$$
- an initial pose
$$(\gamma_0, \theta_0),$$

together with the induced state variables

$$\theta(s) = \theta_0 + \int_0^s \kappa(\sigma)\,\mathrm{d}\sigma,$$

$$\gamma(s) = \gamma_0 + \int_0^s (\cos\theta(\sigma), \sin\theta(\sigma))\,\mathrm{d}\sigma.$$

—

2.11.3. Interpretation

Remark 2.30. An alignment is not defined by its position $\gamma(s)$, but by its curvature $\kappa(s)$.

Remark 2.31. The curve $\gamma(s)$ is a derived quantity obtained by integration.

Remark 2.32. Thus, curvature is the primary design variable, while position is a consequence.

—

2.11.4. State Structure

Definition 2.33 (Alignment state)**.** The **state** of an alignment at position s is given by

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

Remark 2.34. This state couples geometry, orientation, and dynamics in a unified representation.

—

2.11.5. Structural Nature

Proposition 2.35. *An alignment is a low-dimensional structured object, even though its realization may involve high-dimensional data.*

Remark 2.36. This structure enables:

- efficient storage,
- robust optimization,
- meaningful interpretation.

—

2.11.6. Relation to Realization

Remark 2.37. The alignment represents the intended geometry.

Remark 2.38. The physically constructed track is given by the realization operator:

$$\gamma_{\text{real}} = \mathcal{R}(\kappa, \phi).$$

Remark 2.39. Thus, the alignment exists in design space, while the track exists in physical space.

—

2.11.7. Relation to Data

Remark 2.40. Measured data typically provide sampled positions or polylines.

Remark 2.41. An alignment must be reconstructed from such data via inverse problems.

—

2.11.8. Key Insight

Proposition 2.42. *An alignment is not a curve, but a curvature-driven generative model of a curve.*

—

2.11.9. Summary

Summary 2.43. The alignment is the central object of railway geometry.

It is defined by curvature and cant, from which all geometric and dynamic quantities are derived.

This perspective shifts the focus from describing curves to defining generative structures, forming the foundation of alignment-based information modelling.

2.12. Summary

Summary 2.44. Current alignment design methods suffer from fragmentation between geometry, dynamics, and data.

This work addresses the problem of developing a unified, curvature-driven framework that integrates these aspects into a single coherent model.

The goal is to enable consistent modelling, analysis, and optimization of railway alignments.

3. Vision

Why Railway is Different

Designing a railway alignment is fundamentally different from designing a road.

A road vehicle can steer.

A train cannot.

A road alignment may tolerate small geometric imperfections. A railway alignment cannot.

For a train:

- *curvature directly determines lateral acceleration,*
- *curvature variation determines jerk,*
- *and both directly affect safety, comfort, and wear.*

As a consequence:

- *geometry is not just shape,*
- *curvature is not just a derived quantity,*
- *and transitions are not optional smoothing elements.*

They are the system.

3.1. From Problem to Paradigm

Remark 3.1. The previous chapters have identified a fundamental limitation of current alignment modelling approaches: the fragmentation of geometry, dynamics, and data.

Remark 3.2. This motivates not only incremental improvements, but a shift in perspective.

3.2. Core Idea

Remark 3.3. The central idea of this work is to treat railway alignment as a *state trajectory* governed by curvature and its associated quantities.

Remark 3.4. Instead of describing alignment as a sequence of geometric elements, we propose to describe it as a function-based object:

$$(\kappa(s), \phi(s)).$$

Remark 3.5. In this view:

- curvature defines geometry,
- cant defines dynamic balancing,
- and their interaction determines physical behaviour.

3.3. Alignment as a Controlled System

Remark 3.6. The alignment is interpreted as a controlled system evolving along arc length:

$$\begin{aligned}
&\gamma'(s) = (\cos\theta(s), \sin\theta(s)), \\
&\theta'(s) = \kappa(s), \\
&\phi'(s) = \omega(s).
\end{aligned}$$

Remark 3.7. Here:

- $\kappa(s)$ acts as a geometric control,
- $\omega(s)$ governs cant evolution,
- and the resulting state determines both geometry and dynamics.

Remark 3.8. This formulation establishes a direct link between alignment design and optimal control theory.

3.4. Transitions as Primary Objects

Remark 3.9. In classical models, transitions are treated as auxiliary elements connecting geometric primitives.

Remark 3.10. In the proposed framework, transitions become the primary modelling objects, as they define the evolution of curvature.

Remark 3.11. This shifts the focus:

- from static geometry
- to controlled change of state.

3.5. Integration of Reality

Remark 3.12. A central requirement of the proposed approach is compatibility with real-world data.

Remark 3.13. This includes:

- measurement data with uncertainty,
- heterogeneous coordinate systems (CRS),
- legacy alignment descriptions,
- and complex track configurations.

Remark 3.14. The model must therefore support:

- robust reconstruction from imperfect data,
- explicit handling of coordinate transformations,
- and consistent representation across scales.

3.6. Unified Optimization Framework

Remark 3.15. Within this paradigm, alignment design is formulated as an optimization problem:

$$\min J(\kappa, \phi)$$

subject to:

- state dynamics,
- engineering constraints,
- and boundary conditions.

Remark 3.16. This allows:

- direct incorporation of comfort criteria,
- integration of dynamic effects,
- and systematic exploration of design alternatives.

3.7. Towards Alignment-Based Information Modelling

Remark 3.17. The proposed framework forms the basis of *alignment-based information modelling* (AIM).

Remark 3.18. AIM is characterized by:

- curvature-driven representation,
- state-based modelling,
- integration of geometry and dynamics,
- and compatibility with data-driven workflows.

Remark 3.19. In this sense, alignment is not a static object, but a structured, computable entity.

3.8. Implications

Remark 3.20. Adopting this perspective has several implications:

- design becomes a problem of state control,
- transitions emerge from functional requirements,
- optimization replaces iterative manual adjustment,
- and modelling becomes directly compatible with computation.

3.9. Long-Term Perspective

Remark 3.21. The proposed framework is intended as a foundation for future developments, including:

- integration with BIM systems,
- extension to network-level modelling,
- incorporation of vehicle dynamics,
- and real-time optimization.

Remark 3.22. It is not a finished solution, but a structured starting point.

3.10. Summary

Summary 3.23. This work proposes a paradigm shift from element-based alignment modelling to a curvature-driven, state-based representation.

By treating alignment as a controlled system and integrating geometry, dynamics, and data, it establishes a unified foundation for modelling and optimization.

This perspective forms the basis of alignment-based information modelling (AIM).

Teil II.

Mathematical Foundations

Remark 3.24. This part establishes the mathematical foundation of the AIM framework. Its purpose is not to develop abstract mathematics for its own sake, but to identify the intrinsic structures underlying railway alignment modelling.

Remark 3.25. In particular, the focus lies on arc-length based descriptions, curvature-driven geometry, and the distinction between primary and derived quantities.

Remark 3.26. These foundations provide the conceptual and formal basis for all later developments in modelling, realization, and optimization.

4. Axiomatic Foundation

4.1. Primitives

Definition 4.1. $s \in \mathbb{R}$ is the arc-length parameter.

Definition 4.2. $\gamma : I \to \mathbb{R}^2$ is a curve.

Definition 4.3. $\theta : I \to \mathbb{R}$ is the direction angle.

Definition 4.4. $\kappa : I \to \mathbb{R}$ is the curvature.

4.2. Axioms

Axiom 4.5.

$$\|\gamma'(s)\| = 1$$

Axiom 4.6.

$$\theta'(s) = \kappa(s)$$

Axiom 4.7.

$$\gamma'(s) = (\cos\theta(s), \sin\theta(s))$$

4.3. Consequence

Proposition 4.8. *Curvature uniquely defines the curve up to initial conditions.*

5. Notation and Terminology

5.1. Motivation

Remark 5.1. Railway alignment engineering combines concepts from geometry, surveying, dynamics, construction, realization, runtime evaluation, and operational railway systems.

Remark 5.2. Many inconsistencies in classical railway systems arise not from incorrect mathematics, but from ambiguous terminology and mixed conceptual layers.

Remark 5.3. The AIM framework therefore introduces a canonical notation and terminology system.

Remark 5.4. The purpose of this chapter is:

- to define canonical symbols,
- to stabilize terminology,
- to reduce ambiguity,
- and to separate fundamentally different conceptual layers.

5.2. General Principles

Principle 5.5 (Intrinsic-first principle). Whenever possible, railway geometry is formulated intrinsically using arc-length-based quantities.

Principle 5.6 (Runtime distinction principle). A distinction is made between:

- stored representations,
- runtime-evaluated states,
- and physically realized states.

Principle 5.7 (Target vs realization principle). Target geometry and realized geometry are treated as distinct objects.

Principle 5.8 (Canonical layer principle). AIM separates:

- intrinsic geometry,
- metric realization,

- runtime interpretation,
- physical realization,
- operational behaviour,
- and CRS-dependent embedding

into distinct conceptual layers.

5.3. Canonical Spaces

Definition 5.9 (State space notation).

$$\mathcal{X}$$

denotes the canonical railway state space.

Definition 5.10 (Runtime quantity space notation).

$$\mathcal{Y}$$

denotes the space of runtime-evaluated quantities.

Definition 5.11 (Track-space domain notation).

$$\Omega_{\text{track}}$$

denotes the intrinsic railway coordinate domain.

Definition 5.12 (World-space domain notation).

$$\Omega_{\text{world}}$$

denotes the CRS-dependent spatial world domain.

Definition 5.13 (Curvature space notation).

$$\mathcal{K}$$

denotes the space of admissible curvature evolutions and curvature laws.

5.4. Canonical Coordinates

5.4.1. Arc-Length

Definition 5.14 (Arc-length).

$$s$$

denotes intrinsic arc-length along the alignment.

Remark 5.15. Arc-length is the primary intrinsic coordinate within AIM.

5.4.2. Track Coordinates

Definition 5.16 (Track coordinates)**.** Track-space coordinates are:

$$(s, d, h),$$

where:

- s: longitudinal coordinate,
- d: lateral offset,
- h: vertical offset.

Remark 5.17. Track coordinates belong to Ω_{track}.

Remark 5.18. Additional operational coordinates may be introduced by runtime systems.

5.4.3. World Coordinates

Definition 5.19 (World coordinates)**.** World coordinates are denoted by:

$$x \in \Omega_{\text{world}}.$$

Remark 5.20. World coordinates are CRS-dependent.

Remark 5.21. When required, world-space quantities may explicitly use the notation:

$$x_{\text{w}}.$$

5.5. Geometry Symbols

5.5.1. Alignment Curve

Definition 5.22 (Alignment curve)**.**

$$\gamma(s)$$

denotes the intrinsic alignment geometry.

Remark 5.23. The alignment curve represents the geometric backbone of the railway alignment.

Remark 5.24. Depending on context, $\gamma(s)$ may denote planar geometry, spatial embedding, target geometry, or realized geometry. Such distinctions should be marked explicitly when relevant.

5.5.2. Orientation

Definition 5.25 (Orientation angle)**.**

$$\theta(s)$$

denotes horizontal alignment orientation.

5.5.3. Curvature

Definition 5.26 (Curvature)**.**

$$\kappa(s)$$

denotes curvature as function of arc-length.

Remark 5.27. Curvature is the primary intrinsic geometric quantity within AIM.

5.5.4. Cant

Definition 5.28 (Cant angle)**.**

$$\phi(s)$$

denotes cant rotation.

Remark 5.29. Within AIM, $\phi(s)$ is interpreted geometrically as frame rotation. Engineering source data may instead describe cant as rail height difference; the conversion is a convention-dependent runtime or import operation.

Remark 5.30. Depending on context, $\phi(s)$ may also be interpreted as roll angle of the local railway frame.

5.5.5. Cant Evolution

Definition 5.31 (Cant-rate)**.**

$$\omega(s) = \phi'(s)$$

denotes cant-rate evolution.

5.6. Frame Symbols

5.6.1. Tangent Direction

Definition 5.32 (Tangent vector)**.**

$$\mathbf{t}(s)$$

denotes the local tangent direction.

5.6.2. Lateral Direction

Definition 5.33 (Lateral vector)**.**

$$\mathbf{n}(s)$$

denotes the local lateral direction.

5.6.3. Binormal Direction

Definition 5.34 (Binormal vector)**.**

$$\mathbf{b}(s)$$

denotes the local binormal direction.

5.6.4. Local Frame

Definition 5.35 (Local frame)**.**

$$F(s) = (\mathbf{t}(s), \mathbf{n}(s), \mathbf{b}(s))$$

denotes the local pose3 frame.

5.7. Pose and State Terminology

5.7.1. Pose

Definition 5.36 (Pose)**.** A pose denotes a local intrinsic railway state attached to arc-length position s.

5.7.2. pose2

Definition 5.37 (pose2)**.** The pose2 state is:

$$\text{pose2}(s) = (\gamma(s), \theta(s)).$$

Remark 5.38. pose2 represents intrinsic planar railway geometry.

5.7.3. pose3

Definition 5.39 (pose3)**.** The pose3 state extends pose2 by including spatial orientation and cant.

A minimal intrinsic representation is:

$$\text{pose3}(s) = (\gamma(s), \theta(s), \phi(s)).$$

Remark 5.40. More elaborate runtime pose3 interpretations may additionally include:

- local frames,
- profile evaluation,
- metric realization,
- or operational vehicle-space interpretation.

5.7.4. Controls

Definition 5.41 (Control quantities)**.** The canonical control quantities are:

$$\kappa(s), \qquad \omega(s).$$

Remark 5.42. Curvature controls directional evolution, whereas cant-rate controls cant evolution.

5.7.5. State

Definition 5.43 (State)**.** A state denotes a structured railway condition together with its geometric, operational, or physical interpretation.

Remark 5.44. Within AIM, states are not merely stored parameter containers.

5.7.6. Runtime State

Definition 5.45 (Runtime state)**.** A runtime state is a dynamically evaluated operational state generated through runtime operator evaluation.

Remark 5.46. Runtime refers to operational evaluation of alignment-related quantities, independent of implementation details.

5.8. Target, Realized, and Measured Notation

5.8.1. Target Quantities

Definition 5.47 (Target notation)**.** Target quantities use the index:

$$(\cdot)_{\text{target}}.$$

5.8.2. Realized Quantities

Definition 5.48 (Realized notation)**.** Realized quantities use the index:

$$(\cdot)_{\text{real}}.$$

5.8.3. Measured Quantities

Definition 5.49 (Measured notation)**.** Measured quantities use the index:

$$(\cdot)_{\text{meas}}.$$

5.9. Operator Notation

5.9.1. Projection and Transformation Operators

Definition 5.50 (Track-to-world operator notation)**.**

$$\mathcal{P}_{\text{tw}}$$

denotes the track-to-world operator.

Definition 5.51 (World-to-track operator notation)**.**

$$\mathcal{P}_{\text{wt}}$$

denotes the world-to-track projection operator.

Definition 5.52 (Coordinate transformation operator notation)**.**

$$\mathcal{C}$$

denotes coordinate transformation between world-space representations.

5.9.2. Runtime Operators

Definition 5.53 (Evaluation operator notation)**.**

$$\mathcal{E}$$

denotes runtime operator evaluation.

Definition 5.54 (Frame operator notation)**.**

$$\mathcal{F}$$

denotes frame evaluation.

5.9.3. Realization Operators

Definition 5.55 (Sampling operator notation)**.**

$$\mathcal{S}$$

denotes sampling from continuous to discrete representation.

Definition 5.56 (Interpolation operator notation).

$$\mathcal{I}$$

denotes reconstruction from discrete data.

Definition 5.57 (Mechanical operator notation).

$$\mathcal{M}$$

denotes mechanical or structural response of the railway system.

Definition 5.58 (Adjustment operator notation).

$$\mathcal{A}$$

denotes a local construction or maintenance adjustment operator.

Definition 5.59 (Realization operator notation).

$$\mathcal{R}$$

denotes the realization operator.

5.10. CRS and Earth Geometry

5.10.1. Geodesic Curvature

Definition 5.60 (Geodesic curvature).

$$\kappa_g$$

denotes geodesic curvature on the Earth surface.

5.10.2. Normal Curvature

Definition 5.61 (Normal curvature).

$$\kappa_n$$

denotes curvature induced by Earth surface geometry.

5.10.3. Earth Surface Normal

Definition 5.62 (Earth surface normal).

$$\mathbf{n}_{\mathrm{E}}$$

denotes the ellipsoid surface normal.

5.11. Vehicle-Space Terminology

5.11.1. Vehicle Space

Definition 5.63 (Vehicle space)**.** Vehicle space denotes the physically occupied operational space of the railway vehicle.

5.11.2. Operational Geometry

Definition 5.64 (Operational geometry)**.** Operational geometry denotes geometry relevant to:

- passenger comfort,
- platform interaction,
- clearance,
- vehicle behaviour,
- and dynamic vehicle-space interaction.

5.12. Canonical Terminology

Definition 5.65 (Preferred terminology)**.** The following terminology is preferred:

alignment	intrinsic railway geometry system
curve	mathematical geometric object
geometry	geometric description
trajectory	motion-related path
pose	local railway state
frame	local spatial interpretation
state	structured railway condition
runtime state	dynamically evaluated operational state
runtime service	operator-based evaluation process
realization	physical construction or maintenance process
realization operator	mapping from target state to realized state
metric realization	metric-dependent spatial interpretation
world space	CRS-based spatial coordinates
track space	intrinsic alignment coordinates
track	physically realized railway structure

5.13. Avoided Ambiguities

Remark 5.66. The following ambiguities should be avoided:

- alignment $\neq$ merely curve,
- realization $\neq$ visualization,
- runtime $\neq$ software process only,
- pose $\neq$ frame,
- geometry $\neq$ realization,
- target geometry $\neq$ realized geometry,
- metric realization $\neq$ physical realization,
- coordinate transformation $\neq$ world-to-track projection,
- projection $\neq$ CRS transformation,
- world coordinates $\neq$ track coordinates.

5.14. Connection to AIM

Summary 5.67. The AIM notation system separates:

- intrinsic geometry,
- metric realization,
- realized geometry,
- runtime states,
- operational vehicle-space interpretation,
- and CRS-dependent world geometry.

This notation framework forms the canonical language of AIM.

6. Core Equations and Canonical Relations

6.1. Motivation

Remark 6.1. The AIM framework is based on a compact set of canonical equations that connect geometry, dynamics, realization, runtime interpretation, projection, and optimization.

Remark 6.2. These equations do not represent isolated formulas, but a coupled state description for railway alignment modelling.

Remark 6.3. The purpose of this chapter is not to derive all relations in detail, but to define the canonical equations repeatedly used throughout AIM.

Remark 6.4. The equations collected here serve as:

- canonical reference relations,
- shared notation anchors,
- and structural building blocks of the AIM framework.

6.2. Canonical State Representation

6.2.1. Intrinsic State

Definition 6.5 (Canonical intrinsic state)**.** The canonical intrinsic pose3 state is:

$$x(s) = (\gamma(s), \theta(s), \phi(s)). \tag{6.1}$$

Remark 6.6. The state combines:

- intrinsic geometry,
- orientation,
- and cant evolution.

Remark 6.7. This intrinsic state represents the minimal canonical geometric railway state. More elaborate runtime-evaluated spatial interpretations may extend this representation through frame evaluation, profile integration, metric interpretation, and realization-aware runtime services.

6.2.2. Control Variables

Definition 6.8 (Control variables)**.** The canonical control variables are:

$$u(s) = (\kappa(s), \omega(s)), \tag{6.2}$$

where:

$$\omega(s) = \phi'(s).$$

Remark 6.9. Curvature and cant-rate act as the primary controls of railway state evolution.

6.3. Intrinsic Geometry

6.3.1. Arc-Length State Evolution

Definition 6.10 (State evolution relation)**.** The intrinsic geometric state evolves according to:

$$\gamma'(s) = \mathbf{t}(s). \tag{6.3}$$

Remark 6.11. This relation defines the arc-length parameterization of the alignment.

6.3.2. Curvature Relation

Definition 6.12 (Curvature relation)**.** Directional evolution is governed by:

$$\theta'(s) = \kappa(s). \tag{6.4}$$

Remark 6.13. Curvature is the differential quantity governing directional evolution along the alignment.

6.3.3. Planar Tangent Representation

Definition 6.14 (Planar tangent relation)**.** In the planar case:

$$\mathbf{t}(s) = (\cos\theta(s), \sin\theta(s)). \tag{6.5}$$

Remark 6.15. This is the planar specialization of eq. (6.3).

6.3.4. Canonical Pose2 System

Definition 6.16 (Pose2 system)**.** The canonical planar pose2 system is:

$$\begin{aligned} \gamma'(s) &= \mathbf{t}(s), \\ \theta'(s) &= \kappa(s). \end{aligned} \tag{6.6}$$

Remark 6.17. Together with eq. (6.5), this defines the classical curvature-based planar reconstruction system.

6.4. Cant Evolution

6.4.1. Cant State Equation

Definition 6.18 (Cant evolution)**.** Cant evolution is governed by:

$$\phi'(s) = \omega(s), \tag{6.7}$$

where $\omega(s)$ denotes the cant-rate control.

Remark 6.19. Cant is treated as a state quantity rather than as a secondary annotation.

6.4.2. Canonical Pose3 System

Definition 6.20 (Pose3 system)**.** The canonical pose3 system is:

$$\begin{aligned} \gamma'(s) &= \mathbf{t}(s), \\ \theta'(s) &= \kappa(s), \\ \phi'(s) &= \omega(s). \end{aligned} \tag{6.8}$$

Remark 6.21. The controls of the system are given by eq. (6.2).

Remark 6.22. This formulation interprets railway alignment as a controlled dynamical state system.

6.4.3. Generalized State Form

Definition 6.23 (Generalized state equation)**.** The pose evolution may be written abstractly as:

$$x'(s) = f(x(s), u(s)). \tag{6.9}$$

Remark 6.24. Within AIM, the canonical controls are:

$$u(s) = (\kappa(s), \omega(s)).$$

6.5. Dynamic Relations

6.5.1. Effective Lateral Acceleration

Definition 6.25 (Effective lateral acceleration)**.** The effective lateral acceleration is:

$$a_{\text{eff}} = v^2\kappa - g\sin\phi. \tag{6.10}$$

Remark 6.26. This relation expresses the coupling between curvature and cant.

Remark 6.27. The relation represents the classical engineering approximation for moderate cant angles and local engineering geometry. More advanced formulations may additionally incorporate:

- spatial frame dynamics,
- center-of-gravity displacement,
- vehicle suspension,
- and ellipsoid-aware geometry.

6.5.2. Equilibrium Condition

Definition 6.28 (Equilibrium condition)**.** A perfectly balanced alignment satisfies:

$$v^2\kappa = g\sin\phi. \tag{6.11}$$

Remark 6.29. This relation defines ideal cant balance.

6.5.3. Jerk Relation

Definition 6.30 (Jerk relation)**.** The jerk is:

$$j = v^3\kappa' - gv\cos\phi\,\phi'. \tag{6.12}$$

Remark 6.31. The jerk depends jointly on curvature evolution and cant evolution.

6.6. Representation and Projection Relations

6.6.1. Track-to-World Mapping

Definition 6.32 (Track-to-world mapping)**.** The forward transformation is:

$$x_w(s,d) = \gamma(s) + d\,\mathbf{n}(s). \tag{6.13}$$

Remark 6.33. The vector $\mathbf{n}(s)$ denotes the local lateral direction.

6.6.2. World-to-Track Projection

Definition 6.34 (World-to-track projection)**.** The inverse transformation is:

$$(s^*, d^*) = \arg\min_{s,d} \|x_w - (\gamma(s) + d\,\mathbf{n}(s))\|. \tag{6.14}$$

Remark 6.35. This is a nonlinear runtime projection problem onto the alignment.

6.6.3. Coordinate Transformation

Definition 6.36 (Coordinate transformation)**.** Coordinate transformation is represented abstractly by:

$$x_2 = \mathcal{C}(x_1). \tag{6.15}$$

Remark 6.37. Coordinate transformation must not be confused with intrinsic railway projection.

6.7. Realization Relations

6.7.1. Sampling Relation

Definition 6.38 (Sampling relation)**.** Continuous geometry may be sampled as:

$$\mathcal{S}(f) = \{f(s_i)\}. \tag{6.16}$$

6.7.2. Interpolation Relation

Definition 6.39 (Interpolation relation)**.** Continuous reconstruction from sampled data is represented by:

$$\mathcal{I}(\{f(s_i)\}) = \tilde{f}(s). \tag{6.17}$$

6.7.3. Realization Operator

Definition 6.40 (Realization operator)**.** The realization operator maps target states to realized states:

$$\mathcal{R} : \mathrm{pose3}_{\mathrm{target}} \to \mathrm{pose3}_{\mathrm{real}}. \tag{6.18}$$

Remark 6.41. Realization includes:

- construction,
- machine behaviour,
- discretization,
- interpolation,
- and structural response.

6.7.4. Realization Decomposition

Definition 6.42 (Realization decomposition)**.** The realization process may be interpreted as:

$$\mathcal{R} = \mathcal{M} \circ \mathcal{I} \circ \mathcal{S}. \tag{6.19}$$

Remark 6.43. This decomposition separates:

- discretization,
- reconstruction,
- and physical realization effects.

Remark 6.44. Within AIM, this decomposition forms one of the canonical bridges between:

- analytical geometry,
- construction processes,
- runtime reconstruction,
- and physically realized railway behaviour.

6.7.5. Curvature Realization Relation

Definition 6.45 (Curvature realization)**.** At curvature level:

$$\mathcal{R}(\kappa_{\text{target}}) = \kappa_{\text{real}}. \tag{6.20}$$

6.7.6. Inverse Realization Problem

Definition 6.46 (Inverse realization)**.** The inverse realization problem is:

$$\mathcal{R}(\kappa_{\text{target}}) \approx \kappa_{\text{desired}}. \tag{6.21}$$

Remark 6.47. Alignment design therefore becomes an inverse problem under realization constraints.

6.8. Ellipsoid Geometry

6.8.1. Curvature Decomposition

Definition 6.48 (Curvature decomposition)**.** The total curvature satisfies:

$$\kappa^2 = \kappa_g^2 + \kappa_n^2. \tag{6.22}$$

Remark 6.49. The geodesic curvature κ_g is the physically relevant curvature for railway dynamics on the Earth surface.

6.8.2. Intrinsic Surface Curvature

Definition 6.50 (Geodesic curvature)**.** The intrinsic surface curvature is:

$$\kappa_g = \|\nabla_s \mathbf{t}\|. \tag{6.23}$$

Remark 6.51. This replaces the purely planar curvature interpretation on curved reference surfaces.

6.9. Optimization Relations

6.9.1. General Optimization Problem

Definition 6.52 (Optimization problem)**.** A general realization-aware optimization problem is:

$$\min_{u} \; J(\mathcal{R}(x(u))). \tag{6.24}$$

Remark 6.53. The objective is evaluated on realized states rather than purely analytical states.

6.9.2. Least-Squares Structure

Definition 6.54 (Least-squares structure)**.** Many AIM optimization problems take the form:

$$\min_{x} \frac{1}{2}\|r(x)\|^2. \tag{6.25}$$

6.9.3. Gauss–Newton Approximation

Definition 6.55 (Gauss–Newton approximation)**.** The Gauss–Newton approximation is:

$$H \approx J^T J. \tag{6.26}$$

Remark 6.56. This structure appears naturally in:

- curvature fitting,
- realization matching,
- trajectory optimization,
- and runtime adjustment problems.

6.10. Canonical AIM Interpretation

Proposition 6.57. *Within AIM, railway alignment is interpreted as a spatial state trajectory governed by curvature and cant evolution.*

Proposition 6.58. *Curvature, cant, realization, runtime interpretation, projection, metric interpretation, and optimization must therefore be treated as coupled components of one unified system.*

6.11. Connection to AIM

Summary 6.59. The equations collected in this chapter form the mathematical backbone of AIM.

They define:

- intrinsic state evolution,
- cant and dynamic relations,
- runtime projection,
- realization mappings,
- ellipsoid-aware geometry,
- and optimization structures.

Together, these equations establish a unified mathematical framework connecting geometry, realization, runtime interpretation, metric embedding, and optimization.

7. State Model

7.1. Motivation

Remark 7.1. Railway alignment modelling involves interacting descriptions of geometry, orientation, cant, realization, runtime interpretation, and operational behaviour.

Remark 7.2. Classical alignment theory often treats these aspects separately:

- geometry as curves,
- cant as annotation,
- realization as construction detail,
- and optimization as external post-processing.

Remark 7.3. Within AIM, these components are interpreted as parts of a unified state-based system.

Remark 7.4. The purpose of the state model is:

- to define canonical railway states,
- to distinguish intrinsic and extrinsic representations,
- to separate target and realized states,
- and to establish a consistent basis for runtime services, projection, realization, and optimization.

7.2. State Concept

7.2.1. State Definition

Definition 7.5 (State)**.** A state is the minimal set of variables required to describe the geometric and physical condition of a railway alignment at arc-length position s.

Remark 7.6. Within AIM, states are interpreted intrinsically along the alignment rather than globally in world coordinates.

Remark 7.7. The alignment is therefore interpreted as a spatial state trajectory parameterized by arc-length.

7.2.2. Canonical State Variable

Definition 7.8 (Canonical state)**.** The canonical intrinsic railway state is:

$$x(s) = (\gamma(s), \theta(s), \phi(s)). \tag{7.1}$$

Remark 7.9. The state combines:

- intrinsic geometry,
- orientation,
- and cant evolution.

7.3. Pose2 State

7.3.1. Definition

Definition 7.10 (Pose2 state)**.** The planar pose state is:

$$\text{pose2}(s) = (\gamma(s), \theta(s)). \tag{7.2}$$

Remark 7.11. The pose2 state describes:

- planar position,
- and planar orientation.

7.3.2. Pose2 Evolution

Remark 7.12. The pose2 evolution is governed by:

$$\theta'(s) = \kappa(s), \tag{7.3}$$

together with:

$$\gamma'(s) = (\cos\theta(s), \sin\theta(s)). \tag{7.4}$$

Remark 7.13. Curvature therefore acts as the primary geometric control variable.

7.4. Pose3 State

7.4.1. Definition

Definition 7.14 (Pose3 state)**.** The extended railway pose state is:

$$\text{pose3}(s) = (\gamma(s), \theta(s), \phi(s)). \tag{7.5}$$

Remark 7.15. The pose3 state extends the planar state by including cant.

Remark 7.16. Cant is interpreted as a rotation around the local tangent direction.

7.4.2. Controls

Definition 7.17 (Pose3 controls)**.** The canonical controls are:

$$u(s) = (\kappa(s), \omega(s)), \tag{7.6}$$

where:

$$\omega(s) = \phi'(s).$$

Remark 7.18. Curvature controls directional evolution, whereas cant-rate controls cross-level evolution.

7.4.3. Canonical Pose3 System

Remark 7.19. The canonical pose3 evolution system is:

$$\begin{aligned} \gamma'(s) &= (\cos\theta(s), \sin\theta(s)), \\ \theta'(s) &= \kappa(s), \\ \phi'(s) &= \omega(s). \end{aligned} \tag{7.7}$$

Remark 7.20. This formulation interprets railway alignment as a controlled dynamical state system.

7.5. Intrinsic and Extrinsic States

7.5.1. Intrinsic State

Definition 7.21 (Intrinsic state)**.** An intrinsic state is defined relative to the alignment itself and uses arc-length s as primary parameter.

Remark 7.22. Examples include:

- curvature,
- cant,
- tangent direction,
- local railway frames.

7.5.2. Extrinsic State

Definition 7.23 (Extrinsic state)**.** An extrinsic state is defined relative to a global coordinate reference system.

Remark 7.24. Examples include:

- WGS84 coordinates,
- projected CRS coordinates,
- BIM placement coordinates,
- sensor measurements.

7.5.3. Intrinsic–Extrinsic Coupling

Remark 7.25. The world–track transformation couples intrinsic and extrinsic states.

Remark 7.26. World-to-track projection therefore acts as a state conversion process between global and alignment-relative representations.

7.6. Target and Realized States

7.6.1. Target State

Definition 7.27 (Target state)**.** The target state describes the analytically intended alignment:

$$\mathrm{pose3}_{\mathrm{target}}. \tag{7.8}$$

7.6.2. Realized State

Definition 7.28 (Realized state)**.** The realized state describes the physically constructed alignment:

$$\mathrm{pose3}_{\mathrm{real}}. \tag{7.9}$$

7.6.3. Realization Relation

Remark 7.29. The realization operator connects both states:

$$\mathcal{R} : \mathrm{pose3}_{\mathrm{target}} \mapsto \mathrm{pose3}_{\mathrm{real}}. \tag{7.10}$$

Remark 7.30. The realized state includes:

- construction effects,
- machine behaviour,
- discretization,
- maintenance history,
- and measurement uncertainty.

7.7. Runtime States

7.7.1. Runtime Interpretation

Remark 7.31. Within AIM, states are not static stored objects but runtime-evaluated entities.

Remark 7.32. Runtime systems continuously evaluate:

- world-to-track projections,
- local frames,
- realized geometry,
- dynamic quantities,
- and optimization residuals.

7.7.2. Derived Runtime Quantities

Definition 7.33 (Derived runtime quantities)**.** Derived runtime quantities include:

$$a_{\text{eff}}, \qquad j, \qquad \kappa_g, \qquad d, \qquad h. \tag{7.11}$$

Remark 7.34. These are not independent state variables, but quantities derived from the underlying pose and frame structure.

7.8. Sparse and Hybrid States

7.8.1. Sparse Representation

Definition 7.35 (Sparse state)**.** A sparse state is a compact representation using only essential control parameters.

Remark 7.36. Examples include:

- fixed curvature segments,
- transition segments,
- cant transitions,
- anchor poses.

7.8.2. Hybrid Representation

Definition 7.37 (Hybrid state)**.** A hybrid state combines:

- parametric structure,
- and local sampled corrections.

Remark 7.38. Hybrid states combine efficient modelling with local adaptation.

7.9. Optimization States

7.9.1. Optimization Variables

Remark 7.39. Optimization does not necessarily operate on full geometry.

Remark 7.40. Typical optimization variables include:

- curvature parameters,
- transition lengths,
- cant parameters,
- control point positions,
- and realization corrections.

7.9.2. Realization-Aware Optimization

Remark 7.41. Within AIM, optimization is evaluated on realized states:

$$J(\mathcal{R}(\mathrm{pose3})). \tag{7.12}$$

Remark 7.42. Optimization therefore operates indirectly through the realization operator.

7.10. State Hierarchy

Remark 7.43. The AIM framework distinguishes:

- intrinsic states,
- extrinsic states,
- target states,
- realized states,
- runtime states,
- and optimization states.

Remark 7.44. These are not independent models, but interconnected layers of one unified railway state system.

7.11. Canonical Interpretation

Proposition 7.45. *Within AIM, railway alignment is interpreted as a controlled spatial state trajectory evolving along arc-length.*

Proposition 7.46. *Curvature and cant evolution form the primary control structure of this trajectory.*

Proposition 7.47. *Realization, projection, and optimization operate on states rather than on isolated geometric entities.*

7.12. Connection to AIM

Summary 7.48. The AIM state model provides a unified framework linking:

- geometry,
- cant,
- realization,
- runtime projection,
- dynamics,
- and optimization.

It establishes railway alignment as a coupled state-based system rather than merely a collection of curves and coordinates.

8. Operators

8.1. Motivation

Remark 8.1. Railway alignment modelling is not merely based on geometric objects, but on transformations acting on states.

Remark 8.2. Within AIM, geometry, realization, projection, runtime evaluation, metric interpretation, and optimization are formulated operator-theoretically.

Remark 8.3. Operators connect:

- intrinsic geometry,
- metric realization,
- world-space embedding,
- sampled representations,
- runtime evaluation,
- physical realization,
- and optimization states.

Remark 8.4. This chapter introduces the canonical operators and operator spaces of the AIM framework.

8.2. General Operator Interpretation

Definition 8.5 (Operator)**.** An operator is a mapping acting on:

- states,
- functions,
- representations,
- or geometric structures.

Remark 8.6. Operators may:

- transform representations,

- evaluate quantities,
- modify states,
- reconstruct geometry,
- or approximate physical processes.

Principle 8.7 (State-operator principle). Within AIM, operators act on states rather than on isolated geometric objects.

Proposition 8.8. *Within AIM, geometry is not interpreted as static stored data, but as states transformed and interpreted through coupled operator systems.*

8.3. Operator Spaces

Remark 8.9. The canonical spaces used by the operator framework are introduced in chapter 5.

Remark 8.10. The most important spaces are:

$$\mathcal{X}, \qquad \mathcal{Y}, \qquad \Omega_{\text{track}}, \qquad \Omega_{\text{world}}, \qquad \mathcal{K}.$$

Remark 8.11. Here:

- $\mathcal{X}$ denotes railway state space,
- $\mathcal{Y}$ denotes runtime quantity space,
- Ω_{track} denotes intrinsic track space,
- Ω_{world} denotes CRS-dependent world space,
- and $\mathcal{K}$ denotes curvature space.

8.4. Operator Categories

Remark 8.12. The AIM framework distinguishes:

- geometric operators,
- representation operators,
- metric operators,
- runtime operators,
- realization operators,
- and optimization operators.

Remark 8.13. These operator families are conceptually distinct, but operationally coupled.

Principle 8.14 (Operator separation principle). Runtime evaluation, metric realization, coordinate transformation, projection, and physical realization should not be collapsed into one implicit operation.

8.5. Geometric Operators

8.5.1. Curvature Operator

Definition 8.15 (Curvature operator)**.** The directional evolution relation is:

$$\theta'(s) = \kappa(s).$$

Remark 8.16. Curvature acts as the differential generator of directional evolution.

8.5.2. Reconstruction Operator

Definition 8.17 (Reconstruction operator)**.** The intrinsic reconstruction relation is:

$$\gamma'(s) = \mathbf{t}(s).$$

Remark 8.18. Together with curvature evolution, this defines intrinsic alignment reconstruction.

8.5.3. Cant Evolution Operator

Definition 8.19 (Cant evolution operator)**.** Cant evolution is governed by:

$$\phi'(s) = \omega(s).$$

Remark 8.20. Cant-rate acts as the control quantity governing cross-level evolution.

Remark 8.21. The canonical geometric relations are collected in chapter 6.

8.6. Representation Operators

8.6.1. Track-to-World Transformation

Definition 8.22 (Track-to-world operator)**.**

$$\mathcal{P}_{\mathrm{tw}} : \Omega_{\mathrm{track}} \to \Omega_{\mathrm{world}}.$$

Remark 8.23. The operator maps intrinsic railway coordinates into world-space coordinates.

8.6.2. World-to-Track Projection

Definition 8.24 (World-to-track operator)**.**

$$\mathcal{P}_{\text{wt}} : \Omega_{\text{world}} \to \Omega_{\text{track}}.$$

Remark 8.25. Inverse projection is generally nonlinear, metric-dependent, and potentially ambiguous.

Principle 8.26 (Projection principle). World-to-track transformation is an inverse runtime projection problem, not a static coordinate conversion.

8.6.3. Coordinate Transformation Operator

Definition 8.27 (Coordinate transformation operator)**.**

$$\mathcal{C} : \Omega_{\text{world}} \to \Omega_{\text{world}}.$$

Remark 8.28. Coordinate transformations map between CRS-dependent spatial representations.

Remark 8.29. Coordinate transformation does not alter intrinsic railway semantics.

Remark 8.30. Coordinate transformation must not be confused with intrinsic world-to-track projection.

8.7. Metric Operators

Remark 8.31. Within AIM, intrinsic alignment semantics are separated from metric realization.

Principle 8.32 (Metric separation principle). Alignment semantics should not directly depend on a specific metric backend.

Definition 8.33 (Metric realization operator)**.** The metric realization operator is written as:

$$\mathcal{G} : \mathcal{X} \to \mathcal{X}_{\text{metric}}.$$

Remark 8.34. The operator $\mathcal{G}$ expresses how intrinsic railway states are interpreted within a metric geometry model.

Remark 8.35. Metric interpretation may depend on:

- Euclidean engineering geometry,
- projected CRS geometry,
- ellipsoid geometry,
- or generalized differential-geometric manifolds.

Remark 8.36. Metric operators affect:

- distance interpretation,
- curvature interpretation,
- projection behaviour,
- frame construction,
- and stationing-related computations.

Principle 8.37 (Metric backend principle). The alignment principle remains stable while the metric realization may change.

8.8. Runtime Operators

8.8.1. Evaluation Operator

Definition 8.38 (Evaluation operator)**.**

$$\mathcal{E} : \mathcal{X} \times \Omega_{\text{track}} \to \mathcal{Y}.$$

Remark 8.39. Evaluation operators produce:

- runtime geometry,
- frame states,
- dynamic quantities,
- operational quantities,
- and projection results.

Remark 8.40. Runtime operators interpret states. They do not by themselves physically realize or modify the railway infrastructure.

8.8.2. Frame Operator

Definition 8.41 (Frame operator)**.**

$$\mathcal{F} : \mathcal{X}_{\text{metric}} \times \Omega_{\text{track}} \to \text{Frames}.$$

Remark 8.42. The frame operator evaluates:

$$\mathcal{F}(x, s) = (\mathbf{t}(s), \mathbf{n}(s), \mathbf{b}(s)).$$

Remark 8.43. Frames provide local operational spatial interpretation of alignment states.

Remark 8.44. Frame construction may depend on the selected metric realization. For example, a projected engineering frame and an ellipsoid-attached frame need not coincide.

8.8.3. Runtime Quantities

Definition 8.45 (Runtime quantities)**.** Typical runtime-derived quantities include:

$$a_{\text{eff}}, \qquad j, \qquad \kappa_g, \qquad \phi, \qquad F(s).$$

Remark 8.46. These quantities are evaluated dynamically from underlying states, metric interpretation, and runtime operators.

8.9. Realization Operators

8.9.1. Sampling Operator

Definition 8.47 (Sampling operator)**.**

$$\mathcal{S} : f(s) \mapsto \{f(s_i)\}.$$

Remark 8.48. Sampling converts continuous representations into discrete representations.

8.9.2. Interpolation Operator

Definition 8.49 (Interpolation operator)**.**

$$\mathcal{I} : \{f(s_i)\} \mapsto \tilde{f}(s).$$

Remark 8.50. Interpolation reconstructs continuous approximations from sampled data.

8.9.3. Mechanical Operator

Definition 8.51 (Mechanical operator)**.**

$$\mathcal{M}.$$

Remark 8.52. The mechanical operator represents:

- structural response,
- ballast behaviour,
- track elasticity,
- and mechanical smoothing.

8.9.4. Adjustment Operator

Definition 8.53 (Adjustment operator)**.**

$$\mathcal{A}.$$

Remark 8.54. The adjustment operator represents localized construction or maintenance actions.

8.9.5. Realization Operator

Definition 8.55 (Realization operator)**.**

$$\mathcal{R} : \mathcal{X}_{\text{target}} \to \mathcal{X}_{\text{real}}.$$

Principle 8.56 (Realization principle). Realization is interpreted as transformation of alignment states rather than as direct geometric reconstruction.

Remark 8.57. Realization operators describe physical or construction-related transformation. They must be distinguished from runtime operators, which evaluate or interpret states.

8.9.6. Realization Decomposition

Definition 8.58 (Realization decomposition)**.** The realization process may be interpreted as:

$$\mathcal{R} = \mathcal{M} \circ \mathcal{I} \circ \mathcal{S}.$$

Remark 8.59. This decomposition separates:

- discretization,
- reconstruction,
- and physical realization effects.

8.9.7. Iterative Realization

Definition 8.60 (Iterative realization)**.**

$$x_{k+1} = \mathcal{A}(x_k).$$

Remark 8.61. Realization may therefore be interpreted as an iterative operator process.

8.10. Optimization Operators

8.10.1. Residual Operator

Definition 8.62 (Residual operator)**.**

$$r(x).$$

Remark 8.63. Residuals quantify deviations from desired operational, geometric, realization, or metric properties.

8.10.2. Jacobian Operator

Definition 8.64 (Jacobian operator)**.**

$$J(x) = \frac{\partial r}{\partial x}.$$

8.10.3. Gauss–Newton Approximation

Definition 8.65 (Gauss–Newton approximation)**.**

$$H \approx J^T J.$$

Remark 8.66. This approximation appears naturally in least-squares optimization.

8.10.4. Realization-Aware Optimization

Definition 8.67 (Realization-aware optimization)**.**

$$\min_u \; J(\mathcal{R}(x(u))).$$

Remark 8.68. Optimization therefore acts indirectly through realization.

8.11. Operator Composition

8.11.1. Runtime Composition

Definition 8.69 (Runtime operator composition)**.** A simplified runtime evaluation chain may be written as:

$$\mathcal{E} \sim \mathcal{P} \circ \mathcal{F} \circ \mathcal{G}.$$

Remark 8.70. This expresses that runtime evaluation may depend on:

- metric realization,
- frame evaluation,
- and projection services.

8.11.2. Realization Composition

Definition 8.71 (Realization operator composition)**.** The physical realization chain is represented by:

$$\mathcal{R} = \mathcal{M} \circ \mathcal{I} \circ \mathcal{S}.$$

Remark 8.72. This separates physical realization from runtime interpretation.

8.11.3. Full Operational Composition

Definition 8.73 (Full operational operator chain)**.** A schematic full AIM evaluation chain may be written as:

$$\mathcal{E}_{\text{full}} \sim \mathcal{P} \circ \mathcal{F} \circ \mathcal{G} \circ \mathcal{R}.$$

Remark 8.74*.* This expression is not meant as a rigid software pipeline. It describes the conceptual dependency between realization, metric interpretation, frame evaluation, projection, and runtime quantities.

8.12. Runtime versus Realization

Principle 8.75 (Runtime-realization distinction)*.* Runtime operators evaluate states, whereas realization operators transform target states into realized states.

Remark 8.76*.* Runtime evaluation may operate on target, realized, or measured states. Realization describes how target states become realized states.

Remark 8.77*.* Confusing runtime evaluation with physical realization leads to ambiguous models and duplicated geometry.

8.13. Canonical Interpretation

Proposition 8.78. *Within AIM, railway alignment modelling is fundamentally operator-based.*

Proposition 8.79. *Operators transform and evaluate states across:*

- *intrinsic geometry,*
- *metric realization,*
- *world-space embedding,*
- *runtime interpretation,*
- *physical realization,*
- *and optimization.*

Proposition 8.80. *Realization, metric interpretation, projection, runtime evaluation, and optimization are interpreted as coupled operator layers acting on railway states.*

8.14. Connection to AIM

Summary 8.81*.* The AIM operator framework unifies:

- intrinsic geometry,

- metric interpretation,
- world–track projection,
- runtime evaluation,
- physical realization,
- and optimization.

Operators form the dynamic transformation architecture connecting all parts of the AIM framework.

9. System Layers

9.1. Motivation

Remark 9.1. Railway alignment modelling combines multiple fundamentally different domains.

Remark 9.2. Classical approaches often merge:

- intrinsic geometry,
- coordinate reference systems,
- metric interpretation,
- construction realization,
- runtime evaluation,
- dynamics,
- and operational vehicle interpretation

into a single implicit model.

Remark 9.3. Within AIM, these concerns are separated into interacting conceptual and operational system layers.

Remark 9.4. This separation improves:

- conceptual clarity,
- numerical robustness,
- scalability,
- interoperability,
- runtime flexibility,
- and realization-aware modelling.

9.2. Overview

Definition 9.5 (System layer)**.** A system layer is a coherent subsystem operating on its own:

- state representation,
- operators,
- constraints,
- interpretation rules,
- and runtime assumptions.

Remark 9.6. The AIM framework separates railway alignment modelling into:

- geometry layer,
- CRS layer,
- metric realization layer,
- physical realization layer,
- runtime layer,
- vehicle layer,
- and optimization layer.

Remark 9.7. These layers are conceptually distinct, but operationally coupled.

9.3. Geometry Layer

9.3.1. Role

Remark 9.8. The geometry layer defines the intrinsic semantic structure of the alignment.

Remark 9.9. Its central quantities are:

- arc-length s,
- curvature $\kappa(s)$,
- cant evolution,
- transition structure,
- and pose evolution laws.

9.3.2. Core Relations

Remark 9.10. The geometry layer is governed by the canonical intrinsic relations defined in chapter 6.

Remark 9.11. In particular, it uses:

- the curvature relation eq. (6.4),
- the reconstruction relation eq. (6.3),
- and the cant evolution relation eq. (6.7).

9.3.3. Characteristics

Remark 9.12. The geometry layer is:

- intrinsic,
- semantic,
- continuous,
- analytical,
- and independent of a particular metric backend.

Remark 9.13. It defines alignment semantics rather than physically realized track geometry.

9.4. CRS Layer

9.4.1. Role

Remark 9.14. The CRS layer defines external coordinate reference systems and their relations to the Earth.

Remark 9.15. Typical coordinate systems include:

- engineering CRS,
- projected CRS,
- geodetic coordinates,
- local construction coordinate systems,
- and ellipsoid-based representations.

9.4.2. Coordinate Transformation

Remark 9.16. CRS transformations are represented by the coordinate transformation operator $\mathcal{C}$, introduced in theorem 8.27.

Remark 9.17. The CRS layer operates within world space:

$$\mathcal{C} : \Omega_{\mathrm{world}} \to \Omega_{\mathrm{world}}.$$

9.4.3. Fundamental Observation

Proposition 9.18. *A railway alignment is generally not invariant under CRS transformations.*

Remark 9.19. In particular:

- straight lines,
- curvature,
- transition behaviour,
- stationing-related length interpretation,
- and cant-related spatial interpretation

may change under projection or CRS-dependent embedding.

Remark 9.20. The CRS layer therefore affects spatial interpretation, but it is not identical to intrinsic railway projection.

9.5. Metric Realization Layer

9.5.1. Role

Remark 9.21. The metric realization layer defines how semantic alignment structures are interpreted geometrically within a selected metric model.

Remark 9.22. It is governed conceptually by the metric realization operator $\mathcal{G}$, introduced in theorem 10.22.

9.5.2. Core Principle

Principle 9.23 (Metric realization principle). Semantic alignment logic is separated from metric realization behaviour.

Remark 9.24. Thus:

- semantic curvature laws,
- transition structure,

- sparse alignment logic,
- and stationing semantics

remain conceptually stable even when metric realization changes.

9.5.3. Metric Variants

Remark 9.25. Typical metric realization modes include:

- engineering-Euclidean realization,
- projection-aware realization,
- ellipsoid-aware realization,
- and future differential-geometric realization.

9.5.4. Operator Boundary

Remark 9.26. The metric realization layer forms the operator boundary between:

- semantic alignment logic,
- and metric-dependent geometric evaluation.

Remark 9.27. This architecture is introduced formally in chapter 10.

9.6. Physical Realization Layer

9.6.1. Role

Remark 9.28. The physical realization layer models the transformation from analytical target geometry to physically realized railway geometry.

9.6.2. Core Operator

Remark 9.29. The physical realization layer is governed by the realization operator

$$\mathcal{R} : \mathcal{X}_{\text{target}} \to \mathcal{X}_{\text{real}},$$

introduced in theorem 8.55.

9.6.3. Characteristics

Remark 9.30. The physical realization layer includes:

- construction constraints,
- machine behaviour,

- discretization,
- interpolation,
- structural response,
- maintenance processes,
- and measurement feedback.

Remark 9.31. Physical realization acts approximately as a low-pass filter on geometry and curvature.

Remark 9.32. Within AIM, physical realization is interpreted as an explicit operator layer rather than as a secondary engineering detail.

Remark 9.33. This layer is developed in chapter 38.

9.7. Runtime Layer

9.7.1. Role

Remark 9.34. The runtime layer transforms stored railway representations into operationally evaluable geometry.

Remark 9.35. Runtime evaluation is governed by the evaluation operator

$$\mathcal{E} : \mathcal{X} \times \Omega_{\text{track}} \to \mathcal{Y},$$

introduced in theorem 8.38.

9.7.2. Runtime Services

Remark 9.36. Typical runtime services include:

- world–track projection,
- frame evaluation,
- metric-aware geometry evaluation,
- visualization,
- BIM placement,
- spatial queries,
- sensor integration,
- and dynamic quantity evaluation.

9.7.3. Runtime Characteristics

Remark 9.37. The runtime layer is:

- state-dependent,
- scale-dependent,
- metric-aware,
- operator-based,
- computationally adaptive,
- and runtime-evaluated.

Remark 9.38. Many runtime quantities are evaluated dynamically rather than stored explicitly.

Remark 9.39. The runtime layer therefore acts as the operational execution layer of the AIM framework.

Remark 9.40. The runtime architecture is developed further in chapter 31.

9.8. Vehicle Layer

9.8.1. Role

Remark 9.41. The physically relevant object is not merely the alignment centerline, but the railway vehicle moving within space.

9.8.2. Operational Geometry

Remark 9.42. The vehicle layer includes:

- vehicle envelopes,
- bogie motion,
- wheelset orientation,
- center-of-gravity motion,
- operational clearances,
- and platform interaction.

9.8.3. Runtime Dependency

Remark 9.43. Vehicle-space interpretation depends on runtime-evaluated pose3 states, frames, cant, profile, and metric context.

Remark 9.44. Thus, the vehicle layer is not merely an application layer, but an operational interpretation layer built on top of runtime services.

9.8.4. Motivation

Remark 9.45. Classical railway alignment systems often approximate operational geometry using simplified clearance assumptions.

Remark 9.46. Within AIM, operational vehicle-space geometry may instead be modelled explicitly.

9.9. Optimization Layer

9.9.1. Role

Remark 9.47. The optimization layer searches for railway states satisfying:

- geometric,
- metric,
- dynamic,
- operational,
- realization,
- and regulatory constraints.

9.9.2. Realization-Aware Optimization

Remark 9.48. Within AIM, optimization may be evaluated on realized states:

$$J(\mathcal{R}(x(u))).$$

Remark 9.49. This relation is part of the canonical optimization structure introduced in eq. (6.24).

9.9.3. Hybrid Nature

Remark 9.50. Optimization combines:

- analytical models,
- sparse representations,
- discrete constraints,
- runtime evaluation,
- realization operators,
- metric-dependent geometry evaluation,
- and operational vehicle-space constraints.

9.10. Layer Interactions

Remark 9.51. The system layers interact continuously.

Remark 9.52. For example:

- intrinsic geometry feeds metric realization,
- CRS influences metric interpretation and world embedding,
- metric realization affects runtime geometry,
- physical realization modifies target states into realized states,
- runtime supports projection, visualization, and optimization,
- optimization modifies target geometry,
- vehicle-space interpretation depends on pose3, frames, and runtime,
- and physical measurements should generally reference realized geometry.

9.11. Canonical Transformation Structure

Definition 9.53 (Canonical transformation structure)**.** A simplified conceptual AIM structure is:

$$\text{intrinsic geometry} \to \text{metric realization} \to \text{runtime evaluation} \to \text{vehicle-space interpretation.}$$

Remark 9.54. Physical realization is a separate transformation:

$$\text{target state} \to \text{realized state.}$$

Remark 9.55. CRS embedding influences metric realization and world-space representation, but it is not identical to intrinsic world–track projection.

Remark 9.56. Optimization closes the loop by modifying target geometry based on runtime-evaluated and realization-aware objectives.

9.12. Scale Dependence

Remark 9.57. Different layers dominate at different spatial and operational scales.

Remark 9.58. For example:

- CRS effects dominate at large spatial scales,
- metric realization dominates at projection transitions,
- physical realization effects dominate at construction scales,

- runtime costs dominate in large operational systems,
- and vehicle-space effects dominate near platforms, switches, and clearance-critical situations.

9.13. Canonical Layer Separation

Principle 9.59 (Canonical separation principle). Intrinsic geometry, CRS embedding, metric realization, physical realization, runtime interpretation, operational behaviour, and optimization should not be merged into a single implicit model.

Remark 9.60. Many inconsistencies in classical railway systems arise from implicit mixing of fundamentally different conceptual layers.

9.14. Key Insight

Proposition 9.61. *Railway alignment modelling is not a single geometric problem, but a multi-layer interaction system.*

Proposition 9.62. *The operational behaviour of railway systems emerges from interactions between:*

- *intrinsic geometry,*
- *CRS embedding,*
- *metric realization,*
- *physical realization,*
- *runtime evaluation,*
- *vehicle-space interpretation,*
- *and optimization.*

9.15. Connection to AIM

Summary 9.63. The AIM framework separates railway alignment modelling into interacting system layers.

This separation enables:

- rigorous intrinsic geometry,
- CRS-aware embedding,
- metric-aware realization,

- physical realization-aware modelling,
- scalable runtime systems,
- explicit vehicle-space interpretation,
- and realization-aware optimization.

Together, these layers form the conceptual architecture underlying the AIM framework.

10. Metric Realization

10.1. Motivation

Remark 10.1. Classical railway alignment systems often assume that geometry, distance, curvature, projection, and realization are all defined within one implicit Euclidean coordinate model.

Remark 10.2. This assumption is sufficient for many engineering applications, but it becomes problematic when:

- large-scale coordinate systems are involved,
- Earth curvature becomes relevant,
- projection distortions appear,
- multiple CRS layers interact,
- or alignment semantics must remain independent from metric realization.

Remark 10.3. Within AIM, a strict distinction is therefore introduced between:

- alignment semantics,
- CRS embedding,
- and metric realization.

Remark 10.4. This separation enables:

- stable alignment semantics,
- interchangeable metric models,
- realization-aware geometry,
- projection-aware interpretation,
- and future differential-geometric extensions.

10.2. Core Principle

Principle 10.5 (Alignment semantics principle). The semantic structure of an alignment is independent of the metric space in which it is realized.

Remark 10.6. For example:

- straight elements,
- constant-curvature arcs,
- transition elements,
- topology,
- and switch logic

remain semantically stable even when the underlying metric model changes.

Principle 10.7 (Metric realization principle). Geometric realization depends on the metric model used for evaluation.

Remark 10.8. Thus:

- distances,
- curvature,
- projection,
- interpolation,
- frame construction,
- and geodesic behaviour

may differ between metric realizations.

Principle 10.9 (Canonical separation principle). Alignment semantics and metric realization must remain conceptually separable.

10.3. Semantic Alignment Layer

10.3.1. Intrinsic Semantics

Remark 10.10. The semantic alignment layer describes:

- alignment topology,
- curvature laws,
- transition structure,

- stationing semantics,
- continuity conditions,
- and sparse alignment structure.

Remark 10.11. This layer is intentionally metric-independent.

10.3.2. Sparse Representation

Remark 10.12. Within AIM, the sparse alignment model stores:

- fixed curvature elements,
- transition elements,
- lookup-based curvature evolution,
- sparse reconstruction parameters,
- and associated semantic metadata.

Remark 10.13. The sparse representation therefore acts as a semantic alignment description rather than a finalized geometric embedding.

10.4. CRS Embedding versus Metric Realization

10.4.1. Conceptual Distinction

Remark 10.14. Within AIM, CRS embedding and metric realization are related but distinct concepts.

Remark 10.15. The CRS layer defines:

- coordinate reference systems,
- projection rules,
- Earth-related embedding,
- and coordinate transformations.

Remark 10.16. The metric realization layer defines:

- distance interpretation,
- curvature evaluation,
- frame behaviour,
- transport rules,
- and geometric reconstruction behaviour.

10.4.2. Coordinate Transformation

Remark 10.17. Coordinate transformations are represented by:

$$\mathcal{C} : \Omega_{\text{world}} \to \Omega_{\text{world}}.$$

Remark 10.18. Coordinate transformation alone does not define intrinsic railway geometry.

10.4.3. Projection Dependence

Proposition 10.19. *A straight line in one projected CRS is generally not identical to a straight line in another projection space.*

Remark 10.20. Similarly:

- curvature,
- transition behaviour,
- stationing interpretation,
- and lateral offsets

may become projection-dependent.

10.5. Metric Realization Spaces

10.5.1. General Interpretation

Definition 10.21 (Metric realization space)**.** A metric realization space defines:

- distance interpretation,
- directional interpretation,
- curvature evaluation,
- frame transport,
- projection behaviour,
- and geometric reconstruction rules.

10.5.2. Metric Realization Operator

Definition 10.22 (Metric realization operator)**.** Metric realization is represented abstractly by:

$$\mathcal{G} : \mathcal{X} \to \mathcal{X}_{\text{metric}}.$$

Remark 10.23. The operator $\mathcal{G}$ defines how semantic railway states are evaluated geometrically within a selected metric model.

10.5.3. Operator Boundary

Principle 10.24 (Metric operator boundary). Alignment logic should not directly assume a specific metric model.

Remark 10.25. Instead, alignment operations interact with geometry through metric operators and runtime evaluation services.

Remark 10.26. This creates a strict separation between:

- semantic alignment structure,
- and metric realization behaviour.

10.6. Engineering Metric

10.6.1. Role

Remark 10.27. The dominant practical realization mode within AIM is the engineering metric.

10.6.2. Characteristics

Remark 10.28. The engineering metric assumes:

- locally Euclidean geometry,
- Cartesian coordinates,
- fast evaluation,
- simple frame construction,
- and numerically stable reconstruction.

10.6.3. Importance

Principle 10.29 (Engineering priority principle). The engineering metric remains the primary operational mode of AIM and must not suffer unnecessary computational overhead.

Remark 10.30. Advanced metric concepts must therefore remain compatible with highly efficient engineering-scale evaluation.

10.7. Projection-Aware Metric

10.7.1. Motivation

Remark 10.31. Projected coordinate systems introduce:

- scale distortions,

- directional distortions,
- projection-dependent curvature effects,
- and coordinate-dependent metric behaviour.

10.7.2. Interpretation

Remark 10.32. Within AIM, projection-aware realization interprets alignment geometry through projection-dependent metric evaluation.

Remark 10.33. Thus, runtime geometry depends not only on intrinsic alignment semantics but also on projection context.

10.7.3. Operational Consequences

Remark 10.34. Projection-aware realization affects:

- world–track projection,
- nearest-point search,
- stationing interpretation,
- local frame construction,
- and geometric reconstruction.

10.8. Ellipsoid-Based Metric

10.8.1. Motivation

Remark 10.35. At sufficiently large spatial scales, Earth curvature becomes relevant for railway alignment interpretation.

10.8.2. Geodesic Interpretation

Remark 10.36. Within ellipsoid-based geometry:

- Euclidean straightness,
- geodesic straightness,
- and engineering straightness

are no longer identical concepts.

10.8.3. Curvature Interpretation

Remark 10.37. On curved surfaces, curvature decomposes into:

$$\kappa^2 = \kappa_g^2 + \kappa_n^2,$$

where:

- κ_g denotes geodesic curvature,
- κ_n denotes surface-normal curvature.

Remark 10.38. For railway dynamics, geodesic curvature is generally the physically relevant quantity.

10.8.4. Frame Interpretation

Remark 10.39. Within ellipsoid-aware geometry, frame construction may require:

- tangent transport,
- surface-normal evaluation,
- and connection-dependent orientation rules.

10.9. Future Differential-Geometric Layer

10.9.1. Motivation

Remark 10.40. Future railway alignment systems may require:

- intrinsic surface geometry,
- Earth-attached frames,
- geodesic transition interpretation,
- and fully differential-geometric reconstruction methods.

10.9.2. Operator-Based Geometry

Remark 10.41. Within such a framework, geometric operations would no longer be based primarily on Cartesian coordinate algebra.

Remark 10.42. Instead, geometry would emerge from:

- local operators,
- metric tensors,
- connection rules,

- covariant derivatives,
- and differential-geometric transport.

10.10. Runtime Dependency

10.10.1. Runtime Evaluation

Remark 10.43. Metric realization affects runtime services such as:

- projection,
- stationing,
- frame construction,
- nearest-point search,
- pose3 evaluation,
- and geometric reconstruction.

10.10.2. Metric-Aware Runtime

Remark 10.44. Runtime systems should therefore evaluate geometry through metric-aware operator services rather than through hardcoded Euclidean assumptions.

10.10.3. pose3 Dependency

Remark 10.45. Runtime pose3 evaluation depends jointly on:

- semantic alignment states,
- metric realization,
- CRS embedding,
- frame operators,
- profile interpretation,
- and cant interpretation.

Remark 10.46. Thus, pose3 is not a purely intrinsic object, but a runtime-generated metric interpretation of semantic alignment structure.

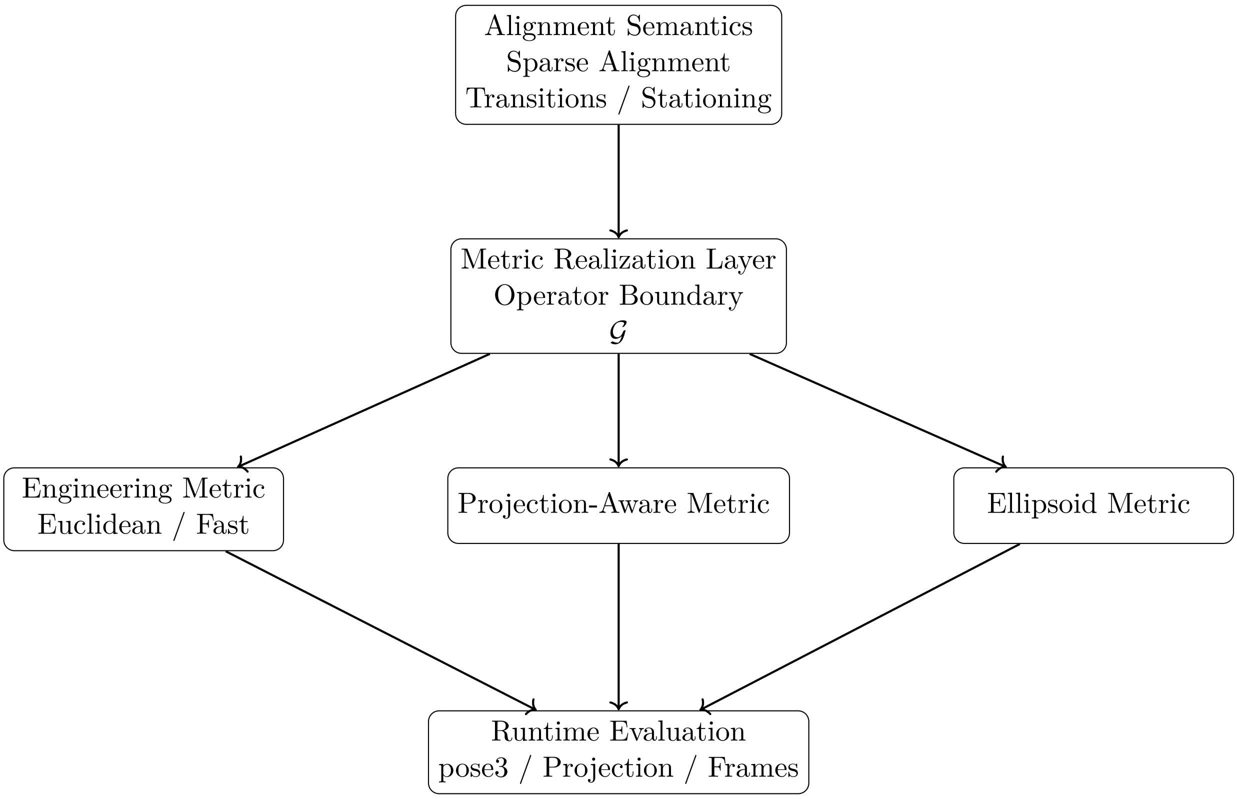

Abbildung 10.1.: AIM separates semantic alignment structure from metric realization. Different metric backends may realize the same semantic alignment model, while runtime services operate on the resulting metric interpretation.

10.11. Conceptual Architecture

10.12. Key Insight

Proposition 10.47. *Railway alignment semantics and metric realization are fundamentally different conceptual layers.*

Proposition 10.48. *The same semantic alignment structure may be realized within different metric spaces.*

Proposition 10.49. *Metric realization therefore acts as an operator layer between semantic alignment logic and spatial geometry evaluation.*

Proposition 10.50. *Runtime geometry emerges from interaction between:*

- *semantic alignment structure,*
- *metric realization,*
- *CRS embedding,*
- *and runtime evaluation operators.*

10.13. Connection to AIM

Summary 10.51. Within AIM, alignment semantics are intentionally separated from metric realization.

This enables:

- stable sparse alignment representations,
- interchangeable metric models,
- realization-aware geometry,
- projection-aware evaluation,
- runtime-dependent pose3 interpretation,
- and future differential-geometric extensions.

The metric realization layer therefore forms the operator boundary between semantic alignment logic and operational geometry.

11. Ontology and Conceptual Structure

11.1. Motivation

Remark 11.1. Railway alignment modelling combines concepts originating from:

- geometry,
- mechanics,
- surveying,
- construction,
- vehicle dynamics,
- and operational railway engineering.

Remark 11.2. Many inconsistencies in classical systems arise because these concepts are mixed implicitly without a rigorous conceptual structure.

Remark 11.3. Within AIM, a distinction is made between:

- mathematical objects,
- physical objects,
- operational objects,
- and runtime representations.

11.2. Core Ontological Distinction

Definition 11.4 (Ontological layers)**.** The AIM ontology distinguishes between:

1. abstract geometry,
2. realized physical geometry,
3. operational railway behaviour,
4. and computational runtime representation.

Remark 11.5. These layers are related, but not identical.

11.3. Alignment as Abstract Geometry

11.3.1. Alignment Concept

Definition 11.6 (Alignment)**.** An alignment is an intrinsic geometric object primarily defined through:

$$\kappa(s), \qquad \phi(s), \qquad \text{pose}(s).$$

Remark 11.7. Within AIM, an alignment is not merely a spatial curve.

Remark 11.8. Instead, it represents a structured operational geometry along arc-length.

11.3.2. Intrinsic Nature

Remark 11.9. The alignment is:

- intrinsic,
- arc-length-based,
- and independent of global coordinates.

11.4. Target Geometry vs Realized Geometry

11.4.1. Target Geometry

Definition 11.10 (Target geometry)**.** The target geometry is the intended analytical railway state defined by the design process.

11.4.2. Realized Geometry

Definition 11.11 (Realized geometry)**.** The realized geometry is the physically existing railway geometry after construction and maintenance.

11.4.3. Fundamental Distinction

Proposition 11.12. *In general:*

$$\text{pose3}_{\text{real}} \neq \text{pose3}_{\text{target}}.$$

Remark 11.13. This distinction is fundamental within AIM.

11.5. Geometry vs Vehicle Space

11.5.1. Classical Simplification

Remark 11.14. Classical railway alignment systems often reduce the railway problem to a centerline geometry problem.

11.5.2. Operational Reality

Remark 11.15. The operationally relevant object, however, is the railway vehicle moving within space.

Remark 11.16. Important operational quantities include:

- center-of-gravity motion,
- vehicle envelope,
- bogie rotation,
- wheelset orientation,
- and platform interaction.

11.5.3. Consequence

Proposition 11.17. *The railway alignment is operationally meaningful only together with its vehicle-space interpretation.*

11.6. Pose, Frame, and State

11.6.1. Pose

Definition 11.18 (Pose)**.** A pose describes a local geometric state attached to arc-length position s.

11.6.2. Frame

Definition 11.19 (Frame)**.** A frame defines a local coordinate interpretation of the pose.

11.6.3. State

Definition 11.20 (State)**.** A state combines:

- geometry,
- orientation,
- and operational interpretation.

11.6.4. Runtime Nature

Remark 11.21. Within AIM, states are runtime-evaluated rather than purely static data objects.

11.7. World Space vs Track Space

11.7.1. Track Space

Definition 11.22 (Track space)**.** Track space is the intrinsic coordinate system attached to the alignment.

11.7.2. World Space

Definition 11.23 (World space)**.** World space is a global spatial coordinate system defined through a CRS.

11.7.3. Transformation

Remark 11.24. The transformation:

$$\text{world} \leftrightarrow \text{track}$$

is nonlinear and context-dependent.

11.8. CRS and Earth Geometry

11.8.1. Projection Dependence

Remark 11.25. Railway geometry depends fundamentally on the underlying coordinate reference system.

11.8.2. Non-Invariance

Proposition 11.26. *Railway alignments are generally not invariant under CRS changes.*

11.8.3. Ellipsoid Reality

Remark 11.27. The physically realized railway exists on the Earth surface rather than within a projected Euclidean plane.

11.9. Runtime Ontology

11.9.1. Runtime Services

Remark 11.28. Many AIM objects exist operationally as runtime services rather than stored static entities.

11.9.2. Examples

Remark 11.29. Examples include:

- world–track projection,
- frame evaluation,
- local coordinate systems,
- and vehicle-space interpretation.

11.10. Realization Ontology

11.10.1. Construction Process

Remark 11.30. The physically realized railway emerges from:

- measurement,
- machine operation,
- maintenance,
- and mechanical response.

11.10.2. Iterative Nature

Remark 11.31. Realization is therefore not a static mapping, but an iterative physical process.

11.11. Optimization Ontology

11.11.1. Classical Optimization

Remark 11.32. Classical optimization often minimizes purely analytical target states.

11.11.2. AIM Optimization

Remark 11.33. Within AIM, optimization operates on:

- realized states,
- operational behaviour,
- and runtime-evaluated constraints.

11.11.3. Inverse Nature

Remark 11.34. Many railway optimization problems are inverse realization problems.

11.12. Sparse vs Runtime Reality

11.12.1. Sparse Representation

Definition 11.35 (Sparse representation)**.** A sparse representation stores only the essential alignment-defining parameters.

11.12.2. Runtime Expansion

Remark 11.36. Operational runtime systems reconstruct:

- geometry,
- frames,
- vehicle-space behaviour,
- and projections

from sparse intrinsic data.

11.13. Fundamental AIM View

Proposition 11.37. *A railway alignment is not merely a geometric curve.*

Remark 11.38. It is a multi-layer operational system connecting:

- intrinsic geometry,
- realization,
- Earth geometry,
- vehicle-space behaviour,
- runtime services,
- and optimization.

11.14. Key Insight

Proposition 11.39. *Many limitations of classical railway systems arise from implicit collapsing of fundamentally different ontological layers into a single geometric abstraction.*

11.15. Connection to AIM

Summary 11.40. The AIM ontology separates:

- target geometry,
- realized geometry,
- operational vehicle behaviour,
- runtime representations,
- and Earth-related coordinate systems.

This separation forms the conceptual foundation for realization-aware, vehicle-aware, and Earth-aware railway alignment modelling.

12. Curvature on the Ellipsoid

12.1. Motivation

Remark 12.1. Railway alignment modelling is traditionally performed in projected coordinate systems such as Gauss–Krüger or UTM.

Remark 12.2. Within these systems, curvature, arc-length, and derived dynamic quantities are evaluated in a planar approximation of the Earth.

Remark 12.3. However, such projections introduce geometric distortions and therefore do not exactly represent the physical geometry on which the railway exists.

Remark 12.4. This motivates a formulation of alignment geometry directly on the reference surface of the Earth, typically modelled as an ellipsoid.

12.2. Curve on a Surface

Definition 12.5 (Reference ellipsoid)**.** Let $\mathcal{E} \subset \mathbb{R}^3$ denote a reference ellipsoid representing the Earth.

Definition 12.6 (Alignment on the ellipsoid)**.** An alignment is defined as a curve

$$\gamma : [0, L] \to \mathcal{E}$$

parameterized by arc-length s.

Remark 12.7. The parameterization satisfies

$$\|\gamma'(s)\| = 1,$$

so that s represents physical distance along the curve.

12.3. Tangent and Normal Decomposition

Definition 12.8 (Tangent vector)**.** The tangent vector is defined as

$$\mathbf{t}(s) = \gamma'(s).$$

Definition 12.9 (Surface normal)**.** Let $\mathbf{n}_{\mathcal{E}}(s)$ denote the unit normal vector of the ellipsoid at $\gamma(s)$.

Remark 12.10. The second derivative $\gamma''(s)$ can be decomposed into:

- a component tangent to the surface,
- a component normal to the surface.

Definition 12.11 (Decomposition)**.**

$$\gamma''(s) = \underbrace{\nabla_s \mathbf{t}(s)}_{\text{tangential}} + \underbrace{(\gamma''(s) \cdot \mathbf{n}_{\mathcal{E}})\, \mathbf{n}_{\mathcal{E}}}_{\text{normal}}.$$

Remark 12.12. Here, $\nabla_s \mathbf{t}$ denotes the covariant derivative of the tangent vector along the curve on the surface.

12.4. Geodesic Curvature

Definition 12.13 (Geodesic curvature)**.** The intrinsic curvature of the alignment on the ellipsoid is defined as

$$\kappa_g(s) = \|\nabla_s \mathbf{t}(s)\|.$$

Remark 12.14. Geodesic curvature measures the change of direction within the surface.

Remark 12.15. It is the surface-intrinsic counterpart of planar curvature and is the relevant curvature notion for alignment geometry constrained to the reference surface.

12.5. Normal Curvature

Definition 12.16 (Normal curvature)**.** The normal curvature is defined as the normal component of the second derivative:

$$\kappa_n(s) = |\gamma''(s) \cdot \mathbf{n}_{\mathcal{E}}(s)|.$$

Remark 12.17. Normal curvature is induced by the curvature of the ellipsoid itself and does not describe the bending of the alignment within the surface.

12.6. Total Curvature

Proposition 12.18. *The total spatial curvature satisfies:*

$$\kappa^2(s) = \kappa_g^2(s) + \kappa_n^2(s).$$

Remark 12.19. Thus, the spatial curvature decomposes into an intrinsic surface contribution and an extrinsic surface-induced contribution.

12.7. Railway-Relevant Curvature

Proposition 12.20. *For railway alignment modelling on a reference surface, the railway-relevant curvature is the geodesic curvature:*

$$\kappa_{\text{rail}}(s) = \kappa_g(s).$$

Remark 12.21. Railway vehicles move along the surface, and dynamic quantities such as lateral acceleration depend on curvature within the surface rather than on the embedding curvature of the Earth.

12.8. Relation to Planar Models

Remark 12.22. In planar geometry, curvature is defined as

$$\kappa(s) = \theta'(s).$$

Remark 12.23. On a curved surface, the heading-angle derivative is replaced by the covariant change of the tangent direction:

$$\kappa_g(s) = \|\nabla_s \mathbf{t}(s)\|.$$

Remark 12.24. Thus, classical planar formulations are a special case of the intrinsic surface formulation.

12.9. Special Case: Geodesics

Definition 12.25 (Geodesic)**.** A curve satisfies

$$\kappa_g(s) = 0$$

if and only if it is a geodesic on the surface.

Remark 12.26. Geodesics represent straightest paths on the ellipsoid in the intrinsic surface sense.

Remark 12.27. They are generally not straight in projected coordinate systems.

Remark 12.28. Railway alignment design is not equivalent to geodesic design. A railway alignment must satisfy engineering, dynamic, operational, and realization constraints.

12.10. Computational Formulation

Remark 12.29. In practical computations, geodesic curvature can be evaluated by projecting the second derivative onto the tangent plane of the surface:

$$\kappa_g = \left\| \gamma'' - (\gamma'' \cdot \mathbf{n}_{\mathcal{E}})\mathbf{n}_{\mathcal{E}} \right\| .$$

Remark 12.30. This formulation separates curvature within the surface from curvature caused by the embedding of the surface in three-dimensional space.

12.11. Implications for Engineering

Proposition 12.31. *Curvature evaluated in projected coordinate systems differs from intrinsic curvature on the ellipsoid.*

Remark 12.32. Consequently:

- arc-length is only approximately preserved,
- curvature is distorted by projection,
- dynamic quantities are evaluated on an approximate geometry.

Proposition 12.33. *Evaluating curvature intrinsically yields more physically consistent results.*

12.12. Vision

Remark 12.34. A consistent modelling framework may formulate alignment geometry directly on the ellipsoid and use projections only for visualization and data exchange.

Remark 12.35. This enables:

- physically meaningful curvature,
- accurate dynamic evaluation,
- improved robustness in optimization,
- and increased safety in alignment design.

Remark 12.36. This is a long-term research direction rather than a prerequisite for practical AIM implementations.

12.13. Connection to AIM

Summary 12.37. Curvature on the ellipsoid must be understood as an intrinsic quantity, measured within the surface rather than in ambient space.

For railway alignment on a reference surface, the relevant curvature is the geodesic curvature.

This distinction is essential for physically consistent modelling and provides a foundation for extending the AIM framework beyond projection-based engineering geometry.

13. Pose3 on the Ellipsoid

13.1. Motivation

Remark 13.1. The planar pose3 model describes railway state by position, direction, curvature, and cant in a projected engineering coordinate system.

Remark 13.2. For physically consistent modelling on the Earth, this state must be defined on the reference ellipsoid rather than in a planar projection.

13.2. Surface-Based Pose

Definition 13.3 (Surface pose)**.** Let $\mathcal{E}$ be a reference ellipsoid and let

$$\gamma : [0, L] \to \mathcal{E}$$

be an arc-length parameterized alignment.

A surface pose is given by

$$\mathrm{pose}_{\mathcal{E}}(s) = \big(\gamma(s), \mathbf{t}(s), \mathbf{n}_{\mathcal{E}}(s)\big),$$

where $\mathbf{t}(s) = \gamma'(s)$ is the tangent vector and $\mathbf{n}_{\mathcal{E}}(s)$ is the ellipsoid normal.

13.3. Railway Surface Frame

Definition 13.4 (Surface railway frame)**.** The local railway frame is defined by

$$\mathbf{t}(s), \qquad \mathbf{b}(s) = \mathbf{n}_{\mathcal{E}}(s) \times \mathbf{t}(s), \qquad \mathbf{n}_{\mathcal{E}}(s).$$

Remark 13.5. Here, $\mathbf{t}$ is the longitudinal direction, $\mathbf{b}$ is the lateral direction within the surface, and $\mathbf{n}_{\mathcal{E}}$ is the local surface normal.

13.4. Cant as Rotation of the Cross-Section

Definition 13.6 (Cant on the ellipsoid)**.** Cant is represented by a rotation of the cross-section around the tangent vector $\mathbf{t}(s)$:

$$R_{\mathbf{t}}(\phi(s)).$$

Remark 13.7. This rotation acts on the lateral–normal plane spanned by

$$\mathbf{b}(s), \quad \mathbf{n}_{\mathcal{E}}(s).$$

Definition 13.8 (Canted lateral direction)**.** The canted lateral direction is

$$\mathbf{b}_{\phi}(s) = R_{\mathbf{t}}(\phi(s))\,\mathbf{b}(s).$$

Definition 13.9 (Canted normal direction)**.** The canted normal direction is

$$\mathbf{n}_{\phi}(s) = R_{\mathbf{t}}(\phi(s))\,\mathbf{n}_{\mathcal{E}}(s).$$

13.5. Gravity Direction

Remark 13.10. On the ellipsoid, gravity is not identical to the ellipsoid normal in full physical precision.

Definition 13.11 (Gravity vector)**.** Let

$$\mathbf{g}(s)$$

denote the local gravity vector at $\gamma(s)$.

Remark 13.12. In a simplified model, one may approximate

$$\mathbf{g}(s) \parallel -\mathbf{n}_{\mathcal{E}}(s).$$

Remark 13.13. A refined model distinguishes between ellipsoid normal, geoid normal, and local gravity direction.

13.6. Curvature and Lateral Acceleration

Definition 13.14 (Railway curvature on the ellipsoid)**.** The railway-relevant curvature is the geodesic curvature

$$\kappa_{\text{rail}}(s) = \kappa_g(s).$$

Remark 13.15. This curvature measures direction change within the reference surface and replaces the planar relation $\kappa = \theta'$.

Definition 13.16 (Surface lateral acceleration)**.** For speed v, the curvature-induced lateral acceleration is

$$a_{\kappa}(s) = v^2 \kappa_g(s).$$

13.7. Effective Acceleration with Cant

Remark 13.17. The effective lateral acceleration is obtained by projecting both curvature-induced acceleration and gravity into the canted lateral direction.

Definition 13.18 (Effective lateral acceleration)**.** A geometrically consistent formulation is

$$a_{\text{eff}}(s) = v^2\kappa_g(s) - \mathbf{g}(s) \cdot \mathbf{b}_\phi(s).$$

Remark 13.19. In the planar approximation with

$$\mathbf{g} \parallel -\mathbf{n}_{\mathcal{E}},$$

this reduces to the familiar expression

$$a_{\text{eff}}(s) = v^2\kappa(s) - g\sin\phi(s).$$

13.8. Pose3 State on the Ellipsoid

Definition 13.20 (Ellipsoidal pose3)**.** The pose3 state on the ellipsoid is

$$\text{pose3}_{\mathcal{E}}(s) = \big(\gamma(s), \mathbf{t}(s), \mathbf{n}_{\mathcal{E}}(s), \kappa_g(s), \phi(s)\big).$$

Remark 13.21. Compared to planar pose3, the heading angle $\theta(s)$ is replaced by a tangent vector field on the surface.

13.9. Implication for AIM

Summary 13.22. Pose3 on the ellipsoid replaces planar heading-based geometry by a surface-based moving frame.

Cant becomes a rotation of the track cross-section around the tangent vector, while curvature is represented by geodesic curvature.

This provides a physically more consistent foundation for evaluating railway dynamics, especially where projection distortions become relevant.

Teil III.

Geometry and Curvature

Introduction

Remark 13.23. Railway alignment geometry is traditionally described through curves, radii, transition elements, and engineering rules.

Remark 13.24. Within AIM, geometry is interpreted differently: not primarily as collections of geometric primitives, but as controlled curvature evolution along arc-length.

Remark 13.25. This shifts the focus:

- from coordinates to intrinsic structure,
- from stored geometry to reconstruction,
- and from isolated curve elements to coupled state evolution.

Remark 13.26. The central geometric quantity is therefore not position itself, but curvature:

$$\kappa(s).$$

Remark 13.27. Within the AIM framework:

- geometry emerges from curvature integration,
- transition curves become curvature-transition laws,
- and sparse alignment models become compact curvature programs.

Remark 13.28. This geometric viewpoint naturally connects:

- reconstruction,
- realization,
- runtime evaluation,
- dynamics,
- and optimization.

Remark 13.29. The chapters of this part introduce the intrinsic geometric structure of railway alignments, beginning with curvature space and continuing toward transition classification and higher-order geometric interpretation.

Proposition 13.30. *Within AIM, railway geometry is fundamentally interpreted as curvature evolution along intrinsic arc-length.*

14. Curvature Space

14.1. Motivation

Remark 14.1. Classical railway alignment theory is usually formulated in position space.

Remark 14.2. Within AIM, the primary geometric object is not the position curve itself, but the curvature evolution along arc-length.

Remark 14.3. This shifts the interpretation of railway geometry from coordinate-based description toward intrinsic curvature structure.

Remark 14.4. The concept of curvature space provides the mathematical setting for:

- transition curves,
- sparse reconstruction,
- realization analysis,
- runtime evaluation,
- and alignment optimization.

Principle 14.5 (Curvature-space principle). Within AIM, railway alignment geometry is interpreted primarily as a problem in curvature space.

14.2. Curvature as Primary Geometry

Definition 14.6 (Curvature law)**.** A curvature law is a function:

$$\kappa : I \to \mathbb{R},$$

defined on an arc-length interval $I \subset \mathbb{R}$.

Remark 14.7. Within AIM, curvature is interpreted as the primary intrinsic geometric quantity.

Remark 14.8. Geometry is reconstructed from curvature through integration.

Principle 14.9 (Curvature-first principle). Within AIM, geometry is generated from curvature rather than curvature being derived from geometry.

14.3. Curvature Space

Definition 14.10 (Curvature space)**.** Curvature space denotes the space of admissible curvature laws:

$$\mathcal{K} = \{\kappa : I \to \mathbb{R}\}.$$

Remark 14.11. The symbol $\mathcal{K}$ does not denote one fixed function class. It denotes the conceptual space in which different admissibility, smoothness, realization, and optimization requirements may be imposed.

Remark 14.12. Different subclasses of curvature space correspond to different geometric and operational properties.

Remark 14.13. Examples include:

- constant curvature,
- piecewise constant curvature,
- continuous curvature,
- differentiable curvature,
- bounded curvature,
- transition curvature,
- and hybrid curvature laws.

OPEN: Later extensions of curvature space include admissible curvature laws, curvature spectra, realization-stable curvature, curvature energy, curvature bandwidth, transition-family subspaces, Vienna-type reinterpretations, and operator deformations.

14.4. Intrinsic Reconstruction

Definition 14.14 (Intrinsic reconstruction)**.** Given a curvature law:

$$\kappa(s),$$

the tangent direction satisfies:

$$\theta'(s) = \kappa(s),$$

and geometry is reconstructed through:

$$\gamma'(s) = \mathbf{t}(s).$$

Remark 14.15. Curvature therefore acts as the differential generator of geometry.

Remark 14.16. The position curve γ is not primary. It is the result of integrating the intrinsic curvature state together with initial pose data.

14.5. Constant Curvature

Definition 14.17 (Constant curvature)**.** A constant curvature law satisfies:

$$\kappa(s) = \kappa_0.$$

Remark 14.18. Special cases are:

- $\kappa_0 = 0$: straight line,
- $\kappa_0 \neq 0$: circular arc.

Remark 14.19. Constant-curvature elements form the fixed parts of the sparse alignment model.

14.6. Transition Curvature

Definition 14.20 (Transition curvature)**.** A transition curvature law connects different curvature states continuously or smoothly.

Remark 14.21. Transition curves are therefore interpreted as curvature-transition processes rather than merely geometric interpolation curves.

Remark 14.22. Typical transition quantities include:

$$\kappa'(s), \qquad \kappa''(s), \qquad \kappa'''(s).$$

Remark 14.23. In AIM, transition families are interpreted as structured subspaces or parameterized paths within curvature space.

14.7. Curvature Continuity

Definition 14.24 (Curvature continuity)**.** A curvature law is:

- C^0-continuous if $\kappa(s)$ is continuous,
- C^1-continuous if $\kappa'(s)$ is continuous,
- C^2-continuous if $\kappa''(s)$ is continuous.

Remark 14.25. Higher continuity improves:

- dynamic smoothness,
- realization stability,
- numerical robustness,
- and passenger comfort.

Remark 14.26. Continuity of position is not sufficient for high-quality railway geometry. The decisive structure lies in the continuity and behaviour of curvature and its derivatives.

14.8. Sparse Curvature Representation

Definition 14.27 (Sparse curvature representation)**.** A sparse representation describes curvature using:

- fixed curvature elements,
- transition elements,
- and compact parameter sets.

Remark 14.28. Within AIM, sparse alignments are interpreted as compact curvature programs.

Remark 14.29. The sparse model stores:

- curvature structure,
- transition families,
- and reconstruction rules,

rather than dense geometric point clouds.

Remark 14.30. This makes sparse alignment suitable for:

- reconstruction,
- import normalization,
- runtime evaluation,
- realization comparison,
- and optimization.

14.9. Hybrid Curvature Representation

Definition 14.31 (Hybrid curvature model)**.** A hybrid curvature model combines:

- parametric curvature structure,
- sampled correction data,
- and local refinement.

Remark 14.32. Hybrid representations allow:

- compact storage,
- efficient reconstruction,

- realization-aware correction,
- measured-state integration,
- and scalable runtime evaluation.

Remark 14.33. Hybrid curvature models are especially relevant when measured or realized geometry cannot be represented adequately by a purely analytical target model.

14.10. Normalized Curvature Space

Definition 14.34 (Normalized curvature law)**.** A normalized curvature law is defined on:

$$w \in [0, 1].$$

Remark 14.35. Normalized representations separate:

- geometric shape,
- from physical scaling.

Remark 14.36. This enables:

- reusable transition families,
- lookup-based transitions,
- family interpolation,
- and normalized optimization.

Remark 14.37. Physical curvature laws are obtained from normalized laws by scaling in length and curvature amplitude.

14.11. Curvature Derivatives

Definition 14.38 (Curvature derivatives)**.** Important higher-order quantities include:

$$\kappa'(s), \qquad \kappa''(s), \qquad \kappa'''(s).$$

Remark 14.39. These quantities influence:

- jerk,
- realization behaviour,
- dynamic smoothness,
- vehicle response,

- and transition quality.

Remark 14.40. Curvature derivatives provide the bridge from intrinsic geometry to runtime dynamics.

14.12. Frequency Interpretation

Remark 14.41. Curvature laws may also be interpreted spectrally.

Remark 14.42. Low-frequency curvature components determine global geometric structure, whereas high-frequency components influence:

- realization sensitivity,
- numerical instability,
- measurement sensitivity,
- and dynamic roughness.

Proposition 14.43 (Spectral realization principle)**.** *Realization acts approximately as a low-pass filter on curvature space.*

Remark 14.44. This spectral view connects curvature space directly to construction, maintenance, and physical realization.

14.13. Curvature Space and Runtime

Remark 14.45. Runtime evaluation operates primarily on curvature structures rather than directly on dense geometry.

Remark 14.46. Projection, reconstruction, frame evaluation, pose3 evaluation, and dynamics are derived from runtime evaluation of curvature-based states.

Remark 14.47. Thus, curvature space is not only a design space. It is also the computational state space underlying runtime reconstruction.

14.14. Curvature Space and Realization

Remark 14.48. Realization transforms target curvature into physically realized curvature.

Remark 14.49. This may be written abstractly as:

$$\kappa_{\text{real}} = \mathcal{R}_{\kappa}(\kappa_{\text{target}}).$$

Remark 14.50. The realization operator may attenuate, deform, or locally modify curvature evolution.

Remark 14.51. Consequently, the physically relevant design object is not only the target curvature law, but also its expected realized curvature state.

14.15. Curvature Space and Optimization

Remark 14.52. Optimization within AIM primarily acts in curvature space.

Remark 14.53. Typical optimization variables include:

- transition parameters,
- curvature amplitudes,
- transition lengths,
- curvature anchors,
- and sparse curvature corrections.

Remark 14.54. AIM optimization may therefore be interpreted as the search for admissible, realizable, and operationally suitable paths in curvature space.

14.16. Canonical Interpretation

Proposition 14.55. *Within AIM, railway alignment geometry is fundamentally a curvature-space problem.*

Proposition 14.56. *Position-space geometry is interpreted as reconstructed geometry derived from curvature evolution.*

Proposition 14.57. *Transition curves are interpreted as controlled transitions within curvature space.*

Proposition 14.58. *Realization, runtime evaluation, and optimization act on curvature-space structures before they act on visualized position geometry.*

14.17. Connection to AIM

Summary 14.59. Curvature space forms the intrinsic geometric foundation of AIM.

Within this framework:

- curvature acts as primary geometry,
- sparse models become curvature programs,
- transition curves become curvature transitions,
- runtime systems operate on intrinsic curvature structure,
- realization transforms curvature states,
- and optimization searches within curvature-space structures.

This interpretation unifies geometry, realization, runtime evaluation, measurement interpretation, and optimization within one intrinsic mathematical framework.

15. Curvature Classes

15.1. Motivation

Remark 15.1. Not all curvature laws possess the same geometric, dynamic, numerical, or physical properties.

Remark 15.2. Within AIM, curvature laws are therefore classified according to:

- continuity,
- differentiability,
- boundedness,
- transition behaviour,
- realization behaviour,
- runtime stability,
- and operational suitability.

Remark 15.3. This classification forms the basis for:

- transition evaluation,
- runtime robustness,
- realization analysis,
- admissibility checks,
- and optimization strategies.

Principle 15.4 (Curvature classification principle). Within AIM, railway alignment elements are classified primarily by their curvature behaviour rather than by their visual position-space shape.

15.2. Basic Curvature Classes

15.2.1. Constant Curvature

Definition 15.5 (Constant curvature)**.** A curvature law is constant if:

$$\kappa(s) = \kappa_0.$$

Remark 15.6. Special cases are:

- $\kappa_0 = 0$: straight line,
- $\kappa_0 \neq 0$: circular arc.

Remark 15.7. Constant curvature forms the fixed-element basis of classical railway geometry.

15.2.2. Piecewise Constant Curvature

Definition 15.8 (Piecewise constant curvature)**.** A curvature law is piecewise constant if:

$$\kappa(s) = \kappa_i$$

on subintervals I_i.

Remark 15.9. This corresponds to classical tangent–arc railway geometry.

Remark 15.10. Piecewise constant curvature is geometrically simple, but curvature jumps are dynamically and operationally problematic.

15.3. Continuity Classes

15.3.1. C^0 Curvature

Definition 15.11 (C^0-continuous curvature)**.** A curvature law belongs to C^0 if:

$$\kappa(s)$$

is continuous.

Remark 15.12. Continuous curvature eliminates curvature jumps.

15.3.2. C^1 Curvature

Definition 15.13 (C^1-continuous curvature)**.** A curvature law belongs to C^1 if:

$$\kappa'(s)$$

exists and is continuous.

Remark 15.14. This improves dynamic smoothness, realization behaviour, and numerical robustness.

15.3.3. C^2 Curvature

Definition 15.15 (C^2-continuous curvature)**.** A curvature law belongs to C^2 if:

$$\kappa''(s)$$

exists and is continuous.

Remark 15.16. Higher continuity reduces excitation effects, numerical instability, and unwanted high-frequency behaviour.

Remark 15.17. In AIM, continuity class is not merely a mathematical property. It is an indicator for dynamic quality, runtime stability, and realization robustness.

15.4. Boundedness Classes

15.4.1. Bounded Curvature

Definition 15.18 (Bounded curvature)**.** A curvature law is bounded if:

$$|\kappa(s)| \leq \kappa_{\max}.$$

Remark 15.19. Bounded curvature corresponds to a minimum admissible radius.

15.4.2. Bounded Curvature Gradient

Definition 15.20 (Bounded curvature gradient)**.** A curvature law has bounded curvature gradient if:

$$|\kappa'(s)| \leq \eta_{\max}.$$

Remark 15.21. This is closely related to jerk limitations in railway dynamics.

15.4.3. Bounded Higher Derivatives

Definition 15.22 (Bounded higher derivatives)**.** Higher-order boundedness conditions include:

$$|\kappa''(s)| < \infty, \qquad |\kappa'''(s)| < \infty.$$

Remark 15.23. Higher-order boundedness becomes relevant for realization behaviour, vehicle response, and transition-family comparison.

15.5. Transition Classes

15.5.1. Monotone Transitions

Definition 15.24 (Monotone transition)**.** A transition curvature law is monotone if:

$$\kappa'(s)$$

does not change sign.

Remark 15.25. Monotone transitions represent direct curvature change between two curvature states.

15.5.2. Symmetric Transitions

Definition 15.26 (Symmetric transition)**.** A normalized transition law is symmetric if its curvature evolution is symmetric with respect to the midpoint of the transition interval.

Remark 15.27. For normalized transition coordinates $w \in [0, 1]$, this may be expressed by a symmetry relation of the transition shape function.

Remark 15.28. Symmetry is not an absolute requirement, but it provides a useful classification property.

15.5.3. Asymmetric Transitions

Definition 15.29 (Asymmetric transition)**.** A transition law is asymmetric if symmetry is intentionally violated.

Remark 15.30. Asymmetric transitions may appear in:

- constrained engineering situations,
- realization-aware optimization,
- vehicle-dynamics optimization,
- and topology-constrained alignments.

15.6. Analytical Classes

15.6.1. Polynomial Curvature Laws

Definition 15.31 (Polynomial curvature)**.** A polynomial curvature law satisfies:

$$\kappa(s) = \sum_{i=0}^{n} a_i s^i.$$

Remark 15.32. Polynomial curvature laws are useful because derivatives and integrals can often be handled systematically.

15.6.2. Trigonometric Curvature Laws

Definition 15.33 (Trigonometric curvature)**.** A trigonometric curvature law contains sinusoidal or cosine-based components.

Remark 15.34. Trigonometric curvature laws are useful for smooth transition behaviour and frequency-oriented interpretation.

15.6.3. Lookup-Based Curvature Laws

Definition 15.35 (Lookup-based curvature)**.** A lookup-based curvature law is defined by sampled or tabulated representations together with interpolation rules.

Remark 15.36. Lookup-based laws allow practical inclusion of transition families or realization effects that are not conveniently represented by a simple closed formula.

15.6.4. Hybrid Curvature Laws

Definition 15.37 (Hybrid curvature law)**.** A hybrid curvature law combines:

- parametric structure,
- sampled corrections,
- measured data,
- and local refinement.

Remark 15.38. Hybrid laws are especially important for measured and realized railway states.

15.7. Sparse Curvature Classes

Definition 15.39 (Sparse curvature representation)**.** A sparse curvature model represents geometry through:

- fixed curvature elements,
- transition elements,
- and compact parameter sets.

Remark 15.40. Sparse classes emphasize:

- compactness,
- runtime efficiency,
- reconstruction stability,

- and semantic clarity.

Remark 15.41. Within AIM, sparse curvature classes are not merely storage formats. They define executable curvature programs.

15.8. Operational Classes

15.8.1. Dynamically Smooth Curvature

Definition 15.42 (Dynamically smooth curvature)**.** A curvature law is dynamically smooth if:

- jerk remains bounded,
- cant-rate remains admissible,
- curvature variation is controlled,
- and high-frequency excitation is limited.

15.8.2. Realization-Stable Curvature

Definition 15.43 (Realization-stable curvature)**.** A curvature law is realization-stable if:

- realization preserves its dominant structure,
- construction processes do not introduce instability,
- and its low-frequency behaviour remains identifiable after physical realization.

Remark 15.44. Realization-stable curvature is a physical property, not merely an analytical smoothness property.

15.8.3. Runtime-Stable Curvature

Definition 15.45 (Runtime-stable curvature)**.** A curvature law is runtime-stable if:

- projection remains numerically stable,
- frame evaluation remains continuous,
- pose reconstruction behaves robustly,
- and local inverse problems remain well-conditioned.

15.9. Frequency Classes

15.9.1. Low-Frequency Curvature

Definition 15.46 (Low-frequency curvature)**.** Low-frequency curvature components determine large-scale geometric behaviour.

Remark 15.47. These components are usually preserved by construction and maintenance processes.

15.9.2. High-Frequency Curvature

Definition 15.48 (High-frequency curvature)**.** High-frequency curvature components influence:

- realization sensitivity,
- dynamic roughness,
- measurement sensitivity,
- and numerical instability.

Proposition 15.49 (Spectral realization principle)**.** *High-frequency curvature components are attenuated by realization.*

Remark 15.50. This frequency-oriented classification connects curvature classes with the realization filter interpretation.

15.10. Admissibility

Definition 15.51 (Admissible curvature law)**.** A curvature law is admissible if it satisfies:

- geometric constraints,
- realization constraints,
- operational constraints,
- dynamic constraints,
- and numerical stability conditions.

Remark 15.52. Admissibility depends on:

- vehicle dynamics,
- realization processes,
- standards,

- topology,
- runtime requirements,
- and intended operational use.

Remark 15.53. Thus, admissibility is context-dependent. A curvature law may be analytically valid but operationally unsuitable or physically unstable under realization.

15.11. Classification versus Quality

Remark 15.54. Classification does not automatically imply quality.

Remark 15.55. A highly smooth curvature law may still be unsuitable if it is:

- difficult to realize,
- dynamically unfavourable,
- numerically unstable,
- incompatible with topology,
- or unsuitable for the intended operation.

Remark 15.56. Within AIM, curvature quality is evaluated by the interaction of class, context, realization, and runtime behaviour.

15.12. Canonical Interpretation

Proposition 15.57. *Different curvature classes correspond to fundamentally different geometric, operational, runtime, and realization behaviours.*

Proposition 15.58. *Within AIM, railway alignment quality is primarily characterized in curvature space rather than position space.*

Proposition 15.59. *Transition families are interpreted as structured subsets of curvature classes.*

Proposition 15.60. *Admissibility is not a purely geometric property, but a combined geometric, dynamic, runtime, and realization property.*

15.13. Connection to AIM

Summary 15.61. Curvature classes provide the structural classification framework of AIM geometry.

They connect:

- continuity,
- differentiability,
- boundedness,
- transition behaviour,
- realization behaviour,
- runtime stability,
- dynamic smoothness,
- and optimization suitability.

Within AIM, transition curves are therefore interpreted not merely as geometric entities, but as members of specific curvature classes with distinct operational properties.

This creates the bridge from intrinsic geometry to classification, realization analysis, runtime evaluation, and optimization.

16. Curvature Transitions

16.1. Motivation

Remark 16.1. Railway alignments rarely consist of isolated constant-curvature segments.

Remark 16.2. Instead, operational geometry requires smooth transitions between different curvature states.

Remark 16.3. Within AIM, transition curves are interpreted fundamentally as curvature-transition processes.

Remark 16.4. This shifts the interpretation:

- from position interpolation,
- to controlled curvature evolution.

Principle 16.5 (Curvature-transition principle). Within AIM, a transition curve is primarily a law of curvature evolution, not a coordinate-space interpolation object.

16.2. Transition Principle

Definition 16.6 (Curvature transition)**.** A curvature transition is a curvature law:

$$\kappa(s)$$

connecting two curvature states:

$$\kappa_A \to \kappa_B.$$

Remark 16.7. The transition determines:

- geometric smoothness,
- dynamic behaviour,
- realization stability,
- runtime robustness,
- and operational comfort.

Principle 16.8 (Transition principle). Within AIM, transition geometry is governed primarily by curvature evolution rather than by coordinate interpolation.

16.3. Normalized Transition Space

Definition 16.9 (Normalized transition parameter)**.** Transitions are commonly normalized to:

$$w \in [0, 1].$$

Definition 16.10 (Normalized curvature law)**.** A normalized transition law satisfies:

$$\hat{\kappa}(w) \in [0, 1].$$

Remark 16.11*.* Normalization separates:

- intrinsic transition shape,
- from physical scaling.

Remark 16.12*.* This enables:

- reusable transition families,
- family interpolation,
- lookup-based evaluation,
- normalized optimization,
- and compact sparse representation.

Remark 16.13*.* A physical transition is obtained by scaling normalized curvature and normalized length to the actual curvature interval and physical arc-length.

16.4. Transition Families

Definition 16.14 (Transition family)**.** A transition family is a class of normalized curvature laws sharing common structural properties.

Remark 16.15*.* Examples include:

- clothoids,
- polynomial transitions,
- trigonometric transitions,
- Bloss transitions,
- Vienna transitions,
- lookup-based transitions,
- and hybrid transition families.

Remark 16.16*.* Within AIM, transition families are interpreted as structured regions or parameterized paths within normalized curvature space.

16.5. Clothoid Transition

Definition 16.17 (Clothoid)**.** The clothoid is defined by linear curvature evolution:

$$\kappa(s) = as + b.$$

Remark 16.18. In normalized form, the clothoid corresponds to:

$$\hat{\kappa}(w) = w.$$

Remark 16.19. The clothoid provides constant curvature gradient.

Remark 16.20. Historically, the clothoid became dominant because:

- it is simple,
- analytically tractable,
- compatible with classical railway practice,
- and dynamically smoother than curvature jumps.

Remark 16.21. Within AIM, the clothoid is not treated as the universal transition truth, but as one important member of curvature-transition space.

16.6. Polynomial Transitions

Definition 16.22 (Polynomial transition)**.** A polynomial transition satisfies:

$$\kappa(s) = \sum_{i=0}^{n} a_i s^i.$$

Remark 16.23. Polynomial transitions allow:

- higher continuity,
- endpoint constraints,
- derivative constraints,
- and optimized dynamic behaviour.

Remark 16.24. Polynomial transitions are especially useful when smoothness conditions are imposed directly on curvature derivatives.

16.7. Trigonometric Transitions

Definition 16.25 (Trigonometric transition)**.** A trigonometric transition contains sinusoidal or cosine-based curvature components.

Remark 16.26. These transitions often provide smoother higher-order behaviour than low-degree polynomial transitions.

Remark 16.27. Trigonometric transitions are naturally connected to spectral interpretation of curvature space.

16.8. Half-Wave Interpretation

Definition 16.28 (Half-wave transition)**.** A half-wave transition is a normalized curvature segment connecting:

$$0 \to 1$$

or

$$1 \to 0.$$

Remark 16.29. Within AIM, many transition families are interpreted compositionally as:

$$\text{halfWave}_1 \to \text{core} \to \text{halfWave}_2.$$

Remark 16.30. This enables:

- modular transition construction,
- asymmetric transitions,
- family interpolation,
- and transition deformation.

Remark 16.31. The half-wave interpretation provides a common language for comparing classical transition families and newly constructed transition laws.

16.9. Curvature Anchors

Definition 16.32 (Curvature anchors)**.** Normalized transition families may define anchor values:

$$0, \quad c_1, \quad c_2, \quad 1.$$

Remark 16.33. Anchors describe:

- transition partitioning,
- continuity conditions,

- normalization structure,
- and family composition.

Remark 16.34. Curvature anchors make it possible to compare transition families by their internal curvature structure rather than by their external position-space appearance.

16.10. Transition Continuity

Definition 16.35 (Transition continuity)**.** Transition quality depends on continuity of:

$$\kappa(s), \qquad \kappa'(s), \qquad \kappa''(s).$$

Remark 16.36. Higher continuity improves:

- jerk behaviour,
- realization smoothness,
- runtime stability,
- vehicle response,
- and passenger comfort.

Remark 16.37. Continuity in position space alone is insufficient for evaluating transition quality.

16.11. Sparse Transition Representation

Definition 16.38 (Sparse transition)**.** A sparse transition representation stores:

- transition family identity,
- normalization parameters,
- curvature anchors,
- scaling parameters,
- and reconstruction rules.

Remark 16.39. Within AIM, sparse transitions are interpreted as compact curvature operators rather than dense geometric curves.

Remark 16.40. A sparse transition therefore stores how curvature shall evolve, not a sampled polyline approximation of the resulting geometry.

16.12. Lookup-Based Transitions

Definition 16.41 (Lookup-based transition)**.** A lookup-based transition uses:

- sampled normalized curvature,
- interpolation rules,
- derivative reconstruction,
- and runtime evaluation.

Remark 16.42*.* This enables:

- arbitrary transition families,
- experimentally designed transitions,
- realization-adapted transitions,
- vehicle-specific transitions,
- and runtime-efficient evaluation.

Remark 16.43*.* Lookup-based transitions are not a fallback for missing theory. They are a practical representation of transition laws in normalized curvature space.

16.13. Transition Derivatives

Definition 16.44 (Transition derivatives)**.** Important quantities include:

$$\kappa'(s), \qquad \kappa''(s), \qquad \kappa'''(s).$$

Remark 16.45*.* These derivatives influence:

- jerk,
- dynamic smoothness,
- realization behaviour,
- vehicle response,
- and optimization stability.

Remark 16.46*.* Derivative behaviour is one of the main differences between transition families that may otherwise appear similar in position space.

16.14. Transition Symmetry

Definition 16.47 (Symmetric transition)**.** A normalized transition is symmetric if its curvature evolution is symmetric with respect to the midpoint of the transition interval.

Definition 16.48 (Asymmetric transition)**.** A transition is asymmetric if this symmetry is intentionally violated.

Remark 16.49. Asymmetric transitions may improve:

- realization quality,
- vehicle dynamics,
- topology fitting,
- or geometric constraint satisfaction.

Remark 16.50. Symmetry is therefore a classification property, not a universal requirement.

16.15. Transition Frequency Behaviour

Remark 16.51. Transitions may be interpreted spectrally through their curvature content.

Remark 16.52. High-frequency curvature behaviour influences:

- realization sensitivity,
- machine smoothing,
- measurement sensitivity,
- and dynamic roughness.

Proposition 16.53 (Spectral transition principle)**.** *Transition quality depends not only on curvature continuity, but also on spectral behaviour within curvature space.*

Remark 16.54. This connects transition theory directly to realization-stable curvature and curvature bandwidth.

16.16. Transition Realization

Remark 16.55. The transition that is designed is not necessarily the transition that is physically realized.

Remark 16.56. Realization may modify transition behaviour through:

- sampling,
- machine response,

- structural filtering,
- ballast mechanics,
- measurement feedback,
- and local correction limits.

Remark 16.57. Thus, transition quality must be evaluated both in target curvature space and in expected realized curvature space.

16.17. Transition Optimization

Remark 16.58. Within AIM, optimization acts primarily on:

- transition families,
- normalization structure,
- transition lengths,
- curvature anchors,
- derivative behaviour,
- and curvature evolution.

Remark 16.59. Thus, optimization is interpreted primarily as optimization in curvature-transition space rather than position space.

Remark 16.60. A realization-aware optimization may optimize not only the target transition law κ_{target}, but also the expected realized transition law:

$$\kappa_{\mathrm{real}} = \mathcal{R}_\kappa(\kappa_{\mathrm{target}}).$$

16.18. Canonical Interpretation

Proposition 16.61. *Transition curves are fundamentally curvature-transition laws.*

Proposition 16.62. *Within AIM, transition geometry is interpreted intrinsically within curvature space.*

Proposition 16.63. *Sparse transition models represent compact transition operators rather than explicit geometric curves.*

Proposition 16.64. *Transition quality depends on curvature evolution, derivative behaviour, spectral content, runtime stability, and realization behaviour.*

16.19. Connection to AIM

Summary 16.65. Curvature transitions form the dynamic geometric core of AIM.

Within this framework:

- transitions are interpreted as curvature evolution processes,
- sparse models become transition operators,
- normalized transition space enables reusable families,
- half-wave decomposition enables modular transition construction,
- lookup-based laws enable experimental and realization-aware transition design,
- and runtime systems reconstruct geometry dynamically from curvature structure.

This interpretation unifies transition theory, curvature classification, realization behaviour, runtime evaluation, and optimization within one intrinsic curvature framework.

17. Curvature Classification

17.1. Motivation

Remark 17.1. Classical railway alignment theory typically classifies transitions by their analytical formulas or historical engineering usage.

Remark 17.2. Within AIM, transition and alignment structures are classified intrinsically within curvature space.

Remark 17.3. This allows classification based on:

- continuity,
- curvature behaviour,
- dynamics,
- realization behaviour,
- spectral structure,
- runtime robustness,
- and operational suitability.

Remark 17.4. Curvature classification therefore becomes a structural interpretation framework rather than merely a catalogue of curve formulas.

Principle 17.5 (Classification as interpretation). Within AIM, classification is not a naming scheme for curve formulas. It is an interpretation layer for curvature behaviour.

17.2. Classification Principle

Principle 17.6 (Intrinsic classification principle). Within AIM, alignments are classified by intrinsic curvature behaviour rather than by reconstructed position geometry.

Remark 17.7. Two geometrically similar curves in position space may possess very different curvature behaviour.

Remark 17.8. Conversely, geometrically different realizations may belong to the same curvature class if their intrinsic curvature evolution behaves equivalently.

Remark 17.9. Classification is therefore performed in curvature space before it is interpreted in position space.

17.3. Continuity Classification

17.3.1. G^0 Geometry

Definition 17.10 (G^0-continuous geometry)**.** A geometry is G^0-continuous if position is continuous.

17.3.2. G^1 Geometry

Definition 17.11 (G^1-continuous geometry)**.** A geometry is G^1-continuous if tangent direction is continuous.

17.3.3. G^2 Geometry

Definition 17.12 (G^2-continuous geometry)**.** A geometry is G^2-continuous if curvature is continuous.

Remark 17.13. Within railway alignment modelling, G^2-continuity is often the minimal requirement for dynamically acceptable transitions.

17.3.4. G^3 Geometry

Definition 17.14 (G^3-continuous geometry)**.** A geometry is G^3-continuous if curvature gradient is continuous.

Remark 17.15. Higher continuity classes improve:

- jerk behaviour,
- realization smoothness,
- runtime stability,
- and vehicle response.

Remark 17.16. Continuity classification bridges classical geometry notation with curvature-space interpretation.

17.4. Curvature Behaviour Classification

17.4.1. Constant Curvature Class

Definition 17.17 (Constant curvature class)**.** A curvature law belongs to the constant class if:

$$\kappa(s) = \kappa_0.$$

17.4.2. Linear Curvature Class

Definition 17.18 (Linear curvature class)**.** A curvature law belongs to the linear class if:

$$\kappa(s) = as + b.$$

Remark 17.19. The classical clothoid belongs to this class.

17.4.3. Polynomial Curvature Class

Definition 17.20 (Polynomial curvature class)**.** A curvature law belongs to the polynomial class if:

$$\kappa(s) = \sum_i a_i s^i.$$

17.4.4. Trigonometric Curvature Class

Definition 17.21 (Trigonometric curvature class)**.** A curvature law belongs to the trigonometric class if it contains sinusoidal or cosine-based components.

17.4.5. Lookup-Based Curvature Class

Definition 17.22 (Lookup-based curvature class)**.** A curvature law belongs to the lookup-based class if it is defined by sampled or tabulated curvature values together with interpolation and reconstruction rules.

17.4.6. Hybrid Curvature Class

Definition 17.23 (Hybrid curvature class)**.** A hybrid curvature law combines:

- parametric structure,
- lookup structure,
- measured data,
- and sampled corrections.

17.5. Symmetry Classification

17.5.1. Symmetric Transitions

Definition 17.24 (Symmetric transition)**.** A normalized transition is symmetric if its curvature evolution is symmetric with respect to the midpoint of the normalized interval.

Remark 17.25. For transition comparison, symmetry should usually be evaluated in normalized transition space rather than in raw physical coordinates.

17.5.2. Antisymmetric Structure

Definition 17.26 (Antisymmetric transition)**.** A transition has antisymmetric structure if its deviation from a reference or midpoint value changes sign under midpoint reflection.

Remark 17.27. Antisymmetry is especially useful when analysing centred transition shape functions or derivative behaviour.

17.5.3. Asymmetric Transitions

Definition 17.28 (Asymmetric transition)**.** A transition is asymmetric if no symmetry condition is imposed.

Remark 17.29. Asymmetry may arise intentionally through:

- realization-aware design,
- vehicle-dynamics optimization,
- topology constraints,
- or local engineering constraints.

17.6. Derivative Classification

17.6.1. Bounded Gradient Class

Definition 17.30 (Bounded gradient class)**.** A curvature law belongs to the bounded-gradient class if:

$$|\kappa'(s)| \leq \eta_{\max}.$$

17.6.2. Bounded Higher-Derivative Class

Definition 17.31 (Bounded higher-derivative class)**.** A curvature law belongs to this class if:

$$|\kappa''(s)| < \infty, \qquad |\kappa'''(s)| < \infty.$$

Remark 17.32. These classes influence:

- dynamic smoothness,
- numerical stability,
- realization robustness,
- and optimization conditioning.

17.7. Spectral Classification

17.7.1. Low-Frequency Structures

Definition 17.33 (Low-frequency curvature class)**.** Low-frequency curvature structures are dominated by slowly varying curvature evolution.

Remark 17.34. Low-frequency structure usually determines the large-scale geometric shape of the alignment.

17.7.2. High-Frequency Structures

Definition 17.35 (High-frequency curvature class)**.** High-frequency curvature structures contain rapidly varying or oscillating components.

Remark 17.36. High-frequency structures tend to:

- amplify realization sensitivity,
- increase dynamic roughness,
- increase measurement sensitivity,
- and reduce numerical stability.

Proposition 17.37 (Spectral realization principle)**.** *Realization suppresses high-frequency curvature components.*

Remark 17.38. Spectral classification therefore links transition theory with physical realization behaviour.

17.8. Realization Classification

17.8.1. Realization-Stable Class

Definition 17.39 (Realization-stable class)**.** A transition belongs to the realization-stable class if:

- dominant curvature structure is preserved under realization,
- low-frequency behaviour remains identifiable,
- and realization does not introduce instability.

17.8.2. Realization-Sensitive Class

Definition 17.40 (Realization-sensitive class)**.** A transition belongs to the realization-sensitive class if small realization perturbations strongly alter curvature behaviour.

Remark 17.41. Realization-sensitive transitions may be mathematically valid but operationally undesirable.

17.9. Runtime Classification

17.9.1. Projection-Stable Class

Definition 17.42 (Projection-stable class)**.** A curvature structure is projection-stable if:

- world-to-track projection remains numerically robust,
- local inverse problems remain well-conditioned,
- and ambiguity remains controllable.

17.9.2. Runtime-Smooth Class

Definition 17.43 (Runtime-smooth class)**.** A curvature structure is runtime-smooth if:

- frame evaluation remains continuous,
- pose reconstruction remains robust,
- visualization remains stable,
- and runtime operators behave predictably.

17.10. Operational Classification

17.10.1. Comfort-Oriented Class

Definition 17.44 (Comfort-oriented class)**.** A transition belongs to the comfort-oriented class if:

- jerk remains low,
- lateral acceleration evolves smoothly,
- and dynamic excitation is minimized.

17.10.2. Realization-Oriented Class

Definition 17.45 (Realization-oriented class)**.** A transition belongs to the realization-oriented class if:

- construction robustness is prioritized,
- machine realization remains stable,
- maintenance sensitivity is reduced,
- and physical filtering preserves the intended behaviour.

17.10.3. Optimization-Oriented Class

Definition 17.46 (Optimization-oriented class)**.** A transition belongs to the optimization-oriented class if:

- parameterization is numerically stable,
- gradients remain well-conditioned,
- constraints are expressible,
- and sparse representation is efficient.

17.11. Family Classification

Definition 17.47 (Transition family classification)**.** Transition families may be classified according to:

- analytical form,
- continuity order,
- derivative behaviour,
- spectral behaviour,
- realization behaviour,
- symmetry,
- runtime stability,
- and operational suitability.

Remark 17.48. Thus, transition classification becomes multidimensional rather than formula-based.

Remark 17.49. A single transition family may be favourable in one classification axis and unfavourable in another.

17.12. Classification Vectors

Definition 17.50 (Classification vector)**.** A transition may be assigned a classification vector:

$$\chi(T) = (\chi_{\mathrm{cont}}, \chi_{\mathrm{deriv}}, \chi_{\mathrm{spec}}, \chi_{\mathrm{real}}, \chi_{\mathrm{run}}, \chi_{\mathrm{op}}),$$

where the components describe continuity, derivative behaviour, spectral structure, realization behaviour, runtime stability, and operational suitability.

Remark 17.51. The classification vector is not intended as a rigid taxonomy. It is a structured way to compare transition families across multiple relevant dimensions.

17.13. Canonical Interpretation

Proposition 17.52. *Within AIM, curvature classification is fundamentally intrinsic rather than coordinate-based.*

Proposition 17.53. *Transition quality cannot be characterized sufficiently by position geometry alone.*

Proposition 17.54. *Realization, runtime stability, dynamics, and optimization require classification directly in curvature space.*

Proposition 17.55. *Transition taxonomy in AIM is multidimensional and context-dependent.*

17.14. Connection to AIM

Summary 17.56. Curvature classification provides the structural interpretation framework of AIM geometry.

Within this framework:

- transitions are classified intrinsically,
- formula names are secondary to curvature behaviour,
- realization behaviour becomes analyzable,
- runtime robustness becomes class-dependent,
- operational suitability becomes comparable,
- and optimization acts directly on curvature structures.

This establishes curvature space as the primary classification domain of railway alignment geometry and provides the basis for a taxonomy of transition families.

Teil IV.

Railway State Model

Remark 17.57. This part introduces the railway state model underlying the AIM framework. Instead of treating the track merely as a geometric curve, the state model captures position, orientation, curvature, and cant in a unified representation.

Remark 17.58. The purpose of this part is to define the variables and transformations required to connect intrinsic alignment modelling with dynamic railway interpretation.

Remark 17.59. This establishes the transition from purely geometric description towards a state-based view of railway alignment.

18. Pose2

18.1. Motivation

Remark 18.1. In railway alignment modelling, the geometric state of the track at a position along its length must be represented in a minimal, stable, and intrinsic form.

Remark 18.2. While full spatial railway modelling involves cant, profile, realization, runtime services, and operational interpretation, many fundamental geometric properties can already be expressed within a planar intrinsic framework.

Remark 18.3. The concept of *pose2* provides this minimal intrinsic representation.

Remark 18.4. Within AIM, pose2 is not merely a geometric helper structure. It forms the persistent intrinsic reconstruction kernel of the alignment model.

18.2. Definition of pose2

Definition 18.5 (pose2)**.** A planar pose state at arc-length position s is defined as

$$\text{pose2}(s) = \big(\gamma(s), \theta(s)\big).$$

Remark 18.6. The pose2 state describes:

- intrinsic planar position,
- and intrinsic tangent orientation.

Remark 18.7. The canonical pose2 system is defined in eq. (6.6).

18.3. Curvature-Driven Reconstruction

Principle 18.8 (Curvature-driven reconstruction)*.* Within AIM, planar geometry is reconstructed intrinsically from curvature evolution.

Remark 18.9. The intrinsic reconstruction chain follows directly from eqs. (6.3) to (6.5).

Remark 18.10. Thus, geometry is not treated as primary. Instead, position and orientation emerge through integration of curvature.

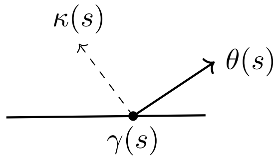

Abbildung 18.1.: Planar geometry emerges through intrinsic curvature-driven reconstruction. Within AIM, curvature is treated as the primary geometric quantity.

18.4. Initial Conditions

Definition 18.11 (Initial pose2 state)**.** A pose2 trajectory is uniquely determined by:

$$\gamma(0), \qquad \theta(0),$$

together with the curvature law

$$\kappa(s).$$

Remark 18.12. The curvature law acts as the geometric control function of planar state evolution.

18.5. Pose2 as Persistent Kernel

Proposition 18.13. *Within AIM, pose2 forms the persistent intrinsic reconstruction kernel of the alignment model.*

Remark 18.14. Pose2 is sufficient for persistent planar reconstruction of intrinsic alignment geometry.

Remark 18.15. Higher-level geometric and operational quantities are derived from pose2 rather than stored redundantly.

18.6. Connection to Sparse Alignment

Remark 18.16. Within the sparse alignment model, cGeom elements are interpreted as local curvature operators contributing to reconstruction of the global pose2 trajectory.

Remark 18.17. Fixed and transition elements therefore do not primarily store geometry itself, but rules governing geometric evolution.

Remark 18.18. The sparse alignment model can thus be interpreted as a compact integration program for reconstructing pose2 trajectories.

18.7. Intrinsic Interpretation

Remark 18.19. pose2 separates:

- intrinsic geometric behaviour,
- from derived geometric quantities.

Remark 18.20. Curvature acts as intrinsic control quantity, whereas position and orientation emerge through reconstruction.

Remark 18.21. This separation forms one of the central structural principles of the AIM framework.

18.8. Relation to pose3

Remark 18.22. Within AIM, pose2 forms the persistent intrinsic reconstruction kernel, whereas pose3 typically represents an extended runtime-evaluated spatial state.

Remark 18.23. pose3 is derived dynamically from:

- pose2 reconstruction,
- cant evaluation,
- profile evaluation,
- frame construction,
- and runtime coordinate services.

Remark 18.24. This avoids redundancy and synchronization instability between planar, cant, profile, and runtime representations.

18.9. Canonical Interpretation

Proposition 18.25. *Within AIM, intrinsic railway geometry is fundamentally curvature-driven.*

Proposition 18.26. *pose2 represents the canonical persistent intrinsic geometry state from which higher-level runtime states are derived.*

18.10. Connection to AIM

Summary 18.27. pose2 is the fundamental intrinsic planar state representation of railway alignment.

Within AIM, pose2 forms the persistent reconstruction kernel from which planar geometry is generated through curvature-driven integration.

Higher-dimensional runtime states such as pose3 are derived dynamically from this kernel together with cant, profile, frame, and runtime evaluation services.

19. Pose3, Cant, and Spatial Railway State

19.1. Motivation

Remark 19.1. Planar railway alignment models describe intrinsic geometry using the pose2 state introduced in theorem 18.5.

Remark 19.2. Operational railway systems are spatial systems involving:

- profile geometry,
- cant,
- local frame orientation,
- vehicle-space interpretation,
- and dynamically relevant quantities.

Remark 19.3. This motivates an extended spatial railway state referred to as *pose3*.

Principle 19.4 (Runtime pose3 principle). Within AIM, pose3 is interpreted primarily as a runtime-evaluated spatial railway state rather than as a persistent geometric storage object.

Principle 19.5 (pose3 storage separation). pose3 is not persisted as independent geometry.

The persistent model stores only the quantities from which pose3 can be reconstructed:

- intrinsic pose2 geometry,
- profile information,
- cant information,
- gauge and convention data,
- and metric/runtime context.

19.2. Relation to pose2

Remark 19.6. The canonical pose2 system is defined in eq. (6.6).

Remark 19.7. Within AIM, pose2 forms the persistent intrinsic reconstruction kernel, whereas pose3 extends this state by runtime-evaluated spatial and operational interpretation.

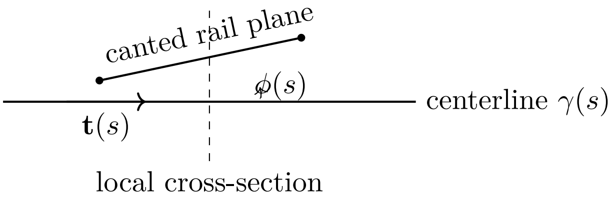

Abbildung 19.1.: pose3 extends planar intrinsic geometry by profile and cant-dependent spatial interpretation.

Remark 19.8. pose3 depends on:

- pose2 reconstruction,
- profile evaluation,
- cant evaluation,
- frame construction,
- runtime metric services,
- and runtime coordinate services.

Remark 19.9. Conceptually:

- pose2 answers: "where is the intrinsic alignment and how does it turn?"
- profile answers: "how does the alignment rise?"
- cant answers: "how is the rail plane rotated?"
- pose3 answers: "what operational spatial railway state results at runtime?"

19.3. Cant Representation

19.3.1. Cant as Persistent Engineering Quantity

Definition 19.10 (Cant height difference)**.** Within AIM, persistent cant is represented as rail height difference:

$$u(s).$$

Remark 19.11. This follows standard railway engineering practice, where cant is typically specified as cross-level difference between rails.

19.3.2. Derived Roll Angle

Definition 19.12 (Derived roll angle)**.** Given a gauge value g_{gauge}, the corresponding roll angle is:

$$\phi(s) = \arctan\left(\frac{u(s)}{g_{\text{gauge}}}\right). \tag{19.1}$$

Remark 19.13. The roll angle is a derived runtime quantity rather than a persistent primary variable.

Remark 19.14. Within AIM, cant is interpreted geometrically as frame rotation, independent of its engineering realization as rail height difference.

19.3.3. Cant Evolution

Remark 19.15. Cant evolution is governed by the canonical relation eq. (6.7).

Remark 19.16. Cant-rate influences:

- dynamic comfort,
- load transfer,
- vehicle response,
- and operational smoothness.

19.4. Cant Reference Conventions

Definition 19.17 (Cant reference convention)**.** A cant convention specifies:

- the reference axis of rotation,
- the sign convention of cant,
- the relation between rail height difference and roll angle,
- and the induced centerline displacement.

Remark 19.18. Railway source data may define cant relative to:

- the centerline,
- one rail,
- a construction axis,
- or another engineering reference.

Remark 19.19. Within AIM, the intrinsic alignment centerline acts as canonical reference geometry.

Remark 19.20. Rail-based cant definitions must therefore be transformed consistently during import, normalization, or runtime evaluation.

19.5. Pose3 Frames

Definition 19.21 (Uncanted geometric frame)**.** The uncanted geometric frame is denoted by:

$$F_0(s).$$

Remark 19.22. $F_0(s)$ is constructed from the reconstructed spatial alignment without cant rotation.

Definition 19.23 (Railway frame)**.** The railway frame is denoted by:

$$F_{\mathrm{rail}}(s).$$

Remark 19.24. $F_{\mathrm{rail}}(s)$ is obtained by cant rotation of the geometric frame $F_0(s)$.

Definition 19.25 (Vehicle frame)**.** A vehicle-dependent runtime frame may additionally be defined as:

$$F_{\mathrm{veh}}(s).$$

Remark 19.26. Vehicle frames may depend on:

- suspension behaviour,
- center-of-gravity location,
- dynamic response,
- and operational conditions.

19.6. Runtime pose3 State

Definition 19.27 (Runtime pose3)**.** The runtime spatial railway state is the station-dependent evaluated state:

$$\mathrm{pose3}(s) = (\Gamma(s), F_{\mathrm{rail}}(s), \phi(s)). \tag{19.2}$$

Remark 19.28. Here:

- $\Gamma(s)$ denotes the reconstructed spatial reference trajectory,
- $F_{\mathrm{rail}}(s)$ denotes the cant-aware railway frame,
- $\phi(s)$ denotes the derived cant rotation.

Remark 19.29. The railway frame is evaluated by the frame operator $\mathcal{F}$ introduced in theorem 8.41.

Remark 19.30. Within projected engineering geometry, the spatial trajectory $\Gamma(s)$ is assembled dynamically from:

- planar pose2 reconstruction,
- profile evaluation,
- metric interpretation,
- and runtime coordinate services.

19.7. Persistent Model versus Runtime State

Remark 19.31. The persistent sparse model stores:

- intrinsic curvature geometry,
- cant bands,
- profile bands,
- and sparse reconstruction parameters.

Remark 19.32. Spatial pose3 states are reconstructed dynamically by runtime services.

Remark 19.33. This avoids:

- redundant spatial storage,
- synchronization instability,
- and unnecessary duplication of derived geometry.

Remark 19.34. The runtime interpretation may vary depending on:

- metric realization,
- CRS interpretation,
- level of detail,
- vehicle-space interpretation,
- and runtime evaluation strategy.

19.8. Reconstruction Principle

Proposition 19.35 (pose3 reconstruction principle)**.** *Given:*

$$\kappa(s), \qquad u(s), \qquad p(s),$$

together with initial pose2 data, gauge information, convention data, and runtime metric context, the runtime pose3 state is determined through evaluation.

Remark 19.36. Thus, pose3 is reconstructed from synchronized runtime evaluation of:

- intrinsic geometry,
- profile,
- cant,
- frame services,
- metric services,
- and coordinate services.

19.9. Dynamic Quantities

19.9.1. Effective Lateral Acceleration

Remark 19.37. The effective lateral acceleration is defined canonically in eq. (6.10).

Remark 19.38. This quantity expresses the operational coupling between curvature and cant.

19.9.2. Jerk

Remark 19.39. The jerk relation is defined in eq. (6.12).

Remark 19.40. Jerk depends jointly on curvature evolution and cant evolution.

Remark 19.41. This is one of the primary reasons why curvature and cant cannot be treated independently in high-quality railway alignment design.

19.10. Cant and Curvature Coupling

Remark 19.42. Curvature and cant are operationally coupled quantities.

Remark 19.43. The equilibrium relation is defined in eq. (6.11).

Remark 19.44. Within AIM, curvature and cant remain persistently separated as distinct bands, whereas their operational interaction is evaluated dynamically.

19.11. Vehicle-Space Interpretation

Remark 19.45. pose3 enables interpretation of:

- vehicle envelopes,
- center-of-gravity motion,
- bogie orientation,

- wheelset behaviour,
- and platform interaction.

Remark 19.46. The physically relevant object is therefore not merely the track centerline, but the operational vehicle-space state generated from the alignment.

Remark 19.47. Vehicle-space interpretation may depend on runtime-generated vehicle frames $F_{\text{veh}}(s)$.

19.12. Towards Schwerpunkt-Trassierung

Remark 19.48. The pose3 framework allows reinterpretation of railway alignment as design of a spatial operational state trajectory.

Remark 19.49. Schwerpunkt-Trassierung does not optimize the rail centerline alone.

Instead, it optimizes vehicle-relevant operational trajectories induced by:

- curvature evolution,
- cant evolution,
- profile evolution,
- frame interpretation,
- and vehicle dynamics.

Remark 19.50. In this interpretation, the railway alignment acts as a generator of runtime vehicle-space states.

OPEN: Vehicle-specific runtime envelopes based on center-of-gravity-dependent cant interpretation.

RESEARCH: Formal definition of Schwerpunkt-based railway alignment optimization.

19.13. Relation to Standards

Remark 19.51. Existing railway standards typically prescribe limits on:

- curvature,
- cant,
- cant-rate,
- jerk,
- and operational acceleration.

Remark 19.52. Within AIM, such rules are interpreted as constraints acting on runtime pose3 states and their derivatives.

19.14. Canonical Interpretation

Proposition 19.53. *Within AIM, pose3 is interpreted as a runtime-evaluated operational spatial railway state.*

Proposition 19.54. *Persistent alignment storage and runtime spatial interpretation are treated as distinct conceptual layers.*

Proposition 19.55. *Curvature, profile, cant, frame evaluation, metric interpretation, and runtime services jointly generate operational railway geometry.*

Proposition 19.56. *The physically relevant railway object is not merely the geometric centerline, but the induced runtime vehicle-space state.*

19.15. Connection to AIM

Summary 19.57. pose3 extends intrinsic planar geometry into operational spatial railway state evaluation.

Within AIM, pose3 is reconstructed dynamically from:

- pose2 geometry,
- cant bands,
- profile bands,
- frame evaluation,
- runtime services,
- metric interpretation,
- and coordinate context.

This provides:

- realization-aware spatial interpretation,
- dynamic operational geometry,
- runtime vehicle-space evaluation,
- and a foundation for advanced alignment optimization concepts.

Within AIM, pose3 is therefore not interpreted as a persistent geometry object, but as a dynamically evaluated operational railway state.

20. Pose3 Frames and Spatial Reference Systems

20.1. Motivation

Remark 20.1. The pose3 state describes railway geometry together with orientation, cant, and spatial interpretation.

Remark 20.2. The state variables alone do not fully determine how the alignment is interpreted in space. For this, local reference frames are required.

Remark 20.3. Within AIM, frames form the operational bridge between:

- intrinsic geometry,
- cant,
- dynamics,
- realization,
- world–track projection,
- and vehicle-space interpretation.

20.2. Pose3 and Frames

Remark 20.4. The canonical pose3 state is defined in eq. (6.1).

Remark 20.5. A pose3 frame provides the local spatial interpretation of this state. It is derived from the alignment and evaluated along arc-length.

20.3. Tangent Direction

Definition 20.6 (Tangent vector)**.** The tangent vector is:

$$\mathbf{t}(s) = \gamma'(s).$$

Remark 20.7. For arc-length parameterization,

$$\|\mathbf{t}(s)\| = 1.$$

Remark 20.8. The tangent direction defines the local forward direction of motion.

20.4. Lateral and Binormal Directions

Definition 20.9 (Lateral direction)**.** The lateral direction is denoted by:

$$\mathbf{n}(s),$$

with

$$\mathbf{n}(s) \cdot \mathbf{t}(s) = 0.$$

Definition 20.10 (Binormal direction)**.** The binormal direction is:

$$\mathbf{b}(s) = \mathbf{t}(s) \times \mathbf{n}(s).$$

Remark 20.11. The lateral and binormal directions define the local cross-section orientation of the railway frame.

20.5. Pose3 Frame

Definition 20.12 (Pose3 frame)**.** The local pose3 frame is:

$$F(s) = (\mathbf{t}(s), \mathbf{n}(s), \mathbf{b}(s)).$$

Remark 20.13. The pose3 frame defines the local operational coordinate system of the railway alignment.

Remark 20.14. It is a runtime-evaluated structure rather than an independently stored geometric object.

20.6. Cant Rotation

Definition 20.15 (Cant rotation)**.** Cant is interpreted as a rotation around the tangent direction:

$$R_\phi(s).$$

Definition 20.16 (Cant-rotated frame)**.** The cant-rotated frame is:

$$F_\phi(s) = (\mathbf{t}(s), \mathbf{n}_\phi(s), \mathbf{b}_\phi(s)).$$

Remark 20.17. The cant-rotated frame determines:

- rail elevations,
- cross-level interpretation,

- vehicle orientation,
- effective gravity direction,
- and platform geometry.

20.7. Rail Plane

Remark 20.18. Gauge, rail elevation, and cross-level geometry are defined relative to the cant-rotated frame.

Remark 20.19. The rail plane is therefore not globally horizontal, but attached to the local pose3 frame and its cant rotation.

20.8. Frenet and Railway Frames

Remark 20.20. Classical Frenet frames are not fully adequate for railway alignment modelling.

Remark 20.21. In particular, Frenet frames become unstable in regions where

$$\kappa(s) \to 0.$$

Remark 20.22. Railway modelling therefore requires frame formulations that remain stable in straight and low-curvature sections.

Remark 20.23. In practice, railway frames are often transported continuously along the alignment rather than recomputed purely from differential geometry.

20.9. Surface and Ellipsoid Frames

Remark 20.24. When alignments are defined on the Earth surface, local frames must respect ellipsoid geometry.

Definition 20.25 (Surface-attached frame)**.** A surface-attached frame consists of:

$$(\mathbf{t}, \mathbf{n}, \mathbf{n}_{\mathrm{E}}),$$

where $\mathbf{n}_{\mathrm{E}}$ denotes the ellipsoid surface normal.

Remark 20.26. In this setting:

- curvature splits into geodesic and normal curvature,
- gravity direction depends on Earth geometry,
- and world-to-track projection becomes surface-dependent.

20.10. Vehicle-Space Interpretation

Remark 20.27. The physically relevant object is not always the track centerline alone, but the motion of the railway vehicle within space.

Remark 20.28. Pose3 frames provide the spatial reference basis for:

- center-of-gravity motion,
- vehicle envelopes,
- bogie rotation,
- wheelset orientation,
- and platform clearance behaviour.

Remark 20.29. Thus, vehicle-space interpretation is derived from alignment states and their associated frames.

20.11. Connection to World–Track Transformation

Remark 20.30. World–track projection relies directly on the local frame.

Remark 20.31. In particular:

- tangent directions define longitudinal coordinates,
- lateral directions define offsets,
- and cant modifies the operational spatial interpretation.

Remark 20.32. This connects pose3 frames to the canonical world–track relation in eq. (6.14).

20.12. Frame Continuity

Remark 20.33. Frame continuity is essential for:

- smooth visualization,
- stable dynamics,
- cant interpretation,
- vehicle-space evaluation,
- and numerical robustness.

Remark 20.34. Discontinuous frame flips may lead to incorrect cant orientation, unstable vehicle interpretation, and invalid projection behaviour.

20.13. Key Insight

Proposition 20.35. *Pose3 frames are operational runtime structures rather than purely geometric differential objects.*

Remark 20.36. They connect intrinsic railway geometry with realization, dynamics, vehicle behaviour, and spatial interpretation.

20.14. Connection to AIM

Summary 20.37. Pose3 frames provide the spatial reference structure underlying railway alignment modelling.

Within AIM, they form the operational bridge between:

- curvature-based geometry,
- cant,
- realization,
- world–track projection,
- vehicle-space interpretation,
- and runtime system behaviour.

21. Dynamics and Railway State Interpretation

21.1. Motivation

Remark 21.1. Geometric alignment alone is not sufficient to describe railway behaviour. The motion of a vehicle depends on both curvature and cant, as well as on operational parameters such as speed.

Remark 21.2. This motivates a dynamic interpretation of alignment in which geometric quantities are translated into physical effects. Such a perspective is consistent with both railway design standards and modern railway vehicle dynamics literature, where curvature, cant, and vehicle response are treated as strongly coupled aspects of the system [6, 4, 16, 49].

21.2. Kinematic Basis

Definition 21.3 (Velocity)**.** Let $v > 0$ denote the velocity of the vehicle along the alignment.

Definition 21.4 (Curvature-induced acceleration)**.** For a planar curve with curvature $\kappa(s)$, the lateral acceleration is

$$a_\kappa(s) = v^2\kappa(s).$$

Remark 21.5. This is the classical centripetal acceleration associated with curved motion and forms the basic link between geometry and dynamics.

21.3. Cant and Effective Acceleration

Definition 21.6 (Cant angle)**.** Let $\phi(s)$ denote the local roll angle induced by cant.

Definition 21.7 (Gravity component)**.** The gravitational acceleration projected onto the lateral direction is

$$a_g(s) = g\sin\phi(s),$$

where g denotes gravitational acceleration.

Definition 21.8 (Effective lateral acceleration)**.** The effective lateral acceleration acting on the vehicle is

$$a_{\text{eff}}(s) = v^2\kappa(s) - g\sin\phi(s).$$

Remark 21.9. This expression captures the fundamental coupling between curvature and cant in railway engineering, where cant is introduced precisely to compensate part of the curvature-induced lateral acceleration [24, 6].

Remark 21.10. From a vehicle-dynamics perspective, this coupling is one of the central mechanisms governing running behaviour in curves [16, 49].

21.4. Equilibrium Condition

Definition 21.11 (Equilibrium cant)**.** A state of equilibrium is achieved when

$$v^2\kappa(s) = g\sin\phi(s).$$

Remark 21.12. In this case, the effective lateral acceleration vanishes:

$$a_{\mathrm{eff}}(s) = 0.$$

Remark 21.13. This condition corresponds to the classical notion of balanced track, where centrifugal and gravitational effects cancel. The same general idea underlies standard cant and cant-deficiency formulations in railway design [6, 24].

21.5. Cant Deficiency and Excess

Definition 21.14 (Cant deficiency)**.** Cant deficiency is defined as

$$\Delta(s) = v^2\kappa(s) - g\sin\phi(s).$$

Remark 21.15. Positive values correspond to insufficient cant, negative values to excess cant.

Remark 21.16. Cant deficiency is a central design parameter in railway engineering, as it directly influences passenger comfort and safety [6, 4].

Remark 21.17. In more detailed vehicle-based interpretations, cant deficiency is also closely related to guiding forces, wheel unloading, and curving behaviour [16, 32].

21.6. Dynamic Smoothness

Definition 21.18 (Jerk)**.** The rate of change of effective acceleration is given by

$$j(s) = \frac{\mathrm{d}}{\mathrm{d}t}a_{\mathrm{eff}}(s).$$

Remark 21.19. Using $\mathrm{d}t = \frac{1}{v}\mathrm{d}s$, one obtains

$$j(s) = v\frac{\mathrm{d}}{\mathrm{d}s}a_{\mathrm{eff}}(s).$$

Proposition 21.20. *The jerk is given by*

$$j(s) = v^3\kappa'(s) - gv\cos\phi(s)\,\phi'(s).$$

Remark 21.21. This shows that both curvature variation and cant variation contribute to dynamic effects. Such higher-order quantities are directly relevant for comfort-oriented transition design [34, 27, 28].

Remark 21.22. In railway vehicle dynamics, smoothness criteria of this kind are important because abrupt changes in curvature or cant excite the vehicle–track system beyond what is visible from geometry alone [25, 16].

21.7. Time Parameterization

Remark 21.23. The alignment is parameterized by arc length s, while dynamic quantities evolve in time t.

Definition 21.24 (Arc-length/time relation)**.** Assuming constant speed $v > 0$, we have

$$\frac{\mathrm{d}s}{\mathrm{d}t} = v.$$

Proposition 21.25. *For any function $f(s)$, the time derivative is given by*

$$\frac{\mathrm{d}f}{\mathrm{d}t} = v\frac{\mathrm{d}f}{\mathrm{d}s}.$$

Remark 21.26. This relation allows all spatial derivatives to be translated into temporal dynamics and provides the mathematical bridge between arc-length-based alignment modelling and time-based vehicle response.

21.8. Dynamic System Formulation

Definition 21.27 (State variables)**.** The railway state is described by

$$x(s), y(s), \theta(s), \phi(s).$$

Proposition 21.28 (Kinematic system)**.** *The system satisfies*

$$\frac{\mathrm{d}x}{\mathrm{d}s} = \cos\theta(s), \qquad \frac{\mathrm{d}y}{\mathrm{d}s} = \sin\theta(s),$$

$$\frac{\mathrm{d}\theta}{\mathrm{d}s} = \kappa(s).$$

Remark 21.29. These equations correspond to the geometric reconstruction of the curve.

Definition 21.30 (Extended dynamic state)**.** The extended state includes cant:

$$(\gamma(s), \theta(s), \phi(s)).$$

Remark 21.31. The functions $\kappa(s)$ and $\phi(s)$ act as control variables.

Remark 21.32. This formulation is compatible with reduced-order vehicle–track interpretations in which track geometry provides structured inputs to a dynamic system rather than merely static shape data [16, 43].

21.9. Time-Based Dynamics

Proposition 21.33. *In time parameterization, the heading angle satisfies*

$$\frac{\mathrm{d}\theta}{\mathrm{d}t} = v\kappa(s).$$

Proposition 21.34. *The lateral acceleration is given by*

$$a_\kappa(t) = v^2\kappa(s).$$

Remark 21.35. This formulation makes explicit that a purely geometric quantity, namely curvature, acquires direct physical meaning once speed is introduced.

21.10. Higher-Order Dynamics

Definition 21.36 (Jerk)**.** The jerk is defined as

$$j(t) = \frac{\mathrm{d}}{\mathrm{d}t} a_{\mathrm{eff}}(t).$$

Proposition 21.37. *The jerk is given by*

$$j(t) = v^3\kappa'(s) - gv\cos\phi(s)\,\phi'(s).$$

Beweis. Using

$$a_{\mathrm{eff}} = v^2\kappa(s) - g\sin\phi(s),$$

we obtain

$$\frac{\mathrm{d}}{\mathrm{d}t} a_{\mathrm{eff}} = v^2\frac{\mathrm{d}\kappa}{\mathrm{d}t} - g\cos\phi(s)\frac{\mathrm{d}\phi}{\mathrm{d}t}.$$

With

$$\frac{\mathrm{d}}{\mathrm{d}t} = v\frac{\mathrm{d}}{\mathrm{d}s},$$

this yields

$$j(t) = v^3\kappa'(s) - gv\cos\phi(s)\,\phi'(s).$$

□

Remark 21.38. This explicit expression shows why transition design cannot be reduced to curvature alone if cant development is part of the system model.

21.11. Control-Theoretic Interpretation

Definition 21.39 (Control system)**.** The railway alignment can be interpreted as a control system with

$$\text{state: } (\gamma, \theta, \phi), \qquad \text{controls: } (\kappa, \phi').$$

Remark 21.40. The curvature $\kappa(s)$ controls the geometric evolution, while $\phi(s)$ controls the dynamic balancing of forces.

Remark 21.41. This formulation establishes a direct connection between alignment design and optimal control theory, and is consistent with the broader optimization-oriented interpretation of railway alignment proposed by Kufver [31, 30].

21.12. Coupled Dynamics

Remark 21.42. The effective acceleration depends on both curvature and cant:

$$a_{\text{eff}} = v^2\kappa - g\sin\phi.$$

Remark 21.43. Thus, curvature and cant must be treated as a coupled system rather than independent design variables.

Proposition 21.44. *Perfect equilibrium requires*

$$v^2\kappa(s) = g\sin\phi(s).$$

Remark 21.45. This coupling is also central from the vehicle–track interaction point of view, since the dynamic response depends on the joint evolution of track geometry, cant, and operating speed [25, 32].

21.13. Coupled State Interpretation

Remark 21.46. The expressions above show that railway behaviour depends on the coupled state variables

$$(\kappa(s), \phi(s)).$$

Remark 21.47. A purely geometric interpretation based on $\kappa(s)$ alone is therefore insufficient for describing the physical behaviour of the system. This is one of the core reasons why a state-based railway model is preferable to a purely planar alignment description [4, 6, 16].

21.14. Extension to Pose3

Remark 21.48. Within the pose3 framework, the railway state is described by

$$(x(s), y(s), z(s), \theta(s), \phi(s)).$$

Remark 21.49. This representation naturally integrates:

- geometry (position and direction),
- vertical profile,
- cant (through ϕ),
- and dynamic interpretation.

Remark 21.50. The extension from planar pose to spatial railway state is therefore not a cosmetic modification, but a structural requirement for coupling alignment description to engineering dynamics.

21.15. Relation to Optimization

Remark 21.51. The dynamic quantities introduced above provide natural objective terms for alignment optimization.

Remark 21.52. Typical optimization criteria include:

- minimizing $\int a_{\text{eff}}(s)^2 \, \mathrm{d}s$,
- minimizing $\int j(s)^2 \, \mathrm{d}s$,
- limiting peak values of cant deficiency.

Such objective terms arise naturally once alignment is interpreted as a coupled geometric–dynamic system rather than as static geometry alone [30, 16].

21.16. Engineering Interpretation

Remark 21.53. In practical railway design, acceptable ranges for cant, cant deficiency, and jerk are prescribed by standards and operational guidelines.

Remark 21.54. The mathematical formulation above provides a unified framework for expressing these rules in analytic form [6, 24, 48].

Remark 21.55. At a more detailed level, such limits are closely related to the running-safety and vehicle–track compatibility questions discussed in railway dynamics literature [48, 16, 49].

21.17. Connection to Schwerpunkt-Trassierung

Remark 21.56. The combined dependence on curvature and cant supports a Schwerpunkt-oriented view of alignment.

Remark 21.57. Rather than describing only a geometric path, alignment is interpreted as a controlled dynamic state along the track.

Remark 21.58. This opens the way toward formulations in which vehicle-relevant derived motions, rather than geometry alone, become the true design object.

21.18. Summary

Summary 21.59. Railway alignment must be interpreted dynamically rather than purely geometrically.

The coupled variables $\kappa(s)$ and $\phi(s)$ determine lateral acceleration, comfort, and operational feasibility.

This provides the foundation for extending alignment modelling toward state-based and optimization-based frameworks.

22. World–Track Coordinate Transformation

22.1. Motivation

Remark 22.1. Railway alignments are formulated intrinsically using arc-length-based coordinates.

Remark 22.2. Real-world data is typically represented in external coordinate reference systems such as:

- WGS84,
- UTM,
- Gauss–Krüger,
- DBRef,
- or local engineering CRS definitions.

Remark 22.3. A transformation between intrinsic alignment coordinates and external world coordinates is therefore required.

Remark 22.4. Within AIM, this transformation is interpreted as a runtime evaluation service connecting:

- intrinsic alignment states,
- measured data,
- visualization,
- realization,
- topology,
- and operational geometry.

Principle 22.5 (Intrinsic-space primacy). Within AIM, intrinsic railway geometry is primary.

World-space coordinates are interpreted as external embeddings generated through runtime evaluation.

22.2. Track Coordinate System

Definition 22.6 (Track coordinate system)**.** The intrinsic track coordinate system consists of:

$$(s, d, h),$$

where:

- s denotes intrinsic arc-length,
- d denotes lateral offset,
- h denotes vertical offset.

Remark 22.7. Track coordinates belong to the intrinsic track-space domain Ω_{track}, introduced in **??**.

Remark 22.8. In planar situations, only:

$$(s, d)$$

is required.

Remark 22.9. Track coordinates are intrinsic alignment coordinates and must not be confused with projected world coordinates.

22.2.1. Intrinsic Geometry versus External Coordinates

Remark 22.10. Intrinsic geometry is independent of any external coordinate reference system.

Remark 22.11. The same intrinsic alignment may be embedded into:

- projected engineering planes,
- ellipsoidal world models,
- local realization frames,
- or visualization coordinate systems.

Remark 22.12. Thus, intrinsic alignment geometry and external CRS representation form distinct conceptual layers.

22.2.2. Arc-Length and Stationing

Remark 22.13. Intrinsic arc-length is not identical to engineering stationing.

Remark 22.14. Operational kilometre systems may contain:

- station equations,

- kilometre jumps,
- overlapping station ranges,
- topology-dependent references,
- and operational branch relations.

Remark 22.15. Stationing systems are operational reference structures layered on top of intrinsic geometry.

Remark 22.16. Consequently, the mapping:

$$s \leftrightarrow \mathrm{km}$$

is generally neither linear nor globally unique.

22.3. Track-to-World Transformation

Definition 22.17 (Track-to-world mapping)**.** The forward transformation is:

$$x(s,d) = \gamma(s) + d\,\mathbf{n}(s). \tag{22.1}$$

Remark 22.18. This relation specializes the canonical track-to-world operator $\mathcal{P}_{\mathrm{tw}}$ introduced in theorem 8.22.

22.3.1. Planar Tangent and Normal Directions

Remark 22.19. In the planar case:

$$\mathbf{t}(s) = (\cos\theta(s), \sin\theta(s)),$$

and:

$$\mathbf{n}(s) = (-\sin\theta(s), \cos\theta(s)).$$

22.3.2. Spatial Runtime Evaluation

Remark 22.20. In spatial runtime evaluation, the mapping depends additionally on:

- profile,
- cant,
- frame construction,
- metric interpretation,
- and runtime realization context.

Remark 22.21. Consequently, the world embedding:

$$x(s, d, h)$$

is generally generated dynamically from runtime pose3 evaluation.

22.3.3. Reference-Line Interpretation

Remark 22.22. The lateral coordinate d depends on the chosen reference interpretation.

Remark 22.23. Possible reference structures include:

- alignment centerline,
- track center,
- rail axis,
- gauge reference,
- or operational offset definitions.

OPEN: Formalize reference-line conventions for:

- multi-track systems,
- cant-dependent offsets,
- and vehicle-space interpretation.

22.4. World-to-Track Transformation

Definition 22.24 (World-to-track projection)**.** The inverse transformation is:

$$(s^*, d^*) = \arg\min_{s,d} \|x - (\gamma(s) + d\,\mathbf{n}(s))\|. \tag{22.2}$$

Remark 22.25. This relation specializes the canonical world-to-track operator $\mathcal{P}_{\mathrm{wt}}$ introduced in theorem 8.24.

Remark 22.26. The term projection is used operationally and does not necessarily imply a unique orthogonal projection in the classical geometric sense.

Remark 22.27. Within AIM, world-to-track projection is interpreted as an inverse runtime evaluation problem.

22.5. Coordinate Transformation versus Projection

Remark 22.28. Coordinate transformation and world-to-track projection are distinct operations within AIM.

Remark 22.29. Coordinate transformations map between world-space representations:

$$\mathcal{C} : \Omega_{\text{world}} \to \Omega_{\text{world}},$$

as introduced in theorem 8.27.

Remark 22.30. World-to-track projection maps between:

$$\Omega_{\text{world}} \quad \text{and} \quad \Omega_{\text{track}}.$$

Principle 22.31 (Projection distinction principle). Coordinate transformation must not be confused with intrinsic world-to-track projection.

Remark 22.32. CRS transformations operate entirely within external coordinate representations, whereas projection accesses intrinsic railway geometry.

22.6. Numerical Evaluation

Remark 22.33. The inverse transformation is generally nonlinear.

Remark 22.34. Typical numerical approaches include:

- nearest-point search,
- local Newton iteration,
- segment-wise projection,
- topology-aware lookup,
- and hybrid runtime strategies.

Remark 22.35. Projection quality depends strongly on:

- curvature behaviour,
- topology,
- initialization,
- runtime context,
- and metric interpretation.

22.7. Ambiguity and Context Dependence

Remark 22.36. World-to-track mapping is generally not globally unique.

Remark 22.37. Ambiguities may occur in:

- loops,
- switches and crossings,
- parallel tracks,
- closely spaced alignments,
- station areas,
- and topology-rich railway networks.

Remark 22.38. Multiple intrinsic states may correspond to nearly identical world-space locations.

Remark 22.39. Consequently, projection is fundamentally context dependent.

Remark 22.40. Runtime projection may require:

- track identity,
- route identity,
- topology context,
- expected station range,
- operational direction,
- runtime interaction history,
- vehicle context,
- or network-state constraints.

Principle 22.41 (Projection ambiguity principle). World-to-track projection cannot generally be interpreted as a purely local geometric operation.

22.8. Topology and Operational References

Remark 22.42. Intrinsic geometry alone is insufficient for operational railway referencing.

Remark 22.43. Operational interpretation additionally depends on:

- topology,
- route membership,

- kilometre references,
- operational direction,
- and network semantics.

Remark 22.44. Thus, a complete projection context may require:

$$(x, \mathcal{T}, \mathcal{R}, \mathcal{O}),$$

where:

- x denotes world-space input,
- $\mathcal{T}$ denotes topology context,
- $\mathcal{R}$ denotes route/reference context,
- $\mathcal{O}$ denotes operational context.

22.9. Relation to Curvature

Remark 22.45. Projection stability depends directly on curvature behaviour.

Remark 22.46. High curvature increases projection sensitivity, whereas low curvature typically yields stable projection behaviour.

Remark 22.47. Errors in intrinsic coordinates propagate nonlinearly into world space.

Remark 22.48. Projection behaviour near transition regions may depend strongly on higher-order curvature behaviour.

22.10. Runtime Dependency

Remark 22.49. Within AIM, world–track transformation depends on runtime evaluation of the underlying alignment representation.

Remark 22.50. In particular:

- cGeom defines intrinsic geometry,
- pose2 defines tangent evolution,
- pose3 defines runtime spatial interpretation,
- metric services define realized embedding,
- CRS services define external representation,
- and runtime operators evaluate resulting states.

Remark 22.51. Projection therefore depends on the currently active runtime evaluation pipeline.

Remark 22.52. Different runtime configurations may yield different projection results, for example:

- projected versus ellipsoidal evaluation,
- target versus realized geometry,
- low-resolution versus high-resolution runtime,
- or simplified versus vehicle-aware interpretation.

Principle 22.53 (Runtime projection principle). World–track transformation is a runtime evaluation problem operating on alignment states rather than a static coordinate utility.

22.11. LOD and Hybrid Evaluation

Remark 22.54. Exact world-to-track projection is computationally expensive.

Remark 22.55. Large-scale systems therefore require:

- LOD-dependent evaluation,
- approximate projection,
- topology-aware filtering,
- and hybrid runtime architectures.

Proposition 22.56 (Selective projection principle). *Exact projection should only be evaluated where high precision is required.*

Remark 22.57. A hybrid strategy combines:

- precomputed samples for coarse lookup,
- topology-aware candidate filtering,
- cached runtime states,
- and local parametric refinement for precision.

Remark 22.58. Hybrid projection architectures are particularly important for:

- large railway networks,
- interactive visualization,
- realtime vehicle-space evaluation,
- and multi-resolution runtime systems.

22.12. Connection to CRS and Realization

Remark 22.59. The complete external transformation chain is:

$$\mathrm{CRS} \to \Omega_{\mathrm{world}} \to \Omega_{\mathrm{track}}.$$

Remark 22.60. For high-precision applications, the alignment itself may be defined on a reference ellipsoid rather than in a projected plane.

Remark 22.61. Projection may also depend on realized geometry:

$$\gamma_{\mathrm{real}}(s) \neq \gamma_{\mathrm{target}}(s).$$

Remark 22.62. Physical measurements should therefore reference realized rather than purely analytical geometry.

Remark 22.63. The distinction between:

$$\text{intrinsic geometry} \quad \text{and} \quad \text{external embedding}$$

is fundamental within AIM.

22.13. Key Insight

Proposition 22.64. *World–track transformation is an inverse runtime service rather than a static coordinate conversion.*

Proposition 22.65. *Within AIM, intrinsic alignment space is primary, whereas world coordinates are interpreted as external embeddings of alignment states.*

Proposition 22.66. *Projection is generally:*

- *nonlinear,*
- *ambiguous,*
- *topology-dependent,*
- *runtime-dependent,*
- *and context-sensitive.*

22.14. Connection to AIM

Summary 22.67. World–track transformation connects intrinsic railway geometry with external coordinate systems, topology, runtime evaluation, and measured reality.

Its nonlinear, ambiguous, topology-dependent, and scale-dependent nature makes it a central runtime component of the AIM framework.

Within AIM, projection is therefore interpreted not as static coordinate conversion, but as contextual runtime reconstruction of intrinsic railway states from external observations. Efficient handling consequently requires:

- hybrid runtime architectures,
- topology-aware projection strategies,
- LOD-dependent evaluation,
- sparse-state reconstruction,
- and metric-aware runtime services.

Teil V.

Dynamic Railway State

Remark 22.68. ...

23. State Taxonomy

23.1. Motivation

Remark 23.1. Within AIM, the term *state* appears in multiple contexts:

- geometric reconstruction,
- runtime evaluation,
- realization,
- operational dynamics,
- and vehicle interpretation.

Remark 23.2. Without a consistent taxonomy, different state concepts may become ambiguous or unintentionally conflated.

Remark 23.3. This chapter therefore introduces a structural classification of state concepts within AIM.

23.2. General State Interpretation

Definition 23.4 (State)**.** A state is a structured description of a system configuration relative to a specified interpretation layer.

Principle 23.5 (State-layer principle). Within AIM, state concepts belong to distinct interpretation layers and must not be identified implicitly.

23.3. Intrinsic Geometric States

23.3.1. pose2

Definition 23.6 (pose2 state)**.** The pose2 state is the intrinsic planar reconstruction state generated from:

$$\kappa(s).$$

Remark 23.7. pose2 represents:

- intrinsic geometry,

- planar tangent evolution,
- and reconstruction state.

Remark 23.8. pose2 is fundamentally an intrinsic geometric state.

23.3.2. pose3

Definition 23.9 (pose3 state)**.** The pose3 state is the runtime-evaluated spatial railway state generated from:

- pose2 reconstruction,
- profile evaluation,
- cant interpretation,
- frame evaluation,
- and runtime coordinate interpretation.

Remark 23.10. pose3 describes the railway runtime state rather than the vehicle itself.

Remark 23.11. pose3 therefore belongs to the runtime railway-state layer.

23.4. Runtime States

Definition 23.12 (Runtime state)**.** A runtime state is a dynamically evaluated state generated during runtime interpretation.

Remark 23.13. Runtime states depend on:

- runtime operators,
- realization,
- coordinate interpretation,
- and operational context.

Remark 23.14. Examples include:

- pose3,
- runtime frames,
- projection states,
- and operational interpretation states.

23.5. Realized States

Definition 23.15 (Realized state)**.** A realized state is a runtime state generated from physically realized rather than purely analytical geometry.

Remark 23.16. Realized states depend on:

- construction,
- measurement,
- maintenance,
- realization filtering,
- and operational degradation.

Remark 23.17. Typical examples include:

$$\mathrm{pose3}_{\mathrm{real}}, \qquad \kappa_{\mathrm{real}}, \qquad \phi_{\mathrm{real}}.$$

23.6. Operational States

Definition 23.18 (Operational state)**.** An operational state is a runtime state relevant for railway operation, comfort, admissibility, or dynamic interpretation.

Remark 23.19. Operational states may include:

- effective acceleration,
- jerk,
- roll evolution,
- cant deficiency,
- and balance trajectories.

Remark 23.20. Operational states are derived from runtime railway states rather than stored directly as intrinsic geometry.

23.7. Vehicle States

Definition 23.21 (Vehicle state)**.** A vehicle state is a runtime-relative operational state describing the vehicle relative to the railway runtime state.

Remark 23.22. Vehicle states depend on:

- railway runtime state,

- operational velocity,
- vehicle interpretation,
- and dynamic response.

Remark 23.23. Thus:

$$\text{vehicleState} = \mathcal{V}(\text{pose3}, v, \dots).$$

Remark 23.24. Vehicle states are therefore not intrinsic railway states.

23.8. Center-of-Gravity States

Definition 23.25 (Center-of-gravity state)**.** A center-of-gravity state is an operational abstraction describing the runtime trajectory of the vehicle center of gravity relative to the railway runtime state.

Remark 23.26. Center-of-gravity states are primarily operational interpretation states rather than full mechanical vehicle models.

Remark 23.27. These states are particularly relevant for:

- comfort interpretation,
- balance interpretation,
- Schwerpunkt-Trassierung,
- and runtime operational optimization.

23.9. Dynamic States

Definition 23.28 (Dynamic state)**.** A dynamic state is a runtime state participating in operational state evolution.

Remark 23.29. Dynamic states evolve according to:

$$x'(s) = f(x, u).$$

Remark 23.30. Dynamic states may include:

- acceleration,
- jerk,
- roll evolution,
- and operational balance quantities.

23.10. Relative States

Definition 23.31 (Relative state)**.** A relative state is a state evaluated relative to another runtime state or runtime reference frame.

Remark 23.32. Within AIM, many operational quantities are fundamentally relative states.

Remark 23.33. Examples include:

- vehicle states relative to track states,
- wheelset states relative to local frames,
- and center-of-gravity trajectories relative to realized track geometry.

23.11. State Transformations

Remark 23.34. Different state classes are connected through runtime operators.

Remark 23.35. Examples include:

$$\mathcal{R}, \qquad \mathcal{D}, \qquad \mathcal{V}.$$

Remark 23.36. Typical transformations include:

- intrinsic reconstruction,
- realization filtering,
- runtime evaluation,
- vehicle interpretation,
- and operational state evolution.

23.12. State Hierarchy

Remark 23.37. The AIM state hierarchy may be interpreted schematically as:

$$\kappa(s) \to \mathrm{pose2} \to \mathrm{pose3} \to x(s) \to x_{\mathrm{vehicle}} \to x_{\mathrm{CG}}.$$

Remark 23.38. Realization operators act throughout this hierarchy:

$$\mathcal{R} : x \mapsto x_{\mathrm{real}}.$$

23.13. Canonical Interpretation

Proposition 23.39. *Within AIM, state concepts belong to distinct interpretation layers.*

Proposition 23.40. *pose3 describes the railway runtime state rather than the vehicle itself.*

Proposition 23.41. *Vehicle states are relative runtime interpretation states generated from interaction with the railway runtime state.*

Proposition 23.42. *Operational and dynamic states are generated through runtime evaluation rather than through static geometric storage.*

23.14. Connection to AIM

Summary 23.43. State taxonomy provides the structural interpretation framework for AIM runtime architecture.

Within this framework:

- pose2 represents intrinsic reconstruction state,
- pose3 represents railway runtime state,
- realized states represent physical railway realization,
- operational states describe runtime behaviour,
- vehicle states describe relative operational interpretation,
- and dynamic states describe runtime state evolution.

This taxonomy prevents conceptual ambiguity between:

- geometry,
- runtime,
- realization,
- dynamics,
- and vehicle interpretation.

24. Vehicle Space and Relative Railway States

24.1. Motivation

Remark 24.1. Classical railway alignment theory often treats the railway track itself as the primary geometric object.

Remark 24.2. Within operational railway systems, however, the physically relevant object is not merely the track geometry, but the evolving operational state of the vehicle relative to the track.

Remark 24.3. This distinction becomes essential for:

- comfort interpretation,
- cant interpretation,
- dynamic behaviour,
- operational admissibility,
- and realization-aware optimization.

Principle 24.4 (Track–vehicle distinction principle). Within AIM, pose3 does not describe the vehicle itself. Instead, pose3 describes the railway runtime state relative to which vehicle states emerge.

Principle 24.5 (Dynamic reference principle). The railway runtime state acts as a moving operational reference system relative to which vehicle states are interpreted.

Remark 24.6. The railway track therefore acts as a dynamic runtime reference system for vehicle-space interpretation.

24.2. Track State versus Vehicle State

Definition 24.7 (Track state)**.** The track state is the runtime-evaluated railway state generated from:

- intrinsic geometry,
- cant,

- profile,
- frame evaluation,
- and runtime reconstruction.

Remark 24.8. Within AIM, this state is represented primarily through:

$$\mathrm{pose3}(s).$$

Definition 24.9 (Vehicle state)**.** A vehicle state describes the operational and geometric state of a vehicle relative to the track state.

Remark 24.10. Vehicle states depend on:

- the track state,
- operational velocity,
- vehicle geometry,
- suspension behaviour,
- local wheel–rail interaction,
- and dynamic interpretation.

Remark 24.11. Thus:

$$\mathrm{vehicleState} \neq \mathrm{pose3}.$$

Remark 24.12. Instead:

$$\mathrm{vehicleState} = \mathcal{V}(\mathrm{pose3}, v, \dots).$$

Remark 24.13. Vehicle states are not intrinsic railway states. Instead, they are runtime-derived relative interpretation states generated from:

- railway runtime state,
- operational velocity,
- vehicle models,
- and dynamic interpretation layers.

24.3. Vehicle Space

Definition 24.14 (Vehicle space)**.** Vehicle space denotes the operational state space of vehicle behaviour relative to the railway runtime state.

Remark 24.15. Vehicle space includes:

- rigid-body interpretation,
- wheelset states,
- bogie states,
- relative suspension behaviour,
- center-of-gravity motion,
- and operational dynamic quantities.

Remark 24.16. Vehicle space is not merely a geometric embedding inside world space, but a runtime-relative operational interpretation space.

Remark 24.17. Vehicle space is generated dynamically from the interaction between:

- runtime railway state,
- realization,
- operational velocity,
- and vehicle interpretation.

24.4. Relative State Interpretation

Definition 24.18 (Relative vehicle state)**.** A relative vehicle state is a vehicle state evaluated with respect to a local railway reference frame.

Remark 24.19. Relative states may include:

- lateral displacement,
- roll angle,
- yaw deviation,
- wheel unloading,
- and center-of-gravity motion.

Remark 24.20. The physically relevant quantity is often not:

$$x_{\text{vehicle}},$$

but:

$$x_{\text{vehicle}} - x_{\text{track}}.$$

Remark 24.21. This motivates interpretation of railway dynamics as relative runtime state evolution inside moving track-attached frames.

24.5. Track Frame versus Vehicle Frame

Definition 24.22 (Track frame)**.** The track frame is the runtime-evaluated local reference frame attached to the railway state.

Remark 24.23. Within AIM, the track frame is generated from:

- pose2 reconstruction,
- pose3 extension,
- cant interpretation,
- and frame evaluation operators.

Definition 24.24 (Vehicle frame)**.** The vehicle frame is the local reference frame attached to the vehicle body or to a vehicle subsystem.

Remark 24.25. Vehicle frames may differ from the track frame due to:

- suspension motion,
- wheelset steering,
- hunting motion,
- roll dynamics,
- and structural flexibility.

Remark 24.26. Thus:

$$F_{\text{vehicle}} \neq F_{\text{track}}.$$

24.6. Wheelset Interpretation

Definition 24.27 (Wheelset state)**.** A wheelset state describes the relative geometric and operational state of a wheelset within the local railway frame.

Remark 24.28. Wheelset interpretation involves:

- yaw behaviour,
- relative wheel guidance,
- local wheel–rail interaction,
- lateral guidance,
- and relative rail geometry.

Remark 24.29. Wheelset states depend directly on:

- realized curvature,
- realized cant,
- local gauge geometry,
- and operational velocity.

Remark 24.30. Within AIM, wheelset interpretation is treated primarily as a runtime interpretation layer rather than as a full contact-mechanics model.

24.7. Bogie States

Definition 24.31 (Bogie state)**.** A bogie state describes the operational and geometric state of a bogie relative to the local railway state.

Remark 24.32. Bogie interpretation may include:

- relative yaw,
- wheelbase effects,
- relative suspension behaviour,
- roll behaviour,
- and wheelset coupling.

Remark 24.33. In transition regions, bogie states are strongly influenced by:

- curvature evolution,
- cant evolution,
- and higher curvature derivatives.

Remark 24.34. Within AIM, bogie interpretation remains primarily a runtime-relative operational abstraction rather than a complete multibody simulation framework.

24.8. Center-of-Gravity Interpretation

Definition 24.35 (Center-of-gravity state)**.** The center-of-gravity state describes the operational motion of the vehicle center of gravity relative to the railway runtime state.

Remark 24.36. This state is particularly relevant for:

- passenger comfort,
- load transfer,

- overturning stability,
- and Schwerpunkt-Trassierung.

Remark 24.37. Within AIM, the center-of-gravity state is interpreted primarily as an operational abstraction layer rather than as a full mechanical vehicle model.

Remark 24.38. Within AIM, cant interpretation is therefore not restricted to rail height difference alone, but extends toward center-of-gravity-dependent runtime interpretation.

Remark 24.39. The operationally relevant object is often not the rail geometry itself, but the trajectory of the vehicle center of gravity through realized runtime space.

24.9. Operational Vehicle Space

Remark 24.40. Operational railway behaviour emerges from the interaction between:

- runtime railway geometry,
- realization,
- operational velocity,
- and vehicle-space interpretation.

Remark 24.41. Thus, operational railway states cannot be interpreted purely through:

- geometry,
- or purely through mechanics,

but only through coupled runtime interpretation.

24.10. Vehicle Space and Realization

Remark 24.42. Vehicle-space interpretation depends on realized rather than purely analytical geometry.

Remark 24.43. In particular:

$$\text{vehicleState}_{\text{real}} = \mathcal{V}(\text{pose3}_{\text{real}}).$$

Remark 24.44. Thus, realization directly affects:

- comfort,
- wheel unloading,
- bogie behaviour,
- and operational smoothness.

24.11. Vehicle Space and Runtime

Remark 24.45. Vehicle-space evaluation is fundamentally a runtime process.

Remark 24.46. It depends on:

- runtime frame evaluation,
- world-to-track transformation,
- realized geometry,
- dynamic state interpretation,
- operational context,
- and runtime operators.

Remark 24.47. Vehicle-space interpretation therefore belongs naturally to the AIM runtime architecture.

24.12. Towards Schwerpunkt-Trassierung

Remark 24.48. The distinction between track state and vehicle state is one of the foundations of Schwerpunkt-Trassierung.

Remark 24.49. Within this interpretation, railway alignment design becomes:

> the design of operational vehicle-state trajectories relative to a realized railway runtime state.

Remark 24.50. Thus, the primary engineering object is no longer merely:

$$\gamma(s),$$

but the dynamically evolving operational state generated from:

$$\mathrm{pose3}_{\mathrm{real}}.$$

24.13. Canonical Interpretation

Proposition 24.51. *Within AIM, pose3 describes the railway runtime state rather than the vehicle itself.*

Proposition 24.52. *Vehicle states are interpreted as relative runtime states generated from interaction with the railway runtime state.*

Proposition 24.53. *Operational railway behaviour therefore emerges from coupled track–vehicle runtime interaction rather than from geometry alone.*

Proposition 24.54. *The center-of-gravity state is interpreted primarily as an operational runtime abstraction rather than as a complete mechanical vehicle model.*

24.14. Connection to AIM

Summary 24.55. Vehicle space extends railway alignment interpretation from pure geometry toward operational runtime-state interaction.

Within AIM:

- pose3 describes the railway runtime state,
- vehicle states are interpreted relatively,
- runtime frames generate operational interpretation,
- realization affects dynamic behaviour directly,
- center-of-gravity trajectories become operationally relevant,
- and runtime interpretation generates vehicle-space behaviour.

This provides the conceptual bridge toward:

- Schwerpunkt-Trassierung,
- realization-aware dynamics,
- runtime vehicle interpretation,
- and operational state optimization.

25. Runtime Dynamics

25.1. Motivation

Remark 25.1. Railway alignments are not merely geometric structures.

Remark 25.2. During operation, railway geometry generates evolving operational states through interaction with moving vehicles.

Remark 25.3. Within AIM, these operational quantities are interpreted as runtime states derived from:

- intrinsic curvature evolution,
- cant evolution,
- runtime frame evaluation,
- realization,
- and vehicle-space interpretation.

Remark 25.4. This motivates interpretation of railway dynamics as runtime state evolution rather than merely as static geometry.

Principle 25.5 (Runtime-dynamics principle). Within AIM, operational railway dynamics are interpreted as runtime evolution of coupled railway and vehicle states.

25.2. Runtime State Evolution

Definition 25.6 (Runtime state)**.** A runtime state is an operationally evaluated state depending on:

- pose reconstruction,
- frame evaluation,
- realization,
- operational context,
- and vehicle interpretation.

Definition 25.7 (Runtime state evolution)**.** Runtime evolution may formally be written as:

$$x'(s) = f(x, u),$$

where:

- $x(s)$ denotes the runtime state,
- $u(s)$ denotes operational or geometric input,
- and f denotes the runtime evolution operator.

Remark 25.8. Within AIM, the evolution parameter is typically arc-length rather than time.

Remark 25.9. The runtime state may include:

- curvature,
- cant,
- roll state,
- effective acceleration,
- vehicle-relative quantities,
- and operational frame states.

25.3. Runtime Dynamics Operator

Definition 25.10 (Runtime dynamics operator)**.** The runtime dynamics operator is denoted by:

$$\mathcal{D}.$$

Remark 25.11. Operational runtime evolution may therefore be interpreted abstractly as:

$$\mathcal{D}(\text{pose3}).$$

Remark 25.12. The operator $\mathcal{D}$ does not represent a single mechanical model. Instead, it denotes a runtime interpretation layer generating operational state evolution from railway geometry.

25.4. Acceleration Interpretation

Definition 25.13 (Runtime acceleration)**.** Runtime acceleration denotes operational acceleration quantities derived from runtime state evaluation.

Remark 25.14. Important quantities include:

- lateral acceleration,
- effective lateral acceleration,
- roll-induced acceleration,
- and vertical acceleration components.

Remark 25.15. Within AIM, acceleration is interpreted operationally relative to the runtime railway frame rather than as a purely world-space quantity.

25.5. Effective Lateral Acceleration

Definition 25.16 (Effective lateral acceleration)**.** The effective lateral acceleration is:

$$a_{\text{eff}}(s) = v^2\kappa(s) - g\sin\phi(s).$$

Remark 25.17. This quantity describes the operational imbalance between:

- curvature-induced acceleration,
- and cant-induced compensation.

Remark 25.18. The effective acceleration is one of the primary runtime quantities governing:

- passenger comfort,
- load transfer,
- and operational admissibility.

25.6. Cant Deficiency

Definition 25.19 (Cant deficiency)**.** Cant deficiency denotes the operational mismatch between:

- required equilibrium cant,
- and realized or applied cant.

Remark 25.20. Cant deficiency is therefore a runtime operational quantity rather than a purely geometric parameter.

Remark 25.21. Within AIM, cant deficiency depends jointly on:

- curvature,
- cant,

- speed,
- realization,
- and runtime interpretation.

25.7. Jerk Interpretation

Definition 25.22 (Runtime jerk)**.** Runtime jerk describes the evolution of acceleration along the runtime trajectory.

Remark 25.23. For railway operation, jerk depends jointly on:

$$\kappa'(s), \qquad \phi'(s), \qquad v.$$

Remark 25.24. One canonical relation is:

$$j(s) = v^3\kappa'(s) - gv\cos\phi(s)\,\phi'(s).$$

Remark 25.25. Jerk is one of the central runtime indicators for:

- comfort,
- transition quality,
- and dynamic smoothness.

25.8. Roll Response

Definition 25.26 (Roll response)**.** Roll response describes the operational rotational response of the vehicle state relative to the track frame.

Remark 25.27. Roll response depends on:

- cant evolution,
- suspension behaviour,
- center-of-gravity position,
- and operational velocity.

Remark 25.28. Within AIM, the roll state is interpreted relative to the runtime track frame generated by pose3 evaluation.

25.9. Dynamic Coupling

Remark 25.29. Operational railway quantities are strongly coupled.

Remark 25.30. In particular:

- curvature influences acceleration,
- cant influences roll response,
- realization influences smoothness,
- and runtime interpretation influences operational admissibility.

Remark 25.31. Thus, railway dynamics cannot be interpreted as isolated scalar quantities.

Proposition 25.32 (Dynamic coupling principle)**.** *Operational railway behaviour emerges from coupled runtime evolution of:*

- *geometry,*
- *cant,*
- *realization,*
- *runtime frames,*
- *and vehicle-space interpretation.*

25.10. Runtime Frames

Remark 25.33. Operational quantities depend on the chosen runtime frame.

Remark 25.34. Different interpretations may use:

- world frames,
- track-attached frames,
- vehicle frames,
- or center-of-gravity frames.

Remark 25.35. Within AIM, operational railway dynamics are primarily interpreted in track-relative runtime frames.

25.11. Runtime Dynamics versus Full Mechanics

Remark 25.36. The AIM runtime-dynamics framework is not intended as a complete multibody simulation model.

Remark 25.37. Instead, it provides:

- operational runtime interpretation,
- state coupling,
- admissibility evaluation,
- and realization-aware state evolution.

Remark 25.38. Detailed vehicle mechanics may later be coupled to this framework through specialized dynamic models.

25.12. Runtime Dynamics and Realization

Remark 25.39. Operational dynamics depend on realized rather than purely analytical geometry.

Remark 25.40. Thus:

$$\mathcal{D}(\mathrm{pose3}_{\mathrm{real}})$$

is operationally more relevant than:

$$\mathcal{D}(\mathrm{pose3}_{\mathrm{target}}).$$

Remark 25.41. Realization therefore directly affects:

- comfort,
- acceleration,
- jerk,
- and operational smoothness.

25.13. Runtime Dynamics and Curvature Space

Remark 25.42. Runtime dynamics emerge fundamentally from curvature evolution.

Remark 25.43. Within AIM, operational dynamics are therefore interpreted primarily as runtime consequences of:

$$\kappa(s), \qquad \kappa'(s), \qquad \kappa''(s).$$

Remark 25.44. Curvature space thus acts not only as a geometric domain, but also as a dynamic state-generation domain.

25.14. Towards Dynamic State Design

Remark 25.45. The runtime-dynamics interpretation shifts railway alignment design from:

curve construction

toward:

runtime operational state design.

Remark 25.46. This prepares the transition toward:

- Schwerpunkt-Trassierung,
- realization-aware dynamics,
- and operational state optimization.

25.15. Canonical Interpretation

Proposition 25.47. *Within AIM, railway dynamics are interpreted as runtime state evolution rather than as static geometric properties.*

Proposition 25.48. *Operational quantities emerge from coupled runtime interaction between:*

- *curvature,*
- *cant,*
- *realization,*
- *runtime frames,*
- *and vehicle interpretation.*

Proposition 25.49. *Runtime dynamics therefore form a state-evolution layer on top of intrinsic geometry.*

25.16. Connection to AIM

Summary 25.50. Runtime dynamics extend AIM geometry toward operational railway-state evolution.

Within this framework:

- runtime states evolve along railway geometry,
- acceleration and jerk become runtime quantities,
- cant deficiency becomes an operational state mismatch,

- realization affects dynamic behaviour directly,
- and curvature evolution generates operational dynamics.

This establishes the bridge between:

- intrinsic geometry,
- runtime evaluation,
- vehicle-space interpretation,
- realization,
- and operational state design.

26. Schwerpunkt States and Operational Balance

26.1. Motivation

Remark 26.1. Classical railway alignment theory primarily interprets transition design through geometric smoothness.

Remark 26.2. Within AIM, however, the operationally relevant object is not merely the track geometry itself, but the dynamically evolving vehicle state generated relative to the railway state.

Remark 26.3. This shifts the interpretation of railway alignment from:

> curve construction

toward:

> trajectory shaping of operational vehicle states.

Remark 26.4. The center-of-gravity interpretation plays a particularly important role within this framework.

Principle 26.5 (Schwerpunkt principle). Within AIM, Schwerpunkt-Trassierung is interpreted as operational trajectory shaping of realized runtime vehicle states.

26.2. Center-of-Gravity State

Definition 26.6 (Center-of-gravity state)**.** The center-of-gravity state describes the operational runtime state of the vehicle center of gravity relative to the railway state.

Remark 26.7. The center-of-gravity state depends on:

- realized railway geometry,
- cant evolution,
- operational speed,
- vehicle configuration,
- suspension behaviour,

- and runtime frame interpretation.

Remark 26.8. Within AIM, the center-of-gravity state is interpreted as a runtime state generated from:

$$\text{pose3}_{\text{real}}.$$

26.3. Center-of-Gravity Trajectory

Definition 26.9 (Center-of-gravity trajectory)**.** The center-of-gravity trajectory is the operational trajectory traced by the vehicle center of gravity within runtime space.

Remark 26.10. This trajectory is generally not identical to:

$$\gamma(s).$$

Remark 26.11. Instead, it emerges from:

- curvature evolution,
- cant evolution,
- roll response,
- vehicle geometry,
- and runtime dynamics.

Remark 26.12. The operational trajectory may therefore differ substantially from the pure geometric centerline.

26.4. Operational Balance

Definition 26.13 (Operational balance)**.** Operational balance denotes the dynamic relationship between:

- curvature-induced acceleration,
- cant-induced compensation,
- and vehicle response.

Remark 26.14. A dynamically balanced state approximately satisfies:

$$v^2\kappa(s) \approx g \sin\phi(s).$$

Remark 26.15. In practice, operational balance is not exact but evolves continuously within admissible runtime ranges.

26.5. Balance Trajectories

Definition 26.16 (Balance trajectory)**.** A balance trajectory is the runtime evolution of operational equilibrium states along the railway alignment.

Remark 26.17. Balance trajectories may be interpreted through:

- effective acceleration,
- roll evolution,
- cant deficiency,
- and center-of-gravity response.

Remark 26.18. Thus, operational balance is interpreted dynamically rather than as a single static equilibrium condition.

26.6. Effective Acceleration Interpretation

Definition 26.19 (Effective operational acceleration)**.** The effective operational acceleration is:

$$a_{\text{eff}}(s) = v^2\kappa(s) - g\sin\phi(s).$$

Remark 26.20. This quantity measures the operational imbalance experienced within the vehicle state.

Remark 26.21. Within Schwerpunkt interpretation, effective acceleration is more relevant than curvature alone.

Remark 26.22. Operational smoothness therefore depends on the trajectory of:

$$a_{\text{eff}}(s),$$

rather than merely on geometric continuity.

26.7. Roll Evolution

Definition 26.23 (Roll evolution)**.** Roll evolution denotes the runtime evolution of vehicle roll state relative to the railway frame.

Remark 26.24. Roll evolution depends jointly on:

- cant evolution,
- suspension response,
- center-of-gravity height,

- and operational velocity.

Remark 26.25. Within operational railway dynamics, abrupt roll evolution may generate:

- discomfort,
- load transfer,
- wheel unloading,
- and dynamic instability.

26.8. Comfort Interpretation

Definition 26.26 (Operational comfort)**.** Operational comfort denotes the dynamic smoothness experienced by the vehicle state during runtime evolution.

Remark 26.27. Comfort depends on:

- effective acceleration,
- jerk,
- roll evolution,
- vibration behaviour,
- and realization smoothness.

Remark 26.28. Comfort is therefore not purely geometric. It emerges from runtime dynamic state evolution.

26.9. Trajectory Shaping

Definition 26.29 (Operational trajectory shaping)**.** Operational trajectory shaping denotes the design of runtime vehicle state evolution through curvature and cant control.

Remark 26.30. Within this interpretation, railway alignment design becomes:

- shaping of effective acceleration trajectories,
- shaping of roll evolution,
- shaping of operational balance,
- and shaping of center-of-gravity motion.

Remark 26.31. Thus, railway alignment design is elevated from:

curve construction

to:

operational state design.

26.10. Schwerpunkt Interpretation

Remark 26.32. Within AIM, Schwerpunkt-Trassierung is not interpreted as a particular curve formula or historical transition family.

Remark 26.33. Instead, it denotes:

> runtime-oriented shaping of operational vehicle-state trajectories.

Remark 26.34. The central engineering question therefore becomes:

> Which operational state trajectory is generated?

rather than:

> Which curve formula is used?

26.11. Schwerpunkt and Realization

Remark 26.35. Operational trajectory shaping depends fundamentally on realized rather than purely analytical geometry.

Remark 26.36. Thus:

$$\mathrm{CG}_{\mathrm{real}}$$

is operationally more relevant than:

$$\mathrm{CG}_{\mathrm{target}}.$$

Remark 26.37. Realization therefore directly influences:

- comfort,
- roll smoothness,
- operational balance,
- and dynamic admissibility.

26.12. Schwerpunkt and Curvature Space

Remark 26.38. Operational state trajectories are generated fundamentally through curvature evolution.

Remark 26.39. Thus, curvature space acts simultaneously as:

- geometric space,
- runtime space,
- and operational state-generation space.

Remark 26.40. Different transition families may therefore generate fundamentally different operational state trajectories even when their position-space geometry appears similar.

26.13. Towards Dynamic State Design

Remark 26.41. The Schwerpunkt interpretation transforms railway alignment theory into a theory of operational runtime-state evolution.

Remark 26.42. This prepares the conceptual transition toward:

- realization-aware dynamics,
- Vienna reinterpretation,
- operational admissibility,
- and runtime-state optimization.

26.14. Canonical Interpretation

Proposition 26.43. *Within AIM, Schwerpunkt-Trassierung is interpreted as shaping of operational runtime vehicle states.*

Proposition 26.44. *The operationally relevant object is not merely the railway curve, but the realized trajectory of vehicle states relative to the railway frame.*

Proposition 26.45. *Operational smoothness depends primarily on runtime state evolution rather than on coordinate geometry alone.*

26.15. Connection to AIM

Summary 26.46. Schwerpunkt states extend railway alignment interpretation from geometry toward operational runtime-state design.

Within AIM:

- center-of-gravity trajectories become operationally relevant,
- balance trajectories replace purely geometric interpretation,
- comfort emerges from runtime evolution,
- effective acceleration becomes a state variable,
- and railway alignment becomes operational trajectory shaping.

This provides the conceptual bridge toward:

- Vienna reinterpretation,
- realization-aware dynamics,
- operational admissibility,
- and runtime-state optimization.

27. Dynamic Balance and Operational Admissibility

27.1. Motivation

Remark 27.1. Railway operation involves continuous interaction between:

- curvature,
- cant,
- vehicle response,
- and operational velocity.

Remark 27.2. Classical railway engineering often expresses this interaction through equilibrium relations between curvature and cant.

Remark 27.3. Within AIM, however, such relations are interpreted not as isolated engineering formulas, but as runtime-state coupling laws.

Principle 27.4 (Dynamic-balance principle). Within AIM, operational balance emerges from runtime coupling between:

- curvature evolution,
- cant evolution,
- vehicle-state response,
- realization,
- and operational context.

27.2. Equilibrium Relation

Definition 27.5 (Equilibrium relation). A dynamically balanced runtime state approximately satisfies:

$$v^2\kappa(s) = g\sin\phi(s).$$

Remark 27.6. This relation couples:

- curvature-induced acceleration,

- and cant-induced compensation.

Remark 27.7. Within AIM, the equilibrium relation is interpreted as a runtime-state coupling law rather than merely as a static engineering equation.

27.3. Operational Imbalance

Definition 27.8 (Operational imbalance)**.** Operational imbalance is the deviation between:

- curvature-induced acceleration,
- and cant-induced equilibrium compensation.

Remark 27.9. The canonical imbalance quantity is:

$$a_{\mathrm{eff}}(s) = v^2\kappa(s) - g\sin\phi(s).$$

Remark 27.10. Operational imbalance is unavoidable in practical railway operation and must therefore be interpreted dynamically rather than eliminated completely.

27.4. Cant Deficiency

Definition 27.11 (Cant deficiency)**.** Cant deficiency denotes insufficient cant relative to the operational equilibrium state.

Remark 27.12. Cant deficiency corresponds operationally to:

$$v^2\kappa(s) > g\sin\phi(s).$$

Remark 27.13. In this situation, uncompensated lateral acceleration remains within the vehicle state.

Remark 27.14. Cant deficiency influences:

- passenger comfort,
- wheel unloading,
- lateral force distribution,
- and operational admissibility.

27.5. Excess Cant

Definition 27.15 (Excess cant)**.** Excess cant denotes cant exceeding the equilibrium requirement for the current operational state.

Remark 27.16. Operationally:

$$v^2\kappa(s) < g \sin \phi(s).$$

Remark 27.17. Excess cant may lead to:

- inward acceleration effects,
- reduced passenger comfort,
- operational limitations at low speed,
- and wheel unloading effects.

27.6. Equilibrium States

Definition 27.18 (Dynamic equilibrium state)**.** A dynamic equilibrium state is a runtime state satisfying operational balance within admissible tolerances.

Remark 27.19. Within practical railway operation, equilibrium is not exact but exists within operational ranges.

Remark 27.20. Thus, equilibrium should be interpreted as a band of admissible runtime states rather than as a single exact condition.

27.7. Equilibrium Bands

Definition 27.21 (Equilibrium band)**.** An equilibrium band is a range of admissible operational imbalance states satisfying:

$$|a_{\text{eff}}(s)| \leq a_{\max}.$$

Remark 27.22. Equilibrium bands depend on:

- vehicle type,
- operational velocity,
- standards,
- realization quality,
- and comfort requirements.

Remark 27.23. Within AIM, operational equilibrium is therefore interpreted as a runtime admissibility region.

27.8. Dynamic Admissibility

Definition 27.24 (Dynamic admissibility)**.** A runtime state is dynamically admissible if operational quantities remain within admissible bounds.

Remark 27.25. Relevant admissibility quantities include:

- effective acceleration,
- jerk,
- roll evolution,
- cant deficiency,
- wheel unloading,
- and operational smoothness.

Remark 27.26. Dynamic admissibility therefore depends on coupled runtime-state evolution rather than on geometry alone.

27.9. Operational Envelopes

Definition 27.27 (Operational envelope)**.** An operational envelope is the admissible region of runtime operational states.

Remark 27.28. Operational envelopes may constrain:

- acceleration,
- jerk,
- roll behaviour,
- cant deficiency,
- speed,
- and realization tolerances.

Remark 27.29. Within AIM, operational envelopes define admissible runtime-state regions rather than purely geometric limits.

27.10. Dynamic Coupling

Remark 27.30. Dynamic balance is fundamentally a coupled runtime phenomenon.

Remark 27.31. In particular:

- curvature influences acceleration,
- cant influences roll response,
- realization influences smoothness,
- and operational speed influences admissibility.

Remark 27.32. Thus:

$$\kappa(s), \qquad \phi(s), \qquad v(s)$$

cannot be interpreted independently.

27.11. Balance Trajectories

Definition 27.33 (Balance trajectory)**.** A balance trajectory is the runtime evolution of operational balance states along the railway alignment.

Remark 27.34. Balance trajectories are generated jointly by:

- curvature evolution,
- cant evolution,
- realization,
- and vehicle-state response.

Remark 27.35. Operational smoothness therefore depends on:

$$a_{\text{eff}}(s), \qquad j(s), \qquad \phi'(s),$$

rather than solely on geometric continuity.

27.12. Runtime-State Coupling

Remark 27.36. The equilibrium relation:

$$v^2\kappa = g\sin\phi$$

is fundamentally a runtime-state coupling law.

Remark 27.37. It connects:

- intrinsic geometry,
- cant evolution,
- runtime vehicle states,
- and operational admissibility.

Remark 27.38. Within AIM, dynamic balance therefore belongs naturally to runtime-state evolution rather than to isolated geometric design formulas.

27.13. Dynamic Balance and Realization

Remark 27.39. Operational balance depends on realized rather than purely analytical geometry.

Remark 27.40. Thus:

$$a_{\mathrm{eff,real}}(s) = v^2\kappa_{\mathrm{real}}(s) - g\sin\phi_{\mathrm{real}}(s)$$

is operationally more relevant than its analytical target counterpart.

Remark 27.41. Realization therefore directly influences:

- comfort,
- admissibility,
- operational smoothness,
- and dynamic balance.

27.14. Dynamic Balance and Curvature Space

Remark 27.42. Operational balance emerges fundamentally from curvature evolution.

Remark 27.43. Different transition families therefore generate different operational balance trajectories even if their position-space geometry appears similar.

Remark 27.44. Curvature space is therefore simultaneously:

- a geometric domain,
- a runtime domain,
- and a dynamic balance domain.

27.15. Towards Runtime-State Design

Remark 27.45. The dynamic-balance interpretation transforms railway alignment from:

curve design

toward:

design of admissible runtime operational states.

Remark 27.46. This prepares the transition toward:

- Vienna reinterpretation,
- realization-aware dynamics,
- Schwerpunkt-Trassierung,
- and operational state optimization.

27.16. Canonical Interpretation

Proposition 27.47. *Within AIM, the equilibrium relation is interpreted as a runtime-state coupling law.*

Proposition 27.48. *Operational railway balance emerges from coupled runtime interaction between:*

- *curvature,*
- *cant,*
- *realization,*
- *speed,*
- *and vehicle-state response.*

Proposition 27.49. *Dynamic admissibility is fundamentally a runtime-state property rather than a purely geometric property.*

27.17. Connection to AIM

Summary 27.50. Dynamic balance extends railway alignment interpretation from geometric equilibrium formulas toward runtime-state coupling.

Within AIM:

- equilibrium becomes a runtime-state relation,
- cant deficiency becomes an operational imbalance state,

- admissibility becomes envelope-based,
- operational balance evolves dynamically,
- and runtime-state trajectories become primary engineering objects.

This establishes the conceptual bridge between:

- curvature evolution,
- runtime dynamics,
- operational admissibility,
- realization-aware behaviour,
- and operational state optimization.

28. Vienna Reinterpretation and Dynamic State Shaping

28.1. Motivation

Remark 28.1. Classical transition theory usually interprets transition curves as special geometric constructions.

Remark 28.2. Within AIM, however, transition structures are interpreted primarily through their generated runtime behaviour.

Remark 28.3. This changes the interpretation of Vienna-style transition concepts fundamentally.

Remark 28.4. Within this framework, Vienna is not interpreted as:

- a specific curve formula,
- a historical construction rule,
- or a particular interpolation method,

but rather as:

> a strategy for shaping operational runtime states.

Principle 28.5 (Vienna reinterpretation principle). Within AIM, Vienna-type transitions are interpreted as dynamic state-shaping strategies operating in curvature-runtime space.

28.2. From Curves to Runtime States

Remark 28.6. Classical railway alignment theory focuses primarily on:

$$\gamma(s).$$

Remark 28.7. Within AIM, however, the operationally relevant quantity is:

$$\mathrm{pose3}_{\mathrm{real}}(s),$$

together with the generated vehicle-space trajectory.

Remark 28.8. Thus, transition interpretation shifts:

- from coordinate geometry,
- toward runtime operational state evolution.

Remark 28.9. The central engineering question therefore becomes:

> Which operational runtime trajectory is generated?

rather than:

> Which geometric curve formula is used?

28.3. Vienna as Runtime-State Strategy

Definition 28.10 (Vienna-type state shaping)**.** A Vienna-type transition strategy is a curvature-evolution strategy designed to shape operational runtime-state trajectories.

Remark 28.11. Relevant runtime quantities include:

- effective acceleration,
- jerk,
- roll evolution,
- operational balance,
- and center-of-gravity motion.

Remark 28.12. Thus, Vienna interpretation operates fundamentally in:

- curvature space,
- runtime space,
- and operational state space.

28.4. Half-Wave Interpretation

Definition 28.13 (Half-wave dynamics)**.** Half-wave dynamics denotes interpretation of transition evolution through normalized curvature half-wave structures.

Remark 28.14. Within AIM, many transition structures may be interpreted compositionally as:

$$\mathrm{halfWave}_1 \to \mathrm{core} \to \mathrm{halfWave}_2.$$

Remark 28.15. This decomposition allows:

- modular runtime shaping,

- asymmetric runtime behaviour,
- spectral control,
- and realization-aware transition design.

Remark 28.16. The half-wave interpretation is not merely geometric. It directly affects:

- acceleration evolution,
- roll evolution,
- and operational smoothness.

28.5. Dynamic State Shaping

Definition 28.17 (Dynamic state shaping)**.** Dynamic state shaping denotes controlled generation of operational runtime trajectories through curvature evolution.

Remark 28.18. Within AIM, transition geometry is interpreted fundamentally as:

> runtime-state shaping through curvature control.

Remark 28.19. Relevant shaped quantities include:

- effective acceleration,
- jerk,
- cant deficiency,
- roll evolution,
- and center-of-gravity trajectories.

28.6. Center-of-Gravity Interpretation

Remark 28.20. Within Vienna reinterpretation, the operationally relevant object is not merely the rail geometry itself.

Remark 28.21. Instead, emphasis shifts toward:

$$\mathrm{CG}(s),$$

the runtime trajectory of the vehicle center of gravity.

Remark 28.22. Different transition families may produce:

- similar geometric centerlines,
- but fundamentally different center-of-gravity trajectories.

Remark 28.23. Thus, Vienna reinterpretation naturally connects transition theory with:

- Schwerpunkt-Trassierung,
- operational balance,
- and runtime comfort interpretation.

28.7. Spectral Interpretation

Remark 28.24. Transition laws may be interpreted spectrally through their curvature content.

Remark 28.25. Low-frequency curvature behaviour primarily influences:

- global operational balance,
- smooth acceleration evolution,
- and large-scale runtime behaviour.

Remark 28.26. High-frequency curvature behaviour tends to influence:

- realization sensitivity,
- machine smoothing,
- dynamic roughness,
- and operational excitation.

Proposition 28.27 (Spectral smoothing principle)**.** *Operationally smooth runtime trajectories correspond to spectrally smooth curvature evolution.*

28.8. Realization Filtering

Remark 28.28. Realization acts approximately as a low-pass filter on curvature space.

Remark 28.29. Thus, highly oscillatory transition structures may collapse into similar realized operational states after construction and maintenance.

Remark 28.30. Within AIM, Vienna reinterpretation therefore includes:

- realization filtering,
- machine smoothing,
- and runtime operational robustness.

Remark 28.31. A transition law is operationally meaningful only if its intended runtime behaviour survives realization.

28.9. Runtime Smoothness

Definition 28.32 (Runtime smoothness)**.** Runtime smoothness denotes smooth operational evolution of runtime states generated through curvature and cant evolution.

Remark 28.33. Runtime smoothness includes:

- smooth acceleration evolution,
- smooth roll evolution,
- smooth operational balance,
- and smooth center-of-gravity trajectories.

Remark 28.34. Within AIM, runtime smoothness is operationally more relevant than purely geometric smoothness.

28.10. Vienna and Curvature Space

Remark 28.35. Vienna reinterpretation operates fundamentally within curvature space.

Remark 28.36. Different curvature structures generate:

- different operational balance trajectories,
- different roll behaviour,
- different jerk evolution,
- and different realization behaviour.

Remark 28.37. Thus, transition quality cannot be characterized sufficiently through position geometry alone.

28.11. Vienna and Runtime Dynamics

Remark 28.38. Vienna reinterpretation is fundamentally a runtime-dynamics concept.

Remark 28.39. It couples:

- curvature evolution,
- runtime-state evolution,
- operational balance,
- realization filtering,
- and vehicle-space interpretation.

Remark 28.40. The transition curve therefore acts as:

a runtime-state generator.

28.12. Operational Interpretation

Remark 28.41. Within AIM, the operational railway problem is interpreted as:

generation of admissible runtime operational trajectories.

Remark 28.42. This transforms railway alignment theory from:

- geometric interpolation,
- toward operational runtime-state design.

Remark 28.43. Vienna reinterpretation therefore acts as a bridge between:

- transition theory,
- runtime dynamics,
- realization,
- and operational optimization.

28.13. Towards Runtime-State Optimization

Remark 28.44. The reinterpretation developed here prepares the transition toward:

- realization-aware optimization,
- operational admissibility optimization,
- runtime-state shaping,
- and Schwerpunkt-based alignment design.

Remark 28.45. Optimization therefore acts not merely on:

$$\gamma(s),$$

but on:

$$\mathrm{pose3}_{\mathrm{real}}(s),$$

and the generated operational vehicle-state trajectories.

28.14. Canonical Interpretation

Proposition 28.46. *Within AIM, Vienna-type transitions are interpreted as runtime-state shaping strategies rather than as isolated geometric curve formulas.*

Proposition 28.47. *Operational transition quality depends fundamentally on generated runtime-state trajectories.*

Proposition 28.48. *Realization filtering and spectral behaviour are central components of transition interpretation.*

Proposition 28.49. *The operationally relevant object is the realized runtime trajectory of vehicle states rather than the abstract geometric curve alone.*

28.15. Connection to AIM

Summary 28.50. Vienna reinterpretation transforms transition theory into a theory of runtime-state shaping.

Within AIM:

- transitions generate operational runtime trajectories,
- half-wave structures shape dynamic behaviour,
- realization filtering determines operational robustness,
- spectral smoothness influences comfort and admissibility,
- and center-of-gravity trajectories become primary operational objects.

This establishes the conceptual bridge between:

- curvature space,
- runtime dynamics,
- realization-aware interpretation,
- Schwerpunkt-Trassierung,
- and operational state optimization.

29. Realization-Aware Runtime Dynamics

29.1. Motivation

Remark 29.1. Classical railway alignment theory often assumes that operational dynamics arise directly from analytical target geometry.

Remark 29.2. In reality, however, railway operation interacts with:

- realized geometry,
- measured states,
- runtime reconstruction,
- and realization-dependent operational behaviour.

Remark 29.3. Thus, operational railway dynamics are fundamentally realization-aware.

Remark 29.4. Within AIM, runtime dynamics are therefore interpreted not as direct consequences of analytical geometry, but as consequences of realized and runtime-evaluated railway states.

Principle 29.5 (Realization-aware dynamics principle). Within AIM, operational railway dynamics emerge from realized runtime states rather than from idealized analytical geometry alone.

29.2. From Target Geometry to Runtime Dynamics

Remark 29.6. The analytical target alignment:

$$\gamma_{\text{target}}(s)$$

is not the operationally relevant railway state.

Remark 29.7. Instead, operational behaviour depends on:

$$\text{pose3}_{\text{real}}(s),$$

including:

- realized geometry,

- realized cant,
- realized curvature,
- runtime reconstruction,
- and operational interpretation.

Remark 29.8. Thus, railway dynamics arise through a chain:

$$\text{target} \to \text{realization} \to \text{runtime} \to \text{operation}.$$

29.3. Realization-Aware Runtime States

Definition 29.9 (Realization-aware runtime state)**.** A realization-aware runtime state is a runtime railway state evaluated from realized rather than purely analytical geometry.

Remark 29.10. Such states depend on:

- realization,
- measurement,
- runtime reconstruction,
- and operational context.

Remark 29.11. Within AIM, operational dynamics are interpreted primarily through:

$$\mathrm{pose3}_{\mathrm{real}}(s).$$

29.4. Realization Operator on Runtime States

Definition 29.12 (Pose3 realization operator)**.** The realization operator acting on runtime states is:

$$\mathcal{R}_{\mathrm{pose3}} : \mathrm{pose3}_{\mathrm{target}} \mapsto \mathrm{pose3}_{\mathrm{real}}.$$

Remark 29.13. The operator includes:

- construction effects,
- measurement effects,
- realization filtering,
- mechanical response,
- and operational reconstruction.

Remark 29.14. Operational runtime dynamics therefore depend on:

$$\mathcal{D}(\mathcal{R}_{\mathrm{pose3}}(\mathrm{pose3}_{\mathrm{target}})).$$

29.5. Measurement and Runtime Reconstruction

Remark 29.15. Operational railway systems do not access exact analytical geometry directly.

Remark 29.16. Instead, runtime systems operate on:

- measurements,
- reconstructed states,
- sensor data,
- estimated geometry,
- and realization-aware runtime evaluation.

Remark 29.17. Thus, runtime dynamics depend fundamentally on:

- measurement interpretation,
- reconstruction quality,
- and realization-aware state estimation.

29.6. Realized Dynamic States

Definition 29.18 (Realized dynamic state)**.** A realized dynamic state is a runtime operational state generated from:

$$\mathrm{pose3}_{\mathrm{real}}(s).$$

Remark 29.19. Important realized operational quantities include:

- effective acceleration,
- jerk,
- roll evolution,
- cant deficiency,
- and operational balance.

Remark 29.20. These quantities differ from their analytical target counterparts due to realization effects.

29.7. Realization-Aware Effective Acceleration

Definition 29.21 (Realized effective acceleration)**.** The realized effective acceleration is:

$$a_{\mathrm{eff,real}}(s) = v^2 \kappa_{\mathrm{real}}(s) - g \sin \phi_{\mathrm{real}}(s).$$

Remark 29.22. This quantity is operationally more relevant than:

$$a_{\mathrm{eff,target}}(s).$$

Remark 29.23. Passenger comfort depends on realized rather than idealized runtime states.

29.8. Realized Jerk

Definition 29.24 (Realized jerk)**.** The realized jerk is:

$$j_{\mathrm{real}}(s) = v^3 \kappa'_{\mathrm{real}}(s) - gv \cos \phi_{\mathrm{real}}(s)\, \phi'_{\mathrm{real}}(s).$$

Remark 29.25. Realized jerk depends jointly on:

- realized curvature evolution,
- realized cant evolution,
- and runtime operational interpretation.

Remark 29.26. Thus, comfort depends fundamentally on realization-aware runtime behaviour.

29.9. Realization Filtering

Remark 29.27. Realization acts approximately as a low-pass filter on:

- curvature,
- cant,
- and higher-order runtime-state evolution.

Remark 29.28. High-frequency operational behaviour may therefore disappear during:

- construction,
- maintenance,
- reconstruction,
- and runtime evaluation.

Proposition 29.29 (Runtime realization-filter principle)**.** *Operational runtime behaviour is generated from filtered realized states rather than directly from analytical target geometry.*

29.10. Operational Admissibility

Definition 29.30 (Realization-aware admissibility)**.** A railway state is realization-aware admissible if:

- realized runtime states remain operationally admissible,
- realization does not introduce instability,
- and operational envelopes remain satisfied.

Remark 29.31. Admissibility therefore depends jointly on:

- geometry,
- realization,
- runtime reconstruction,
- and operational interpretation.

29.11. Operational Runtime Envelopes

Definition 29.32 (Realization-aware runtime envelope)**.** A realization-aware runtime envelope is the admissible region of realized operational runtime states.

Remark 29.33. Such envelopes constrain:

- realized acceleration,
- realized jerk,
- roll evolution,
- cant deficiency,
- and operational smoothness.

Remark 29.34. Operational envelopes therefore belong fundamentally to runtime-state space rather than purely to geometric design space.

29.12. Dynamic State Design

Remark 29.35. The realization-aware interpretation transforms railway alignment from:

> design of analytical curves

toward:

> design of realized operational runtime states.

Remark 29.36. Within AIM, the operationally relevant design object is therefore:

$$\mathcal{R}_{\text{pose3}}(\text{pose3}_{\text{target}}),$$

rather than merely:

$$\text{pose3}_{\text{target}}.$$

29.13. Optimization Interpretation

Remark 29.37. Optimization should therefore operate on:

$$\text{pose3}_{\text{real}},$$

rather than solely on analytical target geometry.

Remark 29.38. Thus, realization-aware optimization becomes:

$$J = J\Big(\mathcal{D}(\mathcal{R}_{\text{pose3}}(\text{pose3}_{\text{target}}))\Big).$$

Remark 29.39. Optimization therefore acts simultaneously on:

- geometry,
- realization behaviour,
- runtime reconstruction,
- and operational dynamics.

29.14. Realization-Aware State Design

Remark 29.40. Within AIM, railway alignment design becomes:

realization-aware runtime-state design.

Remark 29.41. The relevant engineering question is therefore not:

“Which analytical curve should be designed?”

but rather:

“Which realized runtime operational state should emerge?”

Remark 29.42. This interpretation unifies:

- geometry,
- realization,

- measurement,
- runtime dynamics,
- and optimization.

29.15. Canonical Interpretation

Proposition 29.43. *Operational railway dynamics emerge from realized runtime states rather than directly from analytical geometry.*

Proposition 29.44. *The realization operator acts directly on runtime operational behaviour.*

Proposition 29.45. *Passenger comfort, admissibility, and runtime smoothness are therefore realization-aware properties.*

Proposition 29.46. *Within AIM, railway alignment becomes realization-aware operational runtime-state design.*

29.16. Connection to AIM

Summary 29.47. Realization-aware dynamics unify:

- realization,
- measurement,
- runtime reconstruction,
- operational dynamics,
- and optimization.

Within AIM:

- the realization operator acts on runtime states,
- operational behaviour emerges from realized geometry,
- runtime dynamics become realization-aware,
- admissibility becomes state-dependent,
- and railway alignment becomes realization-aware runtime-state design.

This establishes the conceptual bridge between:

- geometry,

- realization,
- runtime evaluation,
- operational dynamics,
- Schwerpunkt-Trassierung,
- and realization-aware optimization.

30. Optimization in Runtime State Space

30.1. Motivation

Remark 30.1. Classical railway alignment optimization is usually formulated as a geometric optimization problem.

Remark 30.2. Typical optimization variables include:

- points,
- radii,
- tangent lengths,
- and geometric transition parameters.

Remark 30.3. Within AIM, however, operational railway behaviour emerges from runtime state evolution rather than from geometry alone.

Remark 30.4. Consequently, the relevant optimization object is not merely geometric shape, but:

runtime operational state trajectories.

Principle 30.5 (State-space optimization principle). Within AIM, railway alignment optimization is interpreted fundamentally as optimization of realized runtime-state trajectories.

30.2. From Geometry to Runtime States

Remark 30.6. Geometric railway alignments generate:

$$\mathrm{pose3}(s),$$

which in turn generates:

- operational balance,
- acceleration evolution,
- jerk evolution,
- roll evolution,

- and vehicle-state trajectories.

Remark 30.7. Thus, geometry acts indirectly through:

$$\kappa(s), \qquad u(s), \qquad \phi(s),$$

to generate operational runtime states.

Remark 30.8. Optimization therefore acts fundamentally on:

$$x(s),$$

where:

$$x(s)$$

denotes the evolving operational runtime state.

30.3. Runtime State Trajectories

Definition 30.9 (Runtime state trajectory)**.** A runtime state trajectory is the operational evolution:

$$x(s)$$

generated along the railway alignment.

Remark 30.10. The runtime state may include:

- effective acceleration,
- jerk,
- roll state,
- cant deficiency,
- operational balance,
- and center-of-gravity behaviour.

Remark 30.11. The operationally relevant optimization object is therefore:

$$x(s),$$

rather than merely:

$$\gamma(s).$$

30.4. State Evolution

Definition 30.12 (Runtime state evolution)**.** Runtime state evolution may be written abstractly as:

$$x'(s) = f(x(s), u(s)),$$

where:

- $x(s)$ denotes the runtime state,
- and $u(s)$ denotes control variables.

Remark 30.13. Typical control variables include:

- curvature evolution,
- cant evolution,
- transition structure,
- and operational velocity.

Remark 30.14. Within AIM, geometry therefore acts as:

a runtime-state generator.

30.5. Operational State Variables

Remark 30.15. Operational state variables may include:

$$x(s) = (a_{\text{eff}}, j, \phi, \phi', \Delta_{\text{cant}}, \ldots).$$

Remark 30.16. The exact state structure depends on:

- operational interpretation,
- realization level,
- and optimization objectives.

30.6. Realized Runtime States

Remark 30.17. Operational railway systems interact with:

$$\text{pose3}_{\text{real}},$$

rather than purely analytical target geometry.

Remark 30.18. Thus, optimization should operate fundamentally on:

$$x_{\mathrm{real}}(s),$$

generated through:

$$\mathcal{R}_{\mathrm{pose3}}.$$

Remark 30.19. This transforms optimization into:

$$J = J\big(x_{\mathrm{real}}(s)\big).$$

30.7. Optimization Functionals

Definition 30.20 (Runtime-state functional)**.** A runtime-state optimization functional may be written as:

$$J[x] = \int_0^L F(x(s), x'(s), u(s))\,\mathrm{d}s.$$

Remark 30.21. Typical optimization targets include:

- minimizing jerk,
- minimizing effective acceleration variation,
- minimizing roll excitation,
- minimizing realization sensitivity,
- and maximizing operational smoothness.

30.8. Trajectory Shaping

Definition 30.22 (Operational trajectory shaping)**.** Operational trajectory shaping denotes optimization of runtime-state evolution through curvature and cant control.

Remark 30.23. Within AIM, optimization therefore acts primarily on:

- acceleration trajectories,
- roll trajectories,
- balance trajectories,
- and center-of-gravity trajectories.

Remark 30.24. This interpretation extends classical railway optimization toward:

> runtime-state trajectory optimization.

30.9. Curvature Space Optimization

Remark 30.25. Runtime-state trajectories are generated fundamentally from curvature evolution.

Remark 30.26. Thus, optimization acts intrinsically within:

$$\mathcal{K},$$

the curvature-space domain.

Remark 30.27. Typical optimization variables include:

- transition families,
- curvature amplitudes,
- transition lengths,
- normalization structure,
- and sparse curvature corrections.

30.10. Sparse Runtime Optimization

Definition 30.28 (Sparse runtime optimization)**.** Sparse runtime optimization uses compact curvature representations to optimize runtime-state trajectories efficiently.

Remark 30.29. Within AIM, sparse models act as:

compact runtime-state generators.

Remark 30.30. This enables:

- scalable optimization,
- efficient runtime evaluation,
- realization-aware optimization,
- and robust parameter control.

30.11. Operational Admissibility

Definition 30.31 (Operationally admissible trajectory)**.** A runtime-state trajectory is operationally admissible if:

- effective acceleration remains bounded,
- jerk remains admissible,

- roll evolution remains smooth,
- and operational envelopes remain satisfied.

Remark 30.32. Admissibility therefore acts directly within runtime-state space.

30.12. Realization-Aware Optimization

Remark 30.33. Optimization should operate on:

$$x_{\text{real}}(s),$$

rather than on purely analytical target states.

Remark 30.34. Thus:

$$J = J\Big(\mathcal{D}(\mathcal{R}_{\text{pose3}}(\text{pose3}_{\text{target}}))\Big).$$

Remark 30.35. Optimization therefore couples:

- geometry,
- realization,
- runtime reconstruction,
- and operational dynamics.

30.13. Center-of-Gravity Optimization

Remark 30.36. Within advanced runtime-state interpretation, optimization may act directly on:

$$\text{CG}(s),$$

the center-of-gravity trajectory.

Remark 30.37. Thus, optimization becomes:

- balance shaping,
- comfort shaping,
- roll shaping,
- and operational trajectory shaping.

Remark 30.38. This interpretation connects directly to:

- Schwerpunkt-Trassierung,
- Vienna reinterpretation,
- and realization-aware runtime dynamics.

30.14. Optimization as Runtime-State Design

Remark 30.39. Within AIM, railway optimization is interpreted fundamentally as:

> design of admissible realized runtime-state trajectories.

Remark 30.40. The central engineering question therefore becomes:

> Which operational state trajectory should emerge?

rather than:

> Which geometric curve should be constructed?

Remark 30.41. Geometry, realization, and runtime dynamics therefore become coupled components of one operational design framework.

30.15. AXTRAN Reinterpretation

Remark 30.42. Classical optimization systems such as AXTRAN already moved partially toward operational optimization beyond purely geometric fitting.

Remark 30.43. Within AIM, this idea is generalized fundamentally:

- from geometry optimization,
- toward runtime-state trajectory optimization.

Remark 30.44. This may be interpreted as:

> AXTRAN-next-generation: runtime-state shaping in curvature space.

30.16. Canonical Interpretation

Proposition 30.45. *Within AIM, railway optimization acts fundamentally on runtime-state trajectories rather than on coordinate geometry alone.*

Proposition 30.46. *Geometry acts as a generator of operational runtime states.*

Proposition 30.47. *Operational admissibility is fundamentally a runtime-state property.*

Proposition 30.48. *Realization-aware runtime-state optimization forms the core of advanced railway alignment design.*

30.17. Connection to AIM

Summary 30.49. Optimization in AIM operates fundamentally in runtime-state space.

Within this framework:

- geometry generates runtime states,
- curvature space generates operational trajectories,
- realization filters runtime behaviour,
- admissibility acts in operational state space,
- and optimization becomes trajectory shaping.

This interpretation unifies:

- curvature evolution,
- runtime dynamics,
- realization,
- Schwerpunkt-Trassierung,
- Vienna reinterpretation,
- and operational optimization.

Railway alignment therefore becomes:

> realization-aware runtime-state trajectory design.

Teil VI.

Runtime and Evaluation

Runtime and Evaluation

Remark 30.50. Classical railway alignment theory primarily focuses on static geometry.

Remark 30.51. Within AIM, however, alignments are not interpreted as passive curves, but as continuously evaluated runtime structures.

Remark 30.52. Projection, frame construction, realization evaluation, dynamic interpretation, and optimization are therefore treated as runtime processes operating on alignment states.

Remark 30.53. This part introduces the computational and conceptual runtime layer that connects:

- intrinsic geometry,
- world-space interaction,
- realization-aware modelling,
- runtime projection,
- and operational evaluation.

Remark 30.54. Within AIM, runtime is not merely software execution. It denotes the dynamic evaluation layer between mathematical alignment models and physically interpreted railway states.

31. Runtime Services

31.1. Motivation

Remark 31.1. Railway alignment systems are not static collections of stored geometry.

Remark 31.2. Within AIM, geometry, frames, projections, realization effects, dynamic quantities, and vehicle-space behaviour are evaluated during runtime.

Remark 31.3. The runtime layer forms the operational bridge between:

- stored alignment semantics,
- metric-aware geometric evaluation,
- visualization,
- optimization,
- and interaction with real-world data.

Remark 31.4. Runtime therefore acts as the operational execution layer of the AIM framework.

31.2. Runtime Interpretation

Definition 31.5 (Runtime service)**.** A runtime service is an operator-based evaluation process acting on alignment states during operational use.

Remark 31.6. Runtime services evaluate:

- geometry,
- projections,
- frames,
- realization-dependent states,
- dynamic quantities,
- and operational vehicle-space behaviour.

Principle 31.7 (Runtime evaluation principle). Operational quantities should be evaluated from underlying state descriptions rather than stored redundantly.

Remark 31.8. Within AIM, runtime does not merely denote software execution. It denotes dynamic operational evaluation of geometric, physical, and spatial railway quantities.

31.3. Runtime State Evaluation

31.3.1. Evaluation Operator

Remark 31.9. Runtime evaluation is governed by the evaluation operator introduced in theorem 8.38.

Definition 31.10 (Runtime evaluation)**.** Runtime evaluation is represented by:

$$\mathcal{E} : \mathcal{X} \times \Omega_{\text{track}} \to \mathcal{Y}.$$

Remark 31.11. Here, $\mathcal{X}$ denotes railway state space, Ω_{track} denotes intrinsic track space, and $\mathcal{Y}$ denotes runtime-evaluated quantities.

Remark 31.12. Runtime evaluation may operate on:

- target states,
- realized states,
- measured states,
- or temporary interaction states.

31.3.2. Typical Runtime Quantities

Remark 31.13. Typical runtime quantities include:

- world coordinates,
- tangent directions,
- local frames,
- curvature,
- cant,
- geodesic curvature,
- dynamic quantities,
- and vehicle-space geometry.

31.4. Persistent State versus Runtime State

Remark 31.14. Within AIM, the persistent model intentionally stores sparse and semantically stable alignment information.

Remark 31.15. Operational geometry is reconstructed dynamically during runtime.

Principle 31.16 (Runtime reconstruction principle). Runtime geometry should be reconstructed from sparse alignment semantics rather than stored redundantly.

Remark 31.17. This avoids:

- synchronization instability,
- redundant geometry storage,
- and unnecessary realization inconsistencies.

31.5. pose3 as Runtime State

Remark 31.18. The minimal intrinsic pose3 notation is introduced in theorem 5.39.

Remark 31.19. Within runtime evaluation, pose3 is interpreted as a spatial operational state generated from:

- intrinsic pose2 reconstruction,
- profile evaluation,
- cant evaluation,
- metric realization,
- and frame construction.

Definition 31.20 (Runtime pose3 evaluation)**.** Runtime pose3 evaluation may be represented schematically as:

$$\mathrm{pose3}_{\mathrm{run}}(s) = \mathcal{E}_{\mathrm{pose3}}(x, s).$$

Remark 31.21. The runtime pose3 state is therefore not persistent truth. It is an evaluated operational state.

31.6. Lazy Evaluation

Definition 31.22 (Lazy evaluation)**.** Lazy evaluation computes quantities only when explicitly required.

Remark 31.23. This avoids unnecessary evaluation of expensive geometric operations.

Proposition 31.24 (Selective evaluation principle)**.** *Exact evaluation should be performed only where precision is required.*

Remark 31.25. This principle becomes essential for:

- world-to-track projection,
- large-scale visualization,

- optimization,
- interactive runtime systems,
- and vehicle-space evaluation.

31.7. Projection Services

Remark 31.26. Runtime projection services are governed by the projection operators introduced in:

- theorem 8.22,
- and theorem 8.24.

31.7.1. Track-to-World Evaluation

Definition 31.27 (Track-to-world runtime service)**.** The track-to-world runtime service evaluates:

$$\mathcal{P}_{\text{tw}} : \Omega_{\text{track}} \to \Omega_{\text{world}}.$$

31.7.2. World-to-Track Evaluation

Definition 31.28 (World-to-track runtime service)**.** The inverse runtime projection service evaluates:

$$\mathcal{P}_{\text{wt}} : \Omega_{\text{world}} \to \Omega_{\text{track}}.$$

Remark 31.29. The inverse projection is:

- nonlinear,
- potentially ambiguous,
- metric-dependent,
- context-dependent,
- and computationally expensive.

Remark 31.30. Projection must not be confused with CRS coordinate transformation. The distinction is established in theorem 8.27.

31.8. Frame Services

Remark 31.31. Runtime frame construction is governed by the frame operator introduced in theorem 8.41.

Definition 31.32 (Frame service)**.** The frame service evaluates:

$$\mathcal{F}(x_{\text{metric}}, s) = (\mathbf{t}(s), \mathbf{n}(s), \mathbf{b}(s)).$$

Remark 31.33. Frame services support:

- cant interpretation,
- pose3 evaluation,
- operational geometry,
- vehicle-space interpretation,
- visualization,
- and runtime dynamics.

Remark 31.34. Frame evaluation may depend on metric realization. For example, projected engineering frames and ellipsoid-attached frames need not coincide.

31.9. Dynamic Runtime Quantities

31.9.1. Operational Dynamics

Remark 31.35. Many operational railway quantities are runtime-derived rather than persistent alignment quantities.

31.9.2. Effective Lateral Acceleration

Definition 31.36 (Effective lateral acceleration)**.**

$$a_{\text{eff}} = v^2\kappa - g\sin\phi.$$

Remark 31.37. This is the simplified engineering relation introduced canonically in eq. (6.10).

31.9.3. Jerk

Definition 31.38 (Runtime jerk)**.**

$$j = v^3\kappa' - gv\cos\phi\,\phi'.$$

Remark 31.39. This is the runtime form of the canonical jerk relation in eq. (6.12).

Remark 31.40. These quantities emerge from:

- curvature evolution,
- cant evolution,

- runtime frame evaluation,
- metric realization,
- and vehicle-state interpretation.

31.10. Hybrid Runtime Evaluation

Principle 31.41 (Hybrid runtime principle)*.* Efficient runtime systems combine:

- approximate coarse evaluation,
- cached representations,
- and exact local refinement.

Remark 31.42*.* Typical hybrid strategies combine:

- sampled lookup structures,
- sparse parametric models,
- local numerical refinement,
- and context-dependent precision selection.

Remark 31.43*.* Hybrid runtime evaluation is especially important for world-to-track projection, frame construction, and large-scale visualization.

31.11. Level-of-Detail Runtime

Definition 31.44 (LOD-dependent evaluation)**.** Runtime evaluation depends on:

- scale,
- precision requirements,
- visibility,
- operational context,
- interaction state,
- and runtime cost.

Remark 31.45*.* Examples include:

- coarse map rendering,
- exact local projection,

- simplified distant geometry,
- high-precision construction-scale evaluation,
- and precise nearby vehicle interaction.

Remark 31.46. LOD-dependent runtime evaluation is one of the central scalability mechanisms of AIM.

31.12. Caching

Definition 31.47 (Runtime cache)**.** A runtime cache stores previously evaluated quantities to avoid repeated computation.

Remark 31.48. Caching may be applied to:

- projection results,
- local frames,
- sampled geometry,
- dynamic quantities,
- nearest-point queries,
- and metric-aware reconstruction results.

Remark 31.49. Caching introduces a trade-off between:

- computational cost,
- memory usage,
- consistency,
- invalidation complexity,
- and numerical freshness.

31.13. Metric-Aware Runtime

Remark 31.50. Runtime evaluation depends on the metric realization layer introduced in chapter 10.

Remark 31.51. Different metric realizations may affect:

- distance interpretation,
- curvature evaluation,

- projection behaviour,
- frame construction,
- stationing,
- and pose3 evaluation.

Principle 31.52 (Metric-aware runtime principle). Runtime systems should evaluate geometry through metric-aware operator services rather than through hardcoded Euclidean assumptions.

Remark 31.53. Metric-aware runtime preserves fast Euclidean engineering evaluation as the default case while allowing projection-aware and ellipsoid-aware extensions.

31.14. Runtime Pipelines

31.14.1. Conceptual Composition

Definition 31.54 (Runtime operator pipeline). A runtime pipeline is a composed operator chain such as:

$$\mathcal{E}_{\text{run}} \sim \mathcal{P} \circ \mathcal{F} \circ \mathcal{G}.$$

Remark 31.55. This pipeline expresses that runtime evaluation may depend on:

- metric realization,
- frame evaluation,
- and projection services.

31.14.2. Realized Runtime Evaluation

Definition 31.56 (Realized runtime pipeline). When realized geometry is required, runtime evaluation may conceptually depend on:

$$\mathcal{E}_{\text{real}} \sim \mathcal{P} \circ \mathcal{F} \circ \mathcal{G} \circ \mathcal{R}.$$

Remark 31.57. This expresses the dependency on physical realization, but it should not be read as a rigid software pipeline.

31.14.3. Runtime Pipeline Variants

Remark 31.58. Different runtime services may activate different operator chains. For example:

- visualization may use coarse metric evaluation,
- projection may require local refinement,

- optimization may require realized-state evaluation,
- and vehicle-space analysis may require frame and cant evaluation.

31.15. Runtime and Realization

Remark 31.59. Runtime systems should distinguish:

- target geometry,
- realized geometry,
- measured geometry,
- and temporary interaction geometry.

Remark 31.60. Operational services often operate on realized states:

$$\mathcal{X}_{\text{real}}.$$

Remark 31.61. Runtime evaluation and physical realization are coupled, but they are not identical. Runtime evaluates states; realization transforms target states into realized states.

31.16. Runtime and Vehicle Space

Remark 31.62. Vehicle-space interpretation requires runtime evaluation of:

- cant,
- local frames,
- vehicle envelopes,
- platform interaction,
- bogie motion,
- center-of-gravity motion,
- and operational clearances.

Remark 31.63. These quantities cannot be represented sufficiently by static centerline geometry alone.

Remark 31.64. Vehicle-space interpretation therefore depends on runtime pose3 evaluation rather than on stored centerline geometry alone.

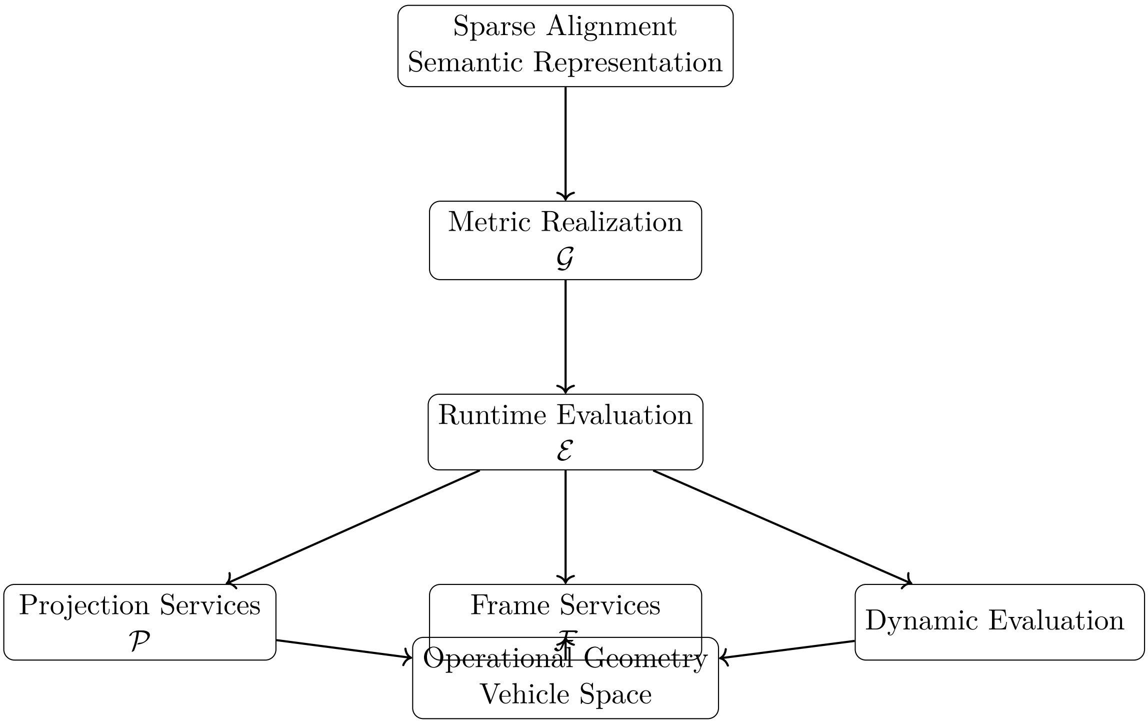

Abbildung 31.1.: Runtime services transform sparse alignment semantics through metric realization into operationally interpretable geometry and vehicle-space behaviour.

31.17. Conceptual Runtime Architecture

31.18. Canonical Interpretation

Proposition 31.65. *Within AIM, runtime services are operator-based evaluation systems acting on alignment states.*

Proposition 31.66. *Runtime geometry is interpreted as dynamically evaluated operational geometry rather than static stored geometry.*

Proposition 31.67. *Projection, metric realization, frame interpretation, dynamics, and realization-aware evaluation form one coupled runtime architecture.*

31.19. Connection to AIM

Summary 31.68. The AIM runtime architecture transforms sparse alignment semantics into operationally interpretable geometry.

Runtime services provide:

- projection,
- frame evaluation,

- pose3 evaluation,
- dynamic evaluation,
- realization-aware geometry,
- metric-aware evaluation,
- and vehicle-space interpretation.

Thus, runtime evaluation forms the operational execution layer of the AIM framework.

AIM is therefore not merely a stored alignment model, but an operational evaluation system.

Teil VII.

Reality and Data

Remark 31.69. This part addresses the relation between the mathematical model and the real world. Railway alignments are not only theoretical objects, but are embedded in coordinate systems, measurement processes, construction workflows, and imperfect data.

Remark 31.70. A realistic alignment framework must therefore account for:

- coordinate reference systems,
- measurement uncertainty,
- special railway elements,
- and the distinction between designed and realized geometry.

Remark 31.71. The following chapters develop this reality layer as an essential part of alignment-based information modelling.

32. Measurement Data and Reality

32.1. Motivation

Remark 32.1. In practical applications, alignment geometry is rarely available in a clean, analytic form. Instead, it is derived from heterogeneous data sources with varying levels of precision, structure, and semantic completeness.

Remark 32.2. Therefore, the modelling process must explicitly distinguish between

- observed data,
- interpreted structure,
- and reconstructed geometry.

—

32.2. Sources of Data

Definition 32.3 (Data source)**.** A data source is any origin of geometric or alignment-related information.

Remark 32.4. Typical sources include:

- survey measurements,
- legacy engineering data,
- container formats such as `LandXML`,
- railway-specific formats,
- point clouds and derived datasets.

—

32.3. Nature of Real-World Data

Remark 32.5. Real-world data exhibits several characteristic properties:

- **Incompleteness:** Not all required information is present.

- **Noise:** Measurements are subject to error.
- **Ambiguity:** Multiple interpretations may be possible.
- **Inconsistency:** Different parts of the data may contradict each other.

Remark 32.6. As a consequence, data cannot be directly equated with geometry.

—

32.4. Coordinate Systems

Remark 32.7. Geometric data is always embedded in a coordinate reference system (CRS).

Definition 32.8 (Coordinate reference system)**.** A coordinate reference system defines how numerical coordinates relate to positions in physical space.

Remark 32.9. In infrastructure applications, multiple systems may coexist, including:

- global systems (e.g. WGS84),
- projected systems (e.g. UTM, Gauss–Krüger),
- domain-specific systems (e.g. railway reference systems).

OPEN: Formal integration of CRS handling into the alignment model.
RESEARCH: Transformation consistency across heterogeneous data sources.

—

32.5. Interpretation Layer

Definition 32.10 (Interpretation)**.** Interpretation is the process of assigning structure and meaning to raw data.

Remark 32.11. Typical interpretation tasks include:

- identifying alignment segments,
- reconstructing curvature behaviour,
- associating auxiliary data such as profile or cant,
- resolving ambiguities in topology.

Remark 32.12. Interpretation is inherently non-unique and often requires domain knowledge.

OPEN: Formal separation between data and interpretation.

—

32.6. From Data to Sparse Model

Remark 32.13. The sparse alignment model provides a target structure for data interpretation.

Definition 32.14 (Reconstruction from data)**.** Reconstruction is the process of deriving a sparse alignment model from a given data source.

Remark 32.15. This process typically involves:

- segmentation of data,
- classification of element types,
- estimation of curvature laws,
- assignment of initial conditions.

TODO: Formal pipeline from raw data to sparse alignment.
OPEN: Handling of incomplete or conflicting input data.
—

32.7. Container Formats

Remark 32.16. Many real-world datasets are provided in structured container formats.

Remark 32.17. Examples include:

- `LandXML`,
- railway-specific XML formats,
- spreadsheet-based engineering datasets.

Remark 32.18. Such formats often mix geometric, semantic, and metadata in ways that are not directly aligned with the sparse model.

RESEARCH: Mapping of container formats to sparse representation.
TODO: Definition of intermediate formats (e.g. landFAT).
—

32.8. Uncertainty and Error

Remark 32.19. All real-world data is subject to uncertainty.

Definition 32.20 (Measurement uncertainty)**.** Measurement uncertainty describes the deviation between observed data and the true physical quantity.

Remark 32.21. Uncertainty propagates through:

- interpretation,
- reconstruction,
- numerical integration.

RESEARCH: Propagation of uncertainty through the reconstruction pipeline.
OPEN: Integration of uncertainty into the sparse model.
—

32.9. Topological vs Geometric Information

Remark 32.22. Data may contain both geometric and topological information.

Definition 32.23 (Geometric information)**.** Geometric information describes positions, directions, and curvature.

Definition 32.24 (Topological information)**.** Topological information describes connectivity and relationships between elements.

Remark 32.25. In many datasets, these two aspects are interwoven and not cleanly separated.

OPEN: Separation and integration of topology and geometry in the model.
—

32.10. Engineering Reality

Remark 32.26. Engineering data is often shaped by practical constraints rather than mathematical clarity.

Remark 32.27. Typical phenomena include:

- piecewise definitions without explicit continuity guarantees,
- implicit assumptions,
- domain-specific conventions,
- historical artefacts in legacy data.

RESEARCH: Systematic classification of real-world alignment data quality.
—

32.11. Role within the Overall Framework

Remark 32.28. The data layer connects the abstract modelling framework to real-world applications.

Remark 32.29. It forms the entry point for:

- import pipelines,
- validation procedures,
- reconstruction workflows.

—

32.12. Summary

Summary 32.30. Real-world alignment data is heterogeneous, noisy, and often ambiguous.

A clear separation between raw data, interpretation, and reconstructed geometry is essential.

The sparse alignment model provides a structured target for reconstruction, while numerical methods and domain knowledge are required to bridge the gap between data and geometry.

33. Coordinate Systems and Spatial Reference

33.1. Motivation

Remark 33.1. Railway alignment data is inherently spatial. Its interpretation depends fundamentally on the coordinate system in which it is expressed.

Remark 33.2. In practice, railway projects involve multiple coordinate reference systems (CRS), often used simultaneously.

Remark 33.3. Failure to handle coordinate systems consistently leads to:

- geometric inconsistencies,
- loss of accuracy,
- and incorrect integration of data sources.

CRS is not Metadata

> *In many software systems, the coordinate reference system (CRS) is treated as metadata.*
>
> *It is stored, displayed, and occasionally converted — but rarely considered part of the model itself.*
>
> *In railway alignment modelling, this view is fundamentally wrong.*
>
> *The CRS determines:*
>
> - *how distances are measured,*
> - *how curvature is interpreted,*
> - *and how different datasets relate to each other.*
>
> *A shift of a few centimeters in coordinates may appear negligible.*
>
> *A change in curvature is not.*
>
> *If coordinate systems are inconsistent:*
>
> - *alignments no longer match,*

- *curvature estimates become unstable,*
- *and optimization results lose physical meaning.*

The problem is not numerical.

It is structural.

A railway alignment does not exist independently of its spatial reference.

The coordinate system is part of the model.

33.2. Coordinate Reference Systems

Definition 33.4 (Coordinate reference system)**.** A coordinate reference system (CRS) is a mathematical framework that defines how spatial coordinates relate to positions in the real world.

Remark 33.5. A CRS typically consists of:

- a reference surface (e.g. ellipsoid),
- a coordinate system (e.g. Cartesian or projected),
- and a transformation to real-world locations.

33.3. Global and Local Systems

Remark 33.6. Railway data commonly uses both global and local coordinate systems:

- global systems (e.g. WGS84),
- projected systems (e.g. UTM, Gauss–Krüger),
- engineering systems (e.g. track-based coordinates).

Remark 33.7. Each system serves a different purpose:

- global systems enable interoperability,
- projected systems provide metric consistency,
- local systems support engineering workflows.

33.4. Engineering Coordinate Systems

Remark 33.8. In railway engineering, alignments are often described in track-relative coordinate systems.

Definition 33.9 (Stationing)**.** Let s denote the arc-length parameter along an alignment. This parameter is referred to as *stationing*.

Remark 33.10. Stationing defines a one-dimensional coordinate system along the track, independent of the global spatial reference.

Remark 33.11. Additional local coordinates may include:

- lateral offsets,
- vertical offsets,
- and local reference frames attached to the track.

33.5. Multiple CRS in Practice

Remark 33.12. Real-world projects frequently involve multiple CRS simultaneously.

Example 33.13. Typical combinations include:

- survey data in local engineering coordinates,
- GIS data in WGS84,
- design data in projected systems (UTM or GK).

Remark 33.14. These systems are often not perfectly consistent, due to:

- measurement errors,
- transformation approximations,
- and legacy definitions.

33.6. Transformations

Definition 33.15 (Coordinate transformation)**.** A coordinate transformation is a mapping

$$T : \mathbb{R}^n \to \mathbb{R}^n$$

that converts coordinates from one CRS to another.

Remark 33.16. Transformations may include:

- translations,
- rotations,
- scaling,

- and projection changes.

Remark 33.17. In practice, transformations are often defined by:

- reference points,
- parameter sets,
- or standardized transformation models.

33.7. Accuracy and Consistency

Remark 33.18. Coordinate transformations introduce uncertainties.

Remark 33.19. Even small inconsistencies can lead to:

- visible misalignment of datasets,
- incorrect curvature estimation,
- and unstable numerical reconstruction.

Remark 33.20. Therefore, a modelling framework must:

- track the CRS of all data,
- apply transformations explicitly,
- and avoid implicit assumptions.

33.8. CRS in Alignment Modelling

Remark 33.21. In a curvature-based alignment model, geometry is reconstructed from functions such as $\kappa(s)$.

Remark 33.22. This reconstruction requires a consistent spatial reference:

- initial position,
- initial orientation,
- and coordinate system definition.

Remark 33.23. Thus, CRS information is not auxiliary, but an essential component of the alignment model.

33.9. Challenges

Remark 33.24. The integration of CRS into alignment modelling raises several challenges:

- handling heterogeneous input data,
- maintaining numerical stability,
- supporting large coordinate values,
- and enabling local computations with high precision.

Remark 33.25. In particular, large global coordinates may lead to numerical issues in floating-point computations.

33.10. Local Coordinate Frames

Remark 33.26. To address numerical issues, local coordinate frames are often used.

Definition 33.27 (Local frame)**.** A local frame is a coordinate system defined relative to a reference point, typically near the area of interest.

Remark 33.28. Using local frames:

- improves numerical stability,
- simplifies geometric computations,
- and allows efficient visualization.

33.11. Towards a Coordinate-Aware Model

Remark 33.29. A modern alignment modelling framework must be coordinate-aware.

Remark 33.30. This includes:

- explicit CRS representation,
- consistent transformation handling,
- integration with geometry reconstruction,
- and support for local and global coordinates.

33.12. Connection to the AIM Framework

Summary 33.31. Coordinate systems are a fundamental aspect of railway alignment data.

A consistent treatment of CRS is essential for integrating real-world data, ensuring numerical stability, and enabling reliable modelling.

Within the AIM framework, CRS is treated as an explicit component of the alignment state, rather than as external metadata.

33.13. Alignment Modelling Beyond Map Projections

33.13.1. Motivation

Remark 33.32. Conventional railway alignment design is typically performed in planar coordinate systems derived from map projections, such as Gauss–Krüger, UTM, or Mercator.

Remark 33.33. These projections introduce distortions in length, angle, and area, which are controlled but fundamentally unavoidable.

Remark 33.34. For large-scale infrastructure or high-precision applications, these distortions become conceptually and computationally relevant.

33.13.2. Intrinsic Surface Formulation

Remark 33.35. An alternative perspective is to formulate alignment geometry directly on the Earth's reference surface, typically modeled as an ellipsoid.

Definition 33.36 (Surface-based alignment)**.** An alignment is defined as a curve

$$\gamma : s \mapsto \mathcal{E},$$

where $\mathcal{E}$ denotes the reference ellipsoid.

Remark 33.37. In this formulation, arc-length is measured intrinsically on the surface, and curvature is defined with respect to the surface geometry.

33.13.3. Geometric Consequences

Remark 33.38. Curves on the ellipsoid are no longer planar:

- geodesics replace straight lines,
- curvature must be interpreted in a surface-intrinsic sense,
- normal vectors depend on the local surface geometry.

Remark 33.39. Thus, the classical decomposition into planar geometry and vertical profile becomes insufficient.

33.13.4. Relation to AIM

Remark 33.40. The AIM framework is inherently curvature-based and therefore not tied to a specific embedding space.

Remark 33.41. In particular, the representation

$$\kappa(s)$$

remains valid on curved surfaces, provided curvature is defined with respect to the surface metric.

33.13.5. Practical Considerations

Remark 33.42. Despite its conceptual appeal, a full ellipsoidal formulation introduces significant complexity:

- nonlinear surface geometry,
- increased computational cost,
- limited compatibility with existing engineering workflows.

Remark 33.43. For this reason, planar projections remain the dominant approach in practice.

33.13.6. Alignment as Curve on a Surface

Remark 33.44. The Earth is modeled as a reference surface $\mathcal{E}$, typically an ellipsoid.

Definition 33.45 (Surface alignment)**.** An alignment can be interpreted as a curve

$$\gamma : s \mapsto \mathcal{E},$$

embedded in the surface.

Remark 33.46. In this setting, curvature decomposes into:

- intrinsic curvature (within the surface),
- normal curvature (with respect to embedding).

Remark 33.47. Straight lines in planar projections correspond to geodesics on the surface, which are not necessarily suitable as railway alignments.

Remark 33.48. Thus, alignment design is not equivalent to geodesic design.

33.13.7. Vision

Summary 33.49. AIM opens the possibility of alignment modelling directly on the Earth's reference surface.

While current engineering practice relies on planar projections, future systems may operate intrinsically on the ellipsoid, avoiding projection distortions and enabling globally consistent modelling.

This perspective represents a long-term research direction rather than an immediate engineering requirement.

Remark 33.50. In this sense, map projections can be understood as implementation choices rather than fundamental modelling constraints.

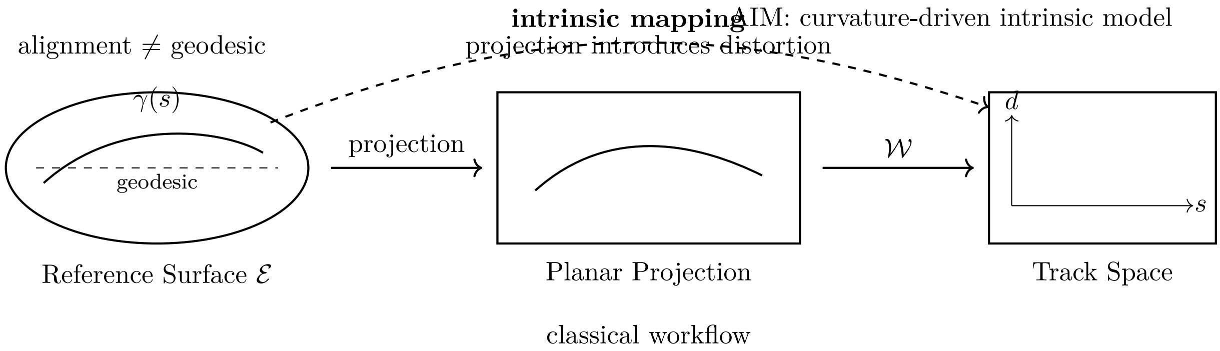

Abbildung 33.1.: Alignment modelling across geometric representations. On the reference surface, alignments are curves that generally differ from geodesics. Classical workflows project geometry to a plane before further processing, introducing distortions. The AIM perspective enables direct intrinsic modelling and mapping to track coordinates, independent of projection.

33.14. Physical Validity and Coordinate Systems

33.14.1. Current Practice

Remark 33.51. In current engineering workflows, railway alignment design and validation are typically performed in projected coordinate systems (e.g. Gauss–Krüger, UTM).

Remark 33.52. All geometric quantities, including curvature, distances, and derived dynamic measures, are evaluated within these planar representations.

Remark 33.53. However, such coordinate systems are approximations of the Earth's geometry and introduce distortions:

- metric distortion (scale),
- angular distortion,
- non-uniform curvature representation.

33.14.2. Fundamental Limitation

Proposition 33.54. *Physical quantities derived in a projected coordinate system are not strictly consistent with the underlying geometry of the Earth.*

Remark 33.55. In particular:

- arc-length is only approximately preserved,
- curvature is distorted by projection,
- dynamic quantities (e.g. acceleration, jerk) are evaluated on an approximate geometry.

Remark 33.56. Thus, current alignment validation is performed in an *engineering approximation space*, not in the physical space in which the railway actually exists.

33.14.3. Ellipsoid-Based Perspective

Remark 33.57. A more physically consistent approach is to formulate alignment geometry directly on a reference ellipsoid.

Remark 33.58. In this setting:

- arc-length is defined intrinsically,
- curvature is defined with respect to the surface geometry,
- transformations to world coordinates are exact (up to numerical precision).

33.14.4. Differential-Geometric Formulation

Definition 33.59 (Intrinsic alignment on a surface)**.** Let $\mathcal{E}$ denote a reference ellipsoid. An alignment is defined as a curve

$$\gamma : s \mapsto \mathcal{E},$$

with curvature defined intrinsically via differential geometry.

Remark 33.60. In this formulation:

- curvature is a geometric invariant,
- arc-length is physically meaningful,
- projection artifacts are eliminated.

33.14.5. Implication for Dynamics

Remark 33.61. Dynamic quantities such as lateral acceleration and jerk depend on curvature and arc-length derivatives.

Remark 33.62. If these are evaluated in a distorted coordinate system, the resulting values are only approximate.

Proposition 33.63. *Evaluating dynamics on an intrinsic (ellipsoid-based) geometry yields more physically accurate and reliable results.*

33.14.6. Safety Perspective

Remark 33.64. Railway alignment design directly affects safety-critical quantities:

- lateral acceleration,
- cant deficiency,
- jerk.

Remark 33.65. Errors introduced by coordinate distortions may be small individually, but can accumulate over long distances or in high-speed scenarios.

Proposition 33.66. *A geometrically consistent modelling framework improves the robustness and safety of alignment design.*

33.14.7. Vision

Remark 33.67. The long-term vision is to move from projection-based engineering geometry to intrinsic, surface-based modelling.

Remark 33.68. This implies:

- alignment design on the ellipsoid,
- differential-geometric formulation of curvature and dynamics,
- projection used only for visualization and data exchange.

Summary 33.69. Current railway alignment validation is performed in projected coordinate systems and is therefore only approximately physical.

A shift toward ellipsoid-based or differential-geometric modelling enables:

- physically consistent curvature,
- accurate dynamic evaluation,
- increased robustness,
- and ultimately safer alignment design.

This represents a fundamental extension of current engineering practice within the AIM framework.

Remark 33.70. Curvature evaluated in projected coordinate systems differs from intrinsic curvature on the ellipsoid.

See chapter 12.

34. Kilometre Line and Stationing

34.1. Motivation

Remark 34.1. Railway infrastructure is not only described geometrically, but also by stationing.

Remark 34.2. The kilometre line defines the longitudinal reference used for addressing positions along a railway route.

Remark 34.3. It must not be confused with the physical track centerline.

34.2. Definition

Definition 34.4 (Kilometre line)**.** A kilometre line is a longitudinal reference alignment that assigns a station value to positions along a railway route.

Remark 34.5. The kilometre line may coincide with a track centerline, but this is not required.

34.3. Relation to Track Alignments

Remark 34.6. Several track alignments may refer to the same kilometre line.

Remark 34.7. This is common in double-track lines, junctions, station areas, and switching zones.

Remark 34.8. In single-track cases, the kilometre line and the track centerline may be identical.

34.4. Outer Stationing

Remark 34.9. When the physical track alignment differs from the kilometre line, positions along the track must be related back to the kilometre reference.

Definition 34.10 (Outer stationing)**.** Outer stationing describes the mapping between a track alignment and an external kilometre reference.

34.5. Topological Role

Remark 34.11. The kilometre line acts as a topological and administrative reference rather than merely a geometric object.

Remark 34.12. It provides continuity across track changes, parallel tracks, and operationally relevant segments.

34.6. Implication for AIM

Summary 34.13. Within AIM, the kilometre line is treated as a reference structure separate from physical track alignments.

This distinction is essential for modelling multi-track situations, switches, stationing jumps, and real-world railway project data.

35. Kilometre Line, Stationing, and Spatial Reference

35.1. Motivation

Remark 35.1. Railway infrastructure is not only described by geometry, but also by stationing.

Remark 35.2. A point on a railway route is usually not addressed by global coordinates, but by a kilometre value along a reference line.

Remark 35.3. Thus, railway modelling must distinguish between:

- geometric location,
- spatial reference system,
- and operational addressing.

35.2. Three Reference Layers

Definition 35.4 (Spatial reference layer)**.** The spatial reference layer defines where geometry lives. It may be based on:

- a planar projection,
- a local engineering coordinate system,
- or, in a long-term view, a reference ellipsoid.

Definition 35.5 (Geometric alignment layer)**.** The geometric alignment layer describes the physical or designed track geometry, typically through curvature, pose, profile, and cant.

Definition 35.6 (Stationing layer)**.** The stationing layer assigns address values to positions along a railway route.

Summary 35.7. Spatial reference answers: where is it? Geometry answers: what shape is it? Stationing answers: how is it addressed?

35.3. Kilometre Line

Definition 35.8 (Kilometre line)**.** A kilometre line is a longitudinal reference structure used to assign station values to positions within a railway route or project.

Remark 35.9. The kilometre line may coincide with a physical track centerline, but this is not required.

Remark 35.10. It is therefore incorrect to treat the kilometre line simply as another track alignment.

35.4. Relation to Track Geometry

Remark 35.11. Several physical track alignments may refer to the same kilometre line.

Remark 35.12. This occurs in:

- double-track lines,
- station areas,
- junctions,
- switches and crossings,
- temporary or construction-stage alignments.

Remark 35.13. Conversely, in simple single-track situations, the kilometre line and track centerline may be identical.

35.5. Outer Stationing

Definition 35.14 (Outer stationing)**.** Outer stationing describes the mapping between a physical track alignment and an external kilometre reference.

Remark 35.15. Outer stationing is required whenever the track geometry and the kilometre reference do not coincide.

Remark 35.16. In such cases, a point on the track has at least two relevant longitudinal coordinates:

- its intrinsic track coordinate s_{track},
- its assigned kilometre value s_{km}.

35.6. Relation to Ellipsoid and CRS

Remark 35.17. The kilometre line is independent of the coordinate reference system in which the geometry is represented.

Remark 35.18. A kilometre value is not a projected coordinate. It is an addressing value along a railway-specific reference structure.

Remark 35.19. Thus, even if geometry is formulated on a reference ellipsoid, in a Gauss–Krüger projection, or in a local engineering CRS, the kilometre line remains a separate semantic layer.

35.7. Reference Separation Principle

Proposition 35.20. *A railway project requires separation of at least three reference concepts:*

$$\textit{spatial reference} \quad \neq \quad \textit{geometric alignment} \quad \neq \quad \textit{stationing reference}.$$

Remark 35.21. Confusing these layers leads to systematic modelling errors, especially in multi-track and junction situations.

35.8. Implication for AIM

Remark 35.22. Within AIM, the kilometre line is treated as a reference object, not as the primary physical track geometry.

Remark 35.23. Track alignments may be assigned to kilometre lines by explicit stationing relations.

Remark 35.24. This allows AIM to model cases where:

- one kilometre line serves multiple tracks,
- a track has no independent kilometre line,
- stationing jumps occur,
- geometric and operational continuity differ.

35.9. Relation to the Seven-Line Model

Remark 35.25. The kilometre line plays a central role in the seven-line import and sorting model.

Remark 35.26. A simplified structure is:

$$\text{km-line} \to \begin{cases} \text{right track} \to (\text{profile}, \text{cant}), \\ \text{left track} \to (\text{profile}, \text{cant}), \\ \text{outer stationing.} \end{cases}$$

Remark 35.27. This expresses that the kilometre line provides the addressing context, while the tracks provide the physical geometry and engineering state.

35.10. Key Insight

Proposition 35.28. *The kilometre line is not primarily a geometric curve, but an addressing structure for railway infrastructure.*

Summary 35.29. The km-line, the geometric alignment, and the spatial reference system serve different roles.

The km-line addresses railway infrastructure.

The alignment describes physical geometry.

The CRS or ellipsoid defines the spatial embedding.

Keeping these layers separate is essential for robust railway modelling, especially in multi-track, station, and junction contexts.

36. Measurement and Imperfect Data

36.1. Motivation

Remark 36.1. Railway alignment modelling is not performed in an ideal mathematical environment, but based on measured and legacy data.

Remark 36.2. Such data is inherently imperfect. It contains noise, inconsistencies, and incomplete information.

Remark 36.3. A realistic modelling framework must therefore account explicitly for data imperfections.

36.2. Sources of Data

Remark 36.4. Typical sources of railway alignment data include:

- geodetic surveys,
- track measurement systems,
- legacy design files,
- and extracted data from infrastructure models.

Remark 36.5. These sources differ significantly in:

- accuracy,
- resolution,
- completeness,
- and coordinate reference systems.

36.3. Types of Imperfections

36.3.1. Measurement Noise

Definition 36.6 (Noise)**.** Let $p_i \in \mathbb{R}^2$ denote measured points. Noise is the deviation between measured and true positions.

Remark 36.7. Noise arises from:

- instrument precision limits,
- environmental effects,
- and data processing steps.

36.3.2. Systematic Errors

Definition 36.8 (Systematic error)**.** A systematic error is a consistent deviation affecting a dataset.

Remark 36.9. Examples include:

- incorrect CRS parameters,
- transformation offsets,
- calibration errors.

36.3.3. Incompleteness

Remark 36.10. Data may be incomplete:

- missing segments,
- missing metadata,
- or missing structural information.

36.3.4. Topological Ambiguity

Remark 36.11. In complex railway systems, data may not explicitly encode topology.

Remark 36.12. This leads to ambiguities such as:

- unclear connectivity of track segments,
- duplicated or overlapping elements,
- inconsistent ordering of points.

36.4. Representation of Measured Geometry

Definition 36.13 (Polyline representation)**.** Measured alignments are often given as a sequence of points

$$P = (p_0, p_1, \dots, p_N), \quad p_i \in \mathbb{R}^2.$$

Remark 36.14. This representation is:

- simple,

- widely used,
- but not directly suitable for analysis or optimization.

Remark 36.15. In particular, curvature is not explicitly given, but must be inferred.

36.5. From Data to Model

Remark 36.16. A key task in alignment modelling is the transformation:

$$\text{raw data} \quad \longrightarrow \quad \text{structured model.}$$

Remark 36.17. This involves:

- filtering noise,
- reconstructing geometry,
- identifying structural elements,
- and assigning semantic meaning.

36.6. Curvature Estimation

Definition 36.18 (Discrete curvature)**.** Given three consecutive points, curvature may be approximated by

$$\kappa_i \approx \frac{\theta_{i+1} - \theta_i}{\Delta s_i},$$

where θ_i denotes the local direction.

Remark 36.19. This approximation is sensitive to noise and sampling density.

Remark 36.20. Therefore, direct use of raw curvature estimates is often unstable.

36.7. Smoothing and Filtering

Remark 36.21. To obtain usable curvature information, smoothing is required.

Definition 36.22 (Smoothing operator)**.** A smoothing operator transforms noisy data into a more regular form:

$$\kappa_i \mapsto \tilde{\kappa}_i.$$

Remark 36.23. Common techniques include:

- moving averages,
- Gaussian filtering,
- spline fitting,
- and regularization methods.

36.8. Model Reconstruction

Remark 36.24. The goal of reconstruction is to obtain a structured alignment model from imperfect data.

Remark 36.25. This includes:

- determining curvature functions,
- identifying transition regions,
- and constructing a consistent representation.

Remark 36.26. Reconstruction is not unique and depends on modelling assumptions.

36.9. Uncertainty

Definition 36.27 (Uncertainty)**.** Uncertainty describes the lack of exact knowledge about the true alignment.

Remark 36.28. Uncertainty arises from:

- measurement errors,
- incomplete data,
- and modelling assumptions.

Remark 36.29. A robust modelling framework should:

- tolerate uncertainty,
- avoid overfitting,
- and provide stable results.

36.10. Impact on Optimization

Remark 36.30. Imperfect data directly affects optimization:

- objective functions may reflect noise,
- constraints may be violated by measurement errors,
- and solutions may become unstable.

Remark 36.31. Therefore, optimization must account for uncertainty, for example by:

- regularization,
- robust objective functions,
- or probabilistic models.

36.11. Engineering Perspective

Remark 36.32. In practice, engineers routinely work with imperfect data.

Remark 36.33. This includes:

- interpreting measurement results,
- reconciling inconsistent datasets,
- and making design decisions under uncertainty.

Remark 36.34. A modelling framework should support these tasks rather than assume ideal data.

36.12. Towards Data-Aware Modelling

Remark 36.35. A modern alignment model must be data-aware.

Remark 36.36. This includes:

- explicit handling of noise,
- robust reconstruction methods,
- integration with coordinate systems,
- and compatibility with real-world workflows.

36.13. Connection to the AIM Framework

Summary 36.37. Railway alignment data is inherently imperfect.

A realistic modelling framework must therefore incorporate measurement noise, incomplete data, and uncertainty.

Within the AIM framework, data is not assumed to be ideal, but is treated as an integral part of the modelling process.

37. Topology and Special Elements

37.1. Motivation

Remark 37.1. Railway systems are not isolated curves, but interconnected networks.

Remark 37.2. While mathematical models often focus on single alignments, real-world railway infrastructure consists of:

- branching tracks,
- merging lines,
- parallel alignments,
- and complex switching structures.

Remark 37.3. A complete modelling framework must therefore go beyond single curves and incorporate topology.

37.2. From Curves to Networks

Definition 37.4 (Alignment)**.** An alignment is a parametrized curve

$$\gamma : [0, L] \to \mathbb{R}^2.$$

Definition 37.5 (Railway network)**.** A railway network is a collection of alignments together with a set of connectivity relations.

Remark 37.6. The transition from a single alignment to a network introduces topological structure in addition to geometry.

37.3. Topological Relations

Remark 37.7. Alignments in a network may be related by:

- continuation (end-to-start connections),
- branching (one alignment splits into several),
- merging (multiple alignments join),

- parallel relations (tracks running side by side).

Definition 37.8 (Connection point)**.** A connection point is a location where two or more alignments meet.

Remark 37.9. At connection points, geometric and dynamic consistency conditions must be satisfied.

37.4. Switches and Turnouts

Remark 37.10. Switches (turnouts) are fundamental elements of railway networks.

Definition 37.11 (Switch)**.** A switch is a structure that allows a vehicle to move from one alignment to another.

Remark 37.12. Switches introduce:

- branching geometry,
- nontrivial curvature transitions,
- and constraints on feasible trajectories.

Remark 37.13. Unlike simple transitions, switches cannot be described by a single curvature function.

37.5. Geometric Complexity

Remark 37.14. Special elements often exhibit geometric properties that differ from standard alignment elements:

- discontinuities in curvature,
- piecewise-defined geometry,
- and locally non-smooth behaviour.

Remark 37.15. These properties must be handled explicitly in a modelling framework.

37.6. Topological Ambiguity in Data

Remark 37.16. Real-world data often does not explicitly encode topology.

Remark 37.17. Typical issues include:

- missing connectivity information,
- duplicated segments,
- and inconsistent ordering of elements.

Remark 37.18. Therefore, topology must often be inferred from geometric and contextual information.

37.7. Representation of Networks

Definition 37.19 (Graph representation)**.** A railway network can be represented as a graph:

- nodes represent connection points,
- edges represent alignments.

Remark 37.20*.* This abstraction separates:

- topology (graph structure),
- and geometry (alignment shape).

37.8. Coupling of Topology and Geometry

Remark 37.21*.* Although topology and geometry can be separated conceptually, they are strongly coupled in practice.

Remark 37.22*.* For example:

- curvature constraints depend on connectivity,
- feasible transitions depend on network structure,
- and optimization must consider multiple interacting alignments.

37.9. Special Engineering Cases

Remark 37.23*.* Railway systems include a variety of special elements, such as:

- switches and crossings,
- sidings and yards,
- station layouts,
- and maintenance tracks.

Remark 37.24*.* These elements often violate assumptions of standard alignment models:

- unique parameterization by arc length,
- smooth curvature evolution,
- and single-track continuity.

37.10. Implications for Modelling

Remark 37.25. The presence of topology and special elements implies that:

- alignment models must support multiple connected curves,
- curvature-based descriptions must be extended,
- and reconstruction algorithms must handle branching structures.

Remark 37.26. In particular, optimization problems must be formulated on networks rather than individual curves.

37.11. Towards Topology-Aware Modelling

Remark 37.27. A topology-aware modelling framework must include:

- explicit representation of connectivity,
- integration of geometry and topology,
- support for special elements,
- and compatibility with real-world data.

37.12. Connection to the AIM Framework

Summary 37.28. Railway systems are inherently network-based and include complex special elements such as switches.

A complete modelling framework must therefore extend beyond single alignments and incorporate topology.

Within the AIM framework, topology is treated as an integral component of the alignment model, enabling consistent representation and optimization of railway networks.

38. Construction and Realization

38.1. Motivation

Remark 38.1. Theoretical railway alignments are defined as continuous geometric and curvature objects.

Remark 38.2. In practice, tracks are not constructed as continuous mathematical curves. They are realized through discrete construction and maintenance processes, local measurements, machine operations, correction limits, and structural response of the track.

Proposition 38.3. *The curve that is constructed is not identical to the curve that is designed.*

Remark 38.4. A realistic modelling framework must therefore include an explicit realization layer between analytical design and built geometry.

Principle 38.5 (Realization principle). Within AIM, realization is treated as an operator layer transforming target alignment states into physically achievable railway states.

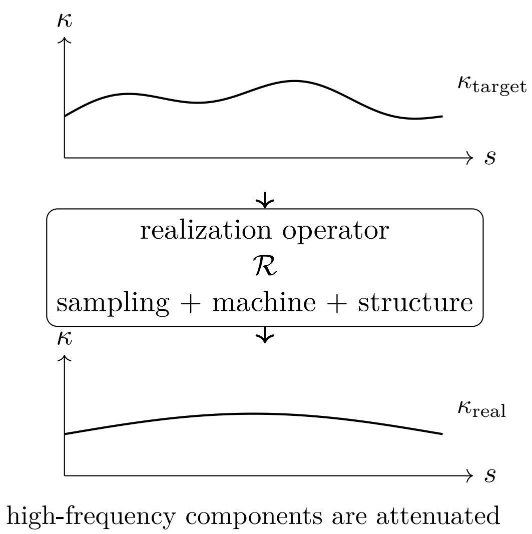

Abbildung 38.1.: Realization acts as a filtering process: target curvature may contain high-frequency components that are attenuated by sampling, machine action, and structural response.

38.2. Realized Alignment

Definition 38.6 (Realized alignment)**.** The realized alignment is the physically constructed railway geometry, denoted by

$$\gamma_{\mathrm{real}}(s).$$

Remark 38.7. It may be represented continuously, but it is usually generated from discrete construction, measurement, and maintenance data.

Definition 38.8 (Realized curvature)**.** The realized curvature is:

$$\kappa_{\mathrm{real}}(s) = \frac{\mathrm{d}\theta_{\mathrm{real}}}{\mathrm{d}s}.$$

Remark 38.9. The realized track should not be judged only by positional agreement, but also by preservation of curvature evolution.

Remark 38.10. Within AIM, realized geometry may be interpreted as:

$$x_{\mathrm{real}} = \mathcal{R}(x_{\mathrm{target}}),$$

where $\mathcal{R}$ denotes the realization operator.

38.3. Realization Error

Definition 38.11 (Curvature realization error)**.** A curvature-based realization error may be written as:

$$E_\kappa = \int_0^L \left(\kappa_{\mathrm{real}} - \kappa_{\mathrm{target}}\right)^2 \mathrm{d}s.$$

Definition 38.12 (Curvature derivative error)**.** A higher-order realization error may be written as:

$$E_{\kappa'} = \int_0^L \left(\kappa'_{\mathrm{real}} - \kappa'_{\mathrm{target}}\right)^2 \mathrm{d}s.$$

Remark 38.13. These measures reflect that different transition families may be close in position space while differing significantly in curvature evolution and dynamic interpretation.

Remark 38.14. Realization error may therefore be evaluated in:

- position space,
- tangent space,
- curvature space,
- curvature-derivative space,
- and runtime pose3 space.

38.4. Sampling and Control Points

Remark 38.15. Construction and maintenance operate through discrete control data.

Definition 38.16 (Control points)**.** Let

$$0 = s_0 < s_1 < \cdots < s_N = L$$

be a strictly increasing sequence of arc-length positions. These positions are called control points.

Remark 38.17. The realized alignment may be reconstructed from these points using:

- piecewise linear interpolation,
- piecewise polynomial interpolation,
- spline-based reconstruction,
- machine-specific reconstruction rules,
- or sparse alignment reconstruction.

Proposition 38.18 (Adaptive sampling principle)**.** *Sampling density should increase in regions of strong curvature variation:*

$$\rho(s) \sim f\big(|\kappa'(s)|, |\kappa''(s)|\big),$$

for a suitable increasing function f.

Remark 38.19. Nearly constant-curvature regions require fewer control points, whereas transition zones require denser control.

38.5. Adjustment as Iterative Process

Remark 38.20. In railway construction and maintenance, track geometry is adjusted through a sequence of localized operations, for example by tamping machines.

Remark 38.21. The canonical adjustment operator is denoted by $\mathcal{A}$, as introduced in theorem 8.53.

Definition 38.22 (Iterative realization)**.** Starting from an initial alignment γ_0, the iterative process is:

$$\gamma_{k+1} = \mathcal{A}_{\gamma_{\text{target}}}(\gamma_k).$$

Remark 38.23. The operator depends on local measurement data, machine parameters, correction limits, and physical properties of the track.

Definition 38.24 (Realized alignment as limit)**.** If the sequence converges, the realized alignment is:

$$\gamma_{\mathrm{real}} = \lim_{k\to\infty} \gamma_k.$$

Proposition 38.25 (Fixed point interpretation)**.** *If the iterative realization converges, then:*

$$\gamma_{\mathrm{real}} = \mathcal{A}_{\gamma_{\mathrm{target}}}(\gamma_{\mathrm{real}}).$$

Remark 38.26. In practice, convergence cannot be assumed a priori, since it depends on measurement feedback, machine limits, and local track conditions.

38.6. Locality and Correction Limits

Definition 38.27 (Local correction)**.** A guided adjustment may be written as:

$$\mathcal{A}_{\gamma_{\mathrm{target}}}(\gamma) = \gamma + \Delta(\gamma, \gamma_{\mathrm{target}}),$$

where Δ is a local correction operator.

Remark 38.28. Typically, corrections are bounded:

$$\|\Delta(\gamma, \gamma_{\mathrm{target}})\| \leq \Delta_{\max}.$$

Proposition 38.29 (Locality)**.** *The adjustment operator is local in the sense that:*

$$(\mathcal{A}_{\gamma_{\mathrm{target}}}\gamma)(s)$$

depends only on γ and γ_{target} in a neighbourhood of s.

Remark 38.30. This reflects the finite working range of construction and maintenance equipment.

38.7. Kernel-Based Realization Model

Definition 38.31 (Kernel-based adjustment)**.** A linearized model of the adjustment process is:

$$(\mathcal{A}_{\gamma_{\mathrm{target}}}\gamma)(s) = \gamma(s) + \int_I K(s,\sigma)\big(\gamma_{\mathrm{target}}(\sigma) - \gamma(\sigma)\big)\,\mathrm{d}\sigma,$$

where $K(s,\sigma)$ is a localized influence kernel.

Remark 38.32. The kernel represents the spatial influence of local construction or maintenance action.

Proposition 38.33 (Localization)**.** *There exists a characteristic length $\ell > 0$ such that:*

$$K(s,\sigma) \approx 0 \qquad \text{for } |s-\sigma| > \ell.$$

Definition 38.34 (Discrete adjustment model)**.** For control points s_i, the kernel model becomes:

$$\gamma_i^{(k+1)} = \gamma_i^{(k)} + \sum_j K_{ij} \big(\gamma_j^{\text{target}} - \gamma_j^{(k)}\big).$$

Remark 38.35*.* This form is compatible with sparse numerical implementation.

Remark 38.36*.* The matrix K_{ij} should not be interpreted merely as numerical smoothing. It represents physical influence: machine action, track-bed response, sleeper spacing, stiffness, local constraints, and measurement feedback.

38.8. Filter Interpretation

Proposition 38.37. *The realization process acts approximately as a low-pass filter on curvature.*

Remark 38.38*.* High-frequency curvature components are attenuated during construction and maintenance, while low-frequency structure is preserved.

Remark 38.39*.* This explains why transition curves with similar low-order behaviour may become nearly indistinguishable in position space after realization, while still differing in curvature evolution and dynamic interpretation.

Remark 38.40*.* In frequency-oriented notation, realization may be idealized as:

$$\widehat{\kappa}_{\text{real}}(\omega) \approx H_{\mathcal{R}}(\omega)\widehat{\kappa}_{\text{target}}(\omega),$$

where $H_{\mathcal{R}}$ denotes a realization transfer function.

Remark 38.41*.* For typical physical realization processes, $H_{\mathcal{R}}$ attenuates high-frequency components.

Principle 38.42 (Realization filter principle)*.* Construction and maintenance do not merely approximate target geometry; they transform it through a physical filter induced by sampling, machinery, structure, and correction limits.

38.9. Relation to the Realization Operator

Remark 38.43*.* In the AIM operator framework, realization is represented by:

$$\mathcal{R} : \mathcal{X}_{\text{target}} \to \mathcal{X}_{\text{real}},$$

as introduced in theorem 8.55.

Remark 38.44*.* The construction process described in this chapter provides one interpretation of how such an operator may arise physically.

Remark 38.45. The simplified realization decomposition:

$$\mathcal{R} \approx \mathcal{M} \circ \mathcal{I} \circ \mathcal{S}$$

separates sampling, reconstruction, and mechanical response.

Remark 38.46. Here:

- $\mathcal{S}$ denotes sampling or control-point extraction,
- $\mathcal{I}$ denotes interpolation or reconstruction,
- $\mathcal{M}$ denotes mechanical and structural response.

Remark 38.47. More generally, $\mathcal{R}$ may depend on:

- machinery,
- measurement process,
- construction method,
- track structure,
- maintenance history,
- local constraints,
- and environmental conditions.

38.10. Sparse Compatibility

Remark 38.48. The realization layer must remain compatible with sparse alignment representation.

Remark 38.49. Within AIM, this means that realization may operate on:

- sparse target parameters,
- sampled runtime states,
- reconstructed curvature functions,
- or realized sparse approximations.

Principle 38.50 (Sparse realization compatibility). Realization-aware modelling must not destroy the sparse structure of the alignment model.

Instead, realized geometry should be interpretable either as a runtime state, a sampled state, or a sparse-compatible approximation.

Remark 38.51. A realized alignment may therefore be represented as:

- measured sample data,
- a reconstructed dense curve,
- a sparse re-fit,
- or a hybrid measured–sparse object.

38.11. Realization-Aware Optimization

Remark 38.52. Classical design optimizes the target curve.

Remark 38.53. Realization-aware design optimizes the target curve with respect to its expected realized outcome.

Definition 38.54 (Realization-aware objective)**.** A realization-aware optimization objective may be written as:

$$\min_{x_{\text{target}}} J(\mathcal{R}(x_{\text{target}})).$$

Remark 38.55. Here, the design variable is the analytical target, while the evaluated object is the realized railway state.

Remark 38.56. This distinction is essential whenever construction, maintenance, machine response, or structural filtering significantly affect the final geometry.

Proposition 38.57 (Realization-aware design principle)**.** *The best target alignment is not necessarily the analytically smoothest alignment, but the target whose realized state best satisfies the operational objective.*

38.12. Design Implication

Remark 38.58. The designed transition curve is not simply the curve that is built. It is a target that produces a realized behaviour after construction and maintenance.

Remark 38.59. Thus, alignment design implicitly includes assumptions about construction, machine behaviour, and structural response.

Proposition 38.60. *The relevant physical design space is the image of the realization operator, not the full space of analytical curvature laws.*

Remark 38.61. This connects construction and realization to realization-aware optimization.

Remark 38.62. For AIM, this means that transition classification, sparse alignment modelling, runtime pose3 evaluation, and optimization must be interpreted not only in analytical target space, but also in realized physical state space.

38.13. Connection to AIM

Summary 38.63. Construction and realization describe the transformation from analytical target alignment to physically realized railway geometry.

Within AIM, this transformation is treated as an explicit operator layer:

$$x_{\text{real}} = \mathcal{R}(x_{\text{target}}).$$

The realization layer explains why physical railway geometry must be understood through construction processes, measurement feedback, local correction limits, machine behaviour, sparse reconstruction, and mechanical response rather than through analytical geometry alone.

Realization is therefore not an implementation detail. It is a central mathematical and physical layer connecting:

- target geometry,
- sparse alignment representation,
- measured reality,
- runtime state evaluation,
- and realization-aware optimization.

39. Realized Pose3 and Dynamic Realization

39.1. Motivation

Remark 39.1. Classical alignment theory typically assumes that the designed geometry is identical to the realized geometry.

Remark 39.2. In reality, railway tracks are produced through iterative and physically constrained realization processes.

Remark 39.3. As a consequence, not only the geometric position of the track differs from the theoretical target alignment, but also its orientation, curvature, cant, and dynamic behaviour.

Remark 39.4. Thus, realization affects the full spatial railway state rather than geometry alone.

Remark 39.5. This motivates the introduction of a realized pose3 state:

$$\mathrm{pose3}_{\mathrm{real}}(s).$$

Principle 39.6 (Realized-state principle). Within AIM, the physically relevant railway state is the realized runtime state, not merely the analytical target state.

39.2. Target and Realized States

Definition 39.7 (Target pose3)**.** The target railway state is defined as

$$\mathrm{pose3}_{\mathrm{target}}(s) = \big(\Gamma_{\mathrm{target}}(s), F_{\mathrm{rail,target}}(s), \phi_{\mathrm{target}}(s)\big).$$

Definition 39.8 (Realized pose3)**.** The realized railway state is defined as

$$\mathrm{pose3}_{\mathrm{real}}(s) = \big(\Gamma_{\mathrm{real}}(s), F_{\mathrm{rail,real}}(s), \phi_{\mathrm{real}}(s)\big).$$

Remark 39.9. The realized state is the physically constructed, measurable, and operationally effective state of the railway track.

Remark 39.10. In practice,

$$\mathrm{pose3}_{\mathrm{real}} \neq \mathrm{pose3}_{\mathrm{target}}.$$

Remark 39.11. The difference is not limited to position. It affects tangent direction, curvature, cant, frame orientation, and derived dynamic quantities.

39.3. Measured States

Definition 39.12 (Measured pose3 state)**.** A measured pose3 state is an observation-derived approximation of the realized railway state:

$$\mathrm{pose3}_{\mathrm{meas}}(s_i) \approx \mathrm{pose3}_{\mathrm{real}}(s_i).$$

Remark 39.13. Measured pose3 states are usually available only at discrete stations, timestamps, or sensor positions.

Remark 39.14. Measurement may provide direct or indirect information about:

- position,
- tangent direction,
- curvature,
- cant,
- gauge,
- vertical profile,
- acceleration,
- and local frame orientation.

Remark 39.15. Measured states are not identical to realized states. They include sensor noise, sampling effects, calibration errors, filtering effects, and interpretation assumptions.

Principle 39.16 (Measurement separation principle). AIM distinguishes target state, realized state, and measured state as separate conceptual layers.

39.4. Sources of Realization Deviations

Remark 39.17. Deviations between target and realized state arise from:

- measurement uncertainty,
- machine limitations,
- discrete control points,
- structural elasticity,
- ballast behaviour,
- rail bending,
- settlement processes,

- maintenance history,
- and operational degradation.

Remark 39.18. Thus, realization is not a purely geometric transformation, but a mechanical and operational process.

39.5. Realized Geometry

Definition 39.19 (Realized spatial trajectory)**.** The realized spatial trajectory is denoted by:

$$\Gamma_{\text{real}}(s).$$

Remark 39.20. $\Gamma_{\text{real}}(s)$ represents the physically realized railway reference trajectory after construction, maintenance, settlement, and operational loading.

Remark 39.21. It may be represented as:

- discrete measured samples,
- a reconstructed dense curve,
- a sparse re-fit,
- or a hybrid measured–analytical object.

Remark 39.22. Within AIM, realized geometry remains compatible with sparse modelling by being interpreted either as reconstructed runtime state or as a sparse-compatible approximation.

39.6. Runtime Reconstruction

Remark 39.23. Realized pose3 is generally not stored directly as a complete continuous object.

Remark 39.24. Instead, it is reconstructed at runtime from:

- measured samples,
- sparse target geometry,
- realization models,
- frame operators,
- cant reconstruction,
- profile reconstruction,
- and metric context.

Definition 39.25 (Runtime realization reconstruction)**.** Runtime reconstruction of realized pose3 may be written abstractly as:

$$\mathrm{pose3}_{\mathrm{real}}(s) = \mathcal{E}_{\mathrm{real}}(s, x_{\mathrm{target}}, m, \mathcal{R}, \mathcal{C}),$$

where m denotes measured data, $\mathcal{R}$ denotes realization model information, and $\mathcal{C}$ denotes coordinate and metric context.

Remark 39.26*.* The operator $\mathcal{E}_{\mathrm{real}}$ is a runtime evaluation operator rather than a persistent storage object.

39.7. Realized Curvature and Cant

Definition 39.27 (Realized curvature)**.** The realized curvature is:

$$\kappa_{\mathrm{real}}(s) = \frac{\mathrm{d}\theta_{\mathrm{real}}}{\mathrm{d}s}.$$

Definition 39.28 (Realized cant)**.** The realized cant angle is:

$$\phi_{\mathrm{real}}(s).$$

Remark 39.29*.* Both quantities differ from their target values due to realization effects.

Remark 39.30*.* In particular,

$$\kappa_{\mathrm{real}} \neq \kappa_{\mathrm{target}}, \qquad \phi_{\mathrm{real}} \neq \phi_{\mathrm{target}}.$$

Remark 39.31*.* Measured curvature and measured cant are further approximations:

$$\kappa_{\mathrm{meas}} \approx \kappa_{\mathrm{real}}, \qquad \phi_{\mathrm{meas}} \approx \phi_{\mathrm{real}}.$$

39.8. Cant Realization

Remark 39.32*.* Cant realization is particularly sensitive because cant is not only a geometric quantity but also a dynamically relevant operational state.

Remark 39.33*.* Cant realization is affected by:

- tamping accuracy,
- rail fastening systems,
- ballast stiffness,
- torsion limits,
- local support conditions,

- measurement procedures,
- and sensor interpretation.

Remark 39.34. In practice, cant ramps are realized only approximately and may exhibit local irregularities.

Remark 39.35. This is especially relevant in:

- transition zones,
- switches and crossings,
- station areas,
- overlapping transition regions,
- and locally constrained construction zones.

39.9. Measurement Ambiguity

Remark 39.36. Measured railway data does not automatically define a unique realized pose3 state.

Remark 39.37. Ambiguities may arise from:

- uncertain station assignment,
- CRS uncertainty,
- projection ambiguity,
- sensor offset from the reference line,
- vehicle motion,
- filtering by onboard systems,
- incomplete cant or profile information,
- and topology ambiguity in multi-track environments.

Remark 39.38. Therefore, measurement interpretation is itself a runtime reconstruction problem.

Principle 39.39 (Measured-state ambiguity). Measured data must be interpreted as evidence for realized pose3, not as realized pose3 itself.

39.10. Sensor Integration

Remark 39.40. Different sensors observe different projections of the realized railway state.

Remark 39.41. Examples include:

- surveying points,
- track geometry cars,
- inertial measurement units,
- GNSS receivers,
- laser scanning,
- photogrammetry,
- gauge and cant measurement systems,
- and vehicle-mounted acceleration sensors.

Remark 39.42. Sensor integration requires mapping heterogeneous observations into a common runtime state interpretation.

Definition 39.43 (Sensor observation model)**.** A sensor observation may be written as:

$$y_i = \mathcal{H}_i(\text{pose3}_{\text{real}}(s_i)) + \varepsilon_i,$$

where $\mathcal{H}_i$ denotes the sensor observation operator and ε_i denotes measurement error.

Remark 39.44. The reconstruction task is therefore an inverse problem: estimate the realized pose3 state from heterogeneous noisy observations.

39.11. Coupling of Curvature and Cant

Remark 39.45. Curvature and cant are dynamically coupled quantities.

Remark 39.46. Therefore, realization errors in curvature and cant cannot be treated independently.

Remark 39.47. The physically relevant quantity is the realized dynamic state:

$$(\kappa_{\text{real}}, \phi_{\text{real}}).$$

Definition 39.48 (Realized equilibrium condition)**.** A realized track is dynamically balanced if:

$$v^2 \kappa_{\text{real}}(s) = g \sin \phi_{\text{real}}(s).$$

Remark 39.49. In practice, deviations from equilibrium are unavoidable and constrained only within admissible limits.

39.12. Dynamic Realization Error

Definition 39.50 (Realized effective lateral acceleration)**.** The realized effective lateral acceleration is:

$$a_{\text{eff,real}}(s) = v^2 \kappa_{\text{real}}(s) - g \sin \phi_{\text{real}}(s).$$

Remark 39.51. This quantity is operationally more relevant than geometric deviation alone.

Definition 39.52 (Realized jerk)**.** The realized jerk is:

$$j_{\text{real}}(s) = v^3 \kappa'_{\text{real}}(s) - gv \cos \phi_{\text{real}}(s)\, \phi'_{\text{real}}(s).$$

Remark 39.53. Passenger comfort depends directly on the realized jerk rather than on the analytical target functions.

39.13. Realization as State Transformation

Definition 39.54 (Pose3 realization operator)**.** The realization process may be written as:

$$\mathcal{R}_{\text{pose3}} : \text{pose3}_{\text{target}} \mapsto \text{pose3}_{\text{real}}.$$

Remark 39.55. This operator includes:

- geometric realization,
- cant realization,
- profile realization,
- mechanical filtering,
- measurement feedback,
- and operational constraints.

Remark 39.56. Thus, realization acts on the full railway state rather than only on position space.

Remark 39.57. In practice, $\mathcal{R}_{\text{pose3}}$ is not directly known. It is approximated through construction knowledge, measurement data, machine models, and runtime reconstruction.

39.14. Filtering Interpretation

Proposition 39.58. *The realization operator acts as a low-pass filter on:*

- *curvature,*
- *cant,*

- *profile irregularities,*
- *and higher-order derivatives.*

Remark 39.59. High-frequency components are attenuated through:

- machine limitations,
- structural smoothing,
- ballast mechanics,
- vehicle-track interaction,
- and measurement discretization.

Remark 39.60. Therefore, highly detailed analytical target states may collapse into similar realized states.

39.15. Operational Interpretation

Remark 39.61. Railway operation interacts with the realized state rather than with the theoretical target geometry.

Remark 39.62. Consequently, the relevant engineering question is not:

> "Was the theoretical geometry built exactly?"

but rather:

> "Does the realized state produce the intended dynamic behaviour?"

Remark 39.63. This makes realized pose3 the natural bridge between construction, measurement, runtime evaluation, and operation.

39.16. Connection to Optimization

Remark 39.64. Optimization should therefore operate on realized states:

$$J = J(\mathrm{pose3}_{\mathrm{real}}) = J(\mathcal{R}_{\mathrm{pose3}}(\mathrm{pose3}_{\mathrm{target}})).$$

Remark 39.65. This extends classical alignment optimization toward realization-aware optimization.

Remark 39.66. When measured states are available, optimization may additionally use:

$$J = J(\mathrm{pose3}_{\mathrm{real}}, \mathrm{pose3}_{\mathrm{meas}}),$$

to calibrate realization models or identify maintenance needs.

39.17. Connection to AIM

Summary 39.67. Real railway tracks are not analytical curves but realized physical states.

The realization process acts on the complete pose3 state, including geometry, curvature, profile, frame orientation, and cant.

Measured railway data provides evidence for this realized state, but it does not define it uniquely. Therefore, AIM distinguishes target pose3, realized pose3, and measured pose3 as separate layers.

Within the AIM framework, realized pose3 forms the bridge between:

- analytical geometry,
- runtime reconstruction,
- sensor integration,
- construction and maintenance,
- physical realization,
- and operational dynamics.

Thus, railway alignment design must be understood as the design of a realizable dynamic state rather than merely a geometric target curve.

Teil VIII.

Alignment Modeling

Remark 39.68. This part develops the internal representation of railway alignments within the AIM framework. The emphasis lies on structured, curvature-based models that are compact, interpretable, and suitable for computation.

Remark 39.69. The goal is not merely to store geometry, but to represent alignments in a way that supports reconstruction, realization, and optimization.

Remark 39.70. The following chapters therefore introduce sparse representations, transition functions, classification principles, and higher-level modelling concepts such as Schwerpunkt-Trassierung.

40. Sparse Alignment Model

40.1. Motivation

Remark 40.1. A continuous curvature law is mathematically natural, but it is not by itself an operational representation for editing, exchange, validation, or reconstruction from heterogeneous engineering data. For this reason, a sparse model is introduced as a structured, piecewise-defined representation of an alignment.

Remark 40.2. The sparse model is not intended as a denial of continuous geometry. On the contrary, it serves as an intermediate layer between continuous curvature theory and practical modelling tasks such as import, computation, optimization, and interoperability.

Remark 40.3. Dense geometric representations such as point clouds or sampled polylines preserve positional information, but largely destroy the structural semantics of the alignment itself.

40.2. General Idea

Definition 40.4 (Sparse alignment model)**.** A sparse alignment model is a structured longitudinal representation consisting of synchronized alignment bands together with sufficient reconstruction information.

Remark 40.5. The adjective *sparse* refers here to the fact that the alignment is not stored as a dense point cloud, but as a compact sequence of semantically meaningful elements.

Remark 40.6. Within AIM, the sparse model is not interpreted as a static 3D geometry, but as a persistent structural kernel from which geometry and dynamics are derived.

40.3. Element Types

Definition 40.7 (Alignment element)**.** An alignment element is a segment of an alignment with a prescribed local curvature behaviour on a parameter interval.

Definition 40.8 (Fixed element)**.** A fixed element is an alignment element with constant curvature

$$\kappa(s) = \kappa_0.$$

Typical special cases are straight segments with $\kappa_0 = 0$ and circular arcs with $\kappa_0 \neq 0$.

Definition 40.9 (Transition element)**.** A transition element is an alignment element with non-constant curvature, i.e.

$$\kappa(s) = f(s)$$

for a suitable curvature law f.

Remark 40.10. Within AIM, transition elements are primarily interpreted through their curvature evolution rather than through explicit geometric shape.

Remark 40.11. At this level of abstraction, no commitment is yet made to a specific family of transition laws. This includes classical clothoids as well as polynomial, lookup-based, or hybrid constructions.

TODO: Formal classification of transition families.

40.4. Minimal Parameterization

Definition 40.12 (Arc length of an element)**.** Each element e_i is associated with an arc length

$$L_i > 0.$$

Definition 40.13 (Local curvature law)**.** Each element e_i is associated with a curvature law

$$\kappa_i : [0, L_i] \to \mathbb{R}.$$

Remark 40.14. For fixed elements, κ_i is constant. For transition elements, κ_i varies along the local parameter.

Definition 40.15 (Sparse alignment)**.** A sparse alignment is an ordered tuple

$$A = (e_1, e_2, \dots, e_n)$$

together with sufficient initial data for geometric reconstruction.

40.5. Initial Data and Pose

Definition 40.16 (Planar start pose)**.** A planar start pose is a tuple

$$P_0 = (x_0, y_0, \theta_0)$$

with position $(x_0, y_0) \in \mathbb{R}^2$ and initial direction $\theta_0 \in \mathbb{R}$.

Proposition 40.17. *The planar alignment is reconstructed by successive integration of the curvature law:*

$$\kappa(s) \to \theta(s) \to \gamma(s).$$

Remark 40.18. Together with the ordered element sequence and the local curvature laws, the start pose determines the alignment geometry by successive integration.

Definition 40.19 (Reconstructable sparse alignment)**.** A sparse alignment is called reconstructable if its element sequence and its initial data suffice to determine a unique planar alignment up to the intended modelling conventions.

40.6. Sparse Longitudinal Band Structure

Principle 40.20 (Sparse longitudinal band principle). Railway alignments are represented as synchronized longitudinal bands rather than as monolithic geometric objects.

Remark 40.21. Within the AIM framework, the persistent alignment model is intentionally kept sparse and decomposed into synchronized longitudinal bands.

40.6.1. cGeom

Definition 40.22 (Core geometric band)**.** The central geometric band is denoted by

$$\mathrm{cGeom}.$$

It represents the curvature-driven planar alignment core and consists of:

- fixed elements,
- transition elements,
- pose2-based reconstruction anchors.

Remark 40.23. The cGeom band forms the primary integration core of the alignment.

40.6.2. Cant Band

Definition 40.24 (Cant band)**.** Cant is represented as an independent longitudinal band correlated with fixed alignment elements.

Remark 40.25. The cant band:

- does not define an independent stationing system,
- stores rail height differences rather than roll angles,
- follows the same normalized curvature family as the associated cGeom element,
- may contain local station offsets at element boundaries.

Remark 40.26. Since cant evolution follows the same normalized curvature family as the associated cGeom element, no independent geometric evolution model is required for the cant band itself.

Remark 40.27. The corresponding roll angle is derived via:

$$\phi(s) = \arctan\left(\frac{u(s)}{g}\right),$$

where:

- $u(s)$: rail height difference,
- g: gauge.

40.6.3. Profile Band

Definition 40.28 (Profile band)**.** Vertical geometry is represented independently through a profile band.

Remark 40.29. Thus, cGeom, cant, and profile are treated as synchronized but distinct bands sharing a common arc-length domain.

40.6.4. Runtime State

Proposition 40.30. *Pose3 is not treated as persistent truth within the sparse model.*

Remark 40.31. Persisting pose3 directly would introduce redundancy and synchronization instabilities between geometry, cant, and profile representations.

Remark 40.32. Instead, pose3 is derived dynamically from:

- cGeom,
- cant,
- profile,
- and runtime evaluation services.

Remark 40.33. Consequently, the primary intelligence of the AIM framework resides not in the storage format itself, but in the mathematical operators and evaluation services acting on the sparse model.

40.7. Concatenation

Definition 40.34 (Element concatenation)**.** Two consecutive elements are concatenated if the end state of the first element provides the start state of the second element.

Definition 40.35 (Positional continuity)**.** A sparse alignment is positionally continuous if consecutive elements meet in a common endpoint.

Definition 40.36 (Directional continuity)**.** A sparse alignment is directionally continuous if consecutive elements share the same tangent direction at their common boundary.

Remark 40.37. In most engineering contexts, positional and directional continuity are minimal requirements. Higher continuity requirements may depend on the chosen transition families and on the intended application.

TODO: Formalize continuity classes such as C^0, C^1, and curvature-related continuity conditions.

40.8. Normal Form and Alternation

Remark 40.38. In practical sparse representations, a frequent pattern is the alternation between fixed elements and transition elements. This corresponds closely to engineering intuition and to many traditional alignment descriptions.

OPEN: Should strict alternation between fixed and transition elements be formulated as a mathematical property, as a preferred normal form, or merely as an implementation convention?

Remark 40.39. It is plausible to regard alternation as a normal form rather than a fundamental axiom. Multiple neighbouring fixed elements may be merged, and certain degenerate transition cases may collapse into fixed elements.

40.9. Normalized Representation of Elements

Definition 40.40 (Normalized local parameter)**.** A normalized local parameter is a variable

$$u \in [0, 1]$$

used to represent an element independently of its physical length.

Definition 40.41 (Normalized curvature law)**.** A normalized curvature law is a law defined on the unit interval, typically of the form

$$\widehat{\kappa} : [0, 1] \to \mathbb{R}.$$

Remark 40.42. Normalization separates shape from scale. It allows the same transition family to be reused for different lengths and different curvature magnitudes.

Remark 40.43. Normalized representations form the basis for reusable transition registries, family classification, and implementation-independent comparison of alignment elements.

40.10. Structural Role of the Sparse Model

Remark 40.44. The sparse model occupies an intermediate conceptual level:

- below the purely abstract curvature-theoretic foundation,

- but above dense numerical sampling, imported point lists, or format-specific container structures.

Remark 40.45. The sparse model is intentionally not a dense geometric storage format, nor a direct copy of external exchange containers.

Remark 40.46. This intermediate status makes the sparse model suitable as

- an internal modelling layer,
- a reconstruction target from imported data,
- a source for numerical evaluation,
- and a bridge toward standards and BIM-related representations.

40.11. Relation to Imported and Standardized Data

Remark 40.47. Existing engineering data rarely arrives directly in sparse form. Instead, it is typically given through container formats, survey data, legacy descriptions, or derived coordinate sequences.

Remark 40.48. Such data may contain:

- inconsistent stationing,
- incomplete cant information,
- broken coordinate reference systems,
- discontinuities,
- partial geometry fragments,
- or non-synchronized alignment components.

Remark 40.49. The sparse model therefore should not be identified with any one exchange standard. It is more appropriate to view it as a compact, internal representation that can be derived from, and mapped back to, multiple external formats.

RESEARCH: Dock sparse model to `railML` alignment structures. **RESEARCH:** Dock sparse model to `LandXML` alignment representations. **OPEN:** Define a minimal common abstraction layer for standards mapping. **OPEN:** Clarify the relation between topology-oriented standards and purely geometric container models.

40.12. Relation to BIM and BIMalignment

Remark 40.50. A sparse alignment model may support BIM-oriented workflows, but it should not merely imitate BIM container logic. Its primary responsibility is to represent alignment structure in a mathematically and operationally coherent way.

RESEARCH: Formulate a BIM-compatible alignment core without losing geometric precision. **OPEN:** Position the sparse model between generic BIM, infrastructure BIM, and explicit BIMalignment concepts. **TODO:** State explicitly where the manuscript aims to push BIMalignment rather than only satisfy BIM conformity.

40.13. Summary

Summary 40.51. The sparse alignment model is a compact, structured representation of an alignment by means of synchronized longitudinal bands together with reconstruction data and concatenation rules.

Within AIM, the sparse alignment model is not interpreted as a static 3D geometry, but as a persistent structural kernel from which geometry, runtime state, and dynamic interpretation are derived.

It serves as a bridge between continuous curvature theory and practical engineering operations such as import, validation, editing, numerical evaluation, optimization, and interoperability.

41. Transition Functions

41.1. Motivation

Remark 41.1. Transitions are the decisive element type for curvature-driven alignment modelling. While fixed elements describe geometric states of constant curvature, transitions describe the controlled change between such states.

Remark 41.2. In railway engineering, transitions are essential for ensuring smooth vehicle motion, limiting dynamic forces, and satisfying comfort and safety requirements, as discussed in both engineering-oriented studies and review literature [26, 4].

Remark 41.3. Within AIM, transitions are not primarily interpreted through explicit geometric shape, but through the evolution of curvature.

41.2. Definition by Curvature Evolution

Principle 41.4 (Curvature evolution principle). Within AIM, transitions are classified primarily by their curvature evolution rather than by explicit geometric shape.

Definition 41.5 (Transition)**.** A transition is an alignment element whose curvature varies over its parameter interval.

Definition 41.6 (Transition curvature law)**.** Let $L > 0$. A transition curvature law is a function

$$\kappa : [0, L] \to \mathbb{R}$$

with prescribed boundary values

$$\kappa(0) = \kappa_0, \qquad \kappa(L) = \kappa_1.$$

Remark 41.7. This formulation directly follows the curvature-based description of planar curves [8].

41.3. Normalized Representation

Definition 41.8 (Normalized transition law)**.** A normalized transition is defined by

$$\widehat{\kappa} : [0, 1] \to \mathbb{R}$$

with

$$\widehat{\kappa}(0) = 0, \qquad \widehat{\kappa}(1) = 1.$$

Remark 41.9. Normalization separates the geometric shape of a transition from its scale and allows systematic comparison of transition families.

Remark 41.10. Normalization separates intrinsic transition behaviour from physical scale and therefore enables reusable transition registries and family-based modelling.

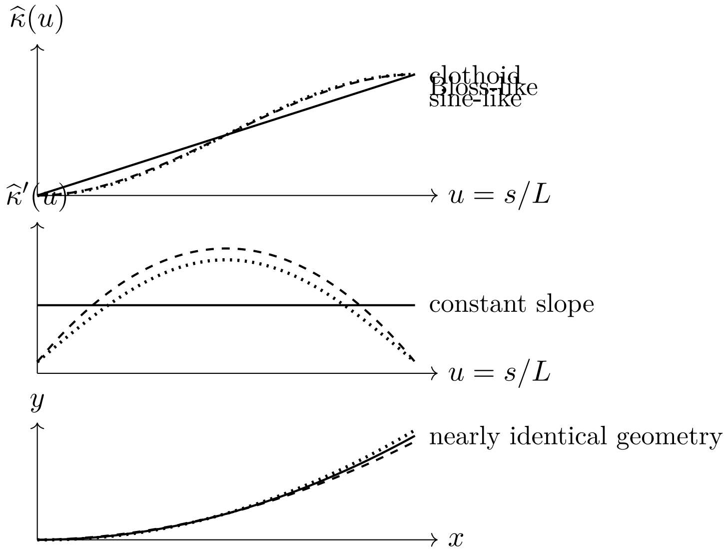

Abbildung 41.1.: Transition families may produce very similar geometric paths while differing strongly in curvature evolution and curvature derivatives. This is why transition design must be evaluated in curvature and dynamic terms, not only in position space.

Remark 41.11. Transitions that appear nearly identical in position space may differ substantially in curvature derivatives and therefore in dynamic behaviour.

41.4. Scaling

Proposition 41.12. *Let $\widehat{\kappa}$ be a normalized transition law. Then a physical transition is given by*

$$\kappa(s) = \kappa_0 + (\kappa_1 - \kappa_0)\,\widehat{\kappa}\left(\frac{s}{L}\right).$$

Remark 41.13. This separates shape from scale and provides the basis for reusable transition families and parameterized design.

41.5. Regularity and Smoothness

Definition 41.14 (Regularity class)**.** A transition is of class C^k if its curvature function is k-times continuously differentiable.

Remark 41.15. In engineering practice, higher-order properties such as curvature derivatives are directly related to ride comfort and dynamic loading, as shown in both theoretical and engineering studies [34, 27].

OPEN: Formal role of κ' and κ'' in engineering rules.

41.6. Transition Families

Definition 41.16 (Transition family)**.** A transition family is a class of curvature laws sharing a common functional structure.

41.6.1. Classical Families

Remark 41.17. The classical transition curve in railway engineering is the clothoid, characterized by linear curvature evolution:

$$\kappa(s) \propto s.$$

Remark 41.18. Historically, this line of development can be traced through early work by Glover [9], Horsburgh [15], and Higgins [13].

41.6.2. Polynomial and Alternative Families

Remark 41.19. Polynomial transition curves provide additional flexibility in shaping curvature evolution and its derivatives.

Remark 41.20. Such approaches are associated with modern polynomial smoothing research [51] as well as control-oriented constructions proposed by Klauder [19, 22].

TODO: Formal definition of polynomial transition families.

41.6.3. Geometric Families

Remark 41.21. Some transition curves are introduced through geometric or implicit relations rather than explicit curvature laws.

Example 41.22. Lemniscate-type or sinusoid-related constructions belong to this class after suitable reparameterization into curvature space [37].

Remark 41.23. These curves require transformation into $\kappa(s)$ representation before they can be integrated into the present framework.

41.6.4. Engineering and Practice-Oriented Families

Remark 41.24. Engineering practice often introduces transition curves through design rules and operational considerations rather than through purely theoretical derivation.

Remark 41.25. The Bloss transition is a notable example of a practical engineering family, originally described by Bloss [3] and further discussed in modern summaries [7].

41.6.5. Modern and Hybrid Families

Remark 41.26. Modern engineering practice considers a variety of transition families with improved dynamic behaviour, smoother endpoint properties, or higher-order control.

Remark 41.27. Examples include smoothed-curvature transitions proposed by Koc [27, 29, 28] and Wiener-Bogen-related approaches [50].

RESEARCH: Systematic classification of modern transition families.

41.7. Decomposition Approaches

Remark 41.28. A transition need not always be treated as a single indivisible object. It may be useful to represent it as a composition of sub-elements.

Remark 41.29. Within AIM, complex transition behaviour may be constructed from composable normalized suboperators such as half-wave and clothoid components.

TODO: Formalize halfWave – clothoid – halfWave decomposition.
OPEN: Is decomposition fundamental or implementation-specific?

41.8. Operator View

Principle 41.30 (Transition operator principle). A transition element acts as a local curvature evolution operator transforming one geometric state into another.

Remark 41.31. Transitions can therefore be interpreted as state evolution operators rather than merely as predefined geometric curve shapes.

RESEARCH: Formal operator algebra of transitions.

41.9. Implementation Perspective

Remark 41.32. In computational systems, transition curves may be represented using:

- analytic expressions,
- polynomial approximations,
- lookup tables,
- compiled hybrid representations.

Remark 41.33. Transition families may be represented through registries containing normalized curvature laws together with derivative and integration operators.

TODO: Registry-based transition system.
OPEN: Trade-off between analytic precision and computational efficiency.

41.10. Relation to Railway Engineering

Remark 41.34. In railway applications, transitions are tightly coupled to operational parameters such as speed and cant, and must satisfy strict engineering constraints [30, 24].

Remark 41.35. The review literature confirms that transition curves must be understood not only geometrically, but also with respect to comfort, maintenance, and dynamic response [4].

41.11. Summary

Summary 41.36. Transitions define the controlled evolution of curvature between geometric states.

Within AIM, they are interpreted as normalized curvature evolution operators rather than merely as predefined geometric curve types.

Their normalized representation enables reusable transition families, operator-based modelling, and runtime reconstruction of alignment geometry and dynamics.

42. Transition Families and Classification

42.1. Motivation

Remark 42.1. Transition curves form a central component of alignment design, yet their treatment in the literature remains fragmented across geometric, engineering, and dynamic perspectives.

Remark 42.2. A unified classification is therefore required in order to compare, evaluate, and optimize transition concepts systematically.

—

42.2. Curvature-Based Representation

Proposition 42.3. *Any transition curve can be represented by a curvature function*

$$\kappa : [0, L] \to \mathbb{R}.$$

Remark 42.4. This representation is independent of the original definition of the curve and provides a common mathematical framework.

—

42.2.1. Normalization

Definition 42.5. A normalized transition is defined on

$$s \in [0, 1]$$

with

$$\kappa(0) = 0, \qquad \kappa(1) = 1.$$

Remark 42.6. Normalization removes scale and allows direct comparison of transition families.

—

42.3. Classification by Functional Form

42.3.1. Linear Curvature: Clothoid

Definition 42.7.

$$\kappa(s) = as.$$

Remark 42.8. The clothoid is characterized by constant curvature derivative

$$\kappa'(s) = \text{const}.$$

Remark 42.9. This simplicity explains its widespread adoption in railway engineering.

—

42.3.2. Polynomial Curvature Families

Definition 42.10.

$$\kappa(s) = \sum_{i=0}^{n} a_i s^i.$$

Remark 42.11. Polynomial transitions allow explicit control of higher-order derivatives of curvature.

Remark 42.12. Such flexibility enables optimization with respect to dynamic criteria.

Remark 42.13. Klauder-type transitions belong to this class, incorporating dynamic considerations directly into the curvature design.

—

42.3.3. Geometric Families

Remark 42.14. Some transition curves are defined implicitly or through geometric relations rather than explicit curvature functions.

Remark 42.15. These curves must be transformed into curvature representation to be integrated into the present framework.

—

42.3.4. Engineering-Based Families

Remark 42.16. Engineering approaches often define transition curves through design rules or empirical considerations.

Remark 42.17. The Bloss transition represents a notable example of such a practical engineering compromise.

—

42.3.5. Optimized and Hybrid Families

Remark 42.18. Modern approaches introduce transition curves designed explicitly to optimize dynamic performance.

Remark 42.19. These include smoothed curvature transitions and jerk-optimized polynomial curves.

—

42.4. Classification by Higher-Order Behaviour

42.4.1. Curvature Derivatives

Definition 42.20.

$$\kappa'(s), \quad \kappa''(s)$$

denote the first and second derivatives of curvature.

—

Remark 42.21. These quantities determine dynamic behaviour, including acceleration variation and smoothness of force development.

—

42.4.2. Smoothness Classes

Definition 42.22. A transition is of class C^k if κ is k-times continuously differentiable.

—

Remark 42.23. Higher smoothness typically improves dynamic performance and reduces wear.

—

42.5. Comparison of Families

Family	κ	κ'	κ''
Clothoid	linear	constant	0
Polynomial	flexible	flexible	flexible
Engineering	implicit	implicit	implicit
Optimized	designed	controlled	controlled

—

42.6. Interpretation as Control Functions

Proposition 42.24. *Transition curves can be interpreted as control functions acting on system dynamics through curvature evolution.*

Remark 42.25. This interpretation links geometric design directly to dynamic performance.

—

42.7. Connection to Railway Engineering

Remark 42.26. Railway transition design is governed by dynamic constraints such as acceleration, jerk, and wear.

Remark 42.27. These constraints translate directly into requirements on

$$\kappa(s), \quad \kappa'(s), \quad \kappa''(s).$$

—

42.8. Main Insight

Theorem 42.28. *Transition curves are not merely geometric connectors, but control functions governing system behaviour.*

—

42.9. Conclusion

Summary 42.29. Transition curves form a unified class of curvature functions.

Their classification by functional form and higher-order behaviour provides a systematic framework for comparison, optimization, and integration into engineering systems.

This perspective establishes a direct link between geometry and system dynamics.

43. Schwerpunkt-Trassierung

43.1. Motivation

Remark 43.1. Classical railway alignment design separates:

- geometric design (alignment),
- cant design,
- dynamic evaluation.

Remark 43.2. This separation leads to suboptimal solutions, as geometry and dynamics are intrinsically coupled. In railway practice, this coupling appears through the interaction of curvature, cant, speed, and vehicle response [24, 6, 4].

Remark 43.3. The concept of *Schwerpunkt-Trassierung* aims to unify these aspects by defining a single design principle based on the motion of a representative point.

Remark 43.4. The idea that railway alignment should be related to the motion of the vehicle centre of gravity rather than merely to the track centerline has also appeared in engineering practice and patent literature [12].

Remark 43.5. This supports the broader AIM interpretation that alignment should be understood as a dynamically relevant spatial state trajectory rather than as a purely geometric curve.

43.2. Conceptual Idea

Definition 43.6 (Reference trajectory)**.** Let

$$\gamma_c : I \to \mathbb{R}^2$$

denote a reference trajectory representing the motion of a characteristic point of the vehicle system.

Remark 43.7. This point may be interpreted as:

- the center of mass,
- a dynamically relevant reference point,
- or an abstract design trajectory.

Remark 43.8. The conceptual shift consists in evaluating track design through the motion of such a reference point rather than through geometry alone.

43.3. Relation to Track Geometry

Remark 43.9. The physical track centerline $\gamma(s)$ and cant angle $\phi(s)$ induce a spatial configuration of the vehicle.

Definition 43.10 (Induced reference trajectory)**.** Given a pose3 state

$$(\gamma(s), \theta(s), \phi(s)),$$

the induced reference trajectory is

$$\gamma_c(s) = \gamma(s) + d\, n(s, \phi(s)),$$

where:

- d is a characteristic offset,
- $n(s, \phi)$ is a direction depending on orientation and cant.

Remark 43.11. The exact form of $n(s, \phi)$ depends on the chosen mechanical model.

RESEARCH: Precise definition of offset direction based on vehicle model.

OPEN: Consistent integration of local track frames and global CRS transformations in the Schwerpunkt model.

43.4. Schwerpunkt Principle

Definition 43.12 (Schwerpunkt principle)**.** A railway alignment is said to satisfy the Schwerpunkt principle if the induced reference trajectory $\gamma_c(s)$ possesses prescribed smoothness and dynamical properties.

Remark 43.13. In this formulation, the primary design object is not the track itself, but the induced trajectory of the reference point.

Remark 43.14. This makes the design task closer to vehicle-oriented engineering than to purely geometric construction.

43.5. Variational Formulation

Definition 43.15 (Schwerpunkt functional)**.** Let

$$J_c[\gamma_c] = \int_0^L \|\gamma_c''(s)\|^2 \,\mathrm{d}s.$$

Remark 43.16. This functional penalizes curvature of the reference trajectory and therefore promotes smooth motion.

Definition 43.17 (Schwerpunkt optimization problem)**.** Find

$$(\kappa, \phi)$$

such that the induced trajectory γ_c minimizes

$$J_c[\gamma_c]$$

subject to the pose3 dynamics.

Remark 43.18. This formulation extends the variational interpretation of transition design from curvature laws alone to vehicle-relevant derived motion.

43.6. Coupling to Pose3

Remark 43.19. The trajectory γ_c depends on:

$$\gamma(s), \quad \theta(s), \quad \phi(s).$$

Remark 43.20. Thus, the Schwerpunkt problem becomes a constrained optimization problem in the variables:

$$\kappa(s), \quad \phi(s).$$

Remark 43.21. This places Schwerpunkt-Trassierung naturally within the pose3-based state model introduced earlier and within the broader optimization perspective of railway alignment [31, 30].

43.7. Interpretation

Remark 43.22. The Schwerpunkt formulation shifts the perspective:

- from track geometry
- to vehicle motion.

Remark 43.23. This leads to a unified framework in which:

- geometry,
- cant,
- and dynamics

are optimized simultaneously.

Remark 43.24. Such a viewpoint is consistent with the general observation that railway transition and alignment quality cannot be judged from geometry alone [4, 6].

43.8. Relation to Classical Design

Proposition 43.25. *Classical transition curves correspond to special cases of the Schwerpunkt formulation where the reference trajectory coincides with the track centerline.*

Remark 43.26. In this case, the problem reduces to curvature-based optimization in the plane.

Remark 43.27. Thus, classical alignment design is not contradicted, but embedded as a special case of a more general formulation.

43.9. Advantages

- unified modelling of geometry and dynamics,
- natural integration of cant,
- compatibility with optimization frameworks,
- extensibility to vehicle-specific models.

Remark 43.28. These advantages derive from the fact that the model treats alignment as a state-governed physical system rather than as a static geometric line.

43.10. Open Problems

OPEN: Precise mechanical interpretation of the reference trajectory.

OPEN: Integration with full vehicle dynamics models.

OPEN: Extension to network-level alignment design.

OPEN: Explicit treatment of CRS, local track frames, and coordinate transformations in large-scale alignment systems.

43.11. Connection to the AIM Framework

Summary 43.29. Schwerpunkt-Trassierung provides a conceptual and mathematical extension of classical alignment design.

Instead of prescribing geometric elements, it defines the design problem in terms of a reference trajectory derived from the railway state.

This aligns with the AIM philosophy of treating alignment as a functional object governed by curvature and its derived quantities.

43.12. Vehicle Model

43.12.1. Rigid Body Approximation

Remark 43.30. We model the railway vehicle as a rigid body with a characteristic center of mass.

Definition 43.31 (Center of mass)**.** Let

$$C(s) \in \mathbb{R}^3$$

denote the spatial position of the vehicle center of mass.

Remark 43.32. For planar track geometry, we consider the projection

$$\gamma_c(s) \in \mathbb{R}^2$$

of the center of mass trajectory.

Remark 43.33. This is intentionally a simplified model. Its purpose is not to replace full multibody simulation, but to introduce a mathematically tractable vehicle-related state layer.

43.12.2. Track Coordinate Frame

Definition 43.34 (Frenet frame)**.** Let

$$t(s) = (\cos\theta(s), \sin\theta(s))$$

be the tangent direction and

$$n(s) = (-\sin\theta(s), \cos\theta(s))$$

the normal direction.

Remark 43.35. The pair (t, n) defines a local coordinate system attached to the track.

Remark 43.36. This local frame is sufficient for the present formulation, but future extensions should consistently relate it to a global project CRS.

43.12.3. Cant Rotation

Definition 43.37 (Rotated normal)**.** Let the cant angle $\phi(s)$ define a rotation of the cross-section.

Remark 43.38. In a simplified model, the lateral offset direction remains aligned with $n(s)$, while vertical effects are captured implicitly.

Remark 43.39. This simplification is acceptable at the present conceptual stage, but a full treatment should explicitly include the three-dimensional rotated cross-section and its transformation into the global frame.

43.12.4. Center of Mass Offset

Definition 43.40 (Offset model)**.** Let $d > 0$ denote a characteristic lateral offset. The center of mass trajectory is approximated by

$$\gamma_c(s) = \gamma(s) + d\, n(s).$$

Remark 43.41. More refined models may include dependence on $\phi(s)$, e.g.

$$\gamma_c(s) = \gamma(s) + d\, \cos\phi(s)\, n(s).$$

RESEARCH: Inclusion of full 3D vehicle geometry.

43.13. Dynamics of the Center of Mass

43.13.1. Velocity and Acceleration

Proposition 43.42. *The velocity of the center of mass is*

$$\gamma_c'(s) = t(s) + d\, n'(s).$$

Remark 43.43. Using

$$n'(s) = -\kappa(s)\, t(s),$$

we obtain

$$\gamma_c'(s) = (1 - d\kappa(s))\, t(s).$$

Proposition 43.44. *The acceleration of the center of mass is proportional to*

$$\gamma_c''(s) = -d\kappa'(s)\, t(s) + (1 - d\kappa(s))\kappa(s)\, n(s).$$

Remark 43.45. Thus, both curvature and its derivative influence the motion of the center of mass.

Remark 43.46. This is one of the key conceptual benefits of the formulation: it makes the dynamic relevance of higher-order geometric quantities explicit.

43.13.2. Effective Forces

Definition 43.47 (Lateral acceleration)**.** The lateral acceleration of the center of mass is

$$a_c(s) = v^2(1 - d\kappa(s))\kappa(s).$$

Remark 43.48. Including cant yields the effective acceleration

$$a_{\text{eff}}(s) = v^2\kappa(s) - g \sin\phi(s).$$

Remark 43.49. The latter term reflects the same curvature–cant coupling that appears in railway design standards and in dynamic alignment interpretation [24, 6].

43.14. Coupled Design Problem

Definition 43.50 (Vehicle-based objective)**.** Define

$$J_{\text{veh}}[\kappa, \phi] = \int_0^L \|\gamma_c''(s)\|^2 \, \mathrm{d}s.$$

Remark 43.51. This functional directly measures the smoothness of the motion of the center of mass.

Definition 43.52 (Dynamic objective)**.** Define

$$J_{\text{dyn}}[\kappa, \phi] = \int_0^L \Big(v^2\kappa(s) - g \sin\phi(s)\Big)^2 \, \mathrm{d}s.$$

Definition 43.53 (Combined vehicle functional)**.** A combined objective is

$$J[\kappa, \phi] = \alpha J_{\text{veh}} + \beta J_{\text{dyn}} + \gamma \int_0^L (\kappa'(s))^2 \, \mathrm{d}s + \delta \int_0^L (\phi'(s))^2 \, \mathrm{d}s.$$

Remark 43.54. This combines geometric smoothing, dynamic balancing, and higher-order regularization within a single optimization framework.

43.15. Schwerpunkt-Trassierung (Formal Definition)

Definition 43.55 (Schwerpunkt alignment)**.** A Schwerpunkt alignment is a pair

$$(\kappa, \phi)$$

that minimizes the functional

$$J[\kappa, \phi]$$

subject to:

- pose3 dynamics,
- boundary conditions,
- engineering constraints.

Remark 43.56. This turns alignment design into the optimization of a vehicle-related state trajectory rather than the mere composition of geometric elements.

43.16. Interpretation

Remark 43.57. In this formulation:

- the track is no longer the primary design object,
- the motion of the vehicle becomes central.

Remark 43.58. Classical alignment design is recovered as a special case where

$$d = 0 \quad \text{and} \quad \phi(s) = 0.$$

43.17. Discussion

Remark 43.59. The presented model is intentionally simplified. However, it already captures:

- coupling between curvature and cant,
- influence of curvature derivatives,
- relation between geometry and dynamics.

Remark 43.60. Its main contribution is therefore conceptual and structural: it defines a mathematically coherent bridge between alignment geometry and vehicle-oriented design thinking.

RESEARCH: Extension to full multibody vehicle models.
RESEARCH: Calibration using measured track data.
OPEN: Integration into practical design workflows.

43.18. Connection to the AIM Framework

Summary 43.61. The combination of pose3 and a vehicle-based model provides a rigorous foundation for Schwerpunkt-Trassierung.

It transforms alignment design into a problem of optimizing the motion of a representative physical system.

This supports the central AIM idea that alignment is not merely a geometric object, but a functional entity governed by curvature and its physical implications.

Teil IX.

System and Pipeline

44. System and Pipeline

Remark 44.1. This part connects the individual components of the AIM framework into a single system view. It describes alignment modelling as a pipeline of transformations linking coordinate systems, intrinsic track coordinates, modelling, realization, and optimization.

Remark 44.2. The purpose of this part is to show that AIM is not merely a collection of methods, but an operator-based framework in which data, geometry, construction reality, and computation interact.

Remark 44.3. The following chapters introduce the AIM pipeline, its interpretation, its failure modes, and the design principles that follow from it.

44.1. AIM Processing Pipeline

44.1.1. Motivation

Remark 44.4. Railway alignment modelling involves multiple domains:

- global coordinate systems (CRS),
- geometric alignment representation,
- physical realization,
- measurement data,
- optimization.

Remark 44.5. These domains are often treated separately, leading to fragmented models and inconsistent results.

Remark 44.6. Within the AIM framework, these components are unified into a single processing pipeline.

44.1.2. Operator Chain

Definition 44.7 (AIM pipeline)**.** The complete modelling and optimization process is described as a composition of operators:

$$\boxed{\mathcal{P} = \mathcal{W}^{-1} \circ \mathcal{R} \circ \mathcal{M} \circ \mathcal{W} \circ \mathcal{C}}$$

Remark 44.8. The operators are:

- $\mathcal{C}$: CRS transformation,
- $\mathcal{W}$: world-to-track transformation,
- $\mathcal{M}$: alignment model (parametric or hybrid),
- $\mathcal{R}$: realization operator,
- $\mathcal{W}^{-1}$: track-to-world transformation.

—

44.1.3. Detailed Structure

Remark 44.9. The pipeline maps raw data to realized geometry:

$$\text{CRS data} \xrightarrow{\mathcal{C}} \text{world coordinates} \xrightarrow{\mathcal{W}} (s,d) \xrightarrow{\mathcal{M}} \kappa(s) \xrightarrow{\mathcal{R}} \gamma_{\text{real}}(s) \xrightarrow{\mathcal{W}^{-1}} \text{world geometry.}$$

—

44.1.4. Hybrid Model Integration

Remark 44.10. The alignment model operator is hybrid:

$$\mathcal{M} : (\mathbf{T}, \delta\kappa) \;\mapsto\; \kappa(s).$$

Remark 44.11. Thus, the pipeline operates on both:

- structured parameters,
- local corrections.

—

44.1.5. Inverse Problem

Definition 44.12 (Pipeline inversion)**.** Given observed data x_i, the inverse problem is:

$$\min_{\mathbf{x}} \sum_i \|\mathcal{P}(\mathbf{x})(s_i) - x_i\|^2.$$

Remark 44.13. This corresponds to fitting a model such that the realized geometry matches observations.

—

44.1.6. Optimization Embedding

Remark 44.14. The pipeline is embedded into an optimization problem:

$$\min_{\mathbf{x}} J(\mathcal{P}(\mathbf{x})).$$

Remark 44.15. The objective is evaluated on the realized geometry, not on the parametric model.

—

44.1.7. Jacobian Structure

Remark 44.16. The derivative of the pipeline follows from the chain rule:

$$\frac{\partial \mathcal{P}}{\partial \mathbf{x}} = D\mathcal{W}^{-1} \cdot D\mathcal{R} \cdot D\mathcal{M} \cdot D\mathcal{W}.$$

Remark 44.17. Each component has specific properties:

- $D\mathcal{M}$: sparse and structured,
- $D\mathcal{R}$: smoothing operator,
- $D\mathcal{W}$: nonlinear projection,
- $D\mathcal{C}$: linear transformation.

—

44.1.8. Computational Implications

Remark 44.18. The pipeline exhibits:

- sparsity (model level),
- nonlinearity (projection level),
- smoothing (realization level),
- scaling effects (CRS level).

Remark 44.19. Efficient implementation requires:

- hybrid representations,
- block-structured solvers,
- LOD-dependent evaluation.

—

44.1.9. Level-of-Detail Interpretation

Remark 44.20. Different parts of the pipeline dominate at different scales:

- large scale: CRS and world geometry,
- medium scale: parametric alignment,
- small scale: corrections and realization.

—

44.1.10. Key Insight

Proposition 44.21. *Alignment modelling is not a single mapping, but a composition of geometric, physical, and computational operators.*

—

44.1.11. Connection to AIM

Summary 44.22. The AIM framework is fundamentally defined by its operator pipeline.

It integrates coordinate systems, geometry, physics, and optimization into a single coherent model.

This unified perspective enables:

- consistent data integration,
- physically meaningful optimization,
- scalable computation.

Thus, AIM transforms alignment modelling from a collection of isolated methods into a structured computational system.

44.2. AIM Pipeline Overview

44.3. AIM Master Architecture

44.4. Pipeline Story: From Measurement to Alignment

44.4.1. A Single Point

Remark 44.23. Consider a single measured point x in the real world.

Remark 44.24. It may originate from:

- a surveying campaign,
- a track recording vehicle,

- or a design dataset.

Remark 44.25. At this stage, the point exists only in a global coordinate system.

—

44.4.2. Step 1: CRS Transformation

Remark 44.26. The point is first interpreted within a coordinate reference system:

$$x_{\text{crs}} \xrightarrow{\mathcal{C}} x_{\text{world}}.$$

Remark 44.27. This step ensures that all data is expressed in a consistent spatial frame.

—

44.4.3. Step 2: Projection onto the Track

Remark 44.28. The point is projected onto the alignment:

$$x_{\text{world}} \xrightarrow{\mathcal{W}} (s, d).$$

Remark 44.29. This is a nonlinear operation:

- the longitudinal position s is determined,
- the lateral deviation d is computed.

Remark 44.30. At this moment, the point becomes part of the intrinsic alignment coordinate system.

—

44.4.4. Step 3: Interpretation through the Model

Remark 44.31. The alignment model provides a curvature description:

$$(s, d) \xrightarrow{\mathcal{M}} \kappa(s).$$

Remark 44.32. Here, two components interact:

- the parametric structure (design intent),
- local corrections (data-driven adjustments).

Remark 44.33. The point is no longer treated as isolated data, but as part of a structured geometric system.

—

44.4.5. Step 4: Physical Realization

Remark 44.34. The theoretical model is transformed by physical processes:

$$\kappa(s) \xrightarrow{\mathcal{R}} \gamma_{\text{real}}(s).$$

Remark 44.35. This step reflects:

- construction constraints,
- mechanical behaviour,
- maintenance operations.

Remark 44.36. The resulting geometry is not identical to the theoretical model.

—

44.4.6. Step 5: Back to World Coordinates

Remark 44.37. The realized alignment is mapped back into world space:

$$\gamma_{\text{real}}(s) \xrightarrow{\mathcal{W}^{-1}} x_{\text{real}}.$$

Remark 44.38. This produces the geometry that is visible and measurable in reality.

—

44.4.7. Step 6: Comparison and Feedback

Remark 44.39. The original measurement and the realized position are compared:

$$x_{\text{real}} \approx x_{\text{world}}.$$

Remark 44.40. The deviation drives optimization:

- parameters are adjusted,
- corrections are updated,
- the pipeline is evaluated again.

—

44.4.8. Interpretation

Remark 44.41. The point has passed through multiple representations:

- global coordinates,
- intrinsic alignment coordinates,

- parametric model,
- realized geometry.

Remark 44.42. Each transformation introduces structure, constraints, and interpretation.

—

44.4.9. Key Insight

Proposition 44.43. *A measured point is not directly part of the alignment.*

It becomes part of the alignment only after passing through the world-to-track transformation and the alignment model.

—

44.4.10. Connection to AIM

Summary 44.44. The AIM framework transforms raw spatial data into structured alignment information through a sequence of operators.

This process integrates:

- coordinate systems,
- geometric modelling,
- physical realization,
- optimization.

Thus, alignment modelling is not a static representation, but a dynamic process linking measurement, theory, and construction.

44.5. Failure Modes and Limitations of the AIM Pipeline

44.5.1. Motivation

Remark 44.45. Any modelling and optimization framework must account not only for its capabilities, but also for its limitations.

Remark 44.46. Within the AIM pipeline, failure modes arise from the interaction of:

- coordinate systems,
- geometric transformations,
- physical realization,
- numerical optimization.

—

44.5.2. CRS-Related Errors

Remark 44.47. Errors in coordinate reference systems include:

- incorrect CRS identification,
- inconsistent transformations,
- mixed coordinate systems within datasets.

Proposition 44.48. *CRS inconsistencies lead to systematic spatial distortions that cannot be corrected by local alignment optimization.*

—

44.5.3. Projection Errors (World-to-Track)

Remark 44.49. The world-to-track transformation may fail due to:

- ambiguous nearest points,
- high curvature regions,
- sparse or noisy alignment representations.

Proposition 44.50. *Projection errors introduce nonlocal distortions in the intrinsic coordinate s, affecting all downstream computations.*

—

44.5.4. Model Mis-Specification

Remark 44.51. The parametric model may fail to represent the true alignment due to:

- insufficient transition families,
- incorrect parameterization,
- overly restrictive structure.

Remark 44.52. Hybrid correction terms mitigate but do not eliminate this issue.

—

44.5.5. Overfitting vs. Underfitting

Remark 44.53. Hybrid models introduce a trade-off:

- underfitting: parametric model too rigid,
- overfitting: correction terms capture noise.

Proposition 44.54. *Regularization is required to balance model fidelity and stability.*

—

44.5.6. Realization Effects

Remark 44.55. The realization operator introduces:

- smoothing of curvature,
- loss of high-frequency information,
- dependence on construction processes.

Proposition 44.56. *Certain features of the target alignment are not physically realizable and will be systematically suppressed.*

—

44.5.7. Inverse Problem Instability

Remark 44.57. The inverse realization problem is ill-posed:

- multiple target alignments yield similar realized geometry,
- small changes in data may cause large parameter variations.

Proposition 44.58. *Regularization and prior knowledge are essential for stable solutions.*

—

44.5.8. Numerical Optimization Issues

Remark 44.59. Optimization may fail due to:

- poor initial guesses,
- nonconvex objective functions,
- ill-conditioned Jacobians.

Remark 44.60. Despite structured sparsity, convergence is not guaranteed.

—

44.5.9. Block Structure Degeneracy

Remark 44.61. The block structure may become degenerate if:

- coupling terms dominate,
- insufficient data separates variables,
- parameters become redundant.

Proposition 44.62. *Loss of block separability reduces computational efficiency and interpretability.*

—

44.5.10. Measurement Noise and Bias

Remark 44.63. Measurement data may contain:

- random noise,
- systematic bias,
- missing or inconsistent segments.

Proposition 44.64. *Noise propagates through the pipeline and may be amplified by inverse operations.*

—

44.5.11. Scale-Dependent Failures

Remark 44.65. Different failure modes dominate at different scales:

- large scale: CRS and projection errors,
- medium scale: model mis-specification,
- small scale: realization and measurement noise.

—

44.5.12. Non-Uniqueness of Solutions

Remark 44.66. Different alignment models may produce nearly identical realized geometry.

Proposition 44.67. *The mapping*

$$\kappa_{\text{target}} \mapsto \gamma_{\text{real}}$$

is not injective.

—

44.5.13. Practical Engineering Limits

Remark 44.68. Certain effects are outside the scope of the model:

- material-dependent behaviour,
- long-term degradation,
- construction variability.

Remark 44.69. These factors must be treated as external constraints.

—

44.5.14. Key Insight

Proposition 44.70. *The AIM pipeline is inherently approximate and limited by both physical and computational constraints.*

—

44.5.15. Connection to AIM

Summary 44.71. Failure modes in the AIM pipeline arise from the interaction of geometry, physics, data, and computation.

Understanding these limitations is essential for:

- robust model design,
- stable optimization,
- reliable engineering application.

Thus, failure analysis is not an auxiliary component, but an integral part of the AIM framework.

44.6. Design Principles of Alignment-Based Information Modelling

44.6.1. Motivation

Remark 44.72. The AIM framework integrates geometry, coordinate systems, realization, and optimization into a unified model.

Remark 44.73. From this integration, a set of fundamental design principles emerges.

—

44.6.2. Principle 1: Curvature-Centric Modelling

Proposition 44.74. *Railway alignments should be modelled primarily through curvature functions $\kappa(s)$, not through point-based geometry.*

Remark 44.75. Curvature provides:

- intrinsic representation,
- direct link to dynamics,
- natural integration with optimization.

—

44.6.3. Principle 2: Separation of Representation and Realization

Proposition 44.76. *The theoretical **alignment** model must be separated from its physical realization.*

Remark 44.77. The realization operator introduces:

- smoothing,
- constraints,
- deviations from the ideal model.

Remark 44.78. Ignoring this separation leads to systematic modelling errors.

—

44.6.4. Principle 3: Hybrid Modelling

Proposition 44.79. *Alignment models should combine parametric structure with local correction terms.*

Remark 44.80. This allows:

- interpretable design intent,
- robust handling of imperfect data,
- flexible refinement.

—

44.6.5. Principle 4: Operator-Based Architecture

Proposition 44.81. *Alignment modelling should be formulated as a composition of operators:*

$$\mathcal{P} = \mathcal{W}^{-1} \circ \mathcal{R} \circ \mathcal{M} \circ \mathcal{W} \circ \mathcal{C}.$$

Remark 44.82. Each operator represents a distinct domain:

- coordinate systems,
- geometric modelling,
- physical processes,
- data transformation.

—

44.6.6. Principle 5: Design in Realized Space

Proposition 44.83. *Optimization objectives must be evaluated on realized geometry, not on theoretical models.*

Remark 44.84. The relevant mapping is:

$$\kappa_{\text{target}} \mapsto \gamma_{\text{real}}.$$

Remark 44.85. Design must account for the transformation induced by realization.

—

44.6.7. Principle 6: Exploitation of Structure

Proposition 44.86. *The intrinsic block and sparsity structure of the problem must be exploited for efficient computation.*

Remark 44.87. This includes:

- locality in arc-length,
- block decomposition (curvature vs. cant),
- sparse Jacobians.

—

44.6.8. Principle 7: Scale Awareness

Proposition 44.88. *Different components of the pipeline must be evaluated at appropriate scales.*

Remark 44.89.

- CRS dominates at large scale,
- parametric modelling at medium scale,
- corrections and realization at small scale.

—

44.6.9. Principle 8: Measurement Integration

Proposition 44.90. *Measured data must be integrated through the world-to-track transformation, not directly in world space.*

Remark 44.91. This ensures consistency with the intrinsic alignment representation.

—

44.6.10. Principle 9: Regularization as Necessity

Proposition 44.92. *Regularization is an essential component of alignment modelling and optimization.*

Remark 44.93. It stabilizes:

- inverse problems,
- hybrid corrections,
- numerical optimization.

—

44.6.11. Principle 10: Acceptance of Non-Uniqueness

Proposition 44.94. *Alignment solutions are not unique and must be evaluated based on engineering criteria.*

Remark 44.95. Different models may produce nearly identical realized geometry.

Remark 44.96. Thus, solution quality must be defined through:

- dynamics,
- constraints,
- robustness.

—

44.6.12. Summary

Summary 44.97. The AIM framework is governed by a set of design principles that emerge from the interaction of geometry, physics, data, and computation.

These principles define:

- how alignments should be represented,
- how they should be transformed,
- how they should be optimized,
- and how their limitations must be handled.

Together, they provide a coherent foundation for alignment-based information modelling as a unified engineering and computational discipline.

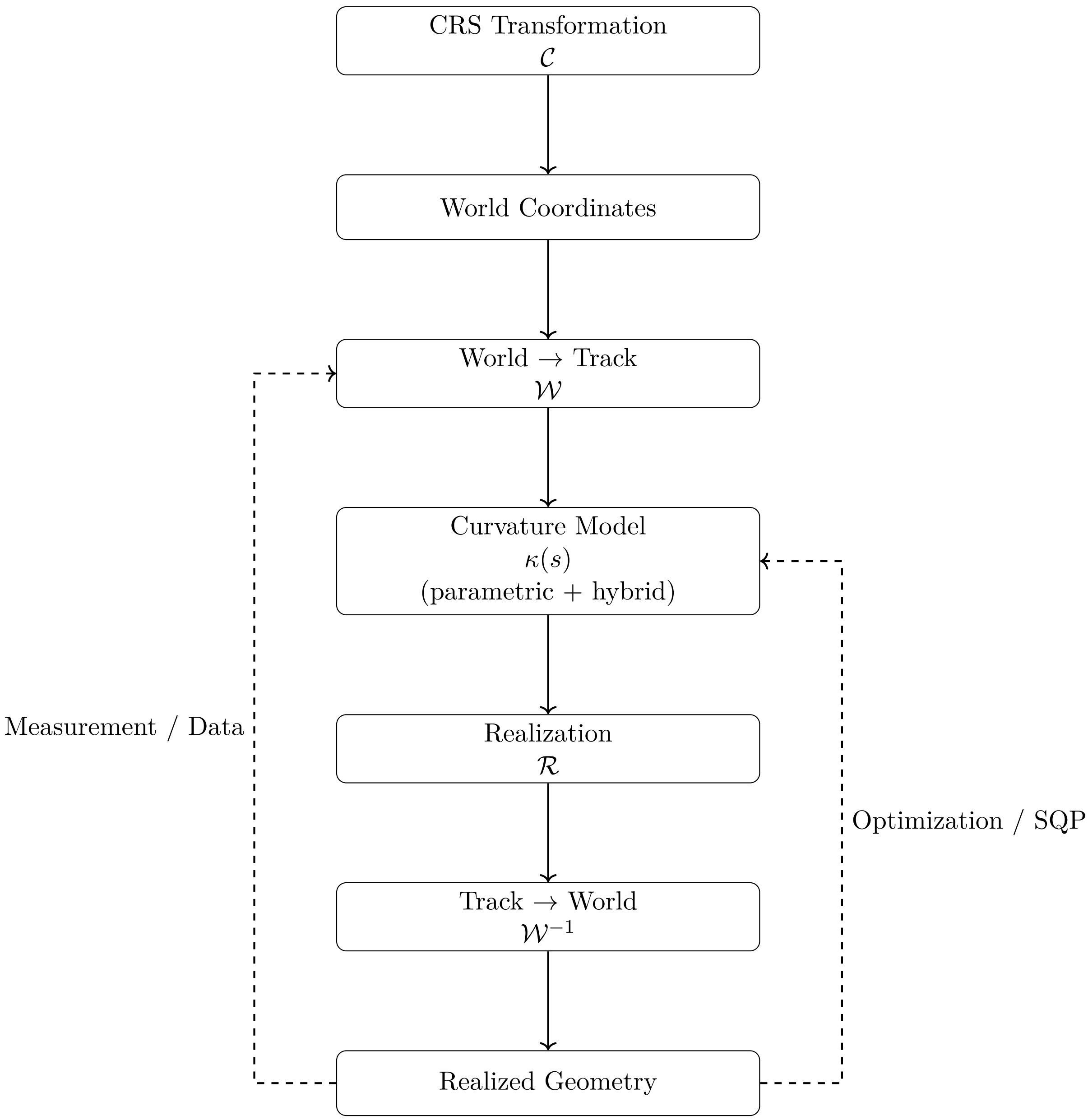

Abbildung 44.1.: AIM processing pipeline as composition of operators linking coordinate systems, alignment modelling, realization, and optimization.

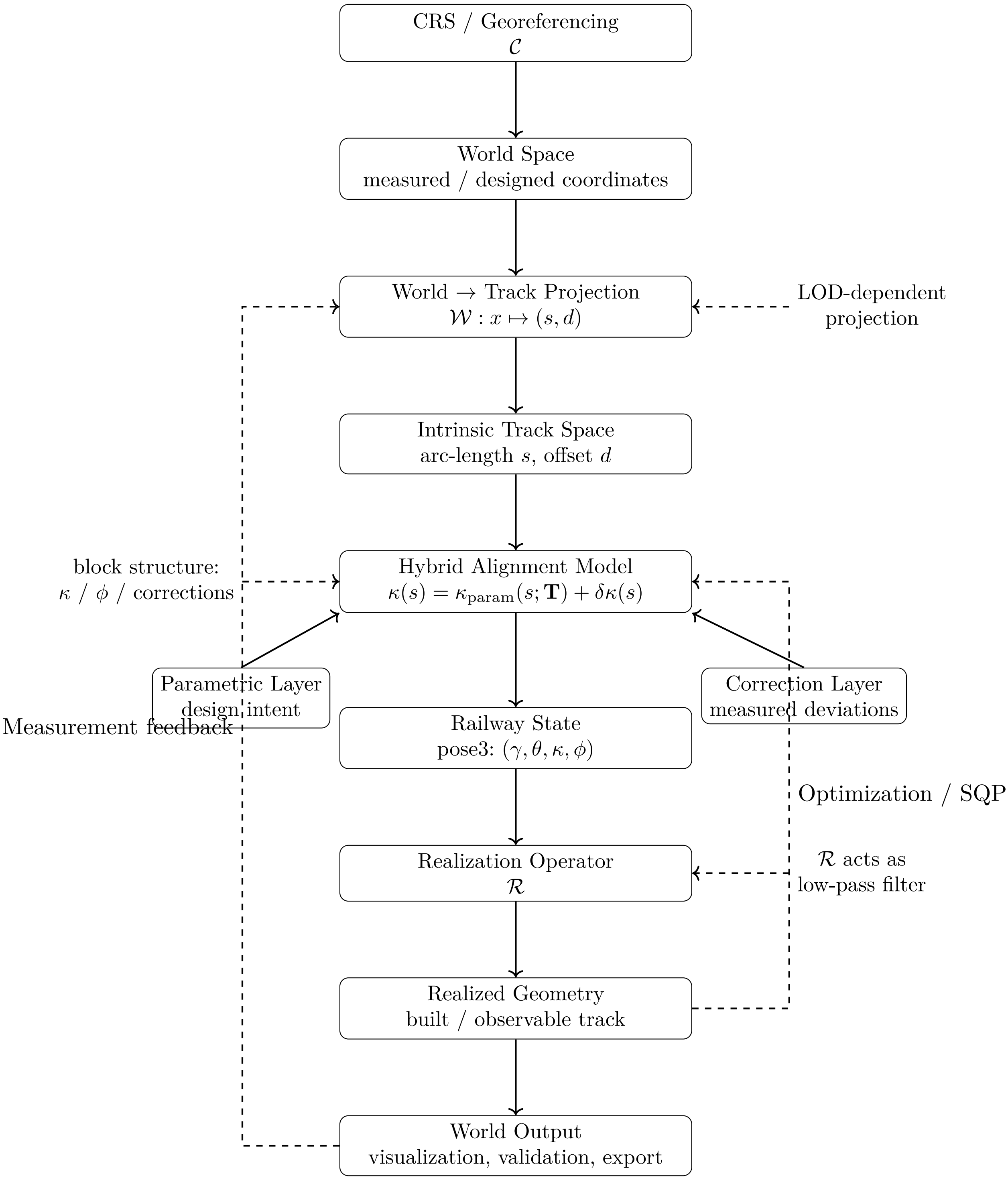

Abbildung 44.2.: Level-2 AIM master architecture. The framework combines CRS handling, nonlinear world-to-track projection, hybrid curvature modelling, pose3 state reconstruction, realization, and optimization feedback. The central modelling object is not the geometric curve itself, but the curvature-driven and realization-aware operator system producing it.

Teil X.

Optimization

Remark 44.98. This part formulates railway alignment design and reconstruction as optimization problems. Within the AIM framework, optimization is not an external add-on, but a natural consequence of curvature-based modelling.

Remark 44.99. The main focus lies on:

- continuous and discrete formulations,
- structured residual models,
- analytical Jacobians,
- sparse and block-structured solution methods.

Remark 44.100. The following chapters show how alignment modelling leads directly to an efficient and interpretable optimization framework.

45. Numerical Reconstruction

45.1. Motivation

Remark 45.1. In the curvature-based framework, geometric objects are not given explicitly as coordinate sets. Instead, they are reconstructed from curvature information by integration.

Remark 45.2. As a consequence, numerical methods are not merely implementation details, but an essential part of the modelling process.

—

45.2. Reconstruction Principle

Definition 45.3 (Reconstruction from curvature)**.** Let $\kappa(s)$ be a curvature function and let

$$(x_0, y_0, \theta_0)$$

be a start pose. The corresponding curve is obtained by solving

$$\theta'(s) = \kappa(s), \qquad \gamma'(s) = (\cos\theta(s), \sin\theta(s)).$$

Remark 45.4. Even if κ is given analytically, closed-form solutions for γ are generally not available.

—

45.3. Discretization

Definition 45.5 (Discretization)**.** A discretization of the parameter interval $[0, L]$ is a finite sequence

$$0 = s_0 < s_1 < \cdots < s_n = L.$$

Definition 45.6 (Discrete reconstruction)**.** A discrete reconstruction computes approximate values

$$\gamma(s_i), \quad \theta(s_i)$$

at the discretization points.

Remark 45.7. The choice of discretization strongly influences both computational cost and geometric accuracy.

TODO: Formal criteria for discretization strategies (uniform vs adaptive).

—

45.4. Integration Methods

Remark 45.8. The reconstruction problem can be formulated as an initial value problem for a system of ordinary differential equations.

45.4.1. Basic Scheme

Definition 45.9 (Explicit step)**.** Given (γ_i, θ_i) at s_i, an explicit step computes

$$\theta_{i+1} = \theta_i + \kappa(s_i)\,\Delta s,$$

$$\gamma_{i+1} = \gamma_i + (\cos\theta_i, \sin\theta_i)\,\Delta s.$$

Remark 45.10. This corresponds to a first-order method and is generally insufficient for high-precision reconstruction.

45.4.2. Higher-Order Methods

TODO: Runge–Kutta methods for coupled systems.

TODO: Romberg integration for curvature integration.

TODO: Adaptive step size control.

OPEN: Separation of curvature integration and geometric integration.

—

45.5. Error Analysis

Definition 45.11 (Local error)**.** The local error of a step is the deviation introduced at a single integration step.

Definition 45.12 (Global error)**.** The global error is the accumulated deviation over the entire interval.

Remark 45.13. Errors arise from:

- discretization,
- numerical integration,
- approximation of curvature laws.

RESEARCH: Quantitative bounds for error propagation in curvature-driven reconstruction.

45.6. Sensitivity

Remark 45.14. The reconstruction process is sensitive to perturbations in curvature. Small deviations in κ may lead to significant geometric drift.

RESEARCH: Sensitivity analysis with respect to curvature perturbations.
OPEN: Relation between curvature noise and positional uncertainty.
—

45.7. Sampling and Representation

Definition 45.15 (Sampling)**.** Sampling refers to the extraction of a finite set of points from a continuous curve representation.

Remark 45.16. Sampling is required for:

- visualization,
- collision detection,
- export to point-based formats.

TODO: Error-controlled sampling strategies.
OPEN: Difference between sampling for visualization and sampling for computation.
—

45.8. Reconstruction in the Sparse Model

Remark 45.17. In the sparse alignment model, reconstruction is performed element-wise. Each element contributes a local curvature law, which is integrated sequentially.

Definition 45.18 (Sequential reconstruction)**.** Let $(e_1, \dots, e_n)$ be a sparse alignment. The reconstruction proceeds by integrating each element in sequence, using the end state of the previous element as initial state for the next.

OPEN: Handling of numerical drift across element boundaries.
—

45.9. Numerical Stability

Remark 45.19. Numerical stability is a central concern, especially for long alignments or highly curved geometries.

RESEARCH: Stability criteria for curvature-based integration.
TODO: Techniques for drift correction and re-normalization.
—

45.10. Connection to Optimization

Remark 45.20. Numerical reconstruction is closely linked to optimization problems. In fitting and design tasks, reconstruction is performed repeatedly within iterative algorithms.

TODO: SQP formulation for alignment fitting.

RESEARCH: Connection to optimal control problems.

—

45.11. Summary

Summary 45.21. Numerical reconstruction is the process of deriving geometric representations from curvature data by integration.

It introduces discretization, approximation, and error propagation as intrinsic components of the modelling framework.

Therefore, numerical methods are not merely technical tools, but an essential layer connecting mathematical definitions with practical geometry.

45.12. Numerical Reconstruction from Curvature

45.12.1. Problem Statement

Remark 45.22. Given a curvature function

$$\kappa : [0, L] \to \mathbb{R},$$

and initial conditions

$$\gamma(0), \quad \theta(0),$$

the geometric alignment is obtained by solving:

$$\gamma'(s) = (\cos\theta(s), \sin\theta(s)), \qquad \theta'(s) = \kappa(s).$$

Remark 45.23. This defines a nonlinear initial value problem.

—

45.12.2. Discrete Approximation

Definition 45.24 (Discretization)**.** Let

$$0 = s_0 < s_1 < \cdots < s_N = L$$

be a partition with step sizes Δs_i.

Remark 45.25. We approximate:

$$\kappa_i \approx \kappa(s_i).$$

—

45.12.3. Forward Integration Scheme

Definition 45.26 (Explicit integration)**.** Given (γ_i, θ_i), define:

$$\theta_{i+1} = \theta_i + \kappa_i \Delta s_i,$$

$$\gamma_{i+1} = \gamma_i + \Delta s_i (\cos\theta_i, \sin\theta_i).$$

Remark 45.27. This corresponds to an explicit Euler scheme.

—

45.12.4. Midpoint Scheme

Definition 45.28 (Midpoint method)**.** Define:

$$\theta_{i+\frac{1}{2}} = \theta_i + \frac{1}{2}\kappa_i \Delta s_i,$$

$$\gamma_{i+1} = \gamma_i + \Delta s_i (\cos\theta_{i+\frac{1}{2}}, \sin\theta_{i+\frac{1}{2}}),$$

$$\theta_{i+1} = \theta_i + \kappa_i \Delta s_i.$$

Remark 45.29. This improves accuracy and stability compared to the explicit scheme.

—

45.12.5. Extension to Pose3

Remark 45.30. Including cant, we obtain:

$$\phi_{i+1} = \phi_i + \omega_i \Delta s_i.$$

Remark 45.31. Thus, the full discrete system is:

$$(\gamma_i, \theta_i, \phi_i) \longrightarrow (\gamma_{i+1}, \theta_{i+1}, \phi_{i+1}).$$

—

45.12.6. Numerical Stability

Remark 45.32. The reconstruction is sensitive to:

- step size Δs,

- noise in κ_i,
- and large coordinate values.

Remark 45.33. To ensure stability:

- sufficiently small step sizes must be used,
- curvature should be smoothed,
- and local coordinate frames are recommended.

—

45.12.7. Floating Origin Technique

Remark 45.34. For large coordinates, numerical precision degrades.

Definition 45.35 (Floating origin). A local coordinate system is introduced such that:

$$\gamma_i^{\text{local}} = \gamma_i - \gamma_{\text{ref}}.$$

Remark 45.36. This reduces numerical errors in computation and visualization.

—

45.12.8. Consistency with CRS

Remark 45.37. Reconstruction depends on:

- initial position,
- initial orientation,
- and coordinate system definition.

Remark 45.38. Thus, numerical reconstruction must be performed within a consistent CRS.

—

45.12.9. Interpretation

Summary 45.39. Numerical reconstruction provides the link between curvature-based models and geometric representation.

It transforms abstract functions into concrete spatial geometry, while preserving the structure required for analysis and optimization.

46. Optimization of Alignments

46.1. Motivation

Remark 46.1. In practical applications, alignments are not only reconstructed from data, but also designed, modified, and improved.

Remark 46.2. This leads naturally to optimization problems in which alignment parameters are adjusted to satisfy geometric, physical, and regulatory constraints, as already emphasized in railway alignment literature [30, 24].

46.2. General Optimization Problem

Definition 46.3 (Alignment optimization problem). An alignment optimization problem consists of:

- a parameterized alignment model,
- an objective function,
- a set of constraints.

Definition 46.4 (Objective function). An objective function is a mapping

$$J : \mathcal{A} \to \mathbb{R}$$

that assigns a cost or quality measure to an alignment $\mathcal{A}$.

Remark 46.5. Typical objectives include:

- minimizing curvature variation,
- minimizing deviation from measurement data,
- minimizing construction cost,
- maximizing comfort.

Such objective classes are consistent with both general numerical optimization literature [36] and railway-specific optimization approaches [30].

46.3. Parameterization

Remark 46.6. The choice of parameters is central to the formulation of the optimization problem.

Definition 46.7 (Parameter vector)**.** An alignment is described by a parameter vector

$$\mathbf{x} \in \mathbb{R}^n$$

encoding quantities such as:

- element lengths,
- curvature values,
- transition parameters,
- geometric constraints.

Remark 46.8. The sparse alignment model provides a natural parameterization by element-wise variables.

TODO: Explicit parameter sets for fixed and transition elements.

46.4. Constraints

Definition 46.9 (Constraint)**.** A constraint is a condition of the form

$$g_i(\mathbf{x}) \leq 0, \qquad h_j(\mathbf{x}) = 0.$$

Remark 46.10. Constraints arise from:

- geometric continuity,
- engineering design rules,
- physical limitations,
- regulatory requirements.

46.4.1. Geometric Constraints

- continuity of position and direction,
- continuity of curvature.

46.4.2. Engineering Constraints

- minimum radius,
- maximum cant,
- limits on curvature derivatives.

Remark 46.11. Such conditions reflect standard railway design practice and are closely related to design rules discussed in engineering literature and standards [24, 6].

OPEN: Formal representation of engineering rules in constraint form.

46.5. Relation to Numerical Reconstruction

Remark 46.12. Evaluation of the objective function requires reconstruction of the alignment geometry from the parameter vector.

Remark 46.13. Thus, optimization is intrinsically linked to numerical reconstruction, as discussed in the previous chapter. From a numerical perspective, this places alignment optimization within the broader context of differential equations and numerical approximation [11, 41].

46.6. Sequential Quadratic Programming

Remark 46.14. Sequential Quadratic Programming (SQP) is a powerful method for solving nonlinear constrained optimization problems [36].

Definition 46.15 (SQP iteration)**.** An SQP method approximates the original problem by a sequence of quadratic subproblems.

Remark 46.16. Each iteration solves a quadratic approximation of the objective function subject to linearized constraints.

TODO: Explicit formulation of SQP for alignment parameters.
RESEARCH: Adaptation of SQP to sparse alignment structures.

46.7. Alignment Optimization as Control Problem

Remark 46.17. The curvature function $\kappa(s)$ may be interpreted as a control variable in a continuous system.

Definition 46.18 (Control formulation)**.** The alignment problem can be formulated as an optimal control problem with state variables (γ, θ) and control κ.

Remark 46.19. This viewpoint is compatible with the curvature-centred perspective on railway alignment advocated by Kufver [31, 30].

RESEARCH: Connection between alignment optimization and optimal control theory.

46.8. Network-Level Optimization

Remark 46.20. Real-world railway systems involve not only single alignments, but networks with branching and merging structures.

RESEARCH: Extension of optimization from single alignments to networks.

OPEN: Handling of switches, crossings, and parallel tracks.

46.9. Robustness and Uncertainty

Remark 46.21. Optimization must account for uncertainty in input data.

RESEARCH: Robust optimization under uncertain geometry.

OPEN: Trade-off between optimality and robustness.

46.10. Implementation Perspective

Remark 46.22. Practical optimization systems must balance:

- numerical stability,
- computational efficiency,
- model expressiveness.

Remark 46.23. This balance is central both in generic numerical optimization [36] and in railway-specific optimization settings [30].

TODO: Integration of optimization into the modelling workflow.

46.11. Historical Context

Remark 46.24. Classical systems such as AXTRAN have demonstrated the feasibility of optimization-based alignment design.

Remark 46.25. Modern approaches aim to extend these ideas using more flexible models and improved numerical methods, in line with the development from classical railway design methods toward explicit optimization formulations [30].

TODO: Reconstruction and generalization of AXTRAN-like methods.

46.12. Formal Optimization in Curvature Space

46.12.1. Problem Setting

Remark 46.26. Let $L > 0$ be fixed and let

$$\kappa : [0, L] \to \mathbb{R}$$

denote the curvature function of a transition.

Remark 46.27. We assume the boundary conditions

$$\kappa(0) = 0, \qquad \kappa(L) = \kappa_1,$$

and optionally

$$\kappa'(0) = 0, \qquad \kappa'(L) = 0.$$

46.12.2. Admissible Set

Definition 46.28 (Admissible transition set)**.**

$$\mathcal{A} = \left\{ \kappa \in H^1(0, L) \mid \kappa(0) = 0,\ \kappa(L) = \kappa_1 \right\}.$$

46.12.3. Variational Problem

Definition 46.29 (Quadratic curvature-gradient functional)**.**

$$J[\kappa] = \int_0^L \left(\kappa'(s)\right)^2 \mathrm{d}s.$$

Definition 46.30 (Optimization problem)**.** Find $\kappa^* \in \mathcal{A}$ such that

$$J[\kappa^*] = \min_{\kappa \in \mathcal{A}} J[\kappa].$$

46.12.4. Euler–Lagrange Equation

Proposition 46.31. *If κ^* is a minimizer, then*

$$\kappa^{*\prime\prime}(s) = 0.$$

46.12.5. Interpretation

Proposition 46.32. *The minimizer is*

$$\kappa^*(s) = \frac{\kappa_1}{L} s.$$

Remark 46.33. Thus, the clothoid appears as the solution of a variational problem. This connects classical railway transition design with an optimization view in curvature space and provides a formal counterpart to the historical role of the clothoid in railway engineering [9, 13].

46.12.6. Higher-Order Model

Definition 46.34.

$$J_2[\kappa] = \int_0^L \left(\kappa''(s)\right)^2 \mathrm{d}s.$$

Proposition 46.35. *Minimizers satisfy*

$$\kappa^{(4)}(s) = 0.$$

Remark 46.36. This explains the occurrence of cubic transition laws and links them to modern polynomial transition design [51].

46.12.7. Generalized View

Remark 46.37. General functionals of the form

$$J[\kappa] = \int_0^L F(\kappa, \kappa', \kappa''; v, c)\, \mathrm{d}s$$

allow incorporation of dynamic and engineering criteria.

46.12.8. Connection to the Main Thesis

Summary 46.38. Transition design can be formulated as an optimization problem in curvature space.

Classical transition curves arise as solutions of such problems, supporting the interpretation of transitions as curvature control functions.

46.13. Coupled Optimization of Curvature and Cant

46.13.1. Motivation

Remark 46.39. In railway applications, curvature and cant cannot be optimized independently. Both jointly determine the effective lateral acceleration and its rate of change.

Remark 46.40. This motivates a coupled optimization problem in which both

$$\kappa(s) \quad \text{and} \quad \phi(s)$$

are treated as design variables.

Remark 46.41. Such coupling is fully consistent with railway alignment design standards, where curvature, speed, and cant are not independent design quantities [6, 24].

46.13.2. Effective Acceleration Functional

Definition 46.42 (Effective lateral acceleration)**.** The effective lateral acceleration is

$$a_{\text{eff}}(s) = v^2\kappa(s) - g\sin\phi(s).$$

Remark 46.43. A natural optimization objective is to reduce the magnitude of unbalanced lateral acceleration.

Definition 46.44 (Acceleration-based functional)**.** Define

$$J_a[\kappa,\phi] = \int_0^L a_{\text{eff}}(s)^2 \,\mathrm{d}s = \int_0^L \left(v^2\kappa(s) - g\sin\phi(s)\right)^2 \mathrm{d}s.$$

46.13.3. Jerk-Based Functional

Definition 46.45 (Jerk)**.** The jerk is given by

$$j(s) = v^3\kappa'(s) - gv\cos\phi(s)\,\phi'(s).$$

Definition 46.46 (Jerk functional)**.** Define

$$J_j[\kappa,\phi] = \int_0^L j(s)^2 \,\mathrm{d}s.$$

That is,

$$J_j[\kappa,\phi] = \int_0^L \left(v^3\kappa'(s) - gv\cos\phi(s)\,\phi'(s)\right)^2 \mathrm{d}s.$$

Remark 46.47. This functional penalizes rapid variation of effective acceleration and therefore provides a natural dynamic smoothness criterion. Such criteria are closely related to engineering concerns about comfort and dynamic compatibility [6, 4].

46.13.4. Combined Objective

Definition 46.48 (Coupled objective functional)**.** A general coupled objective may be written as

$$J[\kappa,\phi] = \alpha J_a[\kappa,\phi] + \beta J_j[\kappa,\phi] + \gamma J_\kappa[\kappa] + \delta J_\phi[\phi],$$

where:

$$J_\kappa[\kappa] = \int_0^L (\kappa'(s))^2 \,\mathrm{d}s, \qquad J_\phi[\phi] = \int_0^L (\phi'(s))^2 \,\mathrm{d}s.$$

Remark 46.49. The weights

$$\alpha,\beta,\gamma,\delta \geq 0$$

determine the relative importance of equilibrium, comfort, curvature smoothness, and cant smoothness.

46.13.5. Admissible Set

Definition 46.50 (Admissible railway state set)**.** Let

$$\mathcal{B} = \left\{ (\kappa,\phi) \;\middle|\; \kappa \in H^1(0,L),\ \phi \in H^1(0,L),\ \kappa(0)=0,\ \kappa(L)=\kappa_1 \right\}.$$

Remark 46.51. Additional boundary conditions may be imposed, for example:

$$\phi(0)=0, \qquad \phi(L)=\phi_1,$$

or derivative constraints such as

$$\kappa'(0) = \kappa'(L) = 0, \qquad \phi'(0) = \phi'(L) = 0.$$

46.13.6. Coupled Optimization Problem

Definition 46.52 (Coupled railway optimization problem)**.** Find

$$(\kappa^*, \phi^*) \in \mathcal{B}$$

such that

$$J[\kappa^*, \phi^*] = \min_{(\kappa,\phi)\in\mathcal{B}} J[\kappa, \phi].$$

46.13.7. Interpretation

Remark 46.53. This problem extends classical transition design in two directions:

- from pure geometry to coupled geometry–cant design,
- from curve selection to functional optimization.

Remark 46.54. In this framework, the transition curve is no longer chosen from a catalogue, but emerges as part of an optimal coupled state.

46.13.8. Special Cases

Proposition 46.55. *If $\phi(s) \equiv 0$, the coupled problem reduces to a pure curvature optimization problem.*

Proposition 46.56. *If $a_{\text{eff}}(s) = 0$ for all s, then*

$$v^2\kappa(s) = g \sin \phi(s),$$

and the design is perfectly balanced.

Remark 46.57. These special cases show that classical equilibrium design appears as a subcase of the coupled optimization framework.

46.13.9. Relation to Optimal Control

Remark 46.58. The coupled optimization problem may be interpreted as an optimal control problem with controls

$$\kappa(s), \quad \phi'(s),$$

state variables

$$x(s), y(s), \theta(s), \phi(s),$$

and cost functional $J[\kappa, \phi]$.

Remark 46.59. This provides a direct bridge from railway alignment design to modern control-theoretic and variational methods.

46.13.10. Connection to the Main Thesis

Summary 46.60. The coupled optimization problem shows that railway alignment design is not a purely geometric task.

Instead, it is the optimization of a coupled state system in which curvature and cant jointly determine dynamic behaviour.

This supports the central thesis of the manuscript that transition curves and cant evolution should be understood as control functions in a curvature-based railway state model.

46.14. Summary

Summary 46.61. Alignment optimization treats geometry as a variable rather than a fixed input.

It combines parameterization, numerical reconstruction, and constrained optimization methods to produce alignments that satisfy both geometric and engineering requirements.

46.15. Discrete Formulation of Alignment Optimization

46.15.1. Discretization of the Domain

Definition 46.62 (Discretization). Let

$$0 = s_0 < s_1 < \cdots < s_N = L$$

be a partition of the interval $[0, L]$.

Remark 46.63. We denote

$$\Delta s_i = s_{i+1} - s_i.$$

46.15.2. Discrete Variables

Definition 46.64 (Discrete curvature). Let

$$\kappa_i \approx \kappa(s_i)$$

denote the curvature values at the nodes.

Definition 46.65 (Discrete cant). Let

$$\phi_i \approx \phi(s_i)$$

denote the cant values.

Definition 46.66 (Parameter vector)**.** The optimization variable is

$$\mathbf{x} = (\kappa_0, \dots, \kappa_N, \phi_0, \dots, \phi_N) \in \mathbb{R}^{2(N+1)}.$$

46.15.3. Discrete Pose Reconstruction

Remark 46.67*.* The pose3 state is reconstructed by numerical integration.

Definition 46.68 (Discrete integration)**.** Given initial conditions

$$\gamma_0,\ \theta_0,\ \phi_0,$$

we compute iteratively:

$$\theta_{i+1} = \theta_i + \kappa_i \Delta s_i,$$

$$\gamma_{i+1} = \gamma_i + \Delta s_i (\cos\theta_i, \sin\theta_i),$$

$$\phi_{i+1} = \phi_i + \omega_i \Delta s_i.$$

Remark 46.69*.* Here, ω_i may be expressed via differences of ϕ_i. This discrete reconstruction corresponds to standard numerical integration ideas [11, 41].

46.15.4. Finite Differences

Definition 46.70 (First derivative)**.** Approximate derivatives by

$$\kappa'(s_i) \approx \frac{\kappa_{i+1} - \kappa_i}{\Delta s_i}, \quad \phi'(s_i) \approx \frac{\phi_{i+1} - \phi_i}{\Delta s_i}.$$

Definition 46.71 (Second derivative)**.**

$$\kappa''(s_i) \approx \frac{\kappa_{i+1} - 2\kappa_i + \kappa_{i-1}}{(\Delta s_i)^2}.$$

46.15.5. Discrete Objective Functions

Curvature Smoothness

$$J_\kappa(\mathbf{x}) = \sum_{i=0}^{N-1} \left(\frac{\kappa_{i+1} - \kappa_i}{\Delta s_i} \right)^2 \Delta s_i.$$

Cant Smoothness

$$J_\phi(\mathbf{x}) = \sum_{i=0}^{N-1} \left(\frac{\phi_{i+1} - \phi_i}{\Delta s_i} \right)^2 \Delta s_i.$$

Dynamic Objective

$$J_{\text{dyn}}(\mathbf{x}) = \sum_{i=0}^{N} (v^2 \kappa_i - g \sin\phi_i)^2 \Delta s_i.$$

Schwerpunkt Objective

Remark 46.72. Using finite differences, we approximate

$$\|\gamma_c''(s_i)\|^2$$

by second-order differences of γ_c.

$$J_{\mathrm{veh}}(\mathbf{x}) = \sum_{i=1}^{N-1} \|\gamma_{c,i+1} - 2\gamma_{c,i} + \gamma_{c,i-1}\|^2.$$

46.15.6. Combined Discrete Problem

Definition 46.73 (Discrete objective)**.**

$$J(\mathbf{x}) = \alpha J_{\mathrm{veh}} + \beta J_{\mathrm{dyn}} + \gamma J_\kappa + \delta J_\phi.$$

46.15.7. Constraints

Definition 46.74 (Equality constraints)**.**

$$\kappa_0 = 0, \quad \kappa_N = \kappa_1.$$

Definition 46.75 (Inequality constraints)**.**

$$|\kappa_i| \leq \kappa_{\max}, \quad |\phi_i| \leq \phi_{\max}.$$

Remark 46.76. Further constraints may include:

- bounds on derivatives,
- continuity conditions,
- alignment anchoring points.

46.15.8. Final Optimization Problem

Definition 46.77 (Discrete optimization problem)**.** Find

$$\mathbf{x}^* \in \mathbb{R}^{2(N+1)}$$

such that

$$J(\mathbf{x}^*) = \min_{\mathbf{x}} J(\mathbf{x})$$

subject to the given constraints.

46.15.9. Structure of the Problem

Remark 46.78. The problem has the structure of a nonlinear least-squares problem.

Remark 46.79. It is therefore well suited for:

- Sequential Quadratic Programming,
- Gauss–Newton methods,
- sparse optimization techniques.

46.15.10. Connection to Implementation

Remark 46.80. The discrete formulation directly corresponds to a computational model:

- κ_i correspond to element parameters,
- ϕ_i correspond to cant samples,
- reconstruction matches numerical evaluation in software.

Summary 46.81. The continuous alignment optimization problem can be reduced to a finite dimensional nonlinear optimization problem.

This provides the direct link between theoretical modelling and practical implementation.

46.16. Sequential Quadratic Programming (SQP)

46.16.1. Problem Structure

Remark 46.82. We consider the discrete optimization problem

$$\min_{\mathbf{x}} J(\mathbf{x}) \quad \text{subject to} \quad g(\mathbf{x}) = 0,\ h(\mathbf{x}) \leq 0.$$

Definition 46.83 (Parameter vector)**.**

$$\mathbf{x} = (\kappa_0, \ldots, \kappa_N, \phi_0, \ldots, \phi_N).$$

46.16.2. Residual Formulation

Definition 46.84 (Residual vector)**.**

$$J(\mathbf{x}) = \frac{1}{2}\|\mathbf{r}(\mathbf{x})\|^2.$$

Remark 46.85. Typical residual components:

$$r_i^{(\kappa)} = \sqrt{\gamma}\frac{\kappa_{i+1} - \kappa_i}{\Delta s}, \quad r_i^{(\phi)} = \sqrt{\delta}\frac{\phi_{i+1} - \phi_i}{\Delta s},$$

$$r_i^{(\text{dyn})} = \sqrt{\beta}(v^2\kappa_i - g\sin\phi_i).$$

46.16.3. Gauss–Newton Step

Definition 46.86 (SQP step)**.** At iteration k, compute $\mathbf{p}_k$ from:

$$(J_r^T J_r)\mathbf{p}_k = -J_r^T \mathbf{r}.$$

Remark 46.87. The Hessian is approximated by:

$$H \approx J_r^T J_r.$$

46.16.4. Sparse Structure

Remark 46.88. The system matrix has band structure:

- tridiagonal coupling for smoothness terms,
- diagonal contributions from dynamic terms.

46.16.5. Constraints Handling

Remark 46.89. Constraints are handled via:

- projection,
- penalty terms,
- Lagrange multipliers.

46.16.6. Iteration Scheme

Algorithm 1 SQP iteration

initialize $\mathbf{x}_0$
for $k = 0, 1, \ldots$ **do**
  compute $\mathbf{r}(\mathbf{x}_k)$
  compute $J_r(\mathbf{x}_k)$
  solve $(J_r^T J_r)\mathbf{p}_k = -J_r^T \mathbf{r}$
  choose step size α_k
  update $\mathbf{x}_{k+1} = \mathbf{x}_k + \alpha_k \mathbf{p}_k$
end for

46.16.7. Interpretation

Summary 46.90. SQP provides a practical and efficient method for solving the discrete alignment optimization problem.

Its efficiency follows from the sparse and local structure of the underlying model.

46.17. Construction of Initial Guesses

46.17.1. Motivation

Remark 46.91. The performance of SQP methods strongly depends on the choice of the initial parameter vector.

Remark 46.92. In railway applications, initial data is often available in the form of:

- measurement polylines,
- existing alignments,
- partially structured design data.

46.17.2. Input Representation

Definition 46.93 (Input polyline)**.** Let

$$P = (p_0, \dots, p_M), \quad p_i \in \mathbb{R}^2$$

denote a measured or imported polyline.

46.17.3. Arc-Length Parameterization

Definition 46.94. Define cumulative arc-length

$$s_0 = 0, \quad s_{i+1} = s_i + \|p_{i+1} - p_i\|.$$

Remark 46.95. This defines a parameterization of the input data.

46.17.4. Initial Curvature Estimation

Definition 46.96 (Discrete curvature)**.** For three consecutive points, define:

$$\kappa_i \approx \frac{\theta_{i+1} - \theta_i}{\Delta s_i},$$

where

$$\theta_i = \arg(p_{i+1} - p_i).$$

Remark 46.97. This provides a first estimate of curvature from geometric data.

46.17.5. Smoothing

Remark 46.98. Raw curvature estimates are typically noisy.

Definition 46.99 (Smoothing operator)**.** Apply a smoothing filter:

$$\kappa_i^{(0)} = \sum_j w_{ij} \kappa_j.$$

Remark 46.100. Possible choices:

- moving average,
- Gaussian filter,
- low-pass filtering.

46.17.6. Initial Cant Estimation

Remark 46.101. Cant may be initialized based on equilibrium:

$$\phi_i^{(0)} = \arcsin\left(\frac{v^2 \kappa_i^{(0)}}{g}\right).$$

Remark 46.102. Alternatively, cant may be:

- set to zero,
- taken from input data,
- approximated by design rules.

46.17.7. Projection to Discrete Grid

Definition 46.103. Interpolate the values $\kappa_i^{(0)}, \phi_i^{(0)}$ onto the optimization grid:

$$s_0, \dots, s_N.$$

46.17.8. Heuristic Segmentation

Remark 46.104. The input alignment may be segmented into:

- straight sections,
- circular arcs,
- transition regions.

Definition 46.105 (Segment detection)**.** A segment is classified based on curvature:

$$\kappa_i \approx 0 \quad \Rightarrow \text{straight},$$

$$\kappa_i \approx \text{const} \quad \Rightarrow \text{arc}.$$

Remark 46.106. This structure can be used to improve the initial guess and is compatible with railway alignment reconstruction approaches such as those discussed by Kufver [31, 30].

46.17.9. Final Initial Guess

Definition 46.107 (Initial parameter vector)**.** The initial guess is

$$\mathbf{x}_0 = (\kappa_0^{(0)}, \dots, \kappa_N^{(0)}, \phi_0^{(0)}, \dots, \phi_N^{(0)}).$$

46.17.10. Interpretation

Remark 46.108. The initial guess combines:

- measured geometry,
- smoothing,
- physical approximation.

Summary 46.109. A robust initial guess transforms raw geometric input into a structured parameter vector suitable for optimization.

This step is essential for bridging real-world data and numerical methods.

46.18. Analytical Structure of the Optimization Problem

46.18.1. Motivation

Remark 46.110. In classical numerical optimization, Jacobians are often computed by finite differences or automatic differentiation.

Remark 46.111. Within the AIM framework, this is not necessary. The alignment model is inherently differentiable.

46.18.2. Fundamental Observation

Proposition 46.112. *All state variables are obtained via arc-length integration:*

$$\theta(s) = \theta_0 + \int_0^s \kappa(\sigma)\,\mathrm{d}\sigma,$$

$$\gamma(s) = \gamma_0 + \int_0^s (\cos\theta, \sin\theta)\,\mathrm{d}\sigma.$$

46.18.3. Parameter Types

Definition 46.113. Primary parameter classes:

- curvature parameters dK,
- length parameters dL,
- transition parameters dT.

46.18.4. Analytical Derivatives

Proposition 46.114.
$$\frac{\partial \theta(s)}{\partial p} = \int_0^s \frac{\partial \kappa}{\partial p} \, \mathrm{d}\sigma.$$

Proposition 46.115.
$$\frac{\partial \gamma(s)}{\partial p} = \int_0^s \begin{pmatrix} -\sin\theta \\ \cos\theta \end{pmatrix} \frac{\partial \theta}{\partial p} \, \mathrm{d}\sigma.$$

Remark 46.116. All derivatives reduce to arc-length integrals.

46.18.5. Key Insight

Proposition 46.117. *The optimization problem admits exact Jacobians without numerical approximation.*

Remark 46.118. This eliminates:

- finite differences,
- numerical instability,
- high computational cost.

46.18.6. Block Structure

Remark 46.119. The parameter vector decomposes as:

$$(\boldsymbol{\kappa}, \boldsymbol{\phi}).$$

Remark 46.120. This induces a block structure:

$$H = \begin{pmatrix} H_{\kappa\kappa} & H_{\kappa\phi} \\ H_{\phi\kappa} & H_{\phi\phi} \end{pmatrix}.$$

46.18.7. Interpretation

Summary 46.121. The structure of the optimization problem is not imposed numerically, but follows directly from the curvature-based modelling approach.

This yields:

- exact derivatives,
- sparse matrices,
- natural block structure.

This is a central advantage of the AIM framework.

46.19. Parametric Representation instead of Sampling

46.19.1. Motivation

Remark 46.122. Classical discretization represents curvature and cant by samples:

$$\kappa_i \approx \kappa(s_i), \qquad \phi_i \approx \phi(s_i).$$

Remark 46.123. This leads to high-dimensional optimization problems with weak structure and limited interpretability.

Remark 46.124. Within the AIM framework, curvature and cant are instead represented by parameterized functions.

—

46.19.2. Parametric Curvature Model

Definition 46.125 (Parametric curvature)**.** A transition is described by:

$$\kappa(s) = \kappa_0 + (\kappa_1 - \kappa_0)\widehat{\kappa}\left(\frac{s}{L}, \mathbf{T}\right),$$

where:

- L is the element length,
- $\mathbf{T}$ is a vector of transition parameters,
- $\widehat{\kappa}$ is a normalized prototype function.

—

46.19.3. Parametric Cant Model

Definition 46.126 (Parametric cant)**.** Cant is represented analogously:

$$\phi(s) = \phi_0 + (\phi_1 - \phi_0)\widehat{\phi}\left(\frac{s}{L}, \mathbf{T}_\phi\right).$$

—

46.19.4. Parameter Vector

Definition 46.127 (Reduced parameter vector)**.** The optimization variable becomes:

$$\mathbf{x} = (L, \kappa_0, \kappa_1, \mathbf{T}, \phi_0, \phi_1, \mathbf{T}_\phi).$$

Remark 46.128. The dimension is independent of discretization density.

—

46.19.5. Dimensional Reduction

Remark 46.129. A sampled representation uses $O(N)$ variables.

Remark 46.130. The parametric representation uses:

$$O(1) \text{ parameters per element.}$$

Remark 46.131. This yields a drastic reduction in problem size.

—

46.19.6. Derivative Structure

Proposition 46.132. *Derivatives reduce to:*

$$\frac{\partial \kappa}{\partial p} = (\kappa_1 - \kappa_0)\frac{\partial \widehat{\kappa}}{\partial p}.$$

Remark 46.133. Thus, all derivatives are computed analytically from prototype functions.

—

46.19.7. Integration into Reconstruction

Remark 46.134. The parametric representation is evaluated through:

$$\theta(s) = \int \kappa(s)\,\mathrm{d}s, \qquad \gamma(s) = \int (\cos\theta, \sin\theta)\,\mathrm{d}s.$$

Remark 46.135. This yields a smooth and consistent geometry without discretization artifacts.

—

46.19.8. Comparison to Sample-Based Methods

	Sample-based	Parametric
dimension	high	low
smoothness	enforced numerically	intrinsic
interpretability	low	high
constraints	many local	few global

—

46.19.9. Interpretation

Remark 46.136. The parametric approach shifts the problem from:

- fitting values

to:

- designing functions.

—

46.19.10. Connection to Transition Families

Remark 46.137. Different transition curves correspond to different prototype functions $\widehat{\kappa}$.

Remark 46.138. Thus, the optimization problem includes selection and tuning of transition families.

—

46.19.11. Computational Implications

- smaller optimization problems,
- better conditioning,
- faster convergence,
- direct integration with analytical Jacobians.

—

46.19.12. Connection to Classical Systems

Remark 46.139. This formulation is conceptually close to classical systems such as AXTRAN, which operate on element parameters rather than sampled data.

—

46.19.13. Connection to AIM

Summary 46.140. The parametric representation replaces discrete sampling by structured functions.

It enables a compact, interpretable, and analytically differentiable model, which forms the basis for efficient and robust alignment optimization.

This approach is central to the AIM philosophy.

46.20. Hybrid Parametric–Discrete Model

46.20.1. Motivation

Remark 46.141. Purely parametric models provide low-dimensional structure but may lack flexibility for fitting real-world data.

Remark 46.142. Purely sampled models provide flexibility but lead to large and ill-conditioned optimization problems.

Remark 46.143. A hybrid approach combines both advantages.

—

46.20.2. Hybrid Representation

Definition 46.144 (Hybrid curvature model)**.**

$$\kappa(s) = \kappa_{\text{param}}(s; \mathbf{T}) + \delta\kappa(s),$$

where:

- κ_{param} is a parametric base model,
- $\delta\kappa(s)$ is a correction term.

Definition 46.145 (Hybrid cant model)**.**

$$\phi(s) = \phi_{\text{param}}(s; \mathbf{T}_\phi) + \delta\phi(s).$$

—

46.20.3. Discretization of Correction

Remark 46.146. The correction term is represented discretely:

$$\delta\kappa_i = \delta\kappa(s_i), \qquad \delta\phi_i = \delta\phi(s_i).$$

—

46.20.4. Extended Parameter Vector

Definition 46.147.

$$\mathbf{x} = (\mathbf{T}, \mathbf{T}_\phi, \delta\boldsymbol{\kappa}, \delta\boldsymbol{\phi}).$$

—

46.20.5. Regularization of Corrections

Remark 46.148. To avoid overfitting, the correction terms are penalized:

$$J_\delta = \int_0^L (\delta\kappa'(s))^2 \, \mathrm{d}s + \int_0^L (\delta\phi'(s))^2 \, \mathrm{d}s.$$

Remark 46.149. This enforces smooth, small deviations from the parametric model.

—

46.20.6. Residual Structure

Remark 46.150. The residual now consists of:

- parametric contributions,
- correction contributions,
- data fitting terms.

—

46.20.7. Block Structure

Remark 46.151. The parameter vector decomposes as:

$$(\mathbf{T}, \delta\boldsymbol{\kappa}) \quad \text{and} \quad (\mathbf{T}_\phi, \delta\boldsymbol{\phi}).$$

Remark 46.152. This yields a multi-level block structure:

- coarse scale (parameters),
- fine scale (corrections).

—

46.20.8. Interpretation

Remark 46.153. The parametric model captures:

- geometric intent,
- transition structure,
- engineering semantics.

Remark 46.154. The correction term captures:

- measurement noise,
- local deviations,
- modelling imperfections.

—

46.20.9. Connection to Measurement Data

Remark 46.155. Measured polylines can be fitted by minimizing:

$$\sum_i \|\gamma(s_i) - p_i\|^2,$$

with the hybrid representation.

Remark 46.156. The parametric part explains the global structure, while the correction absorbs residual errors.

—

46.20.10. Computational Strategy

- first optimize parametric variables,
- then refine using correction terms,
- optionally alternate between both levels.

—

46.20.11. Relation to LOD

Remark 46.157. The hybrid model naturally supports level-of-detail concepts:

- coarse LOD: parametric model only,
- fine LOD: parametric + corrections.

—

46.20.12. Connection to AIM

Summary 46.158. The hybrid representation combines structured modelling and local flexibility.

It enables robust alignment reconstruction from imperfect data while preserving the interpretability of parametric models.

This approach bridges engineering design and data-driven fitting within the AIM framework.

46.21. Hybrid Representation and Block Structure

46.21.1. Motivation

Remark 46.159. Purely parametric models provide a low-dimensional and interpretable representation of alignments, but lack flexibility when fitting real-world data.

Remark 46.160. Purely sampled models provide maximum flexibility, but lead to high-dimensional and often ill-conditioned optimization problems.

Remark 46.161. A hybrid formulation combines both advantages.

46.21.2. Hybrid Representation

Definition 46.162 (Hybrid curvature model)**.**

$$\kappa(s) = \kappa_{\text{param}}(s; \mathbf{T}) + \delta\kappa(s),$$

where:

- κ_{param} is a structured parametric model,
- $\delta\kappa(s)$ is a correction term.

Definition 46.163 (Hybrid cant model)**.**

$$\phi(s) = \phi_{\text{param}}(s; \mathbf{T}_\phi) + \delta\phi(s).$$

46.21.3. Discretization of Corrections

Remark 46.164. The correction terms are represented discretely:

$$\delta\kappa_i = \delta\kappa(s_i), \qquad \delta\phi_i = \delta\phi(s_i).$$

Definition 46.165 (Extended parameter vector)**.**

$$\mathbf{x} = (\mathbf{T}, \mathbf{T}_\phi, \delta\boldsymbol{\kappa}, \delta\boldsymbol{\phi}).$$

46.21.4. Regularization

Remark 46.166. To avoid overfitting, correction terms are regularized:

$$J_\delta = \int_0^L (\delta\kappa'(s))^2 \, \mathrm{d}s + \int_0^L (\delta\phi'(s))^2 \, \mathrm{d}s.$$

Remark 46.167. Thus, the correction captures only smooth, physically meaningful deviations.

46.21.5. Residual Structure

Remark 46.168. The residual vector consists of:

- parametric contributions,
- correction contributions,
- physical constraints,

- data fitting terms.

46.21.6. Block Structure of Variables

Remark 46.169. The parameter vector decomposes naturally as:

$$(\mathbf{T}, \delta\boldsymbol{\kappa}) \quad \text{and} \quad (\mathbf{T}_\phi, \delta\boldsymbol{\phi}).$$

Remark 46.170. This yields a two-level structure:

- coarse scale: parametric variables,
- fine scale: correction variables.

46.21.7. Jacobian Block Structure

Definition 46.171 (Block Jacobian)**.** The Jacobian takes the form:

$$J_r = \begin{pmatrix} J_{\kappa\kappa}^{\text{param}} & J_{\kappa\kappa}^{\text{corr}} & 0 & 0 \\ 0 & 0 & J_{\phi\phi}^{\text{param}} & J_{\phi\phi}^{\text{corr}} \\ J_{\text{dyn},\kappa} & J_{\text{dyn},\kappa}^{\delta} & J_{\text{dyn},\phi} & J_{\text{dyn},\phi}^{\delta} \end{pmatrix}.$$

Remark 46.172. Key properties:

- parametric and correction terms are weakly coupled,
- curvature and cant are largely separable,
- coupling occurs mainly through dynamics.

46.21.8. Hierarchical Interpretation

Remark 46.173. The hybrid model introduces a hierarchical structure:

- parametric layer defines global alignment structure,
- correction layer refines local deviations.

Remark 46.174. This reflects engineering practice:

- design defines intent,
- construction introduces deviations.

46.21.9. Computational Strategy

- optimize parametric variables first,
- introduce correction terms for refinement,
- optionally alternate between both levels.

46.21.10. Connection to Level-of-Detail

Remark 46.175. The hybrid representation naturally supports LOD concepts:

- coarse LOD: parametric model only,
- fine LOD: parametric + corrections.

46.21.11. Key Insight

Proposition 46.176. *The hybrid formulation separates structural modelling from data fitting while preserving a sparse and well-conditioned optimization problem.*

46.21.12. Connection to AIM

Summary 46.177. The combination of hybrid modelling and block structure is a central feature of the AIM framework.

It enables:

- interpretable parametric modelling,
- robust handling of imperfect data,
- efficient optimization due to sparsity and locality.

Thus, it bridges the gap between engineering design and data-driven reconstruction.

47. Optimizer Architecture

47.1. Overview

Remark 47.1. The previous chapters introduced the mathematical and physical foundations of alignment modelling and optimization.

Remark 47.2. In this chapter, these concepts are combined into a concrete system architecture that enables the implementation of a full alignment optimization engine.

Remark 47.3. The architecture consists of the following main components:

- data ingestion (import and preprocessing),
- initial guess construction,
- constraint library,
- objective engine,
- numerical solver,
- live visualization.

—

47.2. Data Flow

Remark 47.4. The overall data flow can be summarized as:

$$\text{Input Data} \to \text{Initial Guess} \to \text{Optimization} \to \text{Result}$$

Remark 47.5. In the implemented system, this flow is realized through a sequence of well-defined transformations.

```
Drop → Parser → Grabbeltisch → Initial Guess → SQP → Model → View
```

—

47.3. Optimization Session

Definition 47.6 (Optimization session)**.** An optimization session encapsulates all data and operations required for a single optimization run.

```
class OptimizationSession {
  constructor({ input, constraints, objectives }) {
    this.input = input
    this.constraints = constraints
    this.objectives = objectives
    this.initialGuess = null
    this.result = null
  }

  buildInitialGuess() {}
  runOptimization() {}
  commitToModel(projectModel) {}
}
```

Remark 47.7. This abstraction separates:

- data preparation,
- numerical optimization,
- integration into the system model.

—

47.4. Initial Guess Construction

Remark 47.8. The initial guess transforms raw geometric data into optimization variables.

```
function buildInitialGuess(polyline) {
  const s = computeArcLength(polyline)

  const theta = computeDirections(polyline)
  const kappa = diff(theta) / ds

  const kappaSmooth = smooth(kappa)

  const phi = kappaSmooth.map(k =>
    Math.asin(clamp(v*v*k/g, -1, 1))
  )
```

```
  return { kappa: kappaSmooth, phi }
}
```

Remark 47.9. This step is essential for robust convergence of the optimization algorithm.

—

47.5. Constraint Library

Definition 47.10 (Constraint)**.** A constraint is a function mapping the current state to a residual.

```
function minRadius(kappa, Rmin) {
  return Math.abs(kappa) - 1/Rmin
}

function parallelConstraint(p1, p2, d) {
  const dx = p1.x - p2.x
  const dy = p1.y - p2.y
  return Math.sqrt(dx*dx + dy*dy) - d
}
```

Remark 47.11. All constraints are expressed uniformly as residuals and integrated into the optimization problem.

—

47.6. Objective Engine

Definition 47.12 (Objective function)**.** The objective function is constructed as a weighted sum of residual terms.

```
function objectiveJerk(kappa, phi, ds, v, g) {
  const r = []
  for (let i = 0; i < kappa.length - 1; i++) {
    const dk = (kappa[i + 1] - kappa[i]) / ds
    const dphi = (phi[i + 1] - phi[i]) / ds
    const j = v*v*v*dk - g*v*Math.cos(phi[i]) * dphi
    r.push(j)
  }
  return r
}
```

```
function buildObjectiveResiduals(ctx, objectives) {
  const r = []

  for (const obj of objectives) {
    const ri = obj.evaluate(ctx)
    const w = Math.sqrt(obj.weight ?? 1)
    for (const v of ri) r.push(w * v)
  }

  return r
}
```

Remark 47.13. This formulation enables flexible adaptation to different design criteria.

—

47.7. Residual Assembly

Remark 47.14. The optimization problem is expressed entirely in terms of residuals.

```
function buildResidualVector(ctx, constraints, objectives) {
  return [
    ...buildConstraintResiduals(ctx, constraints),
    ...buildObjectiveResiduals(ctx, objectives)
  ]
}
```

—

47.8. Numerical Solver

47.8.1. Gauss–Newton / SQP Core

```
async function runSQPInteractive(x0, onStep) {
  let x = x0

  for (let k = 0; k < maxIter; k++) {

    const r = residuals(x)
    const J = jacobian(x)

    const p = solve(J, r)
```

```
    x = add(x, scale(p, alpha))

    onStep({ x, k, r })

    await nextFrame()
  }

  return x
}
```

Remark 47.15. The solver operates on the residual formulation and exploits sparsity.

—

47.9. Live Visualization

Remark 47.16. A key feature of the system is the live visualization of the optimization process.

```
optSession.runOptimization({
  onStep: ({ x, k }) => {
    const geom = reconstruct(x)

    geoRender.updateAlignment(geom)
    plotKappa(x.kappa)
    plotPhi(x.phi)
  }
})
```

Remark 47.17. This enables interactive exploration and debugging of the optimization.

—

47.10. Network Extension

Remark 47.18. The architecture extends naturally to networks of alignments.

```
function residuals(network) {

  const r = []

  for (edge of network.edges) {
    r.push(...smoothness(edge))
  }
```

```
  for (node of network.nodes) {
    r.push(...nodeConstraints(node))
  }

  return r
}
```

Remark 47.19. This allows simultaneous optimization of multiple connected tracks.

—

47.11. System Integration

Remark 47.20. The optimization engine is integrated into the user workflow through the interactive selection interface.

```
onUserFinalizeSelection(selection) {

  const alignmentData =
    buildAlignmentFromSelection(selection)

  const optSession = new OptimizationSession({
    alignmentData
  })

  optSession.buildInitialGuess()
  optSession.runOptimization()
}
```

—

47.12. Summary

Summary 47.21. The presented architecture combines mathematical modelling, numerical optimization, and interactive visualization into a unified system.

By representing all constraints and objectives as residuals, the system achieves a high degree of modularity and extensibility.

This architecture enables the transition from static alignment modelling to dynamic, optimization-driven design of railway infrastructure.

48. Realization-Aware Optimization

48.1. Motivation

Remark 48.1. Classical alignment optimization operates directly on analytical target geometry.

Remark 48.2. Real railway tracks, however, differ from their analytical target states.

Remark 48.3. The physically realized alignment is influenced by:

- construction procedures,
- machine behaviour,
- ballast and track mechanics,
- local correction limits,
- discretization,
- and measurement feedback.

Remark 48.4. Thus, optimization should minimize:

$$J(\mathcal{R}(\kappa, \phi)),$$

rather than:

$$J(\kappa, \phi).$$

Remark 48.5. Within AIM, optimization is therefore interpreted as optimization of expected realized operational behaviour rather than of purely analytical geometry.

48.2. Realization Operator

Definition 48.6 (Realization operator)**.** The realization operator is:

$$\mathcal{R} : \mathrm{pose3}_{\mathrm{target}} \mapsto \mathrm{pose3}_{\mathrm{real}}.$$

Remark 48.7. The operator represents the transformation from analytical target state to physically realized railway state.

Remark 48.8. The realization operator may include:

- sampling,
- machine action,
- smoothing,
- local correction kernels,
- and structural response of the track.

48.3. Optimization on Realized States

Remark 48.9. Objective functions should be evaluated on realized states rather than on analytical target states.

Definition 48.10 (Realization-aware optimization)**.** The general optimization problem is:

$$\min_{\kappa,\phi} J(\mathcal{R}(\kappa,\phi)).$$

Remark 48.11. This formulation explicitly integrates realization effects into the optimization process.

48.4. Pose3 Optimization

Remark 48.12. Optimization operates on the coupled pose3 state:

$$(\gamma,\theta,\phi).$$

Remark 48.13. Curvature and cant must therefore be optimized jointly rather than independently.

Remark 48.14. This reflects the dynamic coupling between geometry and vehicle behaviour.

48.5. Dynamic Objectives

48.5.1. Effective Lateral Acceleration

Definition 48.15 (Effective lateral acceleration)**.** The effective lateral acceleration is:

$$a_{\text{eff}} = v^2\kappa - g\sin\phi.$$

Remark 48.16. This quantity represents the dynamically relevant acceleration acting on the vehicle.

48.5.2. Jerk

Definition 48.17 (Jerk)**.** The jerk is:

$$j = v^3\kappa' - gv\cos\phi\,\phi'.$$

Remark 48.18. Jerk constraints strongly influence transition design and passenger comfort.

48.5.3. Typical Objective Components

Remark 48.19. Typical optimization objectives include:

- curvature smoothness,
- cant smoothness,
- jerk minimization,
- cant deficiency control,
- vehicle comfort,
- energy efficiency,
- realization robustness.

48.6. Realization Constraints

Remark 48.20. Optimization must respect realization constraints such as:

- machine limits,
- tamping limits,
- ballast behaviour,
- local correction ranges,
- and measurement resolution.

Remark 48.21. Thus, the feasible design space is constrained not only by geometry and standards, but also by realizability.

48.7. Inverse Realization

Remark 48.22. The optimization problem may be interpreted as an inverse realization problem.

Definition 48.23 (Inverse realization formulation)**.** The objective is:

$$\mathcal{R}(\kappa_{\text{target}}, \phi_{\text{target}}) \approx (\kappa_{\text{desired}}, \phi_{\text{desired}}).$$

Remark 48.24. The analytical target state therefore acts as a pre-image under the realization operator.

48.8. Frequency Interpretation

Remark 48.25. The realization operator acts approximately as a low-pass filter on curvature and cant.

Remark 48.26. High-frequency components are attenuated during realization.

Remark 48.27. Optimization must therefore avoid introducing geometrically meaningless high-frequency corrections that cannot survive realization.

48.9. Sparse and Hybrid Optimization

Remark 48.28. Sparse and hybrid models are particularly suitable for realization-aware optimization.

Remark 48.29. Sparse structures provide:

- numerical stability,
- low parameter dimension,
- and interpretable geometry.

Remark 48.30. Hybrid models additionally permit:

- local corrections,
- realization compensation,
- and adaptive refinement.

48.10. Runtime Optimization

Remark 48.31. Optimization may operate continuously during:

- construction,
- maintenance,
- measurement feedback,
- and operational monitoring.

Remark 48.32. Thus, optimization becomes a runtime process rather than a purely offline planning step.

48.11. Vehicle-Space Interpretation

Remark 48.33. The physically relevant object is not necessarily the track centerline, but the motion of the railway vehicle within space.

Remark 48.34. This includes:

- vehicle center of gravity,
- bogie motion,
- vehicle envelope,
- platform interaction,
- and clearance behaviour.

Remark 48.35. Such interpretations connect realization-aware optimization to Schwerpunkt-oriented alignment concepts [12].

48.12. Ellipsoid-Aware Optimization

Remark 48.36. For large-scale or high-precision systems, optimization may be performed directly on the Earth reference surface.

Remark 48.37. In this case:

- curvature becomes geodesic curvature,
- frames become surface-attached frames,
- and world-to-track projection depends on ellipsoid geometry.

48.13. Key Insight

Proposition 48.38. *The relevant optimization target is not the analytical alignment itself, but the expected realized operational behaviour.*

Remark 48.39. This fundamentally changes the interpretation of railway alignment optimization.

48.14. Connection to AIM

Summary 48.40. Realization-aware optimization extends classical alignment optimization toward physically realizable railway states.

Within AIM, optimization is formulated on:

- pose3 states,

- realization operators,
- runtime projection structures,
- and dynamically relevant vehicle-space behaviour.

This provides a unified framework linking geometry, realization, dynamics, optimization, and operational interpretation.

Teil XI.

Engineering and Standards

Remark 48.41. This part places the AIM framework into the context of railway engineering practice. Mathematical models alone are insufficient unless they can be related to engineering constraints, standards, and operational rules.

Remark 48.42. The purpose of this part is therefore to connect formal modelling with practical admissibility conditions arising from railway design and operation.

Remark 48.43. The following chapters discuss standards, rule systems, and structured constraint formulations relevant to alignment engineering.

49. Standards and Data Models

49.1. Motivation

Remark 49.1. Railway alignment data does not exist in isolation. It is embedded in a landscape of established standards and engineering data models.

Remark 49.2. Relevant formats include:

- LandXML,
- IFC (Industry Foundation Classes),
- railML,
- and various proprietary or legacy formats.

Remark 49.3. The AIM framework does not replace these standards. Instead, it provides a consistent internal representation that can interpret and integrate them.

—

49.2. General Perspective

Remark 49.4. Most existing standards describe railway geometry in a layered or fragmented way.

- horizontal alignment,
- vertical profile,
- cant (superelevation),
- additional metadata.

Remark 49.5. In contrast, the AIM framework proposes a unified state-based representation:

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

Remark 49.6. This representation serves as a canonical internal model.

—

49.3. LandXML

49.3.1. Overview

Remark 49.7. LandXML is one of the most widely used formats for alignment data exchange.

Remark 49.8. It typically represents:

- horizontal geometry (CoordGeom),
- vertical profiles,
- cant definitions.

49.3.2. Characteristics

- structured XML format,
- hierarchical organization,
- multiple alignments per file,
- explicit naming of geometric elements.

49.3.3. Limitations

Remark 49.9. Despite its structure, LandXML exhibits several limitations:

- separation of horizontal and vertical geometry,
- lack of a unified dynamic interpretation,
- ambiguity in coordinate reference systems,
- inconsistent usage across software systems.

49.3.4. AIM Interpretation

Remark 49.10. Within the AIM framework, LandXML is treated as a *fat container*:

- all available data is parsed,
- relevant components are extracted,
- the result is transformed into a unified state model.

Remark 49.11. The conversion step

$$\text{LandXML} \rightarrow \text{AIM (sparse)}$$

is essential for further processing.

—

49.4. IFC

49.4.1. Overview

Remark 49.12. IFC is a general BIM standard designed for interdisciplinary data exchange.

Remark 49.13. In the context of infrastructure, IFC is extended by infrastructure schemas (IFC Rail, IFC Alignment).

49.4.2. Characteristics

- object-oriented data model,
- rich semantic structure,
- integration of geometry and metadata,
- support for relationships and hierarchies.

49.4.3. Challenges

Remark 49.14. For alignment modelling, IFC presents several challenges:

- complex schema structure,
- multiple representation alternatives,
- inconsistent implementation across tools,
- limited support for advanced alignment concepts.

49.4.4. AIM Interpretation

Remark 49.15. The AIM framework interprets IFC data by extracting:

- alignment geometry,
- reference systems,
- semantic relations.

Remark 49.16. AIM acts as a normalization layer that reduces IFC complexity to a computationally usable form.

—

49.5. railML

49.5.1. Overview

Remark 49.17. railML is a domain-specific XML standard for railway systems.

Remark 49.18. It focuses on:

- infrastructure,
- timetable data,
- rolling stock.

49.5.2. Relevance for Alignment

Remark 49.19. railML provides a more explicit representation of railway topology than most other formats.

Remark 49.20. However, detailed geometric modelling is often secondary to network and operational aspects.

49.5.3. AIM Interpretation

Remark 49.21. Within AIM, railML is particularly relevant for:

- network topology,
- connections between tracks,
- system-level structure.

—

49.6. Legacy and Proprietary Formats

Remark 49.22. In practice, a large amount of alignment data exists in proprietary or legacy formats.

Remark 49.23. Examples include:

- TRA / GRA (Technet, Vermessung),
- spreadsheet-based formats,
- CAD exports.

49.6.1. Characteristics

- incomplete or implicit structure,
- missing type information,
- reliance on conventions rather than formal schema.

49.6.2. AIM Strategy

Remark 49.24. These formats are interpreted using:

- sniffing and classification,
- format-specific parsers,
- reconstruction of implicit structure.

—

49.7. Canonical Internal Model

Definition 49.25 (AIM core model)**.** The AIM framework uses a canonical internal representation based on:

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

Remark 49.26. All external formats are mapped to this representation.

49.7.1. Advantages

- unified handling of geometry and dynamics,
- compatibility with numerical optimization,
- independence from specific file formats,
- extensibility toward new standards.

—

49.8. Transformation Pipeline

Remark 49.27. The transformation from external data to the AIM model follows a pipeline structure.

```
external file
→ parser
→ fat representation
→ validation
→ sparse conversion
→ AIM state model
```

Remark 49.28. This pipeline ensures robustness and traceability of data processing.

—

49.9. Relation to BIM

49.9.1. Conceptual Position

Remark 49.29. BIM (Building Information Modelling) aims to integrate geometry, semantics, and lifecycle information.

Remark 49.30. However, alignment modelling is often underrepresented or treated as a secondary aspect.

49.9.2. AIM Contribution

Remark 49.31. The AIM framework contributes to BIM by:

- providing a rigorous alignment model,
- enabling dynamic interpretation,
- supporting optimization-driven design.

Remark 49.32. In this sense, AIM can be interpreted as a specialization of BIM for alignment-centric infrastructure modelling.

—

49.10. Interoperability

Remark 49.33. A key requirement for practical application is interoperability with existing tools and workflows.

Remark 49.34. The AIM framework supports interoperability through:

- import adapters,
- export converters,
- intermediate canonical representation.

—

49.11. Discussion

Remark 49.35. Existing standards focus primarily on data exchange and documentation.

Remark 49.36. The AIM framework complements these standards by introducing:

- a computationally consistent model,
- a dynamic interpretation,
- integration with optimization methods.

Remark 49.37. Thus, AIM bridges the gap between static data representation and active engineering design.

—

49.12. Summary

Summary 49.38. Railway alignment data is governed by a variety of standards, each with its own strengths and limitations.

The AIM framework provides a unified internal representation that allows these heterogeneous sources to be interpreted consistently.

By combining standard-based data exchange with a state-based, optimization-ready model, AIM enables a transition from static data handling to dynamic and computational alignment design.

50. Constraint Code Architecture

50.1. Motivation

Remark 50.1. Railway alignment rules must be represented in a form that is both mathematically meaningful and computationally tractable.

Remark 50.2. Within the AIM framework, this is achieved by expressing all rules as constraint residuals that can be evaluated uniformly inside an optimization loop.

Remark 50.3. The implementation goal is therefore not a collection of isolated rule checks, but a modular constraint library.

50.2. General Principle

Definition 50.4 (Constraint residual)**.** A constraint is represented as a function

$$c(\mathbf{x}) \in \mathbb{R}^m$$

whose value vanishes in the ideal case and whose magnitude measures violation.

Remark 50.5. This unified representation allows the same mathematical object to be used for:

- feasibility checks,
- penalty terms,
- direct inclusion into SQP solvers.

```
function evaluateConstraint(ctx) {
  return [residual1, residual2, ...]
}
```

50.3. Constraint Registry

Remark 50.6. A practical implementation should organize constraints in a registry.

```
const ConstraintRegistry = {
  geometric: {},
  dynamic: {},
```

```
  topologic: {},
  regulatory: {}
}
```

Remark 50.7. This keeps domain logic extensible and avoids hard-coded monolithic solver structures.

50.4. Constraint Object Model

```
const constraint = {
  id: "min_radius",
  group: "regulatory",
  appliesTo: "edge",
  weight: 1.0,
  evaluate: (ctx) => []
}
```

Remark 50.8. The fields have the following meaning:

- `id`: unique name,
- `group`: semantic class,
- `appliesTo`: edge, node, pair, network,
- `weight`: optional scaling,
- `evaluate`: residual callback.

50.5. Context Object

Remark 50.9. Constraints should not operate directly on raw global arrays. Instead, they receive a normalized context object.

```
const ctx = {
  edge,
  node,
  pair,
  i,
  kappa,
  phi,
  geometry,
  speed,
  params,
```

```
  metadata
}
```

Remark 50.10. This provides a stable interface between model, solver, and rule system.

50.6. Geometric Constraints

50.6.1. Position Continuity

```
function continuityConstraint(p1, p2) {
  return [
    p1.x - p2.x,
    p1.y - p2.y
  ]
}
```

Remark 50.11. This enforces C^0-continuity between adjacent geometric elements or connected alignments.

50.6.2. Direction Continuity

```
function directionConstraint(t1, t2) {
  return [
    t1.dx - t2.dx,
    t1.dy - t2.dy
  ]
}
```

Remark 50.12. This corresponds to tangent continuity and therefore to C^1-style behaviour in geometric terms.

50.6.3. Curvature Continuity

```
function curvatureConstraint(k1, k2) {
  return [k1 - k2]
}
```

Remark 50.13. This enforces continuity of curvature across boundaries.

50.7. Dynamic Constraints

50.7.1. Equilibrium Constraint

```
function equilibriumConstraint(kappa, phi, v, g) {
  return [v * v * kappa - g * Math.sin(phi)]
}
```

Remark 50.14. This residual measures cant deficiency or excess in analytic form.

50.7.2. Jerk Constraint

```
function jerkConstraint(kappa0, kappa1, phi0, phi1, ds, v, g) {
  const dk = (kappa1 - kappa0) / ds
  const dphi = (phi1 - phi0) / ds
  const j = v * v * v * dk - g * v * Math.cos(phi0) * dphi
  return [j]
}
```

Remark 50.15. This constraint reflects dynamic smoothness and comfort-related limits.

50.8. Regulatory Constraints

50.8.1. Minimum Radius

```
function minRadiusConstraint(kappa, Rmin) {
  return [Math.abs(kappa) - 1 / Rmin]
}
```

Remark 50.16. A non-positive value indicates compliance with the radius limit.

50.8.2. Maximum Cant

```
function maxCantConstraint(phi, phiMax) {
  return [Math.abs(phi) - phiMax]
}
```

50.8.3. Cant Gradient

```
function cantGradientConstraint(phi0, phi1, ds, limit) {
  return [Math.abs((phi1 - phi0) / ds) - limit]
}
```

50.9. Topological Constraints

50.9.1. Switch Node Constraint

```
function switchConstraint(main, branch) {
  return [
    main.x - branch.x,
    main.y - branch.y,
    main.dx - branch.dx,
```

```
    main.dy - branch.dy
  ]
}
```

Remark 50.17. This expresses the basic coupling at branching nodes.

50.9.2. Parallel Track Spacing

```
function parallelConstraint(p1, p2, d) {
  const dx = p1.x - p2.x
  const dy = p1.y - p2.y
  return [Math.sqrt(dx * dx + dy * dy) - d]
}
```

50.10. Constraint Assembly

Remark 50.18. The full residual vector is assembled from all active constraints.

```
function buildConstraintResiduals(ctx, constraints) {
  const r = []

  for (const c of constraints) {
    const ri = c.evaluate(ctx)
    const w = Math.sqrt(c.weight ?? 1)
    for (const v of ri) r.push(w * v)
  }

  return r
}
```

Remark 50.19. This produces a single solver-compatible vector independent of the semantic source of the constraints.

50.11. Factory Functions

Remark 50.20. Constraints should be created through factories rather than by writing anonymous closures throughout the code base.

```
function makeMinRadiusConstraint(Rmin, weight = 1) {
  return {
    id: "min_radius",
    group: "regulatory",
    appliesTo: "edge",
```

```
    weight,
    evaluate: ({ kappa, i }) => [
      Math.abs(kappa[i]) - 1 / Rmin
    ]
  }
}

function makeJerkConstraint(v, g, ds, weight = 1) {
  return {
    id: "jerk",
    group: "dynamic",
    appliesTo: "edge",
    weight,
    evaluate: ({ kappa, phi, i }) => {
      const dk = (kappa[i + 1] - kappa[i]) / ds
      const dphi = (phi[i + 1] - phi[i]) / ds
      return [v * v * v * dk - g * v * Math.cos(phi[i]) * dphi]
    }
  }
}
```

50.12. Constraint Presets

Remark 50.21. In practical railway design, different projects require different constraint configurations.

```
const presetHighSpeed = [
  makeMinRadiusConstraint(4000, 10),
  makeJerkConstraint(v, g, ds, 8),
  makeMaxCantConstraint(phiMax, 6)
]

const presetExistingLine = [
  makeMinRadiusConstraint(800, 3),
  makeJerkConstraint(v, g, ds, 2),
  makeParallelConstraint(trackDistance, 5)
]
```

50.13. Integration into the Solver

```
function buildResidualVector(ctx, constraints, objectives) {
  return [
```

```
    ...buildConstraintResiduals(ctx, constraints),
    ...buildObjectiveResiduals(ctx, objectives)
  ]
}
```

Remark 50.22. This shows the central design principle of the architecture: constraints and objectives share the same residual-based interface.

50.14. Discussion

Remark 50.23. The proposed architecture has several advantages:

- modularity,
- extensibility,
- compatibility with least-squares solvers,
- transparent mapping between engineering rules and code.

Remark 50.24. Instead of scattering rule checks throughout the application, the constraint library concentrates domain logic in a single mathematical layer.

50.15. Summary

Summary 50.25. The constraint library translates railway rules, topology, and dynamic requirements into a unified residual-based architecture.

This provides the technical bridge between theoretical modelling, engineering regulations, and computational optimization.

As a result, the AIM framework becomes not only mathematically coherent, but also directly implementable in software.

51. Railway Design Rules

51.1. Motivation

Remark 51.1. Railway alignment design is not solely a geometric or mathematical task. It is governed by engineering rules, safety requirements, and operational constraints.

Remark 51.2. These rules originate from:

- national regulations,
- international standards (e.g. UIC),
- operator-specific guidelines (e.g. Deutsche Bahn),
- empirical engineering practice.

Remark 51.3. Within the AIM framework, such rules are not treated as external constraints only, but are integrated into the modelling and optimization process.

—

51.2. Categories of Rules

51.2.1. Geometric Rules

- minimum radius,
- continuity of alignment,
- transition length requirements,
- curvature monotonicity in transitions.

51.2.2. Kinematic and Dynamic Rules

- limits on lateral acceleration,
- limits on jerk,
- compatibility of curvature and cant,
- speed-dependent requirements.

51.2.3. Operational Rules

- allowable speeds,
- safety margins,
- maintenance considerations,
- interoperability constraints.

51.2.4. Topological Rules

- constraints at switches and crossings,
- parallel track spacing,
- network connectivity conditions.

—

51.3. Geometric Constraints

51.3.1. Minimum Radius

Definition 51.4 (Minimum radius constraint)**.** The curvature must satisfy

$$|\kappa(s)| \leq \kappa_{\max}, \qquad \kappa_{\max} = \frac{1}{R_{\min}}.$$

Remark 51.5. The value of $R_{\min}$ depends on:

- design speed,
- vehicle characteristics,
- regulatory standards.

—

51.3.2. Continuity Conditions

Remark 51.6. Railway alignments must satisfy at least:

- C^0 continuity (position),
- C^1 continuity (direction),
- typically C^2 continuity (curvature).

Remark 51.7. Higher-order smoothness may be required for high-speed lines.

—

51.3.3. Transition Length

Definition 51.8 (Minimum transition length)**.** A transition between curvatures must satisfy

$$L \geq L_{\min}(v, \kappa_1, \kappa_2).$$

Remark 51.9. Typical formulations relate transition length to:

- allowable rate of change of lateral acceleration,
- cant development limits.

—

51.4. Dynamic Constraints

51.4.1. Lateral Acceleration

Definition 51.10 (Lateral acceleration)**.** The lateral acceleration is given by

$$a_y(s) = v^2 \kappa(s).$$

Constraint 51.11.

$$|a_y(s)| \leq a_{\max}.$$

—

51.4.2. Cant and Cant Deficiency

Definition 51.12 (Cant angle)**.** Let $\phi(s)$ denote the cant angle.

Definition 51.13 (Effective acceleration)**.** The effective lateral acceleration is

$$a_{\text{eff}}(s) = v^2 \kappa(s) - g \tan(\phi(s)).$$

Constraint 51.14.

$$|a_{\text{eff}}(s)| \leq a_{\text{eff},\max}.$$

—

51.4.3. Jerk

Definition 51.15 (Jerk)**.** The lateral jerk is defined as

$$j(s) = \frac{\mathrm{d}}{\mathrm{d}t} a_y(s).$$

Remark 51.16. Using s as parameter, one obtains

$$j(s) = v^3 \kappa'(s).$$

Constraint 51.17.

$$|j(s)| \leq j_{\max}.$$

—

51.5. Coupling of Curvature and Cant

Remark 51.18. Curvature and cant must not be treated independently.

Constraint 51.19.

$$|\kappa'(s)| \leq f_1(v, \phi), \qquad |\phi'(s)| \leq f_2(v, \kappa).$$

Remark 51.20. These relations express that:

- curvature changes require corresponding cant development,
- cant changes must respect dynamic limits.

—

51.6. Speed-Dependent Constraints

Remark 51.21. All relevant constraints depend on speed.

Definition 51.22 (Speed profile)**.** A speed profile is a function

$$v : [0, L] \to \mathbb{R}_+.$$

Remark 51.23. Constraints must be evaluated pointwise with respect to $v(s)$.

—

51.7. Constraint Representation in AIM

51.7.1. Abstract Form

Definition 51.24 (Constraint functional)**.** A constraint is represented as a functional

$$C_i[\kappa, \phi, v] \leq 0.$$

Remark 51.25. This allows constraints to be:

- evaluated continuously,
- discretized for numerical optimization,
- combined into a unified constraint system.

—

51.7.2. Discrete Representation

Remark 51.26. In numerical implementation, constraints are evaluated on a grid

$$s_0, \ldots, s_N.$$

Definition 51.27 (Discrete constraint)**.**

$$C_i(\mathbf{x}) = \max_k C_i(s_k).$$

—

51.8. Integration into Optimization

Remark 51.28. Constraints enter the optimization problem as:

$$g_i(\mathbf{x}) \leq 0.$$

Remark 51.29. They may be handled via:

- hard constraints (feasibility),
- penalty terms,
- barrier methods.

—

51.9. Relation to Standards

Remark 51.30. The presented constraints correspond to rules found in:

- UIC codes,
- national railway regulations,
- operator guidelines.

Remark 51.31. The AIM framework provides a mathematical representation that allows these rules to be applied consistently across different data sources.

—

51.10. Discussion

Remark 51.32. Traditional workflows treat rules as external checks applied after design.

Remark 51.33. In contrast, AIM integrates rules directly into:

- modelling,
- reconstruction,
- optimization.

Remark 51.34. This leads to a shift from validation to constraint-driven design.

—

51.11. Summary

Summary 51.35. Railway design rules define the feasible set of alignment solutions.

Within the AIM framework, these rules are formalized as mathematical constraints and integrated into the optimization process.

This allows alignment design to be treated as a constrained optimization problem that respects both geometric and dynamic requirements.

Teil XII.

Applications

Remark 51.36. This part illustrates how the AIM framework can be applied to concrete railway and modelling problems. Its purpose is not only to demonstrate feasibility, but also to show how the theoretical concepts developed earlier become operational in practice.

Remark 51.37. The applications range from classical railway alignment questions to broader perspectives such as BIMalignment and use-case-driven interpretation.

Remark 51.38. Thus, this part serves as the bridge from formal framework to practical engineering relevance.

52. Railway Alignment

52.1. Motivation

Remark 52.1. Railway alignment is one of the most demanding application domains for curvature-based geometric modelling.

Remark 52.2. In railway engineering, alignment is not merely a planar curve design problem. It is a coupled system involving horizontal geometry, vertical profile, cant, operating speed, dynamic vehicle response, safety, and maintenance.

Remark 52.3. This makes railway alignment a natural and severe test case for alignment-based information modelling.

52.2. Railway Alignment as a Multi-Layer Object

Definition 52.4 (Railway alignment)**.** A railway alignment is the spatial path associated with a railway track, typically represented through coordinated horizontal, vertical, and cant-related descriptions.

Remark 52.5. In practical standards and data models, railway alignment is usually not represented as a single monolithic object, but as an ordered system of segments and auxiliary alignment layers.

52.2.1. Horizontal alignment

Remark 52.6. The horizontal alignment is primarily governed by curvature. In its classical engineering decomposition it consists of straight segments, circular arcs, and transition curves.

52.2.2. Vertical alignment

Remark 52.7. The vertical alignment describes elevation and gradient behaviour along the route. In practical data models it is segmented into line, arc, and transition primitives.

OPEN: Unified mathematical treatment of horizontal and vertical alignment.

52.2.3. Cant alignment

Definition 52.8 (Cant)**.** Cant is the elevation difference between the rails, introduced to compensate lateral acceleration in curves.

Remark 52.9. Cant is not an accessory parameter. It forms an alignment layer of its own, with segment structure, continuity rules, and possible smoothing requirements.

Remark 52.10. Recent railway conceptual models explicitly represent cant as an ordered sequence of segments, including line and transition segments, linked to the horizontal alignment by chainage or equivalent position reference.

OPEN: Integrate cant into a unified geometric and kinematic state model.

52.3. Parameter Coupling

Remark 52.11. Railway alignment is characterized by strong coupling between geometric and operational variables.

- curvature,
- cant,
- speed,
- transition length,
- gradient,
- and boundary conditions imposed by rolling stock and operation

Remark 52.12. These quantities are not independent. Increased speed requires either larger radii, larger cant, longer transition lengths, or a different trade-off between these variables.

Remark 52.13. Likewise, high cant or severe curvature may improve one operational aspect while worsening another, for example when stopped trains or restricted sight distances are involved.

52.4. Why Transition Curves Matter in Railways

Remark 52.14. The transition curve is not merely a geometric connector between a straight line and a circular arc.

Remark 52.15. Its primary role is to avoid abrupt changes in lateral forces, improve ride comfort, reduce dynamic excitation, and limit wear on both vehicle and infrastructure.

Remark 52.16. The state-of-the-art review literature explicitly emphasizes that transition curves are crucial for safe and comfortable railway travel and for reducing maintenance demand.

Remark 52.17. This is also why modern railway research increasingly evaluates transition curves not only geometrically, but in terms of lateral change of acceleration, force development, wear, and vehicle dynamics.

52.5. From Classical Geometry to Dynamic Interpretation

Remark 52.18. Classical alignment design often idealizes the influence of curvature, cant, and transition shape through simplified or quasi-static considerations.

Remark 52.19. However, practical evidence and later engineering work show that the vehicle–track system exhibits dynamic responses which depend strongly on the detailed shape of the transition curve.

Remark 52.20. In particular, non-linear transition curves were introduced to mitigate the abrupt onset and disappearance of the variation of unbalanced lateral acceleration that arises with linear transitions at higher speeds.

Remark 52.21. This gives railway transition design a systems character: the geometric curve acts as a control function for dynamic behaviour.

52.6. Curvature, Cant, and Speed

Remark 52.22. Curvature alone does not determine railway behaviour. The relevant quantity is the combined system of curvature, cant, speed, and their rates of change.

Remark 52.23. Cant and cant variation influence lateral acceleration compensation, while transition shape influences how these effects develop along the track.

Remark 52.24. Practical railway design manuals also show that excessive superelevation, insufficient tangent lengths, or poorly located curves can create operational and derailment risks even when the pure geometry is formally traversable.

OPEN: Formal coupled model of curvature, cant, speed, and dynamic response.

52.7. Operational and Maintenance Constraints

Remark 52.25. Railway alignment is constrained not only by construction feasibility, but also by long-term operation and maintenance.

- minimum radii,
- maximum cant,
- limits on cant change,
- curvature and transition limits,
- sight distance restrictions,
- turnout and crossing constraints,
- wear and maintenance considerations

Remark 52.26. Practical design guides repeatedly emphasize that sharper curvature may be physically negotiable, yet still be undesirable because of higher maintenance cost, lower operating efficiency, or increased derailment risk.

52.8. Freight, Passenger, High-Speed, and Special Systems

Remark 52.27. Railway alignment is not uniform across operational domains. Freight, passenger, high-speed, light rail, and special systems such as maglev may lead to different admissible geometry and different transition preferences.

Remark 52.28. This means that an alignment model must be flexible enough to represent domain-specific constraints without losing mathematical coherence.

OPEN: Taxonomy of railway application classes and their geometric consequences.

52.9. Chainage and Engineering Referencing

Definition 52.29 (Chainage)**.** Chainage is the engineering reference system used to identify positions along an alignment.

Remark 52.30. In practice, alignment data is often anchored more robustly by chainage than by raw Cartesian coordinates alone, especially when the alignment is revised or partially reconstructed.

Remark 52.31. Modern railway information models therefore include dedicated chainage mechanisms in order to preserve engineering references despite local alignment changes.

OPEN: Relation between arc-length parameterization and engineering chainage.

52.10. Railway Alignment in Data Models and BIM

Remark 52.32. Railway alignment is increasingly represented within interoperable data models. These models typically distinguish horizontal alignment, vertical alignment, and cant as related but separate geometric layers.

Remark 52.33. Current IFC-related railway modelling efforts explicitly represent transition curves, vertical transition segments, and cant segments, and already reveal that infrastructure alignment cannot be reduced to a single curve primitive.

Remark 52.34. This strongly supports the view adopted in the present manuscript: alignment should be understood as a structured object rather than as an undifferentiated line.

OPEN: Formal docking of the sparse alignment model to IFC/railway data models.

52.11. Why Railway Alignment is a Privileged Use Case

Remark 52.35. Railway alignment combines

- strict geometric structure,
- strong parameter coupling,
- dynamic sensitivity,
- safety implications,
- heavy maintenance consequences,
- and demanding interoperability requirements.

Remark 52.36. For that reason, railway alignment is not merely one application among many, but an especially revealing use case for a general theory of curvature-based structured curve modelling.

52.12. Outlook

Remark 52.37. A mature railway-alignment framework should therefore support:

- structured geometric representation,
- multiple transition families,
- coupled cant and profile modelling,
- numerical reconstruction,
- dynamic and safety-oriented evaluation,
- and interoperability with engineering standards and BIM models.

Summary 52.38. Railway alignment is a multi-layer object rather than a single planar curve. It couples geometry, cant, speed, vehicle dynamics, operational constraints, maintenance, and engineering referencing.

This makes railway alignment a privileged and highly demanding domain for alignment-based information modelling and provides the main application context for the concepts developed in this manuscript.

53. Vehicle Space and Platform Interaction

53.1. Motivation

Remark 53.1. Classical railway alignment modelling is primarily based on the track centerline.

Remark 53.2. In this view, the railway vehicle is treated indirectly through:

- clearance envelopes,
- cant rules,
- safety margins,
- and operational standards.

Remark 53.3. This abstraction has proven remarkably successful and enabled the development of reliable railway systems over many decades.

Remark 53.4. However, the physically relevant object is not the centerline itself, but the spatial motion of the railway vehicle along the track.

Remark 53.5. This becomes particularly visible in:

- tight urban rail systems,
- platform-edge design,
- articulated vehicles,
- narrow tunnel geometries,
- and highly optimized clearance situations.

Remark 53.6. This perspective is closely related to approaches that interpret alignment geometry from the viewpoint of vehicle motion rather than purely from the track centerline.

Such concepts have been explored in advanced railway engineering, including Schwerpunkt-oriented alignment approaches [12].

53.2. Centerline as Reduced Representation

Remark 53.7. The classical alignment model represents the railway track by:

$$\gamma(s), \qquad \kappa(s), \qquad \phi(s).$$

Remark 53.8. This defines:

- position,
- curvature,
- and cant,

along the alignment.

Remark 53.9. The vehicle itself is not explicitly represented.

Remark 53.10. Instead, its behaviour is incorporated indirectly through engineering rules and geometric tolerances.

Proposition 53.11. *The centerline representation is a reduced-order approximation of the actual spatial motion of railway vehicles.*

Remark 53.12. This approximation is generally sufficient for conventional railway engineering.

53.3. Vehicle-Induced Geometry

Remark 53.13. A railway vehicle does not follow the centerline as a rigid point.

Remark 53.14. Its occupied space depends on:

- bogie geometry,
- wheelbase,
- car-body length,
- articulation,
- cant,
- suspension,
- and dynamic motion.

Remark 53.15. As a consequence:

- vehicle ends may swing outward,
- the center of the car body may move inward,
- and the effective occupied space varies along the track.

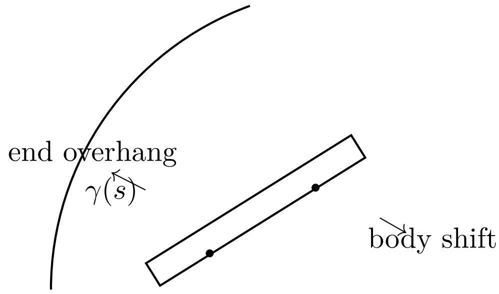

Abbildung 53.1.: Vehicle motion differs from the centerline geometry. Car-body overhang and bogie-constrained motion generate occupied space that cannot be represented by the centerline alone.

53.4. Platform Interaction

Remark 53.16. Platform-edge design is one of the most sensitive examples of vehicle-space interaction.

Remark 53.17. The effective gap between vehicle and platform depends on:

- curvature,
- cant,
- vehicle geometry,
- suspension state,
- and operational conditions.

Remark 53.18. In some urban railway systems, such as light rail and metro systems, vehicle overhang is explicitly considered in platform-edge geometry.

Remark 53.19. This demonstrates that the practically relevant geometry is not merely the track centerline, but the dynamically occupied vehicle space.

53.5. Standards and Conservative Approximation

Remark 53.20. Conventional railway standards often address vehicle motion indirectly through conservative clearance envelopes.

Remark 53.21. This approach increases robustness and simplifies engineering workflows.

Remark 53.22. However, it may also lead to:

- increased platform gaps,
- reduced passenger comfort,
- larger-than-necessary clearance reserves,

- and loss of geometric precision.

Remark 53.23. Thus, many existing systems compensate modelling limitations through additional safety margins.

53.6. Towards Vehicle-Centric Alignment Modelling

Remark 53.24. Future alignment systems may extend the classical centerline model towards vehicle-centric representations.

Remark 53.25. In such a framework, the relevant object would no longer be only:

$$\gamma(s),$$

but the occupied vehicle space evolving along the alignment.

Remark 53.26. Possible extensions include:

- dynamic vehicle envelopes,
- bogie-constrained motion models,
- passenger-space optimization,
- platform interaction models,
- and runtime vehicle-state evaluation.

Remark 53.27. Within the AIM framework, such concepts are regarded as potential future extensions rather than immediate replacement of existing engineering practice.

53.7. Connection to Schwerpunkt-Trassierung

Remark 53.28. The concept of Schwerpunkt-Trassierung already shifts the interpretation of alignment design from purely geometric construction toward the motion of a dynamically relevant state.

Remark 53.29. Vehicle-space modelling can be interpreted as a continuation of this idea.

Remark 53.30. Instead of optimizing only a centerline curve, one may eventually seek to optimize the spatial behaviour of vehicles and passengers along the track.

53.8. Connection to AIM

Summary 53.31. Classical railway alignment modelling represents the railway track by a centerline-based geometric abstraction.

This abstraction is highly successful, but does not explicitly represent the spatial behaviour of railway vehicles.

Practical situations such as platform interaction reveal the limitations of purely centerline-based modelling and motivate future vehicle-centric extensions.

Within the AIM framework, this is regarded as an important long-term research direction.

54. Railway Dynamics versus Aircraft Dynamics

54.1. Motivation

Remark 54.1. Aircraft dynamics and railway dynamics are often treated as fundamentally different engineering domains.

Remark 54.2. However, both systems are governed by related physical principles:

- inertia,
- curvature-driven motion,
- center-of-gravity behaviour,
- rotational response,
- and passenger comfort constraints.

Remark 54.3. The essential difference is not dynamics itself, but the degree of kinematic freedom.

54.2. Aircraft Dynamics

54.2.1. Free Spatial Motion

Remark 54.4. Aircraft move within largely unconstrained three-dimensional space.

Remark 54.5. The aircraft continuously adapts:

- roll,
- yaw,
- pitch,
- and trajectory

to dynamic requirements.

54.2.2. Center-of-Gravity Interpretation

Remark 54.6. Aircraft dynamics are strongly centered around:

- center-of-gravity motion,
- stability,
- inertia tensors,
- and aerodynamic force balance.

54.2.3. Dynamic Freedom

Remark 54.7. The aircraft may continuously adapt:

- orientation,
- banking angle,
- and trajectory curvature

during operation.

54.3. Railway Dynamics

54.3.1. Track-Constrained Motion

Remark 54.8. Railway vehicles are fundamentally different because their motion is strongly constrained by track geometry.

Remark 54.9. The railway vehicle cannot freely select:

- curvature,
- roll angle,
- or spatial trajectory.

Remark 54.10. Instead, these quantities are largely prescribed by:

- alignment geometry,
- cant,
- vehicle suspension,
- and operational velocity.

54.4. The Hidden Similarity

Remark 54.11. Despite these differences, both aircraft and railway systems ultimately attempt to manage:

$$\text{inertial forces}$$

relative to:

$$\text{vehicle orientation and passenger comfort.}$$

Remark 54.12. In aircraft:

- trajectory adapts to dynamics.

Remark 54.13. In railways:

- geometry and cant must be designed in advance to support future dynamic behaviour.

54.5. Cant as Railway Banking

Remark 54.14. Cant may be interpreted as the railway equivalent of aircraft banking.

Remark 54.15. However, unlike aircraft banking:

- railway cant is infrastructure-bound,
- spatially fixed,
- and shared by all vehicles using the track.

Remark 54.16. This creates a fundamental compromise:

- freight trains,
- passenger trains,
- high-speed vehicles,
- and maintenance vehicles

must all operate on the same geometric infrastructure.

54.6. Classical Railway Limitation

Remark 54.17. Classical railway engineering therefore evolved toward strongly coupled design rules:

$$\kappa(s) \leftrightarrow \phi(s).$$

Remark 54.18. In many traditional approaches, curvature and cant evolution are treated almost as inseparable quantities.

Remark 54.19. This coupling simplifies:

- construction,
- maintenance,
- and standards.

Remark 54.20. However, it also limits:

- operational flexibility,
- vehicle-specific optimization,
- and adaptive reinterpretation of existing infrastructure.

54.7. The AIM Perspective

54.7.1. Pose3 Interpretation

Remark 54.21. Within AIM, cant is not interpreted merely as static geometric annotation.

Remark 54.22. Instead, cant contributes to a runtime-derived operational state:

$$\text{pose3}.$$

Remark 54.23. This enables explicit interpretation of:

- vehicle-space behaviour,
- center-of-gravity motion,
- dynamic comfort,
- and operational envelopes.

54.8. From Fixed Geometry to Operational Envelopes

Remark 54.24. Classical railway optimization often attempts to derive:

$$\kappa(s)$$

from:

$$\phi(s).$$

Remark 54.25. Within AIM, the inverse interpretation becomes possible:

- existing curvature geometry remains largely fixed,
- cant becomes partially adjustable,

- and admissible velocity envelopes are evaluated dynamically.

Remark 54.26. Thus, the operational question changes from:

"Which geometry should be built?"

to:

"Which operational behaviour can be supported by existing geometry?"

54.9. Center-of-Gravity Interpretation

Remark 54.27. Different railway vehicles possess different:

- center-of-gravity heights,
- suspension systems,
- roll behaviour,
- and passenger comfort limits.

Remark 54.28. Thus:

- freight trains,
- passenger trains,
- tilting trains,
- and metro systems

may require different operational interpretations of the same physical track geometry.

Remark 54.29. This naturally motivates:

- vehicle-specific runtime evaluation,
- operational velocity envelopes,
- and realization-aware cant interpretation.

54.10. Operational Consequence

Remark 54.30. The operational opportunity is not necessarily:

"new geometry"

but often:

"better interpretation of existing geometry."

Remark 54.31. Small cant adjustments, sometimes only within millimeter or centimeter ranges, may significantly affect:

- admissible speed,
- passenger comfort,
- and operational stability.

Remark 54.32. This creates the possibility of:

- operational upgrades,
- speed improvements,
- and realization-aware optimization

without requiring major reconstruction of alignment geometry.

54.11. AIM Interpretation

Proposition 54.33. *Within AIM, railway alignment is not merely static infrastructure geometry.*

Proposition 54.34. *Railway alignment forms an operationally evaluated dynamic guidance system for constrained vehicle motion.*

Proposition 54.35. *Cant, curvature, vehicle-space behaviour, and runtime evaluation must therefore be interpreted jointly.*

54.12. Connection to AIM

Summary 54.36. Aircraft and railway systems share common dynamic principles related to:

- inertia,
- curvature,
- center-of-gravity behaviour,
- and passenger comfort.

The essential difference lies in the degree of spatial freedom.
Within AIM, this insight motivates:

- runtime-based vehicle-space interpretation,
- realization-aware dynamics,

- and operational evaluation of existing infrastructure.

Thus, railway alignment becomes interpretable as a dynamically evaluated guidance system rather than merely static track geometry.

55. Examples

55.1. Motivation

Remark 55.1. Theoretical concepts become meaningful only when illustrated by concrete examples.

Remark 55.2. The following examples demonstrate how different transition concepts can be interpreted within the curvature-based framework.

—

55.2. Example 1: Clothoid Transition

55.2.1. Definition

Definition 55.3. The clothoid is defined by linear curvature:

$$\kappa(s) = as.$$

55.2.2. Properties

- $\kappa'(s) = \text{const}$
- $\kappa''(s) = 0$

Remark 55.4. The clothoid introduces curvature at a constant rate.

55.2.3. Interpretation

Remark 55.5. In the curvature-based framework, the clothoid represents a linear control function.

Remark 55.6. Its simplicity explains its historical dominance in railway design.

—

55.3. Example 2: Polynomial Transition

55.3.1. Definition

Definition 55.7. A polynomial transition may be defined as

$$\kappa(s) = a_1 s + a_2 s^2 + a_3 s^3.$$

55.3.2. Properties

- $\kappa'(s)$ is variable
- $\kappa''(s)$ is controllable

55.3.3. Interpretation

Remark 55.8. Polynomial transitions allow shaping of higher-order behaviour.

Remark 55.9. They can be used to reduce dynamic excitation by smoothing curvature variation.

—

55.4. Example 3: Comparison of Curvature Profiles

Remark 55.10. Consider two transitions:

- a clothoid,
- a cubic polynomial.

Remark 55.11. Both achieve the same boundary conditions:

$$\kappa(0) = 0, \quad \kappa(L) = \kappa_0.$$

55.4.1. Observation

Remark 55.12. Although geometrically equivalent at the endpoints, the internal behaviour differs.

- clothoid: constant curvature growth,
- polynomial: variable curvature growth.

Remark 55.13. This leads to different dynamic responses.

—

55.5. Example 4: Influence of Speed

Remark 55.14. Lateral acceleration is given by

$$a_{\text{lat}} = v^2 \kappa(s).$$

Remark 55.15. For identical geometry, increasing speed amplifies dynamic effects.

Remark 55.16. This highlights that transition design cannot be separated from operational conditions.

—

55.6. Example 5: Cant Interaction

Remark 55.17. Cant modifies the effective lateral acceleration experienced by the vehicle.

Remark 55.18. The combination of curvature and cant determines the system behaviour.

Remark 55.19. Transition design must therefore consider both geometric and cant evolution.

—

55.7. Example 6: Interpretation as Control Problem

Remark 55.20. Consider $\kappa(s)$ as an input function.

Remark 55.21. The system produces:

- forces,
- vehicle response,
- wear.

Remark 55.22. Different transition curves correspond to different control strategies.

—

55.8. Example 7: Numerical Comparison of Two Transition Laws

55.8.1. Setting

Remark 55.23. Consider two transition curves of equal length

$$L = 100\,\mathrm{m}$$

connecting a straight segment to a circular arc with final curvature

$$\kappa_1 = \frac{1}{1000}\,\mathrm{m}^{-1}.$$

Remark 55.24. Thus, both transitions satisfy

$$\kappa(0) = 0, \qquad \kappa(L) = 0.001.$$

55.8.2. Case A: Clothoid

Definition 55.25. The clothoid is given by

$$\kappa_A(s) = \frac{\kappa_1}{L}\,s = \frac{0.001}{100}\,s = 10^{-5}s.$$

Remark 55.26. Its curvature derivative is constant:

$$\kappa_A'(s) = 10^{-5}\,\mathrm{m}^{-2}.$$

55.8.3. Case B: Cubic Polynomial Transition

Definition 55.27. Consider the normalized cubic law

$$\widehat{\kappa}(u) = 3u^2 - 2u^3, \qquad u = \frac{s}{L}.$$

The corresponding physical curvature law is

$$\kappa_B(s) = \kappa_1 \left(3\left(\frac{s}{L}\right)^2 - 2\left(\frac{s}{L}\right)^3 \right).$$

Remark 55.28. This transition satisfies

$$\kappa_B(0) = 0, \qquad \kappa_B(L) = 0.001,$$

but its curvature derivative is not constant.

55.8.4. Comparison of Curvature Growth

Remark 55.29. At the midpoint $s = 50\,\mathrm{m}$, the clothoid gives

$$\kappa_A(50) = 10^{-5} \cdot 50 = 0.0005.$$

Remark 55.30. For the cubic transition, with $u = 0.5$,

$$\kappa_B(50) = 0.001 \left(3 \cdot 0.5^2 - 2 \cdot 0.5^3\right) = 0.001(0.75 - 0.25) = 0.0005.$$

Remark 55.31. Hence both transitions have the same midpoint curvature, but they differ in the way curvature is introduced.

55.8.5. Comparison of Curvature Derivatives

Remark 55.32. For the clothoid,

$$\kappa_A'(s) = 10^{-5}\,\mathrm{m}^{-2}$$

is constant over the whole interval.

Remark 55.33. For the cubic transition,

$$\kappa_B'(s) = \kappa_1 \left(\frac{6s}{L^2} - \frac{6s^2}{L^3} \right).$$

With $L = 100$ and $\kappa_1 = 0.001$, this becomes

$$\kappa_B'(s) = 0.001 \left(\frac{6s}{10000} - \frac{6s^2}{1000000} \right).$$

Remark 55.34. At the start and end points:

$$\kappa_B'(0) = 0, \qquad \kappa_B'(100) = 0.$$

At the midpoint $s = 50$:

$$\kappa_B'(50) = 0.001\left(\frac{300}{10000} - \frac{15000}{1000000}\right) = 0.001(0.03 - 0.015) = 1.5 \times 10^{-5}\,\mathrm{m}^{-2}.$$

55.8.6. Interpretation

Remark 55.35. The clothoid introduces curvature with constant rate from the very beginning. The cubic transition starts more gently, reaches a higher curvature growth rate in the middle, and then tapers off smoothly toward the end.

Remark 55.36. This shows that identical boundary conditions do not imply identical dynamic behaviour. Even when two transitions connect the same geometric states, their internal curvature evolution may differ substantially.

55.8.7. Dynamic Implication

Remark 55.37. For a vehicle speed v, the lateral acceleration is

$$a_{\mathrm{lat}}(s) = v^2\kappa(s).$$

Hence the rate of change of lateral acceleration is proportional to

$$v^2\kappa'(s).$$

Remark 55.38. The numerical example therefore illustrates a central point of this manuscript: transition curves should be compared not only by endpoint conditions, but by their full curvature evolution and its derivatives.

—

55.9. Example 8: Speed and Dynamic Effects

55.9.1. Setting

Remark 55.39. We consider the same two transition curves as before:

- clothoid (linear curvature),
- cubic transition.

Remark 55.40. Let the vehicle speed be

$$v = 200\,\mathrm{km/h} = 55.56\,\mathrm{m/s}.$$

—

55.9.2. Lateral Acceleration

Proposition 55.41. *The lateral acceleration is given by*

$$a_{\text{lat}}(s) = v^2 \kappa(s).$$

Remark 55.42. With

$$v^2 \approx 3086\,\mathrm{m}^2/\mathrm{s}^2,$$

we obtain for the final curvature

$$a_{\text{lat,max}} = 3086 \cdot 0.001 = 3.086\,\mathrm{m}/\mathrm{s}^2.$$

—

55.9.3. Rate of Change of Acceleration

Definition 55.43. The rate of change of lateral acceleration is

$$\frac{da_{\text{lat}}}{ds} = v^2 \kappa'(s).$$

—

55.9.4. Clothoid

Remark 55.44. For the clothoid:

$$\kappa'(s) = 10^{-5}.$$

Remark 55.45. Thus:

$$\frac{da_{\text{lat}}}{ds} = 3086 \cdot 10^{-5} = 0.0309\,\mathrm{m}/\mathrm{s}^2/\mathrm{m}.$$

Remark 55.46. The acceleration increases at a constant rate along the transition.

—

55.9.5. Cubic Transition

Remark 55.47. At the midpoint $s = 50\,\mathrm{m}$:

$$\kappa_B'(50) = 1.5 \times 10^{-5}.$$

Remark 55.48. Thus:

$$\frac{da_{\text{lat}}}{ds} = 3086 \cdot 1.5 \cdot 10^{-5} = 0.0463\,\mathrm{m}/\mathrm{s}^2/\mathrm{m}.$$

Remark 55.49. At the start and end:

$$\kappa_B'(0) = \kappa_B'(L) = 0$$

and therefore

$$\frac{da_{\text{lat}}}{ds} = 0.$$

—

55.9.6. Interpretation

Remark 55.50. The clothoid introduces acceleration immediately with a constant rate.

Remark 55.51. The cubic transition starts smoothly (zero rate), increases the rate in the middle, and returns smoothly to zero.

—

55.9.7. Engineering Implication

Remark 55.52. From a passenger comfort perspective:

- clothoid: predictable but abrupt start,
- cubic: smoother entry and exit, but stronger variation internally.

Remark 55.53. From a system perspective:

- clothoid: simple control,
- cubic: distributed control with reduced boundary effects.

—

55.9.8. Key Observation

Proposition 55.54. *Even for identical geometry and speed, different curvature laws produce different acceleration profiles.*

—

Summary 55.55. Dynamic behaviour depends not only on curvature magnitude, but on its rate of change.

The numerical example shows that transition curves act as control functions shaping the evolution of forces in the system.

This supports the central thesis that alignment design is fundamentally a problem of curvature control.

—

55.10. Example 9: Optimization Sketch

55.10.1. Motivation

Remark 55.56. Traditional transition design selects predefined curve families.

Remark 55.57. In contrast, the curvature-based framework allows direct optimization of the transition function.

—

55.10.2. Problem Formulation

Definition 55.58. We seek a curvature function

$$\kappa(s), \quad s \in [0, L],$$

such that:

$$\kappa(0) = 0, \quad \kappa(L) = \kappa_1.$$

Remark 55.59. Additionally, we may impose smoothness conditions such as

$$\kappa'(0) = 0, \quad \kappa'(L) = 0.$$

—

55.10.3. Objective

Definition 55.60. A simple objective is to minimize the maximum rate of curvature change:

$$\min_{\kappa} \max_{s \in [0,L]} |\kappa'(s)|.$$

Remark 55.61. This corresponds to minimizing peak dynamic excitation.

—

55.10.4. Candidate Solutions

Remark 55.62. Two candidate solutions are:

- clothoid: constant κ',
- cubic transition: zero boundary derivatives.

—

55.10.5. Qualitative Comparison

Remark 55.63. The clothoid distributes curvature growth evenly over the interval.

Remark 55.64. The cubic transition reduces boundary effects but increases curvature variation in the interior.

—

55.10.6. Optimality Consideration

Proposition 55.65. *An optimal transition balances boundary smoothness and internal variation of curvature.*

Remark 55.66. The optimization problem therefore involves trade-offs between:

- peak curvature derivative,
- smoothness at endpoints,
- distribution of curvature change.

—

55.10.7. General Optimization Framework

Remark 55.67. More general formulations may include objectives such as:

- minimizing jerk,
- minimizing energy,
- minimizing wear-related forces.

Remark 55.68. These objectives can be expressed as functionals of $\kappa(s)$ and its derivatives.

—

55.10.8. Interpretation

Remark 55.69. The optimization problem is defined directly in curvature space.

Remark 55.70. This eliminates the need to select a predefined transition family.

Remark 55.71. Instead, transition curves emerge as solutions of an optimization problem.

—

55.10.9. Connection to AIM

Remark 55.72. Within the AIM framework, the curvature function becomes a parameterized object subject to constraints and optimization.

Remark 55.73. This enables integration with numerical methods and automated design processes.

—

55.10.10. Key Result

Theorem 55.74. *Transition curves can be derived as solutions of optimization problems in curvature space rather than selected from predefined families.*

—

55.11. Summary

Summary 55.75. The present example is formalized in the optimization chapter, where classical transition families are derived as minimizers of curvature-based functionals.

The optimization sketch demonstrates that transition design can be formulated as a mathematical optimization problem.

This shifts alignment design from rule-based construction to model-based optimization and provides a natural link to numerical methods and automated engineering workflows.

—

55.12. Summary

Summary 55.76. The examples demonstrate that:

- transition curves can be compared via $\kappa(s)$,
- different functional forms lead to different system behaviour,
- geometry alone does not determine performance,
- transition design is inherently a control problem.

Two transitions may connect the same straight and circular arc and still exhibit different internal behaviour.

The clothoid is characterized by constant curvature growth, whereas the cubic transition provides a smoother start and end, but stronger variation in the interior.

This confirms that transition design is fundamentally a problem of curvature control rather than mere endpoint matching.

56. Use Cases

56.1. Motivation

Remark 56.1. A theoretical modelling framework becomes substantially more convincing when it is connected to concrete engineering scenarios.

Remark 56.2. The following use cases illustrate how the proposed AIM framework can be applied to realistic railway alignment problems.

Remark 56.3. The examples are not meant as software demonstrations only. They are intended to clarify the interaction between:

- data,
- structured alignment models,
- dynamic interpretation,
- and optimization.

56.2. Use Case 1: Reconstruction of an Existing Alignment

56.2.1. Scenario

Remark 56.4. An existing railway alignment is available only through measured points, legacy file formats, or partially structured engineering data.

Remark 56.5. Typical input may consist of:

- a measured polyline,
- imported TRA/GRA data,
- LandXML fragments,
- or mixed data of uncertain quality.

56.2.2. Task

Remark 56.6. The task is to reconstruct a mathematically coherent alignment model from heterogeneous data.

56.2.3. Pipeline

```
raw data
→ parsing
→ grabbeltisch sorting
→ sparse / pose3 state reconstruction
→ initial guess
→ numerical refinement
```

56.2.4. Interpretation

Remark 56.7. This use case emphasizes that railway geometry is not simply "read from a file". Instead, it is reconstructed as a structured state object.

Remark 56.8. The AIM framework interprets imported data as evidence for an alignment, not as unquestionable truth.

56.3. Use Case 2: Smoothing of an Existing Track

56.3.1. Scenario

Remark 56.9. A measured or existing alignment is geometrically feasible, but exhibits undesirable local irregularities.

Remark 56.10. Examples include:

- noisy curvature estimates,
- abrupt cant changes,
- locally degraded transition zones.

56.3.2. Task

Remark 56.11. The task is to derive an improved alignment preserving the overall route while reducing local dynamic excitation.

56.3.3. Objective Structure

Remark 56.12. Typical objective terms include:

- fitting to the original track,
- smoothness of curvature,
- smoothness of cant,
- reduction of jerk.

56.3.4. Interpretation

Remark 56.13. This is one of the clearest examples showing that alignment should be treated as an optimization problem rather than as a purely geometric description.

56.4. Use Case 3: Design of a New Transition Zone

56.4.1. Scenario

Remark 56.14. A new transition between straight track and circular arc is required.

Remark 56.15. Classically, one would choose a predefined transition family such as a clothoid. In the present framework, the transition may instead be derived through optimization.

56.4.2. Task

Remark 56.16. The task is to construct a transition satisfying:

- boundary curvature conditions,
- cant conditions,
- dynamic smoothness criteria.

56.4.3. AIM Perspective

Remark 56.17. In the AIM framework, the transition is not primarily selected as a named curve family. Rather, it is derived as a controlled curvature evolution.

Remark 56.18. This use case directly illustrates the central thesis that transition curves are control functions rather than merely geometric connectors.

56.5. Use Case 4: Coupled Optimization of Curvature and Cant

56.5.1. Scenario

Remark 56.19. A railway section is to be improved with respect to comfort and dynamic performance.

56.5.2. Task

Remark 56.20. The task is to optimize both:

$$\kappa(s) \quad \text{and} \quad \phi(s),$$

instead of treating cant as a secondary afterthought.

56.5.3. Objective Terms

Remark 56.21. Relevant objective terms include:

- effective lateral acceleration,
- jerk,
- curvature smoothness,
- cant smoothness.

56.5.4. Interpretation

Remark 56.22. This use case demonstrates the transition from pose2-based geometry to pose3-based railway state modelling.

Remark 56.23. It also shows that railway alignment design is inherently a coupled problem.

56.6. Use Case 5: Schwerpunkt-Trassierung

56.6.1. Scenario

Remark 56.24. Instead of designing only the track centerline, one seeks to design the motion of a representative point of the vehicle system.

56.6.2. Task

Remark 56.25. The task is to optimize the trajectory of the center of mass, or of another dynamically relevant reference point, by adjusting curvature and cant.

56.6.3. Model

Remark 56.26. The design variables are:

$$\kappa(s), \quad \phi(s),$$

while the induced trajectory

$$\gamma_c(s)$$

becomes the primary object of evaluation.

56.6.4. Interpretation

Remark 56.27. This use case represents the conceptual extension from classical alignment design toward the new idea of Schwerpunkt-Trassierung.

Remark 56.28. It is one of the clearest expressions of the originality of the present framework.

56.7. Use Case 6: Multi-Track and Network Optimization

56.7.1. Scenario

Remark 56.29. A railway system rarely consists of a single isolated alignment. Instead, one must often deal with:

- parallel tracks,
- switches,
- branching structures,
- coupled corridor geometries.

56.7.2. Task

Remark 56.30. The task is to optimize several alignments simultaneously under topological and geometric coupling constraints.

56.7.3. Typical Constraints

- coincident switch nodes,
- tangent continuity,
- parallel track spacing,
- compatible cant and geometry transitions.

56.7.4. Interpretation

Remark 56.31. This use case shows that the AIM framework extends naturally from single curves to entire railway systems.

56.8. Use Case 7: Interactive Optimization from the Grabbeltisch

56.8.1. Scenario

Remark 56.32. A user imports multiple heterogeneous data sources and arranges them in the grabbeltisch interface according to their semantic role.

56.8.2. Task

Remark 56.33. The system must transform this user-guided sorting into a mathematically meaningful optimization input.

56.8.3. Pipeline

```
drop files
→ detect / parse
→ sort on grabbeltisch
→ build alignment input
→ construct initial guess
→ optimize
→ preview
→ commit to model
```

56.8.4. Interpretation

Remark 56.34. This use case is particularly important because it connects the abstract framework to actual engineering workflow.

Remark 56.35. It also demonstrates that user interaction and mathematical modelling are not opposites, but complementary parts of the same system.

56.9. Use Case 8: Standards and BIM Integration

56.9.1. Scenario

Remark 56.36. An optimized alignment must be exchanged with other systems through standardized or semi-standardized formats.

56.9.2. Task

Remark 56.37. The task is to translate the internal alignment state into layered representations suitable for engineering and BIM workflows.

56.9.3. Interpretation

Remark 56.38. This use case highlights the value of a structured internal model. Only such a model can be mapped consistently to multi-layer data models containing horizontal alignment, profile, and cant.

56.10. Comparative Summary of Use Cases

Use Case	Primary Input	Primary Output
Reconstruction	raw / imported geometry	structured alignment state
Smoothing	measured / degraded track	improved alignment
Transition design	boundary conditions	optimized transition law
Curvature–cant coupling	route + speed + cant	pose3-consistent state
Schwerpunkt	vehicle-based reference	optimized reference trajectory
Network optimization	multi-track graph	globally consistent corridor
Grabbeltisch workflow	user-sorted imports	optimization-ready input
BIM integration	structured state model	exportable layered model

56.11. Conclusion

Summary 56.39. The presented use cases demonstrate that the AIM framework is not limited to theoretical alignment descriptions.

It supports:

- reconstruction,
- optimization,
- dynamic interpretation,
- user-guided engineering workflows,
- and system-level railway design.

Together, these use cases show that alignment can be treated as a structured, dynamic, and optimization-ready object.

57. BIM Alignment Modelling

57.1. Motivation

Remark 57.1. While BIM provides a powerful framework for infrastructure data integration, the modelling of railway alignment within BIM remains structurally fragmented and computationally limited.

Remark 57.2. The AIM framework suggests that alignment should not be treated as a secondary geometric component, but as a primary organizing structure.

Remark 57.3. This chapter introduces the concept of BIMalignment as a synthesis of BIM data modelling and alignment-centred computational representation.

—

57.2. Concept of BIMalignment

Definition 57.4 (BIMalignment)**.** BIMalignment is an alignment-centred modelling approach in which infrastructure geometry, semantics, and dynamics are organized around a canonical alignment state representation.

Remark 57.5. It combines:

- BIM data structures (IFC, LandXML, etc.),
- AIM state representation,
- computational and optimization capabilities.

—

57.3. Core Representation

Remark 57.6. The central object in BIMalignment is the alignment state:

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

Remark 57.7. This representation:

- unifies horizontal and vertical geometry,

- integrates cant,
- enables dynamic interpretation,
- supports optimization.

—

57.4. Alignment as Reference Structure

Remark 57.8. In BIMalignment, infrastructure elements are defined relative to the alignment.

- track geometry follows alignment,
- platforms reference stationing,
- signals are positioned along chainage,
- structures are aligned to geometry.

Remark 57.9. This establishes alignment as the primary coordinate and reference system.

—

57.5. Transformation Pipeline

```
IFC / LandXML / legacy formats
→ parsing
→ normalization
→ AIM state model
→ BIMalignment model
→ analysis / optimization
→ export to BIM
```

Remark 57.10. This pipeline separates:

- data exchange (BIM),
- computation (AIM),
- application (BIMalignment).

—

57.6. Bidirectional Integration

57.6.1. Import

Remark 57.11. BIM data is interpreted and transformed into the AIM state model.

Remark 57.12. This includes:

- reconstruction of alignment geometry,
- interpretation of stationing,
- extraction of cant and profile information.

—

57.6.2. Export

Remark 57.13. Optimized or modified alignments are exported back into BIM formats.

Remark 57.14. This ensures compatibility with:

- existing design tools,
- project workflows,
- lifecycle data systems.

—

57.7. Advantages of BIMalignment

57.7.1. Unified model

Remark 57.15. BIMalignment replaces fragmented representations with a unified alignment-centred model.

—

57.7.2. Computational capability

Remark 57.16. Unlike traditional BIM, the model is directly usable for:

- dynamic evaluation,
- constraint checking,
- optimization.

—

57.7.3. Consistency

Remark 57.17. A single state representation ensures consistency between:

- geometry,
- dynamics,
- constraints,
- and derived quantities.

—

57.7.4. Extensibility

Remark 57.18. The framework allows integration of:

- new transition models,
- advanced vehicle dynamics,
- additional constraint libraries,
- network-level interactions.

—

57.8. Relation to Existing BIM Workflows

Remark 57.19. BIMalignment does not replace existing BIM processes, but augments them.

Remark 57.20. It introduces a computational layer that enables:

- deeper analysis,
- design optimization,
- improved data interpretation.

—

57.9. Role of the Grabbeltisch

Remark 57.21. The interpretation of BIM data often requires human input.

Remark 57.22. The grabbeltisch serves as an intermediate layer where:

- imported data is classified,
- semantic roles are assigned,
- alignment components are structured.

Remark 57.23. This enables robust handling of heterogeneous and imperfect data.

—

57.10. Challenges

57.10.1. Standard complexity

Remark 57.24. BIM standards such as IFC are complex and require careful mapping to alignment-centred models.

—

57.10.2. Data quality

Remark 57.25. Real-world data is often incomplete, inconsistent, or noisy.

—

57.10.3. Adoption

Remark 57.26. Integration into existing workflows requires:

- tool support,
- user acceptance,
- compatibility with established practices.

—

57.11. Outlook

Remark 57.27. BIMalignment provides a foundation for future systems in which:

- alignment is the central modelling object,
- geometry and dynamics are unified,
- optimization becomes an integral part of design,
- BIM evolves from static representation to computational modelling.

—

57.12. Summary

Summary 57.28. BIMalignment extends BIM by introducing an alignment-centred, computationally enabled modelling approach.

It bridges the gap between data representation and engineering computation, enabling consistent, dynamic, and optimization-ready handling of railway alignment within BIM environments.

58. Contribution

58.1. Motivation

Remark 58.1. Railway alignment has traditionally been approached as a geometric design problem, supplemented by engineering rules and dynamic considerations.

Remark 58.2. However, these aspects are often treated separately, leading to fragmented models and limited theoretical coherence.

—

58.2. Central Hypothesis

Theorem 58.3 (Curvature as System Variable)**.** *A railway alignment can be represented as a curvature-driven system in which*

$$\kappa(s)$$

acts as the primary state variable.

Remark 58.4. All geometric, kinematic, and dynamic properties of the alignment follow from $\kappa(s)$ and its derivatives.

—

58.3. System Interpretation of Alignment

Proposition 58.5. *The behaviour of a railway alignment can be described by the mapping*

$$\kappa(s) \longrightarrow \textit{forces} \longrightarrow \textit{system response} \longrightarrow \textit{performance}.$$

Remark 58.6. This establishes alignment as a system rather than a static geometric object.

—

58.4. Unification of Transition Curves

Theorem 58.7 (Unified Transition Representation)**.** *All transition curves can be expressed as curvature functions*

$$\kappa(s)$$

within a normalized domain.

Remark 58.8. This allows comparison of historically distinct transition concepts within a single mathematical framework.

—

58.5. Transition Curves as Control Functions

Theorem 58.9 (Control Interpretation)**.** *Transition curves act as control functions governing the evolution of curvature and thus the dynamic behaviour of the system.*

Remark 58.10. This interpretation replaces the traditional view of transition curves as purely geometric connectors.

—

58.6. Structured Alignment Model

Proposition 58.11. *An alignment can be represented as a structured sequence of elements:*

- *fixed curvature elements,*
- *transition elements,*
- *auxiliary alignment layers such as cant.*

Remark 58.12. This structure separates representation, geometry, and system behaviour.

—

58.7. Coupling with Engineering Variables

Remark 58.13. Alignment design is governed by coupled variables including:

- curvature,
- cant,
- speed,
- and their rates of change.

Proposition 58.14. *Dynamic constraints can be expressed as conditions on*

$$\kappa(s), \quad \kappa'(s), \quad \kappa''(s).$$

—

58.8. Numerical and Optimization Perspective

Remark 58.15. The curvature-based representation allows direct application of numerical integration and optimization methods.

Proposition 58.16. *Alignment design can be formulated as a parameterized optimization problem in curvature space.*

—

58.9. Integration with Data Models

Remark 58.17. Modern railway data models represent alignment as a multi-layer structure consisting of horizontal, vertical, and cant components.

Proposition 58.18. *The structured alignment model proposed here is compatible with such multi-layer representations.*

—

58.10. Main Contributions

Summary 58.19. The contributions of this work can be summarized as follows:

- A curvature-based system interpretation of alignment.
- A unified representation of transition curves.
- The interpretation of transition curves as control functions.
- A structured alignment model separating geometry and behaviour.
- Integration of geometry, dynamics, and engineering constraints.
- A foundation for alignment-based information modelling (AIM).

The numerical comparison of transition laws shows that identical boundary conditions may still lead to substantially different internal curvature evolution and therefore different dynamic behaviour.

—

58.11. Scientific Positioning

Remark 58.20. The presented approach does not introduce a new transition curve, but a new conceptual framework for understanding alignment.

Remark 58.21. It connects previously separated domains:

- geometry,
- dynamics,
- engineering design,
- and data modelling.

—

58.12. Outlook

Remark 58.22. The framework provides a basis for future work in:

- dynamic simulation,
- alignment optimization,
- network-level modelling,
- and BIM integration.

59. Discussion

59.1. Purpose of the Discussion

Remark 59.1. The previous chapters introduced the mathematical foundations, the state-based model, the optimization framework, the software-oriented architecture, and the intended engineering use cases of the AIM approach.

Remark 59.2. The present chapter discusses the significance, scope, and limitations of this framework.

Remark 59.3. Its purpose is not only to summarize previous results, but to position the AIM approach with respect to classical railway alignment design, modern computational methods, and infrastructure data modelling.

59.2. From Geometry to System Modelling

Remark 59.4. A central shift introduced by the AIM framework is the transition from pure geometry to system modelling.

Remark 59.5. Classical alignment theory typically treats the railway path as a curve to be constructed, checked, and documented.

Remark 59.6. In contrast, the AIM framework interprets alignment as a structured state object whose behaviour is governed by curvature, cant, and their derivatives.

Remark 59.7. This leads to a more general view: alignment is not merely a shape, but a functional object with geometric, dynamic, and operational meaning.

59.3. Transition Curves Reinterpreted

Remark 59.8. One of the most important conceptual consequences of the AIM framework is the reinterpretation of transition curves.

Remark 59.9. In many traditional presentations, transition curves appear mainly as named geometric objects, such as clothoids or engineering variants.

Remark 59.10. Within AIM, transition curves are instead understood as control functions for the evolution of curvature and therefore for the dynamic behaviour of the railway system.

Remark 59.11. This reinterpretation is significant because it unifies geometric, dynamic, and optimization-based viewpoints.

59.4. Advantages of the AIM Perspective

59.4.1. Unified internal representation

Remark 59.12. A major strength of AIM lies in the use of a canonical internal model. External data sources may differ substantially in structure and semantics, but can still be mapped into a common state-based representation.

Remark 59.13. This reduces dependence on file formats and enables consistent processing across heterogeneous sources.

59.4.2. Optimization readiness

Remark 59.14. The AIM representation is designed not only for description, but also for computation.

Remark 59.15. Since curvature, cant, and derived quantities are represented explicitly, the model is naturally suited for least-squares methods, SQP, and related optimization approaches.

59.4.3. Dynamic interpretation

Remark 59.16. The framework incorporates dynamics directly through effective lateral acceleration, jerk, cant deficiency, and related quantities.

Remark 59.17. This strengthens the engineering relevance of the model, especially for high-speed and comfort-sensitive applications.

59.4.4. Extensibility

Remark 59.18. The architecture is modular:

- new transition families may be added,
- new constraints may be introduced,
- objective functionals may be reweighted,
- network-level coupling may be extended.

Remark 59.19. This makes AIM suitable not only for one fixed tool, but as a long-term framework.

59.5. Comparison with Classical Approaches

Remark 59.20. Classical railway alignment design often follows a decomposition into:

- horizontal alignment,

- vertical profile,
- cant,
- subsequent rule checking.

Remark 59.21. This decomposition is practical and historically well established. However, it tends to separate modelling from evaluation and often treats dynamics only in a downstream manner.

Remark 59.22. The AIM approach differs by integrating:

- state representation,
- dynamic evaluation,
- and optimization

into a single conceptual structure.

Remark 59.23. This is not intended to invalidate classical methods, but to provide a more general framework into which they can be embedded as special cases.

59.6. Relation to BIM and Data Standards

Remark 59.24. AIM is not designed as a competitor to standards such as LandXML, IFC, or railML.

Remark 59.25. Instead, it serves as a normalization and computational layer between heterogeneous data models and actual engineering reasoning.

Remark 59.26. This is especially important because most standards are primarily designed for exchange and documentation, whereas AIM is designed for interpretation, analysis, and optimization.

Remark 59.27. In that sense, AIM may be viewed as a computational core that complements BIM-style information structures.

59.7. Role of the Grabbeltisch

Remark 59.28. An important practical insight of the present work is that fully automatic interpretation of imported data is neither realistic nor always desirable.

Remark 59.29. The grabbeltisch concept acknowledges that engineering understanding is often needed to assign semantic roles to imported objects.

Remark 59.30. This user-guided intermediate layer is not a weakness of the framework, but a deliberate design choice.

Remark 59.31. It allows the system to combine:

- machine-based parsing,
- human interpretation,
- mathematical reconstruction.

59.8. Limits of the Present Formulation

59.8.1. Physical simplifications

Remark 59.32. The dynamic model introduced in the present manuscript is intentionally simplified.

Remark 59.33. Although effective acceleration and jerk provide meaningful engineering proxies, they do not yet replace full multibody simulation, detailed wheel–rail contact models, or full vehicle-specific dynamics.

59.8.2. Standards formalization

Remark 59.34. While many railway rules can be written as constraint residuals, the full formalization of national and operator-specific regulations remains an open and demanding task.

59.8.3. Topological complexity

Remark 59.35. The extension from single alignments to complete railway networks is one of the strongest features of the framework, but also one of its most challenging aspects.

Remark 59.36. Switches, crossings, branching structures, and corridor-wide coupling introduce substantial mathematical and computational complexity.

59.8.4. Industrial robustness

Remark 59.37. The framework is conceptually strong, but practical deployment requires robust implementations, extensive validation, and careful interface design.

Remark 59.38. In particular, optimization quality depends strongly on:

- good initial guesses,
- stable numerical methods,
- suitable weighting of objectives and constraints.

59.9. Scientific Positioning

Remark 59.39. The contribution of the AIM framework is not the invention of yet another transition curve.

Remark 59.40. Its real contribution lies in the proposal of a broader conceptual framework in which:

- alignment is a state-based object,
- transition curves are control functions,
- geometry and dynamics are treated jointly,
- imported data is interpreted rather than merely accepted,
- and optimization becomes a natural component of design.

Remark 59.41. This positioning places AIM between several established fields:

- railway engineering,
- differential geometry,
- optimization,
- BIM and infrastructure data modelling,
- and interactive engineering software.

59.10. Philosophical Perspective

Remark 59.42. At a deeper level, AIM reflects a shift in engineering modelling: from static representation toward functional interpretation.

Remark 59.43. In this sense, the track is not only a line in space, but a rule-bound, state-dependent object whose meaning arises from its behaviour.

Remark 59.44. This is particularly visible in the move from pose2 to pose3 and from simple curve design to Schwerpunkt-Trassierung.

59.11. Outlook of the Discussion

Remark 59.45. The present discussion suggests that AIM should be understood not as a finished final system, but as a foundational framework.

Remark 59.46. Its long-term value lies in its ability to host:

- richer vehicle models,
- deeper optimization strategies,
- stronger network formulations,
- tighter BIM integration,
- and increasingly automated engineering workflows.

59.12. Summary

Summary 59.47. The AIM framework reinterprets railway alignment as a structured, dynamic, and optimization-ready object.

Its main strengths are the unification of geometry and dynamics, the reinterpretation of transition curves as control functions, and the ability to connect data interpretation, engineering constraints, and numerical optimization.

Its current limitations lie mainly in the simplifications of the dynamic model, the partial formalization of standards, and the remaining challenge of full industrial robustness.

Nevertheless, the framework provides a strong foundation for a new alignment-centred perspective on railway infrastructure modelling.

Teil XIII.

Outlook

Remark 59.48. This part concludes the manuscript by placing the AIM framework into a broader perspective. It discusses how the presented ideas may be extended, generalized, and embedded into future modelling and engineering workflows.

Remark 59.49. The goal is not to close the framework, but to identify its open ends, its possible evolution, and its relevance beyond the present document.

Remark 59.50. The following chapters therefore address BIM-related perspectives and future research directions.

60. BIM and Alignment Modelling

60.1. Motivation

Remark 60.1. Building Information Modelling (BIM) has become a central paradigm in modern infrastructure projects.

Remark 60.2. Its goal is the integration of geometry, semantics, and lifecycle information into a unified digital representation.

Remark 60.3. However, despite its broad scope, the representation of railway alignments within BIM remains conceptually and technically limited.

—

60.2. BIM as a Data Integration Paradigm

Remark 60.4. BIM is fundamentally concerned with:

- object-based modelling,
- semantic enrichment of geometry,
- interoperability across disciplines,
- lifecycle management of infrastructure assets.

Remark 60.5. In this context, geometry is typically associated with objects such as:

- buildings,
- bridges,
- track elements,
- terrain models.

Remark 60.6. Alignment geometry is often embedded implicitly within these objects, rather than treated as a primary modelling entity.

—

60.3. Alignment in BIM Standards

60.3.1. IFC Alignment

Remark 60.7. Recent extensions of IFC introduce explicit alignment entities.

Remark 60.8. These aim to represent:

- horizontal alignment,
- vertical alignment,
- cant (superelevation),
- stationing.

Remark 60.9. While this is a significant step forward, several issues remain:

- separation of components,
- lack of unified state representation,
- limited support for dynamic interpretation,
- complexity of schema usage.

—

60.3.2. Layered Representation

Remark 60.10. In most BIM-related formats, alignment is decomposed into layers:

- horizontal geometry,
- vertical profile,
- cant definition.

Remark 60.11. This decomposition is practical for data exchange, but does not directly support computation or optimization.

—

60.4. Limitations of Current BIM Alignment Modelling

60.4.1. Static representation

Remark 60.12. BIM primarily represents geometry as static data.

Remark 60.13. Dynamic properties such as acceleration, jerk, or vehicle response are not part of the core model.

—

60.4.2. Fragmentation

Remark 60.14. The separation of horizontal, vertical, and cant components leads to a fragmented representation.

Remark 60.15. This fragmentation complicates:

- consistency checks,
- dynamic evaluation,
- optimization-based design.

—

60.4.3. Lack of computational core

Remark 60.16. BIM standards focus on data exchange and documentation.

Remark 60.17. They do not provide a unified computational model for alignment analysis and optimization.

—

60.5. AIM as a Computational Layer for BIM

60.5.1. Conceptual Role

Remark 60.18. The AIM framework can be understood as a computational core that complements BIM data models.

Remark 60.19. Instead of replacing BIM, AIM operates between:

- external data formats,
- and internal engineering reasoning.

—

60.5.2. Canonical Representation

Remark 60.20. AIM introduces a unified representation:

$$(\gamma(s), \theta(s), \kappa(s), \phi(s)).$$

Remark 60.21. This representation:

- integrates horizontal geometry,
- integrates vertical and cant behaviour,
- supports dynamic evaluation,

- is directly usable in optimization.

—

60.5.3. Transformation Pipeline

```
BIM / IFC / LandXML
→ parsing
→ fat representation
→ normalization
→ AIM state model
→ analysis / optimization
→ export
```

Remark 60.22. This pipeline separates:

- data exchange concerns,
- computational modelling,
- engineering reasoning.

—

60.6. Alignment-Centred BIM

60.6.1. Shift of perspective

Remark 60.23. The AIM framework suggests a shift toward alignment-centred modelling.

Remark 60.24. Instead of treating alignment as one component among many, it becomes a primary organizing structure.

—

60.6.2. Implications

- infrastructure elements are referenced to alignment,
- geometry is derived from alignment state,
- dynamic behaviour becomes part of the model,
- optimization becomes a native operation.

—

60.7. Integration with BIM Workflows

60.7.1. Import

Remark 60.25. Existing BIM data is imported via:

- IFC parsing,
- LandXML parsing,
- legacy format interpretation.

—

60.7.2. Processing

Remark 60.26. Within AIM, the imported data is:

- normalized,
- reconstructed,
- enriched with dynamic interpretation,
- potentially optimized.

—

60.7.3. Export

Remark 60.27. Results may be exported back into:

- IFC alignment structures,
- LandXML files,
- other engineering formats.

Remark 60.28. This ensures compatibility with existing BIM ecosystems.

—

60.8. Relation to BIMinfra

Remark 60.29. Infrastructure BIM (often referred to as BIMinfra) extends BIM concepts to linear infrastructure such as railways and roads.

Remark 60.30. AIM can be interpreted as a specialization within BIMinfra focusing on:

- alignment-centric modelling,
- dynamic interpretation,
- optimization-driven design.

—

60.9. Discussion

Remark 60.31. BIM provides a powerful framework for data integration, but lacks a strong computational alignment model.

Remark 60.32. AIM addresses this gap by:

- introducing a unified state representation,
- enabling dynamic evaluation,
- supporting optimization,
- and integrating heterogeneous data sources.

Remark 60.33. Together, BIM and AIM can be seen as complementary:

- BIM for data integration and lifecycle,
- AIM for computation and alignment reasoning.

—

60.10. Summary

Summary 60.34. Current BIM standards provide important structures for data exchange and semantic modelling, but remain limited in their treatment of railway alignment.

The AIM framework complements BIM by introducing a unified, state-based, and optimization-ready representation of alignment.

This enables a transition from static data handling toward dynamic, computational, and alignment-centred infrastructure modelling.

61. Future Work

61.1. Purpose of this Chapter

Remark 61.1. The AIM framework presented in this manuscript establishes a mathematical and conceptual foundation for alignment-centred modelling.

Remark 61.2. However, many aspects remain open and provide opportunities for further research, development, and practical implementation.

Remark 61.3. This chapter outlines potential directions for future work, ranging from mathematical extensions to software architecture and engineering applications.

—

61.2. Extension of the Dynamic Model

61.2.1. From simplified dynamics to vehicle models

Remark 61.4. The current formulation relies on simplified dynamic quantities such as effective lateral acceleration and jerk.

Remark 61.5. Future work should extend this toward more realistic vehicle models, including:

- multi-body vehicle dynamics,
- wheel–rail interaction,
- suspension behaviour,
- track elasticity.

61.2.2. Speed-dependent modelling

Remark 61.6. The integration of variable speed profiles

$$v(s)$$

opens the possibility for more realistic evaluation and optimization of alignments under operational conditions.

—

61.3. Schwerpunkt-Trassierung

61.3.1. Conceptual development

Remark 61.7. The idea of designing the trajectory of a dynamically relevant point, such as the center of mass, represents a fundamental extension of classical alignment design.

Remark 61.8. Further work is required to:

- formalize the mapping from track geometry to vehicle trajectory,
- incorporate full dynamic models,
- define appropriate objective functions.

61.3.2. Integration into optimization

Remark 61.9. A key research direction is the coupling of Schwerpunkt-Trassierung with optimization methods, enabling:

- comfort-driven design,
- dynamic optimization,
- direct evaluation of vehicle response.

—

61.4. Advanced Optimization Methods

61.4.1. SQP extensions

Remark 61.10. While Sequential Quadratic Programming provides a powerful baseline, future work may include:

- tailored SQP formulations for sparse alignment structures,
- improved constraint handling,
- adaptive weighting strategies.

61.4.2. Global optimization

Remark 61.11. Alignment optimization problems are often non-convex.

Remark 61.12. Future research may explore:

- global optimization techniques,
- multi-start strategies,
- hybrid deterministic–stochastic methods.

—

61.5. Network-Level Modelling

61.5.1. From single alignment to network

Remark 61.13. Real railway systems consist of interconnected alignments forming complex networks.

Remark 61.14. Future work should extend the AIM framework to:

- branching structures,
- switches and crossings,
- corridor-wide optimization.

61.5.2. Topological constraints

Remark 61.15. A systematic treatment of topological constraints remains a major challenge.

Remark 61.16. This includes:

- node consistency,
- multi-track coupling,
- network-wide feasibility conditions.

—

61.6. Constraint and Objective Libraries

61.6.1. Constraint formalization

Remark 61.17. Although many railway rules can be expressed as constraints, a complete formalization of regulatory frameworks remains an open task.

Remark 61.18. Future work should aim at:

- systematic encoding of standards,
- validation against real projects,
- parameterization for different railway systems.

61.6.2. Objective design

Remark 61.19. The design of objective functions is crucial for optimization-based alignment design.

Remark 61.20. Potential extensions include:

- cost models,

- maintenance optimization,
- energy efficiency,
- lifecycle considerations.

—

61.7. Integration with BIM and Data Standards

61.7.1. Deeper BIM integration

Remark 61.21. The integration of AIM with BIM workflows can be further developed by:

- tighter coupling with IFC alignment models,
- bidirectional synchronization,
- embedding AIM states within BIM objects.

61.7.2. Standard extensions

Remark 61.22. The AIM framework may also inspire extensions to existing standards, for example:

- inclusion of dynamic properties,
- support for optimization metadata,
- representation of control-based alignment models.

—

61.8. Interactive and Real-Time Systems

61.8.1. Live optimization

Remark 61.23. One of the most promising directions is the development of systems that allow real-time feedback during alignment design.

Remark 61.24. This includes:

- live preview during optimization,
- interactive constraint adjustment,
- immediate visualization of dynamic effects.

61.8.2. User interaction

Remark 61.25. The grabbeltisch concept may be extended to:

- interactive constraint definition,
- visual debugging of alignment states,
- intuitive editing of transition parameters.

—

61.9. AI and Assisted Design

61.9.1. Data-driven approaches

Remark 61.26. Machine learning techniques may complement the AIM framework by:

- learning from existing alignments,
- suggesting initial solutions,
- identifying patterns in design choices.

61.9.2. Hybrid systems

Remark 61.27. A promising direction is the combination of:

- physics-based modelling (AIM),
- data-driven methods (AI),
- interactive user control.

—

61.10. AXTRAN Revival and Beyond

Remark 61.28. Classical systems such as AXTRAN demonstrated the feasibility of optimization-based alignment design.

Remark 61.29. Future work may aim to:

- reconstruct and generalize such systems,
- extend them to network-level problems,
- integrate modern numerical and computational techniques.

—

61.11. Software Architecture and Deployment

61.11.1. Scalability

Remark 61.30. Practical applications require scalable implementations capable of handling large datasets and complex networks.

61.11.2. Multi-view systems

Remark 61.31. Future systems may include:

- synchronized multi-window views,
- combined 2D/3D representations,
- integration of GIS and CAD functionality.

—

61.12. Long-Term Vision

Remark 61.32. In the long term, the AIM framework may contribute to a new paradigm of infrastructure modelling in which:

- alignment becomes the central organizing structure,
- geometry, dynamics, and optimization are unified,
- engineering design becomes an interactive computational process.

—

61.13. Summary

Summary 61.33. The AIM framework opens a wide range of research and development directions.

These include extensions of the dynamic model, integration with BIM, network-level optimization, real-time systems, and AI-assisted design.

Together, these directions point toward a future in which railway alignment is no longer treated as static geometry, but as a dynamic, computational, and optimizable system at the core of infrastructure engineering.

Literaturverzeichnis

[1] E. Andersson, N. G. Nilstam, and L. Ohlsson. Lateral track forces at high speed curving: Comparison of practical and theoretical results of swedish high speed train x2000. *Vehicle System Dynamics*, 25, 1995. Supplement; exact pages to be verified from source.

[2] AREMA. Railway track design, chapter 6, 2003. Manual chapter.

[3] Walter Bloss. *Gleis- und Weichengeometrie*. Transpress, 1978.

[4] Tanita Fossli Brustad and Rune Dalmo. Railway transition curves: A review of the state-of-the-art and future research. *Infrastructures*, 5(43), 2020.

[5] buildingSMART Railway Room. Rwr-ifc rail conceptual model report, 2019. Conceptual model report.

[6] CEN. Bs en 13803:2017 railway applications – track – track alignment design parameters, 2017. Standard.

[7] Constantin Ciobanu. Bloss transition: A short design guide. *PWI Journal*, 133:14–18, 2015.

[8] Manfredo P. do Carmo. *Differential Geometry of Curves and Surfaces*. Prentice-Hall, 1976.

[9] J. Glover. Transition curves for railways. *Proceedings of the Institution of Civil Engineers*, 140:161–179, 1900.

[10] Zulfiqar Habib and Manabu Sakai. Spiral transition curves and their applications. *Scientiae Mathematicae Japonicae*, 59(2):251–262, 2004.

[11] Ernst Hairer, Syvert P. Nørsett, and Gerhard Wanner. *Solving Ordinary Differential Equations I*. Springer, 1993.

[12] Franz Hasslinger. Verfahren zur trassierung eines fahrwegs unter berücksichtigung der schwerpunktbewegung eines fahrzeugs, 2002. Austrian patent AT412975B on Schwerpunkt-based railway alignment.

[13] A. L. Higgins. *The Transition Spiral and Its Introduction to Railway Curves*. Van Nostrand, New York, 1922.

[14] Hofmann-Wellenhof. *GPS: Theory and Practice.* needs to be investigated, 2001.

[15] E. M. Horsburgh. The railway transition curve. *Proceedings of the Royal Society of Edinburgh*, 32:333–347, 1913.

[16] Simon Iwnicki, editor. *Handbook of Railway Vehicle Dynamics.* CRC Press, Boca Raton, 2006.

[17] J. J. Kalker. A fast algorithm for the simplified theory of rolling contact. *Vehicle System Dynamics*, 11(1):1–13, 1982.

[18] J. J. Kalker. *Three-Dimensional Elastic Bodies in Rolling Contact.* Kluwer Academic Publishers, Dordrecht, 1990.

[19] Louis T. Jr. Klauder. A better way to design railroad transition spirals. *ASCE Journal of Transportation Engineering*, 2001. Preprint submitted to ASCE Journal of Transportation Engineering.

[20] Louis T. Jr. Klauder. Railroad curve transition spiral design method based on control of vehicle banking motion, 2001. Patent WO2001098938A1.

[21] Louis T. Jr. Klauder. Railroad curve transition spiral design method based on control of vehicle banking motion, 2007. US application publication US20070027661A9.

[22] Louis T. Jr. Klauder. Railroad curve transition spiral design method based on control of vehicle banking motion, 2009. Patent US7542830.

[23] Louis T. Jr. Klauder. Railroad spiral design and performance, 2012. Conference/publication details to be completed.

[24] Günter Klein. *Trassierung von Verkehrswegen.* Springer, 1998.

[25] Klaus Knothe and Stephen L. Grassie. Modelling of railway track and vehicle/track interaction at high frequencies. *Vehicle System Dynamics*, 22(3–4):209–262, 1993.

[26] W. Koc. Transition curves in railway engineering. *Journal of Transportation Engineering*, 2008. Further metadata to be completed.

[27] W. Koc. Transition curve with smoothed curvature at its ends for railway roads. *Current Journal of Applied Science and Technology*, 22:1–10, 2017.

[28] W. Koc. New transition curve adapted to railway operational requirements. *Journal of Surveying Engineering*, 145, 2019. Issue/pages to be completed.

[29] W. Koc. Smoothed transition curve for railways. *Przeglad Komunikacyjny*, pages 19–32, 2019.

[30] Bengt Kufver. *Optimization of Horizontal Alignment of Railway Tracks.* PhD thesis, Royal Institute of Technology (KTH), 2000.

[31] Björn Kufver. Mathematical description of railway alignments and some preliminary comparative studies. Technical Report 420A, Swedish National Road and Transport Research Institute (VTI), 1997.

[32] Hugo Magalhães, Jorge Ambrósio, José Pombo, and Marco Pereira. Railway vehicle modelling for the vehicle-track interaction compatibility analysis. *Proceedings of the Institution of Mechanical Engineers, Part K: Journal of Multi-body Dynamics*, 230(3):251–267, 2016.

[33] José Martínez-Casas, Egidio Di Gialleonardo, Stefano Bruni, and Luis Baeza. A comprehensive model of the railway wheelset-track interaction in curves. *Journal of Sound and Vibration*, 333:4152–4169, 2014.

[34] E. Mieloszyk. Modern transition curve design. *Transport Problems*, 2012. Further metadata to be completed.

[35] A. W. Miller. The transition spiral. *Australian Surveyor*, 7:518–526, 1939.

[36] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization.* Springer, 2006.

[37] A. Pirti, M. Yücel, and T. Ocalan. Transrapid and the transition curve as sinusoid. *Tehnicki Vjesnik*, 23:315–320, 2016.

[38] Oskar Polach. A fast wheel-rail forces calculation computer code. *Vehicle System Dynamics*, 33, 1999. Supplement; exact issue/pages to be verified from source.

[39] José Pombo and Jorge Ambrósio. Application of a wheel–rail contact model to railway dynamics in small radius curved tracks. *Multibody System Dynamics*, 19(1–2):91–114, 2008.

[40] José Pombo, Jorge Ambrósio, and Manuel Silva. A new wheel–rail contact model for railway dynamics. *Vehicle System Dynamics*, 45(2):165–189, 2007.

[41] William H. Press, Saul A. Teukolsky, William T. Vetterling, and Brian P. Flannery. *Numerical Recipes.* Cambridge University Press, 2007.

[42] Andrew Pressley. *Elementary Differential Geometry.* needs to be investigated, 2010.

[43] Ahmed A. Shabana, Karim E. Zaazaa, and Hiroyuki Sugiyama. *Railroad Vehicle Dynamics: A Computational Approach.* CRC Press, Boca Raton, 2008.

[44] S. Shebl. Geometrical analysis of non-linear curvature transition curves of high speed railways. *Asian Journal of Current Engineering and Maths*, 5:52–58, 2016.

[45] Dirk J. Struik. *Lectures on Classical Differential Geometry.* needs to be investigated, 1961.

[46] E. Tari. The new generation transition curves. *ARI Bulletin, Istanbul Technical University*, 54:35–41, 2004.

[47] TODO. Ifc railway, 2015. Placeholder entry; replace with exact source metadata.

[48] UIC. Uic code 519: Method for determining the running safety of railway vehicles, 2004. International Union of Railways.

[49] A. H. Wickens. *Fundamentals of Rail Vehicle Dynamics.* Swets & Zeitlinger, Lisse, 2003.

[50] R. Wojtczak. *Charakterystyka Krzywej Przejściowej Wiener Bogen.* Publishing House of Poznan University of Technology, Poznan, 2017.

[51] K. Zboinski and P. Woznica. Optimization of polynomial transition curves from the viewpoint of jerk value. *Archives of Civil Engineering*, 63, 2017. Issue/pages to be completed.

[52] Krzysztof Zboinski and Monika Golofit-Stawinska. Dynamics of 2 and 4-axle railway vehicles in transition curves above critical velocity. In *Proceedings of the Second International Conference on Railway Technology: Research, Development and Maintenance*, 2014. Paper 265.

Index